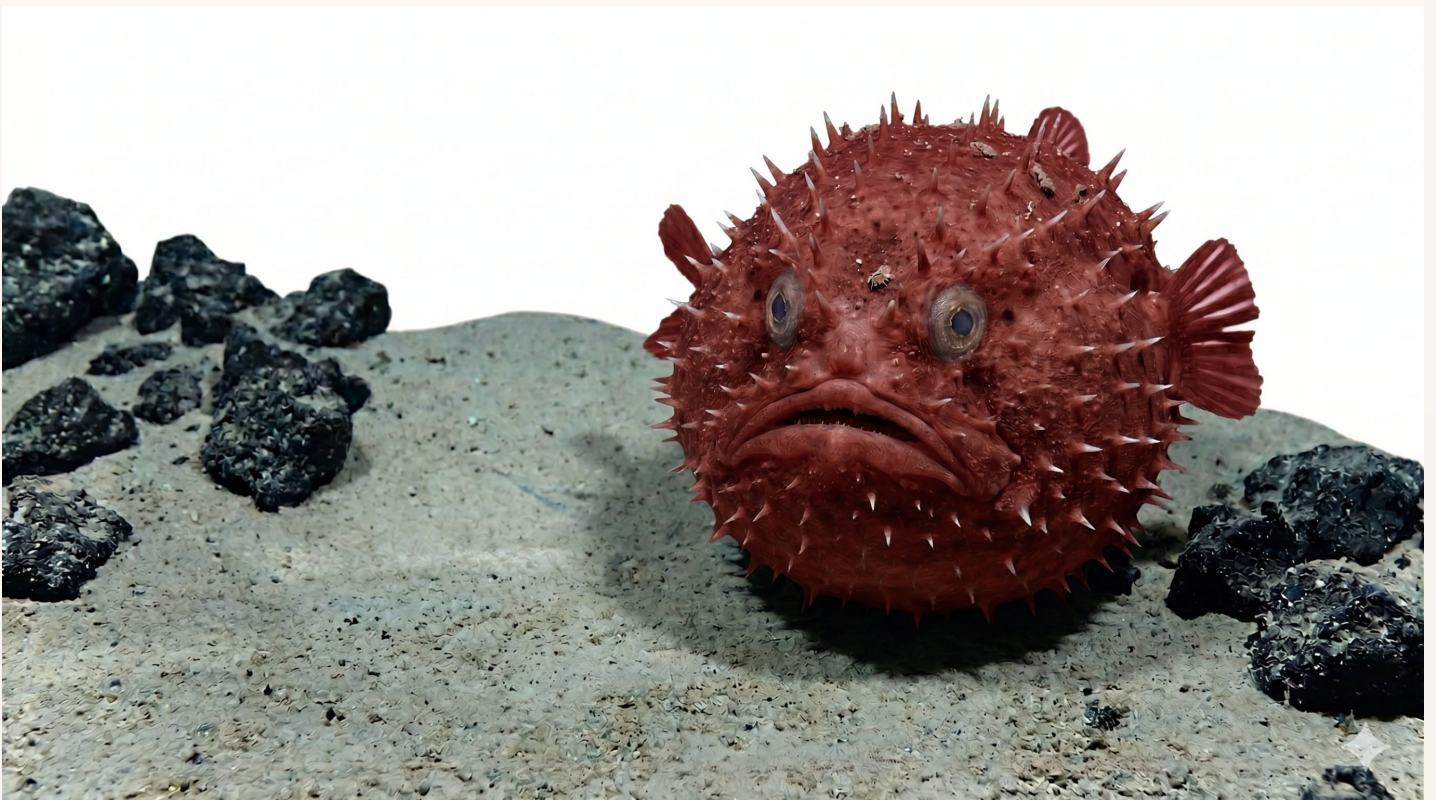


FOUNDATIONS • TRANSFORMERS • SYSTEM DESIGN

Deep Learning for AI Engineers

**The Complete Guide to Neural Networks, Transformers,
Large-Scale AI Systems, and End-to-End Deep Learning
System Design.**

*From a single neuron to a GPT you build from scratch: 17 core chapters,
20 production system case studies, and 6 runnable, gradient-checked
projects, every line of code in the book actually runs.*



Deep Learning for AI Engineers

The Complete Guide to Neural Networks, Transformers, Large-Scale AI Systems, and End-to-End Deep Learning System Design

First edition, 2026.

Copyright © 2026 Lamhot Siagian. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in reviews and certain other noncommercial uses permitted by copyright law.

Published by **AI Engineering Insider**.

Web: <https://aiengineeringinsider.com>

Newsletter: <https://aiengineeringinsider.substack.com/subscribe>

Companion code. All source code in this book is available, and free to use under its accompanying open-source license, at <https://github.com/lamhotsiagian/deep-learning>.

Trademarks. Product and company names mentioned herein (including PyTorch, JAX, TensorFlow, CUDA, and the systems discussed in the case studies) may be the trademarks of their respective owners and are used for identification and editorial purposes only, with no intent to infringe.

Disclaimer of liability. This book is provided for educational purposes on an “as is” basis. While every effort has been made to ensure the technical accuracy of the explanations and code, and every code listing was executed and its output captured directly, the author makes no representations or warranties with respect to the accuracy or completeness of the contents and assumes no liability for any damages arising from the use of the information or code herein.

Preface

Subscribe → aiengineeringinsider.substack.com/subscribe

Most deep-learning books fail in one of two ways. The academic ones bury the intuition under measure theory and leave you unable to ship anything. The applied ones hand you `model.fit()` and leave you helpless the moment it silently doesn't work. This book is written for the engineer stuck between them: someone who can already code, who wants to *understand* what happens between the data and the prediction, and who will eventually have to debug, optimize, and deploy the thing, not just demo it.

What makes this book different

Three commitments shape every chapter.

First, you build before you borrow. You will implement a perceptron, backpropagation, an autograd engine, a CNN, an RNN, and a GPT *from scratch in NumPy* before you ever call a framework. Only then do PyTorch and JAX appear, as tools whose magic you have already removed. When you later type `loss.backward()`, you will know exactly what it does because you wrote it.

Second, every number is real. There are no idealized outputs in this book. Every console block is captured verbatim from a script that runs, and every hand-derived gradient is checked numerically. While writing the book I found, and fixed, several outputs in early drafts that were simply wrong; the discipline of running everything is the only reliable defense, and it is the discipline this book tries to teach. The complete, tested code lives at <https://github.com/lamhotsiagian/deep-learning>.

Third, the model is only one stage of nine. Real systems are a pipeline, objective, data, preprocessing, splitting, architecture, training, tuning, evaluation, and deployment with monitoring. The hands-on projects walk that full lifecycle, and the twenty production case studies show how Tesla, OpenAI, and others realize each stage at scale. The neural network earns you a seat at the table; the pipeline is how you design the table.

Who this book is for

You should be comfortable writing Python and reading basic linear algebra and calculus (matrix multiplication, partial derivatives, the chain rule). You do *not* need prior machine-learning experience, the book starts at a single neuron. It is aimed at AI and machine-learning engineers, software engineers moving into ML, and anyone preparing for deep-learning or AI system-design interviews, where the questions in each chapter's evaluation are deliberately the ones a strong candidate is asked.

How to use it

Read in order, the chapters compound. Keep a Python REPL open; type the from-scratch code yourself at least once (the NumPy MLP and the character RNN are short enough to hold in your head). Treat the warning boxes as hard-won scar tissue and the chapter evaluations as a self-test before you move on. Section [19.7](#) explains how to set up a virtual environment and run everything.

Lamhot Siagian

AI Engineering Insider

About This Book & AI Engineering Insider

Subscribe → aiengineeringinsider.substack.com/subscribe

About this book

Deep Learning for AI Engineers is a complete, practitioner-grade guide that takes you from a single neuron to a GPT you build, train, and run yourself. It is written for readers who already write code and want the layer underneath the abstractions: the mechanics behind backpropagation, the training tricks that make a network actually learn, the architectures (CNNs, RNNs, and Transformers) that power modern AI, and the system-design thinking behind the products that deploy them at scale.

The book is deliberately end-to-end. Part I builds the foundations from scratch; Part II covers the practitioner’s toolkit (activations, losses, optimizers, initialization, regularization, schedules, evaluation); Part III moves to real frameworks (PyTorch and JAX) and the discipline of debugging; Part IV builds the architecture families, convolutional, recurrent, and the Transformer, from first principles. Two bonus parts then connect theory to practice: twenty production **system-design** case studies (Tesla, ChatGPT, YouTube, AlphaFold, and more), and six **hands-on projects** that walk the full data → train → evaluate → deploy lifecycle, culminating in a Transformer trained from scratch.

★ Key Insight

All code in this book is real, runnable, and tested. Every console output is captured verbatim from a script that actually executes, and every from-scratch gradient is verified numerically. The complete companion repository, per-chapter code, the `coredl` mini-framework, and all six projects, is available at:

<https://github.com/lamhotsiagian/deep-learning>

About AI Engineering Insider

AI Engineering Insider is a technical publication for software engineers and AI practitioners who are tired of content that is long on enthusiasm and short on substance. Every guide is written for readers who already know how to write code and want to go deeper: the mechanics behind the abstractions, the failure modes the documentation doesn’t mention, and the judgment calls that only show up after you’ve debugged the thing at 2 a.m.

Other titles in the AI Engineering Series include:

- *The AI Engineer’s System Design Interview Guide*

- *The AI Engineer's Reinforcement Learning Interview Guide*
- *Transformers & Vision Transformers: 50 Interview Questions*
- *The AI Product Manager's Career Guide*

For new releases, technical deep-dives, and frontier-model analysis, subscribe at <https://aiengineeringinsider.substack.com> or visit <https://aiengineeringinsider.com>.

How to Use This Book

Subscribe → aiengineeringinsider.substack.com/subscribe

This book is built to be read with a pencil, or a Python REPL, in hand. Each chapter follows roughly the same arc: intuition first, then the math, then the mechanics, then where it breaks in practice. Several recurring box types flag different kinds of content so you can scan for what you need:

★ Key Insight

Key Insight marks the one idea in a section that, if you remember nothing else, you should remember this.

! Pitfall / Warning

Pitfall / Warning flags a mistake that is common enough, and costly enough, that it earns its own box.

✓ Best Practice

Best Practice is a concrete, actionable habit worth adopting, not just a thing to know but a thing to do.

From the Field

From the Field is a first-person aside: a war story, a debugging session, or a hard-won lesson that didn't fit neatly into the main text.

Definition

A precise, citable definition of a term, given the first time it matters, separate from the surrounding narrative explanation.

Q0

Interview Q&A boxes pose a question the way an interviewer might ask it.

The answer that follows is written the way a strong candidate would actually say it out loud, not as a textbook proof. Every chapter carries several of these, and they are listed individually in the Table of Contents so you can use this book as interview-prep reference material, not just a cover-to-cover read.

Code listings use a labeled header bar (e.g., `example.py`) and are written to run as-is in a standard Python environment with NumPy and, where relevant, PyTorch or JAX installed. Console output blocks show the actual output you should expect to see, not idealized output, so you can use them to check your own implementation against a known-good run.

Contents

Preface	i
About This Book & AI Engineering Insider	iii
How to Use This Book	v
Part I Foundations	1
1 The Perceptron: Where It All Started	2
1.1 The Biological Inspiration	2
1.2 The Mathematical Perceptron	2
1.3 Architecture Diagram	3
1.4 Geometric Intuition: A Linear Decision Boundary	4
1.5 The Perceptron Learning Rule	5
1.6 The XOR Problem	5
1.7 From Perceptron to Modern Artificial Neuron	6
1.8 Chapter Summary	6
1.9 Chapter Evaluation	6
1.10 Answers	8
2 Multi-Layer Networks & Forward Pass	11
2.1 Why Depth: The Universal Approximation Theorem	11
2.2 Layers, Weights, and Biases as Matrices	11
2.3 The Forward Pass	12
2.4 Architecture Diagram	13
2.5 Width vs. Depth	15
2.6 Computational Graphs	15
2.7 Chapter Summary	16
2.8 Chapter Evaluation	16
2.9 Answers	18
3 Backpropagation from Scratch	21
3.1 The Credit Assignment Problem	21
3.2 Chain Rule Refresher	21
3.3 Backprop Through a Single Neuron	22
3.4 Backprop Through a Layer (Vectorized)	22
3.5 Backprop Through Multiple Layers	23
3.6 Architecture Diagram	24
3.7 Computational Graphs and Reverse-Mode Autodiff	24
3.8 Common Derivation Mistakes	25
3.9 Vanishing and Exploding Gradients: First Encounter	25

3.10	Chapter Summary	26
3.11	Chapter Evaluation	26
3.12	Answers	27
4	Build Your Own Mini Framework	30
4.1	Why Build a Toy Framework	30
4.2	Designing a Value Class	30
4.3	Implementing Forward Operations	31
4.4	The Backward Pass: Topological Sort	33
4.5	Architecture Diagram	33
4.6	Building a Tiny MLP	34
4.7	Training It End to End	35
4.8	Gradient Checking Your Engine	36
4.9	Chapter Summary	37
4.10	Chapter Evaluation	37
4.11	Answers	39
Part II	Making Networks Learn Well	42
5	Activation Functions: ReLU, Sigmoid, GELU & Why	43
5.1	Sigmoid and Tanh: The Original Choices	43
5.2	ReLU: Simplicity, Sparsity, and Dying Units	44
5.3	Leaky ReLU and Parametric ReLU	45
5.4	GELU and Swish/SiLU	45
5.5	Architecture Diagram	46
5.6	Softmax for Output Layers	46
5.7	Choosing an Activation by Layer Position and Task	47
5.8	Chapter Summary	48
5.9	Chapter Evaluation	48
5.10	Answers	49
6	Loss Functions: MSE, Cross-Entropy, Contrastive	52
6.1	Regression Losses	52
6.2	Classification Losses	53
6.3	Why Cross-Entropy Pairs with Softmax	53
6.4	Contrastive Loss	54
6.5	Architecture Diagram	55
6.6	Choosing a Loss Function for Your Task	56
6.7	Diagnosing a Loss That Won't Go Down	56
6.8	Chapter Summary	57
6.9	Chapter Evaluation	57
6.10	Answers	58
7	Optimizers: SGD, Momentum, Adam, AdamW	61
7.1	Batch, Stochastic, and Mini-Batch Gradient Descent	61
7.2	Momentum	61
7.3	Adagrad and RMSProp: Adaptive Learning Rates	62
7.4	Adam: Combining Momentum and Adaptive Rates	62

7.5	AdamW: Decoupling Weight Decay	64
7.6	Architecture Diagram	64
7.7	Practical Optimizer Selection	65
7.8	Chapter Summary	65
7.9	Chapter Evaluation	66
7.10	Answers	67
8	Weight Initialization & Training Stability	70
8.1	Why Naive Initialization Fails	70
8.2	Signal Propagation: Keeping Variance Stable	71
8.3	Xavier/Glorot Initialization	71
8.4	He Initialization	71
8.5	Architecture Diagram	72
8.6	Diagnosing Bad Initialization in Practice	73
8.7	Chapter Summary	74
8.8	Chapter Evaluation	75
8.9	Answers	76
9	Regularization: Dropout, Weight Decay, BatchNorm	79
9.1	Overfitting vs. Underfitting, Revisited	79
9.2	Architecture Diagram	79
9.3	L1 and L2 Weight Regularization	80
9.4	Dropout	80
9.5	Batch Normalization	81
9.6	Layer Normalization vs. Batch Normalization	82
9.7	Data Augmentation	82
9.8	Early Stopping	83
9.9	Chapter Summary	83
9.10	Chapter Evaluation	84
9.11	Answers	85
10	Learning Rate Schedules & Warmup	88
10.1	Why a Fixed Learning Rate Is Rarely Optimal	88
10.2	Step Decay and Exponential Decay	88
10.3	Cosine Annealing	88
10.4	Warmup	89
10.5	Cyclical Learning Rates and the LR Range Test	90
10.6	Architecture Diagram	91
10.7	Practical Scheduling Recipes	92
10.8	Chapter Summary	92
10.9	Chapter Evaluation	92
10.10	Answers	94
11	Evaluation & Diagnosis	96
11.1	Train, Validation, and Test Splits	96
11.2	Accuracy, Precision, Recall, F1	96
11.3	Confusion Matrices	97
11.4	ROC Curves and AUC	98

11.5	k-Fold Cross-Validation	98
11.6	Reading Loss Curves	99
11.7	Architecture Diagram	99
11.8	Hyperparameter Tuning Workflow	100
11.9	Chapter Summary	100
11.10	Chapter Evaluation	101
11.11	Answers	102
Part III Real Frameworks		105
12	Introduction to PyTorch	106
12.1	Tensors: PyTorch's Core Data Structure	106
12.2	Autograd: Comparing to Your Chapter 4 Engine	106
12.3	nn.Module: Building Networks Declaratively	107
12.4	Optimizers and the Training Loop	108
12.5	DataLoader and Dataset	109
12.6	GPU Placement	110
12.7	Saving, Loading, and Checkpointing	111
12.8	A Minimal, Correct Training Loop Template	111
12.9	Architecture Diagram	112
12.10	Chapter Summary	112
12.11	Chapter Evaluation	113
12.12	Answers	114
13	Introduction to JAX	117
13.1	JAX's Philosophy: Pure Functions, No Hidden State	117
13.2	grad: Automatic Differentiation as a Function Transformation	117
13.3	jit: Just-In-Time Compilation	119
13.4	vmap: Automatic Vectorization	120
13.5	PyTrees	120
13.6	A Minimal Training Loop in JAX	121
13.7	Architecture Diagram	122
13.8	JAX vs. PyTorch: When Each Makes Sense	123
13.9	Chapter Summary	123
13.10	Chapter Evaluation	123
13.11	Answers	125
14	Debugging Neural Networks	128
14.1	The Debugging Mindset: Isolate Before You Tune	128
14.2	Shape Errors and How to Localize Them Fast	128
14.3	NaN/Inf Losses: Common Causes and Fixes	129
14.4	Loss Not Decreasing: A Decision Tree	130
14.5	Loss Decreasing but Model Not Generalizing	131
14.6	Gradient Checking and Gradient Flow Visualization	131
14.7	Overfitting a Single Batch as a Sanity Test	132
14.8	Reproducibility: Seeding and Determinism	133
14.9	Architecture Diagram	133

14.10 Chapter Summary	133
14.11 Chapter Evaluation	134
14.12 Answers	135
Part IV Architectures	138
15 Convolutional Neural Networks	139
15.1 Why Fully Connected Layers Fail on Images	139
15.2 The Convolution Operation	139
15.3 Stride, Padding, and Output Size	141
15.4 Pooling: Max and Average	141
15.5 Receptive Fields	142
15.6 The Canonical CNN Pattern	143
15.7 Architecture Diagram	144
15.8 A Brief Tour of Milestone Architectures	144
15.9 Transfer Learning	145
15.10 Chapter Summary	145
15.11 Chapter Evaluation	145
15.12 Answers	147
16 Recurrent Neural Networks, LSTM & GRU	150
16.1 Sequential Data and the Limits of Feedforward Networks	150
16.2 The Vanilla RNN: Hidden State as Memory	150
16.3 Backpropagation Through Time	151
16.4 Vanishing Gradients in RNNs, Revisited	151
16.5 LSTM: Forget, Input, and Output Gates	152
16.6 Architecture Diagram	153
16.7 GRU: A Simplified Gating Alternative	153
16.8 LSTM vs. GRU: When Each Is Preferable	154
16.9 Where RNNs Still Matter Today	154
16.10 Chapter Summary	155
16.11 Chapter Evaluation	155
16.12 Answers	156
17 Transformers & Attention	159
17.1 Transformer Foundations	159
17.2 The Self-Attention Mechanism	164
17.3 Multi-Head Attention	168
17.4 Encoder–Decoder Architecture	172
17.5 Vision Transformers (ViT)	175
17.6 CNN vs. ViT, Hybrids, and Modern Techniques	177
17.7 Quick Reference	181
Part V Bonus: Systems at Scale	182
18 Bonus: 20 Deep Learning Systems in Production	183
18.1 The Twenty Systems at a Glance	183

18.2 Case 1: Tesla Autopilot & Full Self-Driving	185
18.3 Case 2: OpenAI ChatGPT	186
18.4 Case 3: Google Search Ranking	187
18.5 Case 4: YouTube Recommendations	188
18.6 Case 5: Netflix Recommendations	189
18.7 Case 6: Meta Feed & Ads Ranking (DLRM)	189
18.8 Case 7: Spotify Music Recommendation	190
18.9 Case 8: Uber DeepETA	191
18.10 Case 9: Amazon (Recommendations & Alexa)	192
18.11 Case 10: TikTok / ByteDance Monolith	193
18.12 Case 11: DeepMind AlphaFold 2	193
18.13 Case 12: Google Translate	194
18.14 Case 13: Pinterest PinSage	195
18.15 Case 14: Waymo Autonomous Driving	196
18.16 Case 15: Stable Diffusion / Latent Diffusion	196
18.17 Case 16: GitHub Copilot	197
18.18 Case 17: Stripe Radar (Fraud Detection)	198
18.19 Case 18: LinkedIn Feed & Connections	199
18.20 Case 19: DoorDash / Instacart Logistics	199
18.21 Case 20: NVIDIA DLSS	200
18.22 The Twenty Systems as End-to-End Pipelines	201
18.23 Cross-Cutting Lessons	202
 Part VI Bonus: Build It Yourself	 203
19 Bonus: Six Hands-On Projects, Step by Step	204
19.1 Project 1: A Multi-Layer Perceptron in Pure NumPy	205
19.2 Project 2: A Custom ResNet on CIFAR-10	208
19.3 Project 3: A Character-Level RNN in Pure NumPy	211
19.4 Project 4: Seq2Seq with Bahdanau Attention	214
19.5 Project 5: A VAE and a Diffusion Model	216
19.6 Project 6: A GPT-Style Transformer From Scratch	219
19.7 Where to Go Next	222
Running the Code	223
19.8 Repository Layout	223
19.9 Step 1: Create a Virtual Environment	223
19.10 Step 2: Install the Requirements	223
19.11 Step 3: Run It	224
Appendix A: Notation Reference	226
Common Abbreviations	227
Appendix B: Glossary	228
Appendix C: Recommended Next Steps	231
If You Want to Go Deeper on Architecture	231
If You Want to Go Deeper on Training at Scale	231
If You Want to Go Deeper on LLMs Specifically	231

If You Want Interview-Ready Depth	232
A Closing Note	232
References	233

PART I

Foundations

Every architecture, every optimizer, every training trick in modern deep learning rests on four ideas: a neuron, a forward pass, backpropagation, and the discipline of building it yourself before you trust a framework to do it for you. This part is slow on purpose. If Chapters 1 through 4 are solid, everything else in this book is just variation on a theme.

The Perceptron: Where It All Started

Subscribe → aiengineeringinsider.substack.com/subscribe

The original 1958 idea, why it failed, and why it still matters.

Every neural network you will ever train, from a two-layer toy model to a hundred-billion-parameter transformer, is built out of the same atomic unit. That unit was first formalized in 1958 by Frank Rosenblatt as the **perceptron**. It is worth spending real time here, not because the perceptron itself is useful (it isn't, on its own), but because every confusion you will later have about activations, decision boundaries, and why depth matters traces back to understanding this one simple model precisely.

1.1 The Biological Inspiration

The perceptron was loosely inspired by the biological neuron: dendrites receive signals, the cell body aggregates them, and if the aggregate signal is strong enough, the neuron fires an output along its axon. Rosenblatt's insight was to formalize “aggregate, then decide” as a mathematical operation that could, in principle, be learned from data rather than hand-programmed.

! Pitfall / Warning

The biological analogy is useful for building intuition once, but stop using it as soon as the math is in front of you. Real biological neurons are vastly more complex (dendritic computation, spike timing, plasticity rules that bear no resemblance to gradient descent), and over-relying on the analogy tends to produce sloppy intuitions about what artificial neurons actually compute.

1.2 The Mathematical Perceptron

Strip away the biology and the perceptron is a precise, small piece of math. Given an input vector $\mathbf{x} \in \mathbb{R}^n$, a weight vector $\mathbf{w} \in \mathbb{R}^n$, and a scalar bias b , the perceptron computes:

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b$$

and then applies a step function to produce a binary output:

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Perceptron

A perceptron is a linear classifier: it computes a weighted sum of its inputs plus a bias, then applies a hard threshold (the step function) to produce a binary decision. The weights and bias are the only learnable parameters.

That's the entire forward computation. No hidden layers, no nonlinearity beyond the final threshold, just a weighted sum and a yes/no decision.

1.2.1 A Worked Example

Suppose you want a perceptron to decide whether to approve a small loan based on two features: credit score (normalized to $[0, 1]$) and existing debt (normalized to $[0, 1]$, higher is worse). A plausible (untrained, hand-picked for illustration) set of weights might be $w_1 = 3$ (credit score, positive influence), $w_2 = -2$ (debt, negative influence), and $b = -1$.

perceptron_example.py

```
def perceptron(x1, x2, w1=3.0, w2=-2.0, b=-1.0):
    z = w1 * x1 + w2 * x2 + b
    return 1 if z >= 0 else 0

# High credit score, low debt -> approve
print(perceptron(0.9, 0.1))    # z = 2.7 - 0.2 - 1 = 1.5 -> 1

# Low credit score, high debt -> deny
print(perceptron(0.2, 0.8))    # z = 0.6 - 1.6 - 1 = -2.0 -> 0
```

Output

1
0

1.3 Architecture Diagram

Figure 1.1 shows the full perceptron forward pass as a data-flow diagram: each input feature x_i is scaled by its weight w_i , the scaled signals (plus the bias b) are summed into the pre-activation z , and a hard step function maps z to a binary decision. Read it left to right; this is the exact computation every later neuron in this book generalizes.

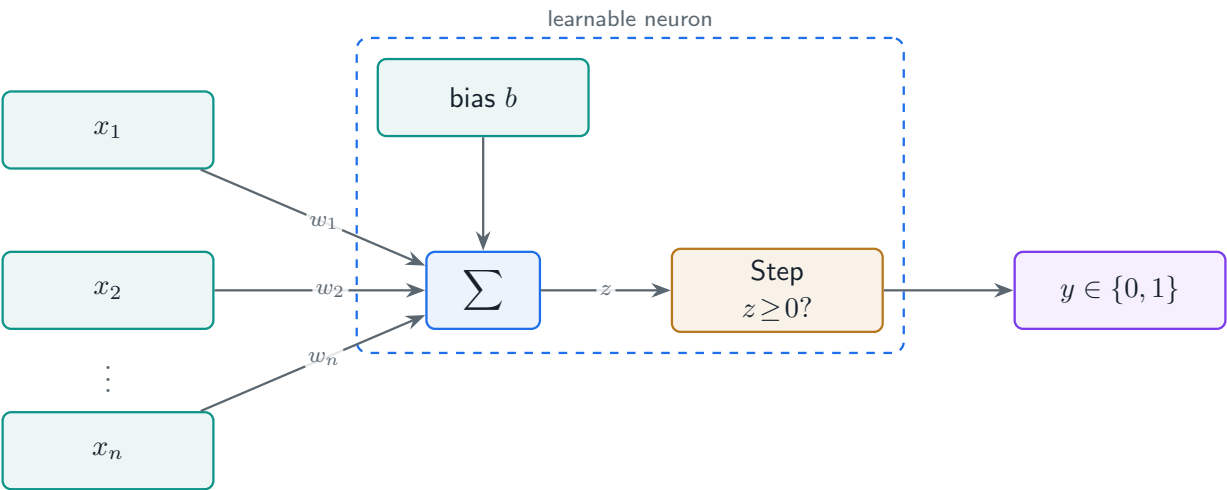


Figure 1.1: The perceptron forward pass: weighted sum followed by a hard threshold. The dashed box is the single learnable “neuron”; everything inside it is reused, unchanged, by every modern artificial neuron.

Table 1.1 contrasts the perceptron with the two models it is most often confused with. The structural core (a weighted sum) is identical; the output map and training rule are what differ, and those differences are exactly why gradient-based deep learning exists.

Table 1.1: Perceptron versus logistic regression versus the modern artificial neuron.

Property	Perceptron (1958)	Logistic Regression	Modern Neuron
Output map	Hard step $\{0, 1\}$	Sigmoid $\rightarrow (0, 1)$	Smooth activation (ReLU, GELU, ...)
Output meaning	Class label	Calibrated probability	Pre/post-activation feature
Differentiable?	No (zero a.e.)	Yes	Yes
Trained by	Perceptron rule	Gradient descent on cross-entropy	Backprop + SGD/Adam
Convergence	Guaranteed iff linearly separable	Convex, global optimum	Non-convex (in a network)
Composable into layers?	Not usefully	Rarely	Yes — the entire point

1.4 Geometric Intuition: A Linear Decision Boundary

The equation $\mathbf{w}^\top \mathbf{x} + b = 0$ defines a hyperplane in input space. In two dimensions, that hyperplane is just a line. Everything on one side of the line gets classified as 1; everything on the other side gets classified as 0. The weight vector $\mathbf{w}$ is normal (perpendicular) to that line, and the bias b shifts the line away from the origin.

★ Key Insight

A perceptron can only ever carve up its input space with a single straight line (or, in higher dimensions, a single flat hyperplane). If your data isn’t **linearly separable**, no setting of $\mathbf{w}$ and b will ever classify it perfectly. This single fact is the entire reason multi-layer networks exist.

1.5 The Perceptron Learning Rule

Rosenblatt also gave the perceptron a learning algorithm, and it predates backpropagation by decades. For each misclassified training example $(\mathbf{x}, y_{\text{true}})$, update the weights:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y_{\text{true}} - y_{\text{pred}}) \mathbf{x} \quad b \leftarrow b + \eta(y_{\text{true}} - y_{\text{pred}})$$

where η is the learning rate. If the prediction was correct, the update is zero and nothing changes. If the perceptron predicted 0 but the true label was 1, the weights nudge toward the input. If it predicted 1 but the true label was 0, the weights nudge away.

✓ Best Practice

Implement the perceptron learning rule from scratch at least once, on a tiny 2D linearly separable dataset, and plot the decision boundary after each epoch. Watching the line rotate and shift into place is the single fastest way to build real intuition for what “learning” means in a neural network, before gradients and loss surfaces enter the picture.

The **Perceptron Convergence Theorem** guarantees that if the data is linearly separable, this algorithm will find a separating hyperplane in a finite number of steps. It says nothing about what happens if the data is not linearly separable, and that gap turned out to be the whole problem.

1.6 The XOR Problem

In 1969, Marvin Minsky and Seymour Papert published *Perceptrons*, a rigorous analysis of what single-layer perceptrons could and could not compute. Their most famous result: a single perceptron cannot learn the XOR function.

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Plot these four points. The two points labeled 1 sit on opposite corners from each other, and so do the two points labeled 0. No single straight line can separate the 1s from the 0s. XOR is not linearly separable, so by the geometric argument above, no perceptron, no matter how it’s trained, can solve it.

! Pitfall / Warning

This result is widely (and somewhat unfairly) credited with triggering the first “AI winter.” Funding for neural network research dried up for roughly a decade, in part because Minsky and Papert’s result was read more broadly than it actually applied, it’s a statement about *single-layer* perceptrons, not neural networks in general. The fix (stacking layers) was already conceptually available; what was missing was an efficient way to train multi-layer

networks, which is exactly the gap backpropagation later filled.

1.7 From Perceptron to Modern Artificial Neuron

The modern artificial neuron used in deep learning keeps the weighted-sum structure but changes two things. First, the hard step function is replaced with a smooth, differentiable activation function (Chapter 5), because a discontinuous step function has a derivative of zero almost everywhere, which makes gradient-based learning impossible. Second, neurons are stacked into layers, and layers are stacked into networks, which is exactly the escape hatch from the XOR problem (Chapter 2).

Perceptron vs. Modern Artificial Neuron

A **perceptron** uses a hard step function and the historical perceptron learning rule. A **modern artificial neuron** uses the same weighted-sum computation but pairs it with a smooth, differentiable activation function and is trained with gradient descent and backpropagation, not the original perceptron rule. The terms are often used loosely as synonyms in casual conversation, but the distinction matters the moment you start asking “how is this trained?”

1.8 Chapter Summary

- A perceptron computes a weighted sum of its inputs plus a bias, then applies a hard threshold.
- Geometrically, a perceptron defines a single linear decision boundary, a hyperplane.
- The perceptron learning rule provably converges, but only on linearly separable data.
- XOR is the canonical example of a non-linearly-separable function, and it cannot be learned by a single perceptron.
- Modern neurons replace the step function with a smooth activation function and are trained with gradient descent, not the original rule, setting up everything in Chapter 2 onward.

1.9 Chapter Evaluation

Ten questions spanning the whole chapter: conceptual foundations, a production-style design call, and a debugging scenario. Try each with only the hint first; full worked answers follow in section 1.10.

Exercise 1.1

Why can a single perceptron never learn the XOR function, and what is the smallest change that fixes it?

Hint. Think about what shape a single decision boundary can take, then plot the four XOR points.

Exercise 1.2

Structurally a perceptron and logistic regression both compute $\mathbf{w}^\top \mathbf{x} + b$. Name the two things that differ, and why each matters in practice.

Hint. One difference is in the output map; the other is in the training signal. Both trace back to differentiability.

Exercise 1.3

The perceptron learning rule is $\mathbf{w} \leftarrow \mathbf{w} + \eta(y_{\text{true}} - y_{\text{pred}})\mathbf{x}$. Explain geometrically what each update does to the decision boundary.

Hint. $\mathbf{w}$ is normal to the boundary. A misclassified positive example pushes $\mathbf{w}$ which way?

Exercise 1.4

The Perceptron Convergence Theorem guarantees a solution. State its precondition, and explain why the guarantee is nearly useless in practice.

Hint. Real data is almost never perfectly separable. What does the theorem promise when it isn't?

Exercise 1.5

Why is the step function fatal for gradient-based learning, and what replaced it?

Hint. Differentiate the step function. What is its slope almost everywhere, and what does that do to a gradient update?

Exercise 1.6

(Production) You are asked to ship a binary fraud classifier. A junior engineer proposes “a perceptron, it’s the simplest neural net.” Give two concrete technical reasons to reject that and one to keep the linear model anyway.

Hint. Think calibration, thresholds, and the cost of a non-differentiable output in a monitored production system.

Exercise 1.7

(Debugging) You implement the perceptron rule and the weights oscillate forever without converging. List three distinct causes and how you’d confirm each.

Hint. One cause is in the data, one in the learning rate, one in your stopping condition.

Exercise 1.8

Implement the perceptron learning rule from scratch in NumPy and train it on a 2D linearly separable set. What single line decides convergence?

Hint. The loop should stop when no example is misclassified in a full pass.

Exercise 1.9

Explain how stacking perceptrons solves XOR at the level of decision regions, not just “add a hidden layer.”

Hint. Each hidden unit draws one line. What does the output unit do with two half-planes?

Exercise 1.10

(Technical) The bias b is often folded into $\mathbf{w}$ by appending a constant-1 feature. Show the algebra and state one practical reason frameworks keep bias separate anyway.

Hint. Augment $\mathbf{x} \rightarrow [\mathbf{x}, 1]$ and $\mathbf{w} \rightarrow [\mathbf{w}, b]$. Then think about weight decay.

1.10 Answers

Answer 1.1

Why can a single perceptron never learn XOR, and what is the smallest fix?

XOR is not **linearly separable**. A perceptron's rule $\mathbf{w}^\top \mathbf{x} + b \geq 0$ defines a single hyperplane (a line in 2D) splitting input space into two half-planes. Plot the four points: the positive class $\{(0, 1), (1, 0)\}$ sits on opposite corners from the negative class $\{(0, 0), (1, 1)\}$, so no single line can separate them. The smallest fix is one hidden layer with two units: each hidden unit learns a different line (e.g. " x_1 OR x_2 " and "NOT (x_1 AND x_2)"), and the output unit ANDs those two half-planes, carving out the XOR region. This is precisely the motivation for chapter 2.

Answer 1.2

What differs between a perceptron and logistic regression?

Two things. (1) **Output map:** the perceptron applies a hard step and emits a class label; logistic regression applies a sigmoid and emits a calibrated probability in $(0, 1)$. Calibration matters whenever a downstream system thresholds on confidence (ranking, cost-sensitive decisions). (2) **Training signal:** the perceptron updates only on misclassification by a fixed amount; logistic regression minimizes cross-entropy by gradient descent. Because the sigmoid is smooth, its loss is differentiable everywhere, which is what lets the same machinery scale to every deep network in this book. The perceptron's hard threshold has zero gradient almost everywhere, so it cannot be trained that way.

Answer 1.3

Geometric meaning of each perceptron update.

$\mathbf{w}$ is normal to the decision boundary, so changing $\mathbf{w}$ rotates the boundary and changing b translates it. For a false negative ($y_{\text{true}} = 1, y_{\text{pred}} = 0$) the update adds $\eta \mathbf{x}$, rotating $\mathbf{w}$ toward the misclassified point so its projection $\mathbf{w}^\top \mathbf{x}$ grows and eventually crosses zero. A false positive subtracts $\eta \mathbf{x}$, rotating away. A correct prediction yields a zero update. Each step reduces the margin violation of the offending point, though possibly at the expense of others, which is why it only provably terminates on separable data.

Answer 1.4*Precondition and limitation of the Convergence Theorem.*

Precondition: the training set is *linearly separable*. Under that assumption the rule finds a separating hyperplane in a finite number of updates (bounded by $(R/\gamma)^2$ for radius R and margin γ). It is nearly useless in practice because (a) real data is almost never perfectly separable, and on non-separable data the rule *never* converges, it cycles forever; (b) it says nothing about the *quality* of the boundary (no max-margin guarantee like an SVM); and (c) the bound depends on $1/\gamma^2$, which can be astronomically large for nearly-overlapping classes.

Answer 1.5*Why the step function breaks gradient learning.*

The step function is flat everywhere except at $z = 0$, where it is discontinuous. Its derivative is therefore 0 almost everywhere (and undefined at the jump). Backpropagation multiplies by this local derivative, so every gradient that flows through a step unit is annihilated, the network receives no learning signal. It was replaced by smooth activations (sigmoid, tanh, and later ReLU/GELU in chapter 5) whose nonzero derivatives let gradient information propagate.

Answer 1.6*Production: perceptron for a fraud classifier?*

Reject because: (1) the hard $\{0,1\}$ output gives no score to threshold, fraud systems need a tunable probability so you can trade precision against recall and feed a calibrated risk into downstream rules; (2) the non-differentiable output blocks gradient-based training and modern tooling (early stopping on a smooth validation loss, probability calibration, monitoring of score drift). **Keep a linear model anyway because:** if the signal is largely linear, *logistic regression* (same linear core, sigmoid output) is interpretable, cheap, well-calibrated, and easy to monitor, often the right first production model before reaching for depth.

Answer 1.7*Debugging: weights oscillate and never converge.*

Three distinct causes: (1) **Non-separable data**, the rule cannot converge in principle; confirm by training to a fixed epoch budget and checking whether the misclassification count plateaus above zero, or by trying a soft (logistic) model. (2) **Learning rate too large**, each update overshoots; confirm by logging $\|\mathbf{w}\|$ and the update norm, and by shrinking η to see if oscillation damps. (3) **No proper stopping condition / shuffling artifact**, you keep looping even after separation, or a fixed example order induces a limit cycle; confirm by adding “stop when a full pass has zero errors” and shuffling each epoch.

Answer 1.8*Implement the perceptron rule in NumPy.*

```
perceptron_fit.py
```

```

import numpy as np

def perceptron_fit(X, y, lr=1.0, max_epochs=100):
    """X: (n, d) features; y: (n,) labels in {0,1}. Returns w, b, epochs."""
    n, d = X.shape
    w, b = np.zeros(d), 0.0
    for epoch in range(max_epochs):
        errors = 0
        for xi, yi in zip(X, y):
            pred = 1 if (w @ xi + b) >= 0 else 0
            update = lr * (yi - pred)          # 0 when correct
            w += update * xi
            b += update
            errors += int(update != 0)
        if errors == 0:                        # <- the convergence test
            return w, b, epoch + 1
    return w, b, max_epochs

```

The single deciding line is `if errors == 0`: a full pass with no misclassification means a separating hyperplane has been found and training can stop. On non-separable data this is never reached and the loop runs to `max_epochs`.

Answer 1.9

How stacking perceptrons solves XOR at the level of decision regions.

A single hidden unit draws one line, splitting the plane into two half-planes and outputting which side a point is on. With two hidden units you get two independent lines, partitioning the plane into up to four regions. XOR needs the region “inside exactly one of the two half-planes.” Choose $h_1 = \mathbf{1}[x_1 + x_2 - 0.5 \geq 0]$ (OR) and $h_2 = \mathbf{1}[x_1 + x_2 - 1.5 \geq 0]$ (AND); then $\text{XOR} = h_1 \wedge \neg h_2$, which the output unit realizes as $\mathbf{1}[h_1 - h_2 - 0.5 \geq 0]$. The hidden layer *re-represents* the inputs into a space where the classes *are* linearly separable, which is the essence of what depth buys you.

Answer 1.10

Folding bias into the weight vector.

Augment every input with a constant feature: $\tilde{\mathbf{x}} = [\mathbf{x}, 1]$ and $\tilde{\mathbf{w}} = [\mathbf{w}, b]$. Then $\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} = \mathbf{w}^\top \mathbf{x} + b \cdot 1 = \mathbf{w}^\top \mathbf{x} + b$, so the bias becomes an ordinary weight on a constant input, which simplifies the math and the gradient bookkeeping. Frameworks nonetheless keep bias as a separate parameter mainly because of **weight decay**: you almost always want to regularize $\mathbf{w}$ but *not* b (penalizing the bias just shifts the model’s output baseline for no benefit), and a separate parameter makes that exclusion trivial. It also keeps shapes clean for batched matmuls and broadcasting.

Multi-Layer Networks & Forward Pass

Subscribe → aiengineeringinsider.substack.com/subscribe

Stacking neurons into something that can actually learn nonlinear functions.

Chapter 1 ended with a wall: a single perceptron cannot learn XOR, or anything else that isn't linearly separable. This chapter is about the fix, stack neurons into layers, and stack layers into a network, and shows exactly why that fix works, not just that it does.

2.1 Why Depth: The Universal Approximation Theorem

The **universal approximation theorem** (Cybenko, 1989; Hornik et al., 1989) states, informally, that a feedforward network with a single hidden layer of sufficient width, using a non-constant, bounded, continuous activation function, can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary precision.

★ Key Insight

The universal approximation theorem is a statement about what's *representable*, not what's *learnable*. It guarantees a network of the right shape exists; it says nothing about whether gradient descent will actually find it, how much data you'd need, or how wide "sufficient width" really has to be. In practice, that width can be exponentially large for a single hidden layer, which is the entire practical argument for depth over width: deeper networks can represent the same functions with dramatically fewer total parameters.

2.2 Layers, Weights, and Biases as Matrices

A single neuron computes $z = \mathbf{w}^\top \mathbf{x} + b$. A **layer** is just many neurons computing this in parallel, each with its own weight vector, sharing the same input. Stack those weight vectors as rows of a matrix W , and the entire layer's computation becomes a single matrix-vector product:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}$$

For a layer with n inputs and m outputs (neurons), $W \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$, and $\mathbf{z} \in \mathbb{R}^m$.

Hidden Layer

Any layer in a network that is neither the input layer nor the output layer. Its outputs are not directly observed; they're an internal, learned representation that the network constructs to make the final task easier.

2.3 The Forward Pass

The **forward pass** is the sequence of computations that turns an input into a prediction. For a network with L layers, each layer applies an affine transformation followed by a nonlinear activation function f :

$$\mathbf{a}^{(0)} = \mathbf{x}, \quad \mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f(\mathbf{z}^{(l)})$$

for $l = 1, \dots, L$, where $\mathbf{a}^{(L)}$ is the network's final output. Each $\mathbf{a}^{(l)}$ is called an **activation**, and it's exactly the nonlinearity f applied between layers that breaks the “single straight line” limitation from Chapter 1.

forward_pass.py

```
import numpy as np

def relu(z):
    return np.maximum(0, z)

def forward_pass(x, W1, b1, W2, b2):
    z1 = W1 @ x + b1
    a1 = relu(z1)
    z2 = W2 @ a1 + b2
    # No activation on the final layer here; raw score (e.g. for regression)
    return z2

# A tiny 2-input, 3-hidden-unit, 1-output network
W1 = np.array([[1.0, -1.0], [0.5, 0.5], [-1.0, 1.0]])
b1 = np.array([0.0, 0.0, 0.0])
W2 = np.array([[1.0, 1.0, 1.0]])
b2 = np.array([0.0])

x = np.array([1.0, 0.0])
print(forward_pass(x, W1, b1, W2, b2))
```

Output

[1.5]

2.3.1 Solving XOR with Two Layers

It's worth seeing the resolution to Chapter 1's cliffhanger explicitly. A two-layer network with two hidden units can solve XOR: each hidden unit learns one of the two lines needed to separate the classes, and the output layer combines them.

xor_solution.py

```
import numpy as np

def step(z):
    return (z >= 0).astype(float)

# Hand-picked weights (not learned here, just to show representability)
W1 = np.array([[1.0, 1.0], [1.0, 1.0]])
b1 = np.array([-0.5, -1.5])
W2 = np.array([[1.0, -1.0]])
b2 = np.array([-0.5])

def xor_net(x):
    a1 = step(W1 @ x + b1)
    return step(W2 @ a1 + b2)

for x1 in [0, 1]:
    for x2 in [0, 1]:
        x = np.array([x1, x2])
        print(x1, x2, "->", int(xor_net(x)[0]))
```

Output

```
0 0 -> 0
0 1 -> 1
1 0 -> 1
1 1 -> 0
```

✓ Best Practice

Don't just read this example, run it, then change the hand-picked weights and watch it break. Understanding *why* a particular weight setting solves XOR (what line each hidden unit draws, and how the output layer combines the two half-plane decisions) is worth more than memorizing that a two-layer network “can” solve it.

2.4 Architecture Diagram

Figure 2.1 draws the forward pass of a small multi-layer perceptron as the layered, fully-connected graph frameworks actually build. Inputs enter on the left; each arrow carries a weight; each hidden and output node applies the affine map then the activation f . The vectorized form $\mathbf{a}^{(l)} = f(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$ is just this picture written as matrices.

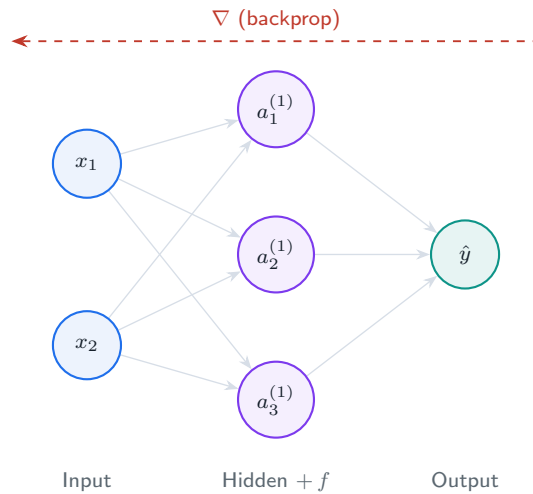


Figure 2.1: Forward pass through a $2 \rightarrow 3 \rightarrow 1$ MLP ($W^{(1)} \in \mathbb{R}^{3 \times 2}$, $W^{(2)} \in \mathbb{R}^{1 \times 3}$). Each layer is a matrix multiply plus bias followed by a nonlinearity; the dashed arrow marks the direction gradients flow during backpropagation (chapter 3).

Figure 2.2 is not a schematic, it is the actual decision boundary a $2 \rightarrow 64 \rightarrow 32 \rightarrow 3$ MLP learns on the three-class “spiral” dataset, produced by the runnable code in [code/gen_figures.py](#). A linear model scores about 33% here; the network reaches 99.3% by bending its boundary into the data’s spiral structure, the visual payoff of stacking nonlinear layers.

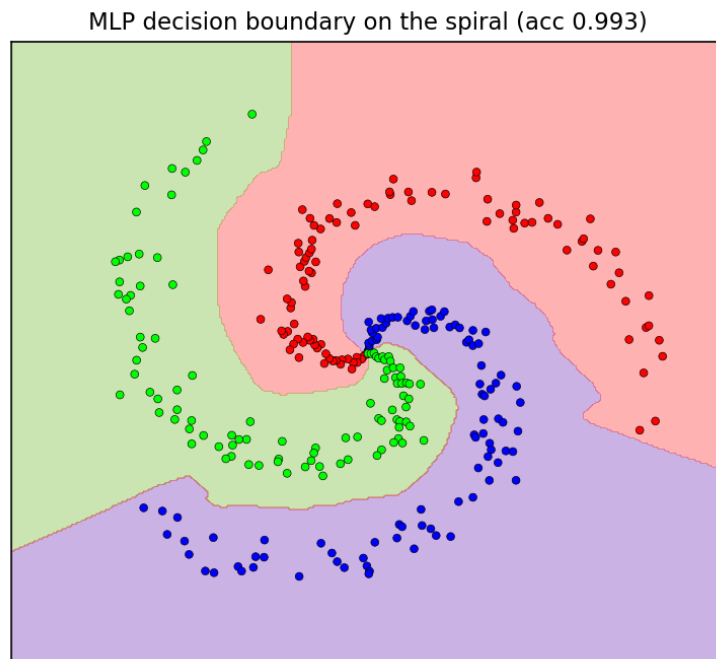


Figure 2.2: Real output: the nonlinear decision regions a trained MLP carves on the three-class spiral (99.3% accuracy). The smooth, curved class boundaries are exactly what a single linear layer (chapter 1) cannot represent.

2.5 Width vs. Depth

Given a fixed parameter budget, should you add more neurons per layer (width) or more layers (depth)? There’s no universal answer, but some general patterns hold in practice:

- **Depth** tends to build hierarchical representations: early layers learn simple features, later layers compose them into increasingly abstract ones (this is most visible in CNNs, Chapter 15).
- **Width** adds raw representational capacity at a given layer but doesn’t on its own encourage hierarchical composition.
- Very deep networks without countermeasures (Chapters 7-10) are harder to train: gradients can vanish or explode as they propagate backward through many layers.
- In practice, architecture search and empirical results for your specific task matter more than either extreme; most production architectures are a deliberate, tuned mix of both.

Table 2.1: Width versus depth at a fixed parameter budget: what each buys and what each costs.

Dimension	Add Width (neurons/layer)	Add Depth (more layers)
Representational gain	Raw capacity at one level	Hierarchical, compositional features
Parameter efficiency	Can need exponentially many units	Often exponentially fewer for same function
Trainability	Easy; gradients stay short	Vanishing/exploding gradients without care
Required tooling	Little	Residuals, normalization, careful init (chapters 8 to 10)
Typical use	Wide heads, embeddings	Backbones (CNNs, transformers)

2.6 Computational Graphs

Frameworks like PyTorch and JAX don’t think about a network as a sequence of equations, they think about it as a **computational graph**: a directed acyclic graph (DAG) where nodes are operations (matrix multiply, add, ReLU) and edges are the tensors flowing between them. The forward pass is a traversal of this graph from inputs to outputs.

Computational Graph

A directed acyclic graph representation of a sequence of mathematical operations, where each node is an operation and each edge carries a tensor (the output of one operation, the input to the next). Building the network this way, rather than as opaque code, is what makes automatic differentiation possible: the graph can be traversed backward to compute gradients, which is exactly the subject of Chapter 3.

This representation matters practically, too: it’s the reason a shape mismatch three layers deep produces an error message pointing at the specific operation node that failed, not a vague failure at the end of the network.

From the Field

A genuinely common first-week bug: defining a layer as `nn.Linear(128, 64)` when the actual input has 256 features, then staring at a cryptic matrix-multiplication shape error for twenty minutes. The fix is almost always faster than the panic suggests: print `x.shape` immediately before the line that fails, and work backward from there. Treat shape-checking as a default debugging reflex, not a last resort.

2.7 Chapter Summary

- The universal approximation theorem guarantees a sufficiently wide single-hidden-layer network can represent any continuous function, but says nothing about learnability or practical width.
- A layer's forward computation is a matrix-vector product followed by a nonlinear activation: $\mathbf{a}^{(l)} = f(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$.
- Two layers are enough to solve XOR: each hidden unit learns one linear boundary, and the output layer combines them nonlinearly.
- Depth tends to build hierarchical representations; width adds raw capacity at a given layer. Most real architectures tune both.
- Frameworks represent networks as computational graphs (DAGs), which is what makes automatic differentiation, and useful shape-mismatch error messages, possible.

2.8 Chapter Evaluation

Ten questions covering representational theory, the forward-pass mechanics, and the practical failure modes of depth. Hints first; answers in section 2.9.

Exercise 2.1

Why do we need a nonlinear activation between layers? Prove it.

Hint. Compose two affine maps and simplify. What kind of function do you get?

Exercise 2.2

The universal approximation theorem says one hidden layer suffices. Why, then, does anyone use deep networks?

Hint. Distinguish “representable” from “learnable,” and think about how width scales.

Exercise 2.3

Write the forward pass of an L -layer MLP in vectorized form and give the shape of every matrix and vector for layer l .

Hint. $\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$. What are the row/column counts of $W^{(l)}$?

Exercise 2.4

Explain, at the level of decision regions, how exactly two hidden units solve XOR.

Hint. Each unit is one line. The output unit combines two half-planes with which Boolean op?

Exercise 2.5

(Production) A teammate adds 8 more `nn.Linear` layers “to increase capacity” but forgets the ReLUs between them. The model trains and loss drops. What’s wrong and how would you catch it in review?

Hint. The loss dropping doesn’t mean capacity increased. What is this 10-layer stack equivalent to?

Exercise 2.6

(Debugging) `nn.Linear(128, 64)` throws a shape error three layers deep. Give your exact debugging procedure.

Hint. The computational graph points at the failing node. What’s the fastest thing to print?

Exercise 2.7

Why is the computational-graph (DAG) view of a network more than a pedagogical nicety? Name two concrete things it enables.

Hint. One is gradients; one is error messages / shape inference.

Exercise 2.8

Given a fixed parameter budget, state two regimes where you’d prefer width and two where you’d prefer depth.

Hint. Embeddings and output heads vs. feature backbones.

Exercise 2.9

Implement a generic L -layer forward pass in NumPy that accepts a list of (W, b) pairs and an activation, with no activation on the final layer.

Hint. Loop over layers; apply f on all but the last.

Exercise 2.10

(Technical) Activations $\mathbf{a}^{(l)}$ are called a “learned representation.” What does that mean operationally, and how would you inspect it?

Hint. Think of the hidden layer as remapping inputs into a space where the task is easier; you can extract and probe those vectors.

2.9 Answers

Answer 2.1

Why a nonlinearity is required, with proof.

A linear layer computes $W\mathbf{x} + \mathbf{b}$. Stack two with nothing between: $W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2W_1)\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2)$, which is again affine, the product of matrices is a matrix. By induction, any number of stacked linear layers collapses to a single equivalent affine map, so depth adds zero representational power. A nonlinear f between layers breaks the collapse, giving the network strictly more expressive power than any single layer.

Answer 2.2

If one hidden layer is universal, why go deep?

The theorem is about *representability*, not *learnability*, and it gives no width bound. For many functions a single hidden layer needs exponentially many units, whereas a deep network represents the same function with far fewer parameters by composing features hierarchically. Depth also tends to produce representations that gradient descent actually finds and that generalize, the theorem promises existence of weights, not that SGD will reach them.

Answer 2.3

Vectorized L -layer forward pass with shapes.

With $\mathbf{a}^{(0)} = \mathbf{x}$, for $l = 1 \dots L$: $\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$ and $\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)})$. If layer l maps n_{l-1} inputs to n_l outputs, then $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$, $\mathbf{b}^{(l)}, \mathbf{z}^{(l)}, \mathbf{a}^{(l)} \in \mathbb{R}^{n_l}$. With a batch of B examples, $A^{(l-1)} \in \mathbb{R}^{n_{l-1} \times B}$ and $Z^{(l)} = W^{(l)}A^{(l-1)} + \mathbf{b}^{(l)}$ (bias broadcast over columns).

Answer 2.4

Decision-region account of two-unit XOR.

Pick $h_1 = \mathbf{1}[x_1 + x_2 - 0.5 \geq 0]$ (fires unless both inputs are 0, an OR) and $h_2 = \mathbf{1}[x_1 + x_2 - 1.5 \geq 0]$ (fires only when both are 1, an AND). XOR is “OR but not AND” = $h_1 \wedge \neg h_2$, which the output unit computes as $\mathbf{1}[h_1 - h_2 - 0.5 \geq 0]$. The hidden layer re-represents the four points into an (h_1, h_2) space where the two classes are linearly separable.

Answer 2.5

Production: stacked Linear layers with no activations.

Without nonlinearities the 10-layer stack is mathematically identical to a single linear layer (Answer 2.1), so it has the representational ceiling of logistic/linear regression: loss can still drop toward the best *linear* fit, masking the bug. Catch it in review by (a) checking every **Linear** is followed by an activation, (b) comparing against an actual one-layer linear baseline, parity in val metrics is the tell, and (c) a unit test that asserts the model can fit a nonlinear toy task (e.g. XOR) it should not be able to fit if it’s secretly linear.

Answer 2.6

Debugging a deep shape error.

The DAG points the traceback at the failing op. Procedure: (1) read the error, it names the operation and the two incompatible shapes; (2) print `x.shape` immediately before that line; (3) walk backward layer by layer checking each `Linear(in,out)`'s `in` against the previous `out`; (4) the mismatch is almost always a hardcoded dimension that doesn't match the real input width. Add a one-time `assert x.shape[-1] == expected` at the model entry to fail fast and clearly next time.

Answer 2.7

Why the DAG view matters.

Two concrete enablers: (1) **Automatic differentiation**, because the graph records every operation and its inputs, it can be traversed in reverse to apply the chain rule (chapter 3); without the recorded graph there is nothing to differentiate through. (2) **Shape inference and precise errors**, the framework propagates tensor shapes through nodes and can report exactly which operation received an incompatible shape, instead of a vague end-of-network failure. (It also enables graph optimizations like operator fusion.)

Answer 2.8

When to prefer width vs depth.

Prefer **width** for: (1) embedding/projection layers and wide output heads where you need raw capacity at one level; (2) shallow tabular models where hierarchy buys little. Prefer **depth** for: (1) perceptual data (images, audio, text) where features are naturally compositional; (2) when parameter efficiency matters, depth represents many functions with exponentially fewer weights. Real architectures tune both (see table 2.1).

Answer 2.9

Generic L-layer forward pass in NumPy.

`mlp_forward.py`

```
import numpy as np

def relu(z): return np.maximum(0.0, z)

def mlp_forward(x, params, activation=relu):
    """params: list of (W, b). No activation on the final layer."""
    a = x
    last = len(params) - 1
    for i, (W, b) in enumerate(params):
        z = W @ a + b
        a = z if i != last else activation(z)    # raw scores on output
    return a
```

This is the exact shape of every framework's **forward**: an affine map per layer, a nonlinearity on all but the output, returning logits/scores.

Answer 2.10

What “learned representation” means, and how to inspect it.

Operationally, $\mathbf{a}^{(l)}$ is the network’s re-encoding of the input into coordinates that make the remaining task easier, e.g. moving XOR points into a linearly-separable (h_1, h_2) space. You inspect it by running inputs forward, capturing a hidden layer’s activations (a forward hook in PyTorch), and probing them: visualize with t-SNE/UMAP, train a cheap linear classifier on them (a “linear probe”), or measure nearest-neighbor class purity. Cleanly separable hidden activations indicate the representation has done useful work.

Backpropagation from Scratch

Subscribe → aiengineeringinsider.substack.com/subscribe

The algorithm that makes deep learning possible, derived step by step.

Chapter 2 showed that stacking layers with nonlinear activations gives a network real representational power, enough to solve XOR and, in principle, approximate any continuous function. But representational power is useless without a way to find the right weights. **Backpropagation** (Rumelhart et al., 1986) is that way, and it is, despite its reputation, nothing more than a disciplined application of the chain rule from calculus.

3.1 The Credit Assignment Problem

Training a network means adjusting every weight, in every layer, to reduce the loss. The hard part isn't computing the loss, it's figuring out *how much each individual weight contributed to that loss*. A weight in the first layer of a ten-layer network affects the output only indirectly, through nine layers of subsequent transformation. This is the **credit assignment problem**: distributing blame (or credit) for the final error back to every parameter that helped cause it.

★ Key Insight

Backpropagation doesn't compute anything new, mathematically. Every gradient it produces could, in principle, be computed by hand with enough patience and the chain rule. What backpropagation provides is an *efficient, systematic algorithm* for computing all of those gradients in a single backward pass, reusing intermediate results instead of recomputing them from scratch for every parameter.

3.2 Chain Rule Refresher

If $y = f(u)$ and $u = g(x)$, then:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

This generalizes to longer chains. If $y = f(g(h(x)))$:

$$\frac{dy}{dx} = \frac{dy}{du_1} \cdot \frac{du_1}{du_2} \cdot \frac{du_2}{dx}$$

where $u_1 = g(h(x))$ and $u_2 = h(x)$. A neural network's forward pass is exactly this kind of nested composition, layer after layer, so its gradient is exactly this kind of chained product.

3.2.1 A Worked Scalar Example

Let $x = 2$, and consider $f(x) = (3x + 1)^2$. Let $u = 3x + 1$, so $f = u^2$.

$$\begin{aligned}\frac{df}{du} &= 2u, & \frac{du}{dx} &= 3 \\ \frac{df}{dx} &= \frac{df}{du} \cdot \frac{du}{dx} = 2u \cdot 3 = 6u = 6(3x + 1)\end{aligned}$$

At $x = 2$: $u = 7$, so $\frac{df}{dx} = 42$. Verify numerically: $f(2) = 49$, $f(2.001) \approx 49.042$, and $\frac{49.042 - 49}{0.001} \approx 42$. The chain rule and the numerical derivative agree.

✓ Best Practice

Whenever you derive a gradient by hand, especially for the first time on a new function, check it against a numerical estimate: $\frac{f(x+\epsilon) - f(x-\epsilon)}{2\epsilon}$ for a small ϵ (e.g., 10^{-5}). This single habit, called **gradient checking**, will catch sign errors and dropped terms before they cost you a debugging session three layers deep in a real network.

3.3 Backprop Through a Single Neuron

Consider a single neuron: $z = wx + b$, $a = \sigma(z)$ (using sigmoid for concreteness), feeding into a loss L . We want $\frac{\partial L}{\partial w}$ and $\frac{\partial L}{\partial b}$. By the chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Each factor is a local, easy derivative: $\frac{\partial z}{\partial w} = x$, $\frac{\partial a}{\partial z} = \sigma(z)(1 - \sigma(z))$ (the sigmoid's own derivative, covered fully in Chapter 5), and $\frac{\partial L}{\partial a}$ depends on the specific loss function (Chapter 6). The same pattern gives $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot 1$, since $\frac{\partial z}{\partial b} = 1$.

Local Gradient

The derivative of a single operation's output with respect to its own input, computed without reference to anything upstream or downstream in the network. Backpropagation works by computing local gradients at every node and chaining them together, rather than trying to derive one giant end-to-end expression by hand.

3.4 Backprop Through a Layer (Vectorized)

Real networks operate on vectors, not scalars, so the gradients become matrix and vector quantities. For a layer $\mathbf{z} = W\mathbf{a}_{in} + \mathbf{b}$, given the upstream gradient $\delta = \frac{\partial L}{\partial \mathbf{z}}$ (often called the layer's **error signal**), the gradients with respect to the layer's own parameters are:

$$\frac{\partial L}{\partial W} = \delta \mathbf{a}_{in}^\top, \quad \frac{\partial L}{\partial \mathbf{b}} = \delta, \quad \frac{\partial L}{\partial \mathbf{a}_{in}} = W^\top \delta$$

That last term, $\frac{\partial L}{\partial \mathbf{a}_{in}}$, is exactly the upstream gradient that gets passed to the *previous* layer, becoming its δ . This is the recursive structure that makes backpropagation a single backward pass rather than N independent derivations for N layers: each layer only needs to know how to compute its own local gradients and pass one quantity backward.

manual_backprop.py

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_grad(z):
    s = sigmoid(z)
    return s * (1 - s)

# Single layer: z = Wx + b, a = sigmoid(z), loss = 0.5*(a - target)^2
W = np.array([[0.5, -0.3]])
b = np.array([0.1])
x = np.array([1.0, 2.0])
target = np.array([0.8])

# Forward pass
z = W @ x + b
a = sigmoid(z)
loss = 0.5 * (a - target) ** 2

# Backward pass
dL_da = (a - target)           # dLoss/da
da_dz = sigmoid_grad(z)       # da/dz
delta = dL_da * da_dz         # combined: dLoss/dz

dL_dW = np.outer(delta, x)     # dLoss/dW
dL_db = delta                  # dLoss/db

print("loss:", loss)
print("dL/dW:", dL_dW)
print("dL/db:", dL_db)
```

Output

```
loss: [0.045]
dL/dW: [[-0.075 -0.15 ]]
dL/db: [-0.075]
```

3.5 Backprop Through Multiple Layers

Stack two layers and the recursion continues exactly as promised: the gradient with respect to layer 2's input becomes layer 1's δ . The full algorithm, for a network with L layers, is:

1. Run the forward pass, caching every $\mathbf{z}^{(l)}$ and $\mathbf{a}^{(l)}$ along the way (you need them again on the backward pass).
2. Compute $\delta^{(L)} = \frac{\partial L}{\partial \mathbf{a}^{(L)}} \odot f'(\mathbf{z}^{(L)})$ at the output layer.
3. For $l = L - 1, \dots, 1$: compute $\delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot f'(\mathbf{z}^{(l)})$.
4. At every layer, $\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} \mathbf{a}^{(l-1)\top}$ and $\frac{\partial L}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$.

($\odot$ denotes element-wise multiplication.) Notice the caching requirement in step 1: this is why training a network uses meaningfully more memory than just running inference, every intermediate activation has to stay in memory until the backward pass consumes it.

3.6 Architecture Diagram

Figure 3.1 shows the two passes that define training. The forward pass (top, blue) flows left to right, caching each $\mathbf{z}^{(l)}$ and $\mathbf{a}^{(l)}$. The backward pass (bottom, red) flows right to left, threading the error signal δ through the transpose-weight recursion $\delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot f'(\mathbf{z}^{(l)})$ and reading the cached values to form $\partial L / \partial W^{(l)}$.

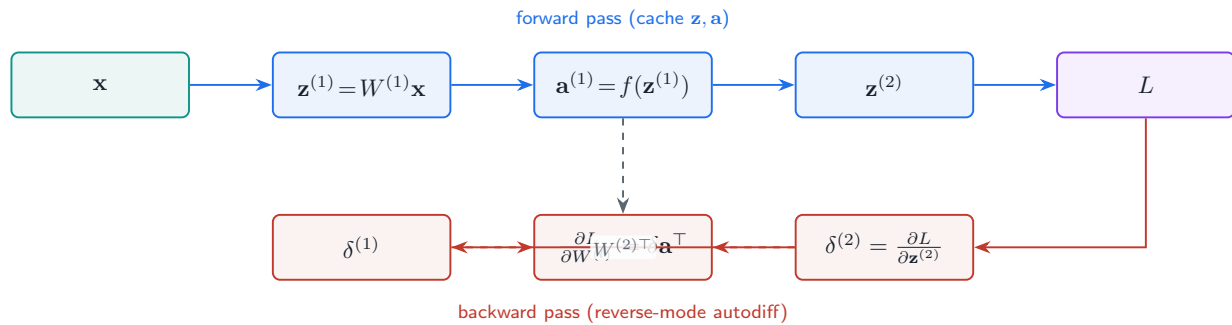


Figure 3.1: Forward and backward passes over a two-layer network. The cache (dashed, center) is why training costs more memory than inference: every activation must survive until the backward pass consumes it.

Table 3.1: Reverse-mode versus forward-mode automatic differentiation, and why deep learning uses reverse mode.

Property	Reverse mode (backprop)	Forward mode
Sweep direction	Outputs $\rightarrow$ inputs	Inputs $\rightarrow$ outputs
Cost per pass	One scalar loss $\rightarrow$ all params	One input $\rightarrow$ all outputs
Efficient when	Many params, few outputs (DL)	Few inputs, many outputs
Memory	High (caches activations)	Low (no cache)
Used by	PyTorch <code>.backward()</code> , JAX <code>grad</code>	JAX <code>jvp</code> , Jacobian columns

3.7 Computational Graphs and Reverse-Mode Autodiff

The manual derivation above generalizes to arbitrary computational graphs (Chapter 2), not just stacks of identical layers. **Reverse-mode automatic differentiation** traverses the graph

backward from the loss, applying the chain rule at each node using only that node’s local gradient and the gradient flowing in from downstream. This is exactly what `.backward()` does in PyTorch and what `grad()` does in JAX, and it’s exactly what you’ll implement yourself in Chapter 4.

Reverse-Mode Automatic Differentiation

An algorithm for computing exact gradients of a scalar output (typically the loss) with respect to every input or parameter in a computational graph, by traversing the graph in reverse topological order and applying the chain rule at each node. It is called “reverse-mode” because it propagates from outputs back to inputs, which is the efficient direction when there are many parameters and one scalar loss, exactly the situation in deep learning.

3.8 Common Derivation Mistakes

! Pitfall / Warning

The single most common hand-derivation mistake is dropping a transpose or getting matrix dimensions backward, $\frac{\partial L}{\partial W} = \delta \mathbf{a}_{in}^\top$ and $\frac{\partial L}{\partial \mathbf{a}_{in}} = W^\top \delta$ use the transpose in different places for a reason: the dimensions have to work out so the result matches the shape of W or $\mathbf{a}_{in}$, respectively. When in doubt, write out the dimensions explicitly ($W \in \mathbb{R}^{m \times n}$, $\delta \in \mathbb{R}^m$, $\mathbf{a}_{in} \in \mathbb{R}^n$) and check that every product is dimensionally valid before trusting the derivation.

The second most common mistake: forgetting that $\delta^{(l)}$ depends on $f'(\mathbf{z}^{(l)})$, the derivative of *that layer’s own* activation function, not the next layer’s. Mixing these up silently produces a network that trains, just incorrectly, which is far harder to debug than a network that crashes outright. This is precisely why vanishing and exploding gradients (introduced next) are so insidious: the symptom (a network that won’t learn) looks identical whether the cause is a mathematical bug or simply many layers of small derivatives multiplying together.

3.9 Vanishing and Exploding Gradients: First Encounter

Look again at the recursive formula for $\delta^{(l)}$: it involves a product of $f'(\mathbf{z})$ terms across every layer from L back to l . If those derivatives are consistently less than 1 (true of sigmoid almost everywhere, where the maximum derivative is just 0.25), the product shrinks exponentially with depth, the **vanishing gradient problem**. Early layers in a deep network can receive a gradient signal so small that they effectively stop learning. The reverse problem, derivatives consistently greater than 1, causes gradients to grow exponentially instead, the **exploding gradient problem**, leading to unstable, divergent training.

! Pitfall / Warning

This isn’t a one-time, historical concern, it’s the underlying reason several entire chapters of this book exist: activation function choice (Chapter 5), careful initialization (Chapter 8), normalization layers (Chapter 9), and architectures like LSTM (Chapter 16) are all, in part, direct responses to vanishing and exploding gradients. If a network refuses to train and the loss is flat or NaN, this family of problems should be among your first suspects.

3.10 Chapter Summary

- Backpropagation is the chain rule, applied systematically and efficiently across a computational graph, nothing more exotic than that.
- Each node only needs its own local gradient; the chain rule combines local gradients into the full end-to-end gradient.
- The backward pass is recursive: each layer's output gradient becomes the previous layer's incoming error signal δ .
- Reverse-mode automatic differentiation is the general algorithm underlying `.backward()` in PyTorch and `grad()` in JAX.
- Vanishing and exploding gradients arise from repeated multiplication of per-layer derivatives across depth, and motivate much of the rest of this book's training-mechanics content.

3.11 Chapter Evaluation

Ten questions on the chain rule, the vectorized recursion, memory cost, and the numerical failure modes backprop is prone to. Hints first; answers in section 3.12.

Exercise 3.1

Derive $\partial L / \partial z$ for cross-entropy with a sigmoid output and explain why the result is so clean.

Hint. Multiply $\partial L / \partial a$ by the sigmoid derivative $a(1 - a)$ and watch terms cancel.

Exercise 3.2

What is the precise difference between backpropagation and gradient descent?

Hint. One computes gradients; the other consumes them. They are separable.

Exercise 3.3

For a layer $\mathbf{z} = W\mathbf{a}_{in} + \mathbf{b}$, derive the three gradients and explain why the transpose appears where it does.

Hint. Match shapes: $\partial L / \partial W$ must be shaped like W ; $\partial L / \partial \mathbf{a}_{in}$ like $\mathbf{a}_{in}$.

Exercise 3.4

Why does training require caching forward activations, and what does that imply for memory at scale?

Hint. Step 4 of the algorithm uses $\mathbf{a}^{(l-1)}$. When is it consumed?

Exercise 3.5

Explain vanishing and exploding gradients from the recursion, and name three countermeasures and the chapters that cover them.

Hint. $\delta^{(l)}$ multiplies $f'(\mathbf{z})$ across every layer. What if each factor is < 1 ? > 1 ?

Exercise 3.6

(Debugging) Your loss goes to NaN after a few hundred steps. Walk through how you'd localize whether it's exploding gradients, a bad learning rate, or a numerical issue in the loss.

Hint. Log gradient norms; clip; check for $\log(0)$ in the loss.

Exercise 3.7

Implement gradient checking for an arbitrary scalar function and state the tolerance you'd accept.

Hint. Central difference $(f(x + \epsilon) - f(x - \epsilon))/2\epsilon$; compare relative error.

Exercise 3.8

Why is reverse mode (not forward mode) the right autodiff direction for deep learning? When would forward mode win?

Hint. Count the cost in terms of number of parameters versus number of outputs.

Exercise 3.9

(Debugging) Your hand-derived gradient trains a network that learns, but slightly worse than the framework. The most likely class of bug?

Hint. A network that trains-but-wrongly usually has a subtle, silent error, e.g. the wrong layer's f' .

Exercise 3.10

(Technical) Gradient checkpointing trades compute for memory. Explain the mechanism in terms of the activation cache.

Hint. If you don't store an activation, how do you get it back during the backward pass?

3.12 Answers

Answer 3.1

Gradient of cross-entropy w.r.t. the logits.

With $a = \sigma(z)$ and $L = -[y \log a + (1 - y) \log(1 - a)]$: $\frac{\partial L}{\partial a} = -\frac{y}{a} + \frac{1-y}{1-a}$ and $\frac{\partial a}{\partial z} = a(1 - a)$. Their product is $(-\frac{y}{a} + \frac{1-y}{1-a})a(1 - a) = -y(1 - a) + (1 - y)a = a - y$. So $\frac{\partial L}{\partial z} = a - y$, prediction minus label. It is clean because the sigmoid's $a(1 - a)$ exactly cancels the $1/a$ and $1/(1 - a)$ from the log loss; this is why sigmoid/softmax is paired with cross-entropy and why you should feed *logits* (not probabilities) into the loss for numerical stability.

Answer 3.2

Backprop vs gradient descent.

Backpropagation computes $\partial L / \partial \theta$ for every parameter by reverse-mode autodiff. Gradient descent uses those gradients to update: $\theta \leftarrow \theta - \eta \partial L / \partial \theta$. Backprop tells you the ascent

direction; gradient descent is the policy of stepping opposite to it. They are separable, you can pair backprop's gradients with momentum, Adam, or AdamW (chapter 7) instead of vanilla descent.

Answer 3.3

Layer gradients and the transpose.

$\frac{\partial L}{\partial W} = \delta \mathbf{a}_{in}^\top$, $\frac{\partial L}{\partial \mathbf{b}} = \delta$, $\frac{\partial L}{\partial \mathbf{a}_{in}} = W^\top \delta$. With $W \in \mathbb{R}^{m \times n}$, $\delta \in \mathbb{R}^m$, $\mathbf{a}_{in} \in \mathbb{R}^n$: the outer product $\delta \mathbf{a}_{in}^\top$ is $m \times n$ (matches W); $W^\top \delta$ is $n \times 1$ (matches $\mathbf{a}_{in}$). The transpose appears wherever needed to make the product dimensionally valid and shaped like the quantity being differentiated, the single fastest derivation sanity check.

Answer 3.4

Why caching, and the memory implication.

$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} \mathbf{a}^{(l-1)\top}$ needs the *forward* activation $\mathbf{a}^{(l-1)}$, which isn't available until the backward pass reaches layer l . So every activation must be retained from the forward pass until consumed, training memory scales with depth $\times$ batch $\times$ activation size, far above inference. This is why batch size is memory-bound and why gradient checkpointing (Answer 3.10) exists.

Answer 3.5

Vanishing/exploding gradients and countermeasures.

$\delta^{(l)}$ is a product of $f'(\mathbf{z})$ and weight terms across layers $L \rightarrow l$. If those factors are consistently < 1 (sigmoid's max is 0.25) the product shrinks exponentially with depth, early layers stop learning (vanishing). If consistently > 1 , it grows exponentially (exploding, often NaN). Countermeasures: non-saturating activations like ReLU/GELU (chapter 5), variance-preserving initialization (chapter 8), normalization layers (chapter 9), residual connections, gradient clipping, and gated recurrence/LSTM (chapter 16).

Answer 3.6

Debugging a loss that goes NaN.

Localize in order: (1) **Loss math**, check for $\log(0)$ / $0/0$, use the logits-based loss (`BCEWithLogits`, `CrossEntropyLoss`) rather than logging a probability. (2) **Exploding gradients**, log the global grad norm each step; if it spikes before the NaN, add `clip_grad_norm` and/or lower the LR. (3) **Learning rate**, halve it; if NaN disappears it was a step-size/stability issue. (4) Check inputs/targets for NaN/Inf and that you're not dividing by a near-zero std in normalization.

Answer 3.7

Gradient checking.

`grad_check.py`

```
import numpy as np

def grad_check(f, x, analytic_grad, eps=1e-5, tol=1e-6):
    """f: scalar fn of vector x. analytic_grad: your computed gradient."""
    num = np.zeros_like(x, dtype=float)
    for i in range(x.size):
        xp, xm = x.copy(), x.copy()
        xp[i] += eps; xm[i] -= eps
        num[i] = (f(xp) - f(xm)) / (2 * eps)      # central difference
    rel = np.abs(num - analytic_grad) / (np.abs(num) + np.abs(analytic_grad) +
    ↪ 1e-12)
    return rel.max(), rel.max() < tol              # accept if max rel err < tol
```

Use the *central* difference (error $O(\epsilon^2)$, not $O(\epsilon)$), and judge by *relative* error; $< 10^{-6}$ is solid for double precision, relax to $\sim 10^{-3}$ for float32.

Answer 3.8

Why reverse mode for deep learning.

One reverse pass yields the gradient of a single scalar (the loss) w.r.t. *all* parameters at cost $\approx$ one forward pass, ideal when params are many and outputs are one. Forward mode costs one pass *per input direction*, so it wins only when inputs are few and outputs many (e.g. a Jacobian of a function $\mathbb{R}^k \rightarrow \mathbb{R}^m$ with $k \ll m$, or Jacobian-vector products). Deep nets have millions of params and a scalar loss, reverse mode is asymptotically cheaper.

Answer 3.9

Debugging trains-but-worse.

A silent gradient bug, almost always using the wrong layer's activation derivative (applying $f'(\mathbf{z}^{(l+1)})$ where $f'(\mathbf{z}^{(l)})$ is required), a dropped transpose, or accumulating with $=$ instead of $+=$ on a shared parameter. The network still descends *some* surrogate, so loss falls, just along the wrong direction. Confirm with gradient checking (Answer 3.7) on a tiny instance against numerical gradients.

Answer 3.10

Gradient checkpointing.

Normally every forward activation is cached for the backward pass. Checkpointing stores only a sparse subset (the “checkpoints”) and *recomputes* the dropped activations on the fly during the backward pass by re-running the forward segment from the nearest checkpoint. It trades extra compute (one partial re-forward) for a large drop in peak memory ($O(\sqrt{L})$ instead of $O(L)$ for an L -layer net), which is how very deep/large models fit on a given GPU.

Build Your Own Mini Framework

Subscribe → aiengineeringinsider.substack.com/subscribe

A micrograd-style autograd engine, to prove you actually understand Chapter 3.

There's a specific kind of understanding that only comes from implementation. You can read Chapter 3's derivations, nod along, and still not really *know* backpropagation the way you will after building it yourself, end to end, in maybe 100 lines of Python. This chapter walks through exactly that exercise, in the tradition popularized by Andrej Karpathy's `micrograd`.

4.1 Why Build a Toy Framework

★ Key Insight

The goal here is not to produce something useful, PyTorch and JAX already exist, and Chapters 12 and 13 cover them in depth. The goal is to remove every shred of “magic” from automatic differentiation by implementing it yourself, so that when you later call `loss.backward()` in PyTorch, you have a precise mental model of what's actually happening underneath, not a black box you trust on faith.

4.2 Designing a Value Class

The core idea: wrap every number in an object that remembers (a) its current value, (b) its gradient (initialized to zero), and (c) how it was produced, which other `Value` objects, and which operation, created it. That third piece of information is what lets us walk the computational graph backward later.

value_class.py

```
class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None  # how to propagate gradient backward
        self._prev = set(_children)    # parent nodes in the graph
        self._op = _op                 # which operation produced this value

    def __repr__(self):
        return f"Value(data={self.data}, grad={self.grad})"
```

Every arithmetic operation we implement will do two things: compute the forward result as a new `Value`, and attach a `_backward` function that knows how to push gradient from that result back onto its inputs.

4.3 Implementing Forward Operations

Start with addition and multiplication, the building blocks of $z = wx + b$.

`ops_add_mul.py`

```
class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), '+')

        def _backward():
            self.grad += out.grad
            other.grad += out.grad
            out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), '*')

        def _backward():
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
            out._backward = _backward
        return out

    def __repr__(self):
        return f"Value(data={self.data}, grad={self.grad})"
```

✓ Best Practice

Notice the `+=` in every `_backward` function, not `=`. If a single `Value` feeds into more than one downstream operation (a very common situation, e.g. a bias term used in multiple places), its gradient must *accumulate* contributions from every path. Using plain assignment here is one of the most common bugs when implementing autograd from scratch, and it fails silently: the forward pass still works, the gradients are just quietly wrong.

Next, add power (for things like squared error) and a nonlinearity. We'll implement `tanh` here as a concrete, smooth, differentiable example.

ops_pow_tanh.py

```

import math

class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op

    # ... __add__ and __mul__ as above ...

    def __pow__(self, other):
        assert isinstance(other, (int, float))
        out = Value(self.data ** other, (self,), f'**{other}')

        def _backward():
            self.grad += (other * self.data ** (other - 1)) * out.grad
            out._backward = _backward
        return out

    def tanh(self):
        t = math.tanh(self.data)
        out = Value(t, (self,), 'tanh')

        def _backward():
            self.grad += (1 - t ** 2) * out.grad
            out._backward = _backward
        return out

```

Each `_backward` function above is exactly the local gradient from Chapter 3, written in code instead of on paper: $\frac{d}{dx}x^n = nx^{n-1}$, and $\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x)$.

Finally, add a few convenience operators so a `Value` behaves like an ordinary number, these are what let the training loop write `yp - yt` and `sum(...)` directly. Without them the loop below raises a `TypeError`.

value_ops_extra.py

```

def __neg__(self):                # -self
    return self * -1

def __sub__(self, other):         # self - other
    return self + (-other)

def __radd__(self, other):       # other + self (so sum() works)
    return self + other

```

4.4 The Backward Pass: Topological Sort

Here's the part that ties everything together. To correctly accumulate gradients, we must process nodes in *reverse topological order*, every node's gradient must be fully accumulated (from all its consumers) *before* we call its own `_backward` to push gradient further upstream. Getting this order wrong silently produces incorrect gradients on any graph with branching.

`backward_method.py`

```
def backward(self):
    topo = []
    visited = set()

    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build_topo(child)
            topo.append(v)
    build_topo(self)

    self.grad = 1.0 # seed: dL/dL = 1
    for v in reversed(topo):
        v._backward()
```

`Value.backward = backward`

From the Field

The most common bug students hit when building this for the first time isn't in the math, it's exactly this ordering. If you call `_backward()` in the wrong order (say, forward topological order instead of reversed), some nodes will push gradient before they've received all of their own incoming gradient, and the result is wrong in a way that often *looks* plausible. If your hand-computed gradients don't match this engine's output, check the topological sort before you suspect the math.

4.5 Architecture Diagram

Figure 4.1 shows the computational graph our engine builds for the expression $c = a \cdot b + a^2$ (the one we gradient-check at the end of the chapter). Solid arrows are the forward pass that records the graph; dashed red arrows are the reverse-topological backward pass that accumulates `.grad` on each node. Note that a feeds two paths, exactly why gradients must `+=` rather than overwrite.

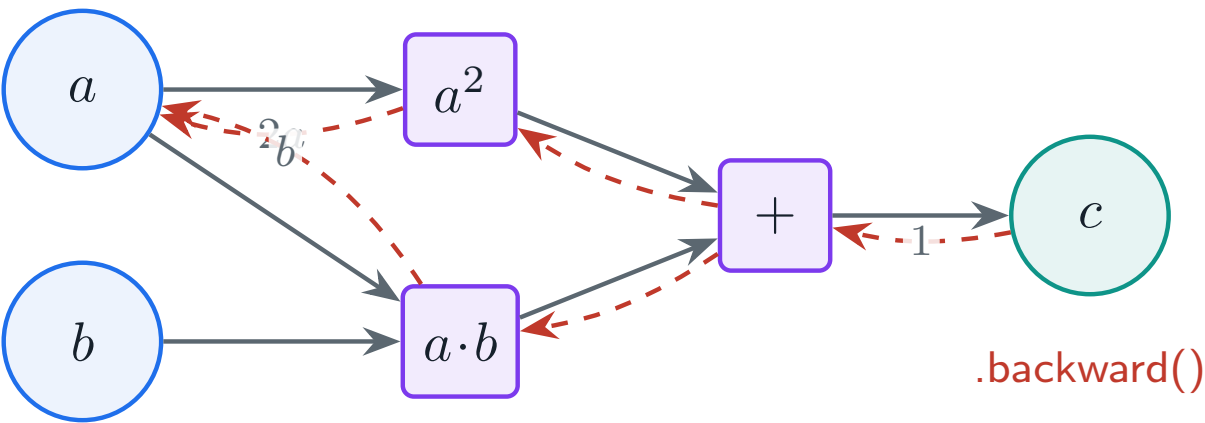


Figure 4.1: The autograd graph for $c = a \cdot b + a^2$. Each node stores `data`, `grad`, its parents, and a `_backward` closure. The node `a` has two consumers, so its gradient is the sum of both incoming contributions.

Table 4.1: Our scalar engine versus production tensor frameworks: same principles, very different engineering.

Aspect	This chapter’s engine	PyTorch / JAX
Granularity	Scalar Value	N-dim tensors (vectorized)
Backward	Python closures, recursion	Fused C++/CUDA kernels
Graph	Built every forward (define-by-run)	Eager (PyTorch) / traced & compiled (JAX)
Hardware	CPU, one number at a time	GPU/TPU, massively parallel
Purpose	Understanding	Production performance

4.6 Building a Tiny MLP

With `Value` as the foundation, a neuron, a layer, and a full MLP are thin wrappers around composing `Value` operations.

```
mini_mlp.py

import random

class Neuron:
    def __init__(self, nin):
        self.w = [Value(random.uniform(-1, 1)) for _ in range(nin)]
        self.b = Value(0.0)

    def __call__(self, x):
        act = sum((wi * xi for wi, xi in zip(self.w, x)), self.b)
        return act.tanh()

    def parameters(self):
        return self.w + [self.b]

class Layer:
    def __init__(self, nin, nout):
```

```

        self.neurons = [Neuron(nin) for _ in range(nout)]

    def __call__(self, x):
        outs = [n(x) for n in self.neurons]
        return outs[0] if len(outs) == 1 else outs

    def parameters(self):
        return [p for n in self.neurons for p in n.parameters()]

class MLP:
    def __init__(self, nin, nouts):
        sizes = [nin] + nouts
        self.layers = [Layer(sizes[i], sizes[i+1]) for i in range(len(nouts))]

    def __call__(self, x):
        for layer in self.layers:
            x = layer(x)
        return x

    def parameters(self):
        return [p for layer in self.layers for p in layer.parameters()]

```

4.7 Training It End to End

A complete training loop: forward pass, compute loss, zero gradients, backward pass, update weights. This is the exact same loop you'll write in PyTorch in Chapter 12, just with your own engine underneath instead of PyTorch's.

`train_loop.py`

```

import random
random.seed(42)    # reproducible run (the output below is from this seed)

# Toy dataset: 4 examples, 3 input features, binary-ish targets
xs = [
    [2.0, 3.0, -1.0],
    [3.0, -1.0, 0.5],
    [0.5, 1.0, 1.0],
    [1.0, 1.0, -1.0],
]
ys = [1.0, -1.0, -1.0, 1.0]

model = MLP(3, [4, 4, 1])

for step in range(50):
    # Forward pass
    ypred = [model(x) for x in xs]
    loss = sum((yp - yt) ** 2 for yp, yt in zip(ypred, ys))

    # Zero gradients

```

```

for p in model.parameters():
    p.grad = 0.0

# Backward pass
loss.backward()

# Update (plain SGD, Chapter 7 covers better optimizers)
for p in model.parameters():
    p.data += -0.05 * p.grad

if step % 10 == 0:
    print(f"step {step:2d}  loss {loss.data:.4f}")

```

Output

```

step 0  loss 5.2788
step 10  loss 0.3966
step 20  loss 0.1106
step 30  loss 0.0577
step 40  loss 0.0377

```

★ Key Insight

Watch the loss column. It should decrease, monotonically or close to it, exactly the same signature you'll learn to read in real PyTorch training runs in Chapter 11 and Chapter 14. If you implement this yourself and the loss does *not* decrease, that's valuable: it means you have a real bug to chase down with a 100-line, fully-inspectable codebase, far easier debugging practice than chasing the same kind of bug inside a production PyTorch model.

4.8 Gradient Checking Your Engine

Before trusting any of this, verify it against numerical differentiation, exactly the technique introduced in Chapter 3.

gradient_check.py

```

def numerical_grad(f, x, eps=1e-6):
    return (f(x + eps) - f(x - eps)) / (2 * eps)

# Compare engine gradient to numerical gradient for a small expression
a = Value(2.0)
b = Value(-3.0)
c = a * b + a ** 2
c.backward()

def f(a_val):
    return a_val * (-3.0) + a_val ** 2

```

```
print("engine dC/da:    ", a.grad)
print("numerical dC/da: ", numerical_grad(f, 2.0))
```

Output

```
engine dC/da:      1.0
numerical dC/da:  1.000000000139778
```

✓ Best Practice

Make gradient checking a permanent part of your workflow, not a one-time exercise. Any time you implement a new operation in a custom autograd system, or suspect a gradient bug in any framework, write a five-line numerical check like the one above before you spend an hour staring at training curves.

4.9 Chapter Summary

- A minimal autograd engine wraps scalars in a `Value` class that tracks data, gradient, and the operation that produced it.
- Every operation defines both a forward computation and a local backward function, exactly the local gradients derived by hand in Chapter 3.
- Gradients must accumulate (`+=`), not overwrite, since a value can feed into multiple downstream operations.
- The backward pass requires reverse topological order; getting this wrong is the most common bug when implementing autograd from scratch.
- A full MLP, and a full training loop, are thin compositions on top of this engine, the exact same shape you'll see in PyTorch in Chapter 12.

4.10 Chapter Evaluation

Ten questions on autograd internals, the classic from-scratch bugs, and how the toy engine maps to real frameworks. Hints first; answers in section 4.11.

Exercise 4.1

Walk through how autograd works under the hood, from forward graph construction to `.backward()`.

Hint. Record nodes on the forward pass; topologically sort; seed $dL/dL = 1$; chain local gradients in reverse.

Exercise 4.2

Why must the computational graph be a DAG, and how do RNNs “loop” without violating that?

Hint. A cycle has no valid topological order. RNNs unroll across time.

Exercise 4.3

Why is `+=` (not `=`) mandatory inside every `_backward`? Give a concrete graph where `=` silently gives the wrong answer.

Hint. Consider a node with two consumers, like a in $c = a \cdot b + a^2$.

Exercise 4.4

Why does the backward pass require *reverse topological* order specifically? What breaks with the wrong order?

Hint. A node must have its full incoming gradient before it pushes upstream.

Exercise 4.5

Implement the `_pow_` and `tanh` backward closures and state each local derivative.

Hint. $\frac{d}{dx}x^n = nx^{n-1}$; $\frac{d}{dx}\tanh x = 1 - \tanh^2 x$.

Exercise 4.6

(Debugging) Your toy MLP’s loss is flat. The math looks right. What two single-line bugs are most likely?

Hint. Forgot to zero grads each step? Overwrote instead of accumulated?

Exercise 4.7

Why must you zero gradients each training step? What happens if you don’t?

Hint. Gradients accumulate by design; the next step would add to last step’s.

Exercise 4.8

(Production) Name three concrete reasons this scalar engine is unusable for real training, mapping each to what PyTorch/JAX do instead.

Hint. Granularity, hardware, kernel fusion.

Exercise 4.9

Implement a numerical gradient check against the engine for $c = a \cdot b + a^2$ at $a = 2, b = -3$. What should dc/da be?

Hint. $dc/da = b + 2a$. Plug in.

Exercise 4.10

(Technical) Extend the engine with a `relu` op. Give forward and backward, and note the subtlety at $z = 0$.

Hint. $\text{relu}'(z) = 1[z > 0]$; the choice at exactly 0 is a convention.

4.11 Answers

Answer 4.1

How autograd works under the hood.

On the forward pass each operation creates a node recording its inputs, its result, and a `_backward` closure encoding the local derivative. Calling `.backward()` on the scalar loss: (1) build a topological order of all ancestors; (2) seed the loss's gradient to 1.0 (dL/dL); (3) walk the order in reverse, at each node multiply the incoming downstream gradient by the local gradient (chain rule) and *add* it to each input's `.grad`. The engine does no calculus at run time beyond evaluating pre-written local gradients, its real job is bookkeeping: record the graph, process it in the right order.

Answer 4.2

Why a DAG, and how RNNs loop.

The graph is *directed* (data flows inputs→outputs) and must be *acyclic* because a cycle would make a node depend on its own output, no valid evaluation order and no topological sort exists. RNNs (chapter 16) don't introduce a true cycle; they *unroll* the recurrence across T time steps into a deep, strictly acyclic chain of distinct nodes (one per step), which is exactly what makes backprop-through-time possible.

Answer 4.3

Why += and a graph where = fails.

A value feeding multiple consumers receives a gradient contribution from each path; the chain rule sums them. In $c = a \cdot b + a^2$, a feeds both the multiply and the square, so $dc/da = b + 2a$. With overwriting `=`, whichever `_backward` runs last wins and `a.grad` would equal only b or only $2a$, silently wrong, forward still works, so the bug hides. `+=` accumulates both contributions correctly.

Answer 4.4

Why reverse topological order.

A node's `_backward` pushes gradient to its parents, so it must already hold its *complete* incoming gradient (summed over all its consumers) before it fires. Reverse topological order guarantees every consumer of a node runs before the node itself. The wrong order makes a node push a partial gradient upstream, producing results that are wrong but often plausible-looking, the classic from-scratch autograd bug.

Answer 4.5

--pow-- and tanh backward.

```
pow_tanh_backward.py
```

```

def __pow__(self, k):
    # k is int/float
    out = Value(self.data ** k, (self,), f'**{k}')
    def _backward():
        self.grad += (k * self.data ** (k - 1)) * out.grad # n x^(n-1)
    out._backward = _backward
    return out

def tanh(self):
    t = math.tanh(self.data)
    out = Value(t, (self,), 'tanh')
    def _backward():
        self.grad += (1 - t * t) * out.grad # 1 - tanh^2
    out._backward = _backward
    return out

```

Each closure is the Chapter-3 local gradient written in code.

Answer 4.6

Debugging a flat toy-MLP loss.

Two one-line culprits: (1) **not zeroing grads**, you forgot `p.grad = 0.0` before `loss.backward()`, so stale gradients accumulate across steps and the update direction is garbage; (2) **overwrite vs accumulate**, a `_backward` uses `=` instead of `+=`, silently dropping contributions on any shared node. Both leave the forward pass working, so the only symptom is a loss that won't move.

Answer 4.7

Why zero gradients each step.

Because `.grad` accumulates by design (to support shared parameters and multi-path graphs). If you don't reset it, step $t + 1$'s gradient is added on top of step t 's leftover, so you descend along a stale, ever-growing direction and training destabilizes. The loop is always: zero grads $\rightarrow$ backward $\rightarrow$ step.

Answer 4.8

Production: why the scalar engine is unusable.

(1) **Scalar granularity**, one Python object per number means millions of Python-level ops; PyTorch/JAX operate on whole tensors in fused kernels. (2) **No hardware acceleration**, it runs serially on CPU; real frameworks dispatch to GPU/TPU. (3) **No kernel fusion / graph optimization**, every op is an interpreted closure; JAX traces and XLA-compiles, PyTorch uses optimized C++/CUDA. The *principles* are identical (see table 4.1); the engineering is not.

Answer 4.9

Numerical gradient check.

`engine_grad_check.py`

```

a = Value(2.0); b = Value(-3.0)
c = a * b + a ** 2
c.backward()                                # engine gradient

f = lambda av: av * (-3.0) + av ** 2
num = (f(2.0 + 1e-6) - f(2.0 - 1e-6)) / (2e-6)
print(a.grad, num)                         # dc/da = b + 2a = -3 + 4 = 1

```

$dc/da = b + 2a = -3 + 4 = 1$; the engine prints 1.0 and the numerical estimate ≈ 1.0000000 .

Answer 4.10

Adding a relu op.

relu_op.py

```

def relu(self):
    out = Value(self.data if self.data > 0 else 0.0, (self,), 'relu')
    def _backward():
        self.grad += (1.0 if self.data > 0 else 0.0) * out.grad
    out._backward = _backward
    return out

```

Forward: $\max(0, z)$. Backward: pass the gradient through where $z > 0$, block it where $z < 0$. At exactly $z = 0$ the derivative is undefined; by convention frameworks use 0 (sometimes 1). The discontinuity is measure-zero and rarely matters in practice, but it's why ReLU is “non-differentiable at the origin.”

PART II

Making Networks Learn Well

A network that can compute a forward pass and a gradient is not yet a network that learns well. This part covers the practitioner's toolkit: the choices around activations, losses, optimizers, initialization, regularization, and schedules that separate a model that trains from one that doesn't, and a model that generalizes from one that memorizes.

Activation Functions: ReLU, Sigmoid, GELU & Why

Subscribe → aiengineeringinsider.substack.com/subscribe

Why activation choice quietly determines whether your network trains at all.

Chapter 2 established that nonlinear activations are what give a multi-layer network real representational power, without them, any stack of layers collapses algebraically into a single linear transformation. This chapter is about which nonlinearity to actually use, and why the field's default answer has changed more than once.

5.1 Sigmoid and Tanh: The Original Choices

The sigmoid function was the default activation for decades:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

It squashes any real input into $(0, 1)$, which made it a natural choice both as a hidden-layer nonlinearity and, later, as the natural output function for binary classification (Chapter 6). Tanh is a rescaled sibling:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1, \quad \tanh'(z) = 1 - \tanh^2(z)$$

squashing inputs into $(-1, 1)$ instead, zero-centered, which tends to make optimization slightly better behaved than sigmoid in hidden layers.

! Pitfall / Warning

Look at $\sigma'(z)$ across its domain: it peaks at $z = 0$ with a value of exactly 0.25, and decays toward zero as $|z|$ grows in either direction. Stack several sigmoid layers and, per Chapter 3's vanishing gradient discussion, the backward pass multiplies several of these sub-0.25 terms together. Even in the best case, a 10-layer sigmoid network's gradient at the first layer is attenuated by a factor of at most $0.25^{10} \approx 10^{-6}$ before accounting for the weights at all. This single fact is most of the reason sigmoid and tanh were displaced as default hidden-layer activations.

5.2 ReLU: Simplicity, Sparsity, and Dying Units

The Rectified Linear Unit (Nair & Hinton, 2010) reset expectations for what a “good” activation function needed to look like:

$$\text{ReLU}(z) = \max(0, z), \quad \text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$$

★ Key Insight

ReLU’s derivative is exactly 1 for any positive input, not a fraction less than one. That means gradients pass through active ReLU units completely unattenuated, which is the single biggest reason ReLU enabled training meaningfully deeper networks than sigmoid or tanh ever could. It’s also computationally trivial, a max operation, with none of sigmoid’s exponentials.

The cost of that simplicity shows up on the negative side: the derivative is exactly zero for any negative input. A neuron whose weighted input is consistently negative across the entire training set will receive zero gradient, forever, and stop updating. This is the **dying ReLU problem**.

dying_relu_demo.py

```
import numpy as np

def relu(z):
    return np.maximum(0, z)

def relu_grad(z):
    return (z > 0).astype(float)

# A neuron whose weights have drifted so its pre-activation is always negative
z = np.array([-3.0, -1.5, -0.2, -5.0]) # all negative, e.g. bad init + large lr
print("activations:", relu(z))
print("gradients: ", relu_grad(z))
```

Output

```
activations: [0. 0. 0. 0.]
gradients:   [0. 0. 0. 0.]
```

! Pitfall / Warning

A large learning rate combined with a large negative bias update can push a ReLU unit’s weights into a region where it never activates again on any training example, zero output, zero gradient, permanently dead. In a wide enough layer, losing some fraction of units to this failure mode often goes unnoticed because the rest of the layer compensates; in a narrow layer, it can visibly degrade capacity.

5.3 Leaky ReLU and Parametric ReLU

The direct fix: give the negative side a small, nonzero slope instead of a hard floor at zero.

$$\text{LeakyReLU}(z) = \begin{cases} z & z \geq 0 \\ \alpha z & z < 0 \end{cases}$$

with α typically a small fixed constant like 0.01. **Parametric ReLU** (PReLU) makes α a learned parameter instead of a fixed hyperparameter, letting the network decide, per channel, how much negative signal to preserve.

✓ Best Practice

Leaky ReLU is a reasonable default substitution any time you suspect dying ReLUs are limiting a network, large negative biases in saved checkpoints, or a sizable fraction of zero activations when you inspect a layer's output distribution. It rarely hurts and costs essentially nothing extra to compute.

5.4 GELU and Swish/SiLU

Modern architectures, transformers especially, have largely moved to smoother activations. The **Gaussian Error Linear Unit** (GELU; Hendrycks & Gimpel, 2016) weights its input by the standard Gaussian CDF:

$$\text{GELU}(z) = z \cdot \Phi(z), \quad \Phi(z) = P(X \leq z), \quad X \sim \mathcal{N}(0, 1)$$

In practice this is usually computed with a fast tanh-based approximation rather than the exact Gaussian CDF. **Swish** (also called SiLU, Sigmoid Linear Unit) takes a related form:

$$\text{Swish}(z) = z \cdot \sigma(z)$$

Saturating vs. Non-Saturating Activations

An activation function **saturates** when its derivative approaches zero as the input grows large in magnitude (sigmoid and tanh both saturate on both sides). It is **non-saturating** when the derivative stays bounded away from zero for at least one direction of large input (ReLU and its variants are non-saturating for positive inputs). Saturation is precisely the mechanism behind vanishing gradients introduced in Chapter 3; non-saturating activations were adopted specifically to avoid it.

Unlike ReLU, both GELU and Swish are smooth everywhere (no sharp corner at zero) and non-monotonic, they dip slightly negative just below zero before rising, which empirically seems to help optimization in very deep networks, though the precise theoretical reason remains an active area of debate rather than settled science.

! Pitfall / Warning

Resist the urge to treat “GELU is what transformers use, so GELU is strictly better” as a universal rule. The empirical gains of GELU/Swish over ReLU are real but modest, and come at a real computational cost (exponentials or error functions instead of a single comparison). For many CNN architectures and smaller models, plain ReLU remains a perfectly strong, faster-to-compute default.

5.5 Architecture Diagram

Figure 5.1 plots the four activations side by side. Read it for two things: where each function *saturates* (flat regions kill the gradient) and where it stays *active* (slope near 1 lets gradient flow). Sigmoid and tanh flatten on both ends; ReLU is flat only on the left (the dying-unit zone); GELU is smooth and lets a little negative signal through.

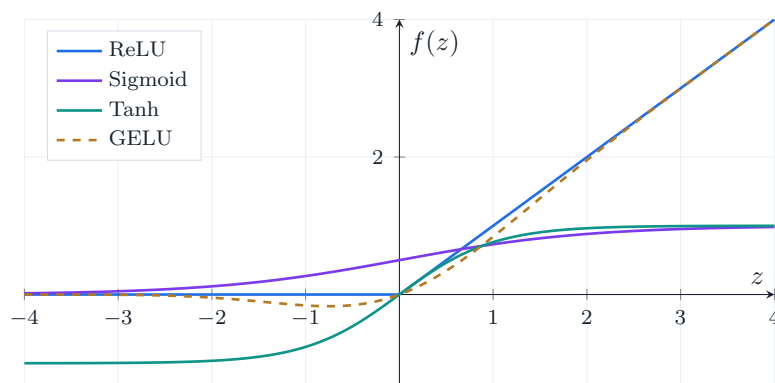


Figure 5.1: Activation functions compared. Flat regions are where the derivative $\rightarrow 0$ and gradients vanish; the steep central region is where learning signal flows. ReLU and GELU stay non-saturating for positive inputs.

Table 5.1: Activation properties at a glance. “Saturates” marks where the derivative collapses to zero.

Function	Range	Saturates	Smooth	Main risk
Sigmoid	$(0, 1)$	both sides	yes	vanishing gradient
Tanh	$(-1, 1)$	both sides	yes	vanishing gradient
ReLU	$[0, \infty)$	left only	no (corner)	dying units
Leaky ReLU	$(-\infty, \infty)$	no	no (corner)	α tuning
GELU/Swish	$\approx [-0.17, \infty)$	left only	yes	compute cost

5.6 Softmax for Output Layers

Softmax is a special case: it’s not used in hidden layers, only at the output layer of a multi-class classifier, and it operates on an entire vector at once rather than element-wise.

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

This produces a valid probability distribution: every output is positive and the whole vector sums to exactly 1. Chapter 6 covers exactly why softmax is almost always paired with cross-entropy loss, and Chapter 3’s interview question already previewed the elegant gradient that results.

softmax_demo.py

```
import numpy as np

def softmax(z):
    z_shifted = z - np.max(z)  # numerical stability, see warning below
    exp_z = np.exp(z_shifted)
    return exp_z / np.sum(exp_z)

logits = np.array([2.0, 1.0, 0.1])
print(softmax(logits))
print("sums to:", softmax(logits).sum())
```

Output

```
[0.65900114 0.24243297 0.09856589]
sums to: 1.0
```

! Pitfall / Warning

Always subtract $\max(\mathbf{z})$ before exponentiating, exactly as the code above does. Without it, large logits overflow `float32` or `float64` silently, producing `inf` or `nan` instead of a clean probability distribution. This shift doesn’t change the mathematical result (it cancels in the ratio), but it changes whether the computation is numerically stable in practice. Every production framework implements softmax this way internally; if you’re ever implementing it by hand, do the same.

5.7 Choosing an Activation by Layer Position and Task

Position / Task	Typical Choice
Hidden layers, general default	ReLU, or Leaky ReLU if dying units are observed
Hidden layers, transformers	GELU (or Swish/SiLU)
Binary classification output	Sigmoid
Multi-class classification output	Softmax
Regression output	None (linear/identity)
Recurrent network gates	Sigmoid and tanh, by convention (Chapter 16)

From the Field

A genuinely common mistake among engineers new to the field: applying an activation function, often ReLU, to a regression output layer “for consistency” with the hidden layers.

This silently caps every prediction at zero, which is catastrophic if the true target values can be negative. If you're getting suspiciously many zero predictions on a regression task, check the output layer's activation before you check anything else.

5.8 Chapter Summary

- Sigmoid and tanh saturate on both sides, which causes vanishing gradients in deep networks and is why they were displaced as default hidden-layer activations.
- ReLU passes gradient through unattenuated for positive inputs but can permanently “die” for units whose input is consistently negative.
- Leaky ReLU and PReLU fix dying units by giving the negative side a small nonzero slope.
- GELU and Swish/SiLU are smooth, non-monotonic activations favored in modern transformer architectures, at a higher computational cost than ReLU.
- Softmax is reserved for multi-class output layers, producing a valid probability distribution, and always needs the max-subtraction trick for numerical stability.

5.9 Chapter Evaluation

Ten questions on saturation, the dying-ReLU failure mode, numerical stability, and how to choose an activation by layer and task. Hints first; answers in section 5.10.

Exercise 5.1

Why were sigmoid/tanh displaced as default hidden activations? Quantify the problem.

Hint. $\max \sigma' = 0.25$. What does multiplying many such factors do across depth?

Exercise 5.2

Why did ReLU enable training much deeper networks than sigmoid?

Hint. What is $\text{ReLU}'(z)$ for $z > 0$, and what does that do to gradient magnitude?

Exercise 5.3

What causes a ReLU unit to “die,” and how do you prevent and detect it?

Hint. $\text{ReLU}'(z) = 0$ for $z < 0$. What if $z < 0$ for every example, forever?

Exercise 5.4

Distinguish saturating from non-saturating activations and tie it to vanishing gradients.

Hint. Saturation = derivative $\rightarrow 0$ for large $|z|$.

Exercise 5.5

Why is softmax always implemented with a max-subtraction? Show it doesn't change the result.

Hint. $e^{z_i} / \sum e^{z_j}$ vs $e^{z_i - c} / \sum e^{z_j - c}$. What cancels? What overflows without it?

Exercise 5.6

Why did transformers adopt GELU/Swish over ReLU, and is it “strictly better”?

Hint. Smooth, non-monotonic, modest empirical gains, negligible cost relative to attention.

Exercise 5.7

(Debugging) A regression model outputs suspiciously many exact zeros. First thing to check?

Hint. What activation is on the output layer?

Exercise 5.8

(Production) You inspect a hidden layer and 80% of activations are exactly 0. Is that necessarily bad? What would you do?

Hint. Some sparsity is healthy; near-total sparsity is dying units. Distinguish the two.

Exercise 5.9

Implement numerically-stable softmax and Leaky ReLU in NumPy.

Hint. Subtract the max before exp; LeakyReLU uses αz for $z < 0$.

Exercise 5.10

(Technical) Give a decision rule for choosing an activation by layer position and task.

Hint. Hidden: ReLU/GELU. Output: depends on the loss/target.

5.10 Answers

Answer 5.1

Why sigmoid/tanh were displaced.

Both saturate on both sides, so their derivatives collapse toward 0 for large $|z|$. Sigmoid's derivative peaks at just 0.25. The backward recursion (chapter 3) multiplies one such factor per layer, so a 10-layer sigmoid net attenuates the first layer's gradient by at most $0.25^{10} \approx 10^{-6}$ before weights, early layers barely learn. ReLU's derivative of 1 for $z > 0$ removes that attenuation.

Answer 5.2*Why ReLU enabled depth.*

For any positive pre-activation, $\text{ReLU}'(z) = 1$ exactly, so gradients pass through active units *unattenuated*, no per-layer shrink factor. That breaks the multiplicative vanishing that sigmoid/tanh impose, letting useful gradient reach early layers of deep networks. It's also cheap (a single comparison, no exponential).

Answer 5.3*Dying ReLU: cause, prevention, detection.*

A unit dies when $z = \mathbf{w}^\top \mathbf{x} + b < 0$ for *every* training example; since $\text{ReLU}' = 0$ there, it gets zero gradient forever and never recovers. Usual cause: too-large LR + an unlucky large negative update pushing the bias far negative. Prevent with smaller LR / warmup (chapter 10), He init (chapter 8), or Leaky ReLU/PReLU (nonzero negative slope). Detect by monitoring the fraction of zero activations per layer, creeping toward 100% is the warning.

Answer 5.4*Saturating vs non-saturating.*

An activation *saturates* where its derivative $\rightarrow 0$ for large $|z|$ (sigmoid, tanh, both sides). It is *non-saturating* when the derivative stays bounded away from 0 for at least one direction (ReLU for $z > 0$). Saturation is the direct mechanism of vanishing gradients, repeated near-zero derivatives multiply to nothing across depth, so non-saturating activations were adopted precisely to avoid it.

Answer 5.5*Stable softmax.*

$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}} = \frac{e^{z_i - c}}{\sum_j e^{z_j - c}}$ for any constant c (factor e^{-c} cancels top and bottom). Choosing $c = \max_j z_j$ makes the largest exponent 0, so $e^{z_i - c} \in (0, 1]$, no overflow. Without it, a logit like 1000 makes $e^{1000} = \text{inf}$ and the result is **nan**. Mathematically identical; numerically essential.

Answer 5.6*GELU/Swish in transformers.*

GELU is smooth (no corner), non-monotonic (dips slightly negative below 0, acting like a soft probabilistic gate), and empirically gives modestly better-conditioned training at transformer scale. Its extra cost (a tanh/erf) is negligible next to attention and the big feedforward matmuls. It is *not* strictly better, for many CNNs and small models plain ReLU is as good and faster. The transformer adoption is largely empirical convention from BERT/GPT, not a settled theorem.

Answer 5.7*Debugging zero-heavy regression output.*

Check the *output* activation first. A ReLU (or any nonnegative activation) on a regression head clamps every prediction to ≥ 0 , producing many exact zeros and making negative

targets unreachable. Regression outputs should be linear (identity). This is a classic “applied ReLU for consistency” mistake.

Answer 5.8

Production: 80% zero activations.

Not necessarily bad, ReLU networks are *designed* to be sparse, and moderate sparsity improves efficiency and can aid generalization. It’s a problem only if those zeros are *permanent* (dying units). Distinguish by checking whether the same units are zero across *all* batches (dead) or different units per input (healthy sparsity). If dead: lower LR, switch to Leaky ReLU, re-init the affected layer, or check for a too-negative bias.

Answer 5.9

Stable softmax + Leaky ReLU in NumPy.

`activations.py`

```
import numpy as np

def softmax(z, axis=-1):
    z = z - np.max(z, axis=axis, keepdims=True) # stability shift
    e = np.exp(z)
    return e / np.sum(e, axis=axis, keepdims=True)

def leaky_relu(z, alpha=0.01):
    return np.where(z >= 0, z, alpha * z)

def leaky_relu_grad(z, alpha=0.01):
    return np.where(z >= 0, 1.0, alpha)
```

The `keepdims=True` broadcasting makes softmax work on batched logits along any axis.

Answer 5.10

Choosing an activation.

Hidden layers: ReLU by default (Leaky ReLU if you see dying units), GELU/Swish for transformers. Output layer is dictated by the task and loss: *sigmoid* for binary classification, *softmax* for multi-class, *identity/linear* for regression. Recurrent gates use sigmoid/tanh by convention (chapter 16). The rule of thumb: pick hidden activations for gradient flow, output activations for the target’s range and the paired loss (table 5.1).

Loss Functions: MSE, Cross-Entropy, Contrastive

Subscribe → aiengineeringinsider.substack.com/subscribe

How a network knows it's wrong, and why the loss function shapes everything downstream.

A network's loss function is the single number that gradient descent (and, underneath it, backpropagation) is trying to minimize. Pick the wrong loss for your task and no amount of architecture tuning or hyperparameter search will fix the underlying mismatch between what you're optimizing and what you actually want the model to do.

6.1 Regression Losses

6.1.1 Mean Squared Error

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

MSE penalizes large errors disproportionately, doubling an error quadruples its contribution to the loss, which makes it sensitive to outliers but gives it a smooth, well-behaved gradient everywhere: $\frac{\partial}{\partial \hat{y}_i} \text{MSE} = \frac{2}{N} (\hat{y}_i - y_i)$.

6.1.2 Mean Absolute Error and Huber Loss

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

MAE is robust to outliers (a single huge error contributes linearly, not quadratically) but its gradient has a constant magnitude regardless of error size, and is undefined exactly at zero error. **Huber loss** interpolates between the two, quadratic for small errors, linear for large ones, controlled by a threshold δ :

$$L_{\delta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & |y - \hat{y}| \leq \delta \\ \delta (|y - \hat{y}| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

✓ Best Practice

Default to MSE for regression unless you have a specific reason not to. Reach for Huber loss when your data has occasional large outliers you don't want to dominate training (sensor glitches, data entry errors), and MAE when you want the model to optimize for median rather than mean behavior.

6.2 Classification Losses

6.2.1 Binary Cross-Entropy

For a binary target $y \in \{0, 1\}$ and predicted probability $\hat{y} \in (0, 1)$ (typically the output of a sigmoid):

$$L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

This is the negative log-likelihood of the true label under the model's predicted Bernoulli distribution. When $y = 1$, the loss reduces to $-\log \hat{y}$, which is small when $\hat{y}$ is close to 1 and grows without bound as $\hat{y} \rightarrow 0$, the loss punishes confident wrong answers far more severely than a squared-error loss would.

6.2.2 Categorical Cross-Entropy

The multi-class generalization, for a one-hot true label vector $\mathbf{y}$ and predicted probability vector $\hat{\mathbf{y}}$ (typically the output of softmax):

$$L = - \sum_{k=1}^K y_k \log \hat{y}_k$$

Since $\mathbf{y}$ is one-hot, only one term in the sum survives, this simplifies to $-\log \hat{y}_c$ where c is the correct class index. The loss is exactly the negative log-probability the model assigned to the correct answer.

6.3 Why Cross-Entropy Pairs with Softmax

Chapter 3's interview question previewed this result, and it's worth stating in full here. When categorical cross-entropy is paired with a softmax output layer, the gradient with respect to the pre-softmax logits $\mathbf{z}$ simplifies to:

$$\frac{\partial L}{\partial z_k} = \hat{y}_k - y_k$$

★ Key Insight

This is remarkably clean: the gradient flowing back into the network is just “predicted probability minus true label,” for every class simultaneously. No leftover sigmoid or softmax derivative terms survive in the final expression. This is the real, mathematical reason cross-entropy and softmax (or sigmoid, in the binary case) are used together almost universally, the combination produces well-scaled, well-behaved gradients that don’t suffer the vanishing-gradient issues a mismatched loss/activation pairing (like MSE with a sigmoid output) can run into.

cross_entropy_grad.py

```
import numpy as np

def softmax(z):
    z = z - np.max(z)
    e = np.exp(z)
    return e / e.sum()

def cross_entropy(y_true_idx, probs):
    return -np.log(probs[y_true_idx])

logits = np.array([2.0, 0.5, -1.0])
y_true_idx = 0 # correct class is index 0

probs = softmax(logits)
loss = cross_entropy(y_true_idx, probs)

# Analytical gradient: probs - one_hot(y_true)
y_onehot = np.zeros_like(probs)
y_onehot[y_true_idx] = 1.0
grad = probs - y_onehot

print("probs:", probs)
print("loss: ", loss)
print("grad: ", grad)
```

Output

```
probs: [0.78559703 0.17529039 0.03911257]
loss:  0.24131129665715703
grad:  [-0.21440297 0.17529039 0.03911257]
```

6.4 Contrastive Loss

Not every task fits the “predict a label” mold. **Contrastive loss** (Hadsell et al., 2006) is designed for tasks where the goal is learning a useful embedding space, pulling representations of similar items together and pushing dissimilar items apart, rather than predicting a class directly.

Contrastive Loss

A loss function that operates on pairs of examples rather than individual labeled examples, minimizing distance between embeddings of similar (“positive”) pairs and maximizing distance, up to some margin, between embeddings of dissimilar (“negative”) pairs. It underlies self-supervised representation learning methods like SimCLR and is foundational to how embedding models (including the encoders behind retrieval and search systems) are trained.

A common form, given a distance function D (often Euclidean distance between embeddings), a binary similarity label y (1 if the pair is similar, 0 if dissimilar), and a margin m :

$$L = y \cdot D(\mathbf{a}, \mathbf{b})^2 + (1 - y) \cdot \max(0, m - D(\mathbf{a}, \mathbf{b}))^2$$

For similar pairs ($y = 1$), only the first term is active, it directly penalizes distance, pulling the embeddings together. For dissimilar pairs ($y = 0$), only the second term is active, and it only penalizes pairs that are *closer than the margin* m , pairs already far apart contribute zero loss. This margin is what prevents the loss from pushing dissimilar embeddings infinitely far apart, wasted optimization effort once they’re sufficiently separated.

6.4.1 Triplet Loss and Hard Negative Mining

Triplet loss extends the same idea to triplets: an anchor $\mathbf{a}$, a positive example $\mathbf{p}$ (similar to the anchor), and a negative example $\mathbf{n}$ (dissimilar):

$$L = \max(0, D(\mathbf{a}, \mathbf{p}) - D(\mathbf{a}, \mathbf{n}) + m)$$

This directly optimizes a relative constraint: the anchor should be closer to the positive than to the negative, by at least the margin m .

! Pitfall / Warning

Triplet loss is notoriously sensitive to *how negatives are chosen*. Random negatives are usually already far from the anchor and contribute near-zero gradient (the loss is already satisfied), wasting most of training on uninformative triplets. **Hard negative mining**, explicitly selecting negatives that are currently close to the anchor (and therefore violating, or nearly violating, the margin constraint) is often the difference between a contrastive model that trains efficiently and one that plateaus early. It is also easy to overdo: mining only the very hardest negatives can destabilize training, since some of those “hard” negatives may be mislabeled or genuinely ambiguous pairs.

6.5 Architecture Diagram

Figure 6.1 plots the three regression losses against the error $r = y - \hat{y}$. The shapes encode their behavior: MSE’s parabola punishes large errors quadratically (outlier-sensitive); MAE’s V grows linearly (robust, but a constant-magnitude gradient and a kink at 0); Huber is the deliberate compromise, quadratic near zero for a smooth gradient, linear in the tails for robustness.

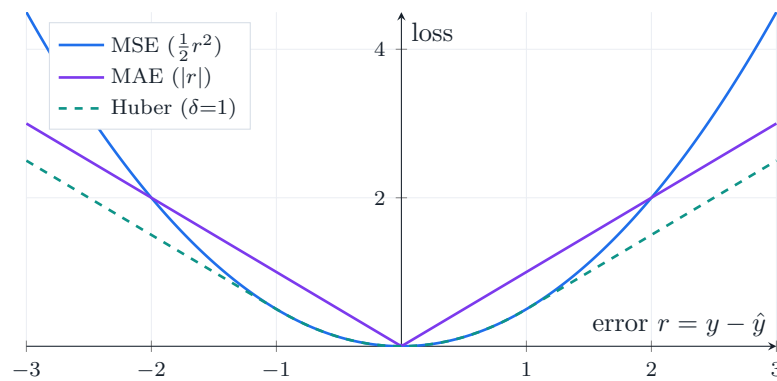


Figure 6.1: Regression losses versus error r . MSE (parabola) over-weights outliers; MAE (V) is robust but non-smooth at 0; Huber ($\delta=1$) is quadratic in the middle and linear in the tails.

Table 6.1: Loss-function selection map: task, output activation it pairs with, and the gradient signature.

Loss	Pairs with	Key property
MSE	linear output	smooth; outlier-sensitive; $\text{grad} \propto r$
Huber	linear output	robust + smooth; grad capped at δ
Binary cross-entropy	sigmoid (logits)	$\text{grad} = \hat{y} - y$; punishes confident wrong
Categorical CE	softmax (logits)	$\text{grad} = \hat{\mathbf{y}} - \mathbf{y}$
Contrastive / triplet	embedding (L2)	margin-based; needs negative mining

6.6 Choosing a Loss Function for Your Task

Task	Typical Loss
Regression	MSE (default), Huber (outlier-robust)
Binary classification	Binary cross-entropy
Multi-class classification	Categorical cross-entropy
Multi-label classification	Binary cross-entropy, applied per label
Embedding / representation learning	Contrastive or triplet loss
Ranking	Pairwise or listwise ranking losses (margin-based, related to contrastive)

6.7 Diagnosing a Loss That Won't Go Down

✓ Best Practice

When a loss refuses to decrease, check the loss/output pairing before suspecting anything more exotic. The most common real-world causes, in roughly descending order of frequency: a mismatched loss and output activation (e.g., MSE on raw logits instead of cross-entropy, or a sigmoid output with a loss that expects raw logits), a learning rate that's orders of magnitude too high or too low, label encoding bugs (off-by-one class indices, or labels that aren't actually one-hot when the loss expects them to be), or unnormalized input features

with wildly different scales across dimensions. Chapter 11 and Chapter 14 build out a full diagnostic workflow for exactly this situation.

6.8 Chapter Summary

- MSE is the regression default; Huber loss trades some sensitivity for outlier robustness.
- Binary and categorical cross-entropy are the standard classification losses, derived as the negative log-likelihood under the model's predicted distribution.
- Cross-entropy paired with softmax (or sigmoid) produces a remarkably clean gradient, predicted probability minus true label, which is the real reason this pairing is the near-universal default.
- Contrastive and triplet losses operate on pairs or triplets rather than single labeled examples, optimizing an embedding space directly rather than a classification boundary.
- Hard negative mining is often essential for contrastive and triplet losses to train efficiently; random negatives contribute little useful gradient once a model is even moderately well-trained.

6.9 Chapter Evaluation

Ten questions on choosing and pairing losses, the cross-entropy/softmax gradient, contrastive learning, and diagnosing a stuck loss. Hints first; answers in section 6.10.

Exercise 6.1

Why is MSE a poor choice for classification? Give the geometric and the gradient argument.

Hint. Confident-wrong should hurt a lot. What does $\text{MSE} \times \sigma'$ do when the prediction is confidently wrong?

Exercise 6.2

Derive why cross-entropy + softmax yields $\partial L / \partial z_k = \hat{y}_k - y_k$, and why that matters.

Hint. The softmax derivative cancels the $1/\hat{y}$ from the log.

Exercise 6.3

When would you choose Huber or MAE over MSE? What does each optimize for?

Hint. Outliers, and mean vs median behavior.

Exercise 6.4

Explain contrastive loss and three production systems it underpins.

Hint. Pull positives together, push negatives apart up to a margin. Think search/RAG/recsys.

Exercise 6.5

What is the margin m for in contrastive/triplet loss, and what breaks without it?

Hint. Without a margin, how far does the optimizer push already-separated negatives?

Exercise 6.6

Why is hard-negative mining often essential for triplet loss, and how can it backfire?

Hint. Random negatives are already satisfied (zero grad); the very hardest may be mislabeled.

Exercise 6.7

(Production) A multi-label classifier (an image can be cat *and* dog) uses softmax + categorical CE. What's wrong, and the fix?

Hint. Softmax forces the labels to compete (sum to 1). Multi-label labels don't.

Exercise 6.8

(Debugging) Your loss is flat from step 0. List the four most common causes in order of frequency.

Hint. Loss/activation mismatch, LR, label encoding, input scaling.

Exercise 6.9

Implement numerically-stable categorical cross-entropy directly from logits.

Hint. Use log-sum-exp; don't exponentiate then log.

Exercise 6.10

(Technical) Why do frameworks fuse softmax and cross-entropy into one op (e.g. CrossEntropyLoss)?

Hint. Numerical stability and the clean fused gradient $\hat{y} - y$.

6.10 Answers

Answer 6.1

Why MSE is bad for classification.

Geometrically, MSE under-penalizes confident-wrong predictions: squared error grows gently as a probability approaches the wrong extreme, whereas log-loss $\rightarrow \infty$. Mechanically, MSE $\times$ a sigmoid/softmax output multiplies in $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, which is *tiny* exactly when the model is confidently wrong, so the gradient stalls when you most need it. Cross-entropy removes that factor: its gradient is $\hat{y} - y$, large for confident-wrong, giving fast correction.

Answer 6.2*Softmax + CE gradient.*

With $\hat{y}_k = \text{softmax}(\mathbf{z})_k$ and $L = -\sum_k y_k \log \hat{y}_k$ for one-hot $\mathbf{y}$: the softmax Jacobian $\partial \hat{y}_k / \partial z_j = \hat{y}_k(\delta_{kj} - \hat{y}_j)$ combines with $\partial L / \partial \hat{y}_k = -y_k / \hat{y}_k$ to give $\partial L / \partial z_j = \sum_k (-y_k / \hat{y}_k) \hat{y}_k (\delta_{kj} - \hat{y}_j) = \hat{y}_j - y_j$ (using $\sum_k y_k = 1$). It matters because the backward signal is just “probability minus label,” well-scaled, no vanishing activation term.

Answer 6.3*Huber/MAE vs MSE.*

Use **MSE** by default, smooth, optimizes the conditional *mean*. Use **Huber** when occasional large outliers (sensor glitches, data-entry errors) shouldn’t dominate, it’s quadratic near zero (smooth gradient) and linear in the tails (bounded influence per example). Use **MAE** when you want the conditional *median* and full robustness, accepting a constant-magnitude, non-smooth-at-zero gradient.

Answer 6.4*Contrastive loss and where it’s used.*

It trains embeddings so similar inputs land close and dissimilar inputs land far (up to a margin), instead of predicting a class. Production uses: (1) **semantic search / retrieval** (and RAG) where query and document encoders must put related text nearby; (2) **recommendation** (two-tower user/item embeddings); (3) **self-supervised vision** (SimCLR: positive pairs are two augmentations of one image). Face verification (FaceNet) is another classic.

Answer 6.5*Role of the margin.*

The margin m caps how far dissimilar pairs need to be pushed: a negative pair already farther than m contributes zero loss. Without it, the optimizer keeps pushing already-separated negatives infinitely apart, wasting capacity and destabilizing the embedding geometry (and for positives-only objectives, collapsing everything). The margin defines “separated enough.”

Answer 6.6*Hard-negative mining.*

Once a model is even moderately trained, random negatives are already beyond the margin, so they yield zero gradient and training stalls on uninformative triplets. Mining negatives that currently *violate* the margin concentrates gradient where learning is needed. It backfires if you mine *only* the hardest: those are disproportionately mislabeled or genuinely ambiguous, so the model chases noise and training destabilizes. Semi-hard mining is the usual compromise.

Answer 6.7*Production: multi-label with softmax.*

Softmax forces the class probabilities to sum to 1, so the labels *compete*, raising “cat” must lower “dog,” which is wrong when both are present. Replace it with *per-label sigmoids*

and *binary cross-entropy applied independently to each label* (BCEWithLogitsLoss over K outputs). Each label then gets an independent presence probability.

Answer 6.8

Debugging a flat loss.

In rough frequency order: (1) **loss/activation mismatch**, e.g. feeding probabilities into a logits-loss, or MSE on logits; (2) **learning rate** orders of magnitude off; (3) **label-encoding bug**, off-by-one class index, labels not one-hot when expected, wrong dtype; (4) **unnormalized inputs** with wildly different feature scales. Check them in that order before suspecting the architecture (chapter 14).

Answer 6.9

Stable cross-entropy from logits.

ce_from_logits.py

```
import numpy as np

def cross_entropy_logits(logits, y_idx):
    """logits: (N, K) raw scores; y_idx: (N,) correct class indices."""
    m = logits.max(axis=1, keepdims=True) # stability
    logsumexp = m.squeeze(1) + np.log(np.exp(logits - m).sum(axis=1))
    correct = logits[np.arange(len(y_idx)), y_idx]
    return np.mean(logsumexp - correct) # -log softmax(correct)
```

This computes $-\log \text{softmax}$ via log-sum-exp, never forming a probability that could underflow to 0 before the log.

Answer 6.10

Why fuse softmax + CE.

Two reasons. (1) **Numerical stability**: the fused op uses log-sum-exp and never materializes a probability that could underflow to 0 and then be $\log$ 'd to $-\infty$. (2) **Clean fused gradient**: the backward is simply $\hat{\mathbf{y}} - \mathbf{y}$, computed directly without separately differentiating softmax then the log. That's why PyTorch's `CrossEntropyLoss` and `BCEWithLogitsLoss` take *logits*, not probabilities.

Optimizers: SGD, Momentum, Adam, AdamW

Subscribe → aiengineeringinsider.substack.com/subscribe

How the gradient actually gets turned into a weight update.

Backpropagation (Chapter 3) computes a gradient. An **optimizer** decides what to do with it: how big a step to take, in what direction, and how to adapt that decision as training progresses. The naive answer, “just subtract the gradient,” works, but every refinement in this chapter exists because that naive answer has real, specific failure modes.

7.1 Batch, Stochastic, and Mini-Batch Gradient Descent

Batch gradient descent computes the loss gradient over the entire training set before taking a single step. It produces an accurate, low- variance gradient estimate but is often computationally infeasible at scale and, somewhat counterintuitively, can converge to worse solutions, since it never benefits from the noise that helps escape shallow local minima.

Stochastic gradient descent (SGD) computes the gradient from a single example at a time. It’s noisy but cheap, and that noise is often a feature, not a bug, it helps the optimizer escape sharp, narrow minima that generalize poorly.

Mini-batch gradient descent, the practical default, splits the difference: compute the gradient over a small batch (commonly 32 to a few thousand examples). This gets a much less noisy estimate than single- example SGD while still being far cheaper than a full-batch pass, and it maps efficiently onto GPU parallelism.

Epoch vs. Step

A **step** (or iteration) is one parameter update, computed from one mini-batch. An **epoch** is one full pass through the entire training set. With a dataset of 10,000 examples and a batch size of 100, one epoch consists of 100 steps.

7.2 Momentum

Plain mini-batch SGD updates each parameter independently at each step: $\theta \leftarrow \theta - \eta \nabla_{\theta} L$. This ignores everything about the trajectory the optimizer has taken so far, which causes visible problems in practice: oscillation across narrow ravines in the loss surface, and slow progress across long, shallow plateaus.

Momentum fixes this by accumulating a velocity term, an exponential moving average of past gradients, and stepping in that direction instead of the raw current gradient:

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} L, \quad \theta \leftarrow \theta - \eta v_t$$

with β typically around 0.9. Gradients that consistently point the same direction across steps reinforce each other and accelerate progress; gradients that oscillate (pointing in alternating directions across a narrow ravine) partially cancel out, damping the oscillation.

★ Key Insight

A useful physical intuition: think of the loss surface as a landscape and the optimizer as a ball rolling across it. Plain SGD is a ball with no mass, it stops instantly whenever the local slope is zero, even if it's sitting in a long, shallow valley. Momentum gives the ball mass and inertia, it keeps moving through flat regions and small bumps based on where it's already been heading, exactly the behavior that helps escape shallow local minima and traverse plateaus faster.

7.3 Adagrad and RMSProp: Adaptive Learning Rates

Momentum addresses the *direction* of the update. A separate problem is that a single global learning rate η is rarely ideal for every parameter simultaneously, some parameters' gradients are consistently large, others consistently small, and a learning rate tuned for one can be wrong by orders of magnitude for the other.

Adagrad adapts the learning rate per parameter, dividing by the square root of the sum of all past squared gradients for that parameter:

$$G_t = G_{t-1} + (\nabla_{\theta} L)^2, \quad \theta \leftarrow \theta - \frac{\eta}{\sqrt{G_t} + \epsilon} \nabla_{\theta} L$$

Parameters with large historical gradients get their effective learning rate shrunk; parameters with small historical gradients keep a relatively larger effective learning rate. The flaw: G_t only ever grows, so the effective learning rate monotonically shrinks toward zero over a long training run, eventually halting learning entirely regardless of whether the model has actually converged.

RMSProp fixes this by replacing the cumulative sum with an exponential moving average, so old gradients decay away instead of permanently inflating the denominator:

$$G_t = \gamma G_{t-1} + (1 - \gamma) (\nabla_{\theta} L)^2, \quad \theta \leftarrow \theta - \frac{\eta}{\sqrt{G_t} + \epsilon} \nabla_{\theta} L$$

7.4 Adam: Combining Momentum and Adaptive Rates

Adam (Adaptive Moment Estimation; Kingma & Ba, 2015) combines both ideas: a momentum-style moving average of the gradient itself (the first moment) and an RMSProp-style moving average of the squared gradient (the second moment), plus a bias-correction term to account for both averages starting at zero.

$$\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} L, & v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} L)^2 \\
\hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, & \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \\
\theta &\leftarrow \theta - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t
\end{aligned}$$

Default hyperparameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$) work reasonably well across a very wide range of problems, which is the core reason Adam became the default starting point for most deep learning practitioners: it requires comparatively little tuning to get a model training reasonably well.

adam_from_scratch.py

```
import numpy as np

class Adam:
    def __init__(self, params, lr=1e-3, beta1=0.9, beta2=0.999, eps=1e-8):
        self.lr, self.b1, self.b2, self.eps = lr, beta1, beta2, eps
        self.m = {id(p): np.zeros_like(p) for p in params}
        self.v = {id(p): np.zeros_like(p) for p in params}
        self.t = 0

    def step(self, params, grads):
        self.t += 1
        for p, g in zip(params, grads):
            key = id(p)
            self.m[key] = self.b1 * self.m[key] + (1 - self.b1) * g
            self.v[key] = self.b2 * self.v[key] + (1 - self.b2) * (g ** 2)
            m_hat = self.m[key] / (1 - self.b1 ** self.t)
            v_hat = self.v[key] / (1 - self.b2 ** self.t)
            p -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)

# Minimize f(x) = x^2, gradient is 2x
x = np.array([5.0])
opt = Adam([x], lr=0.1)
for step in range(10):
    grad = 2 * x
    opt.step([x], [grad])
    if step % 2 == 0:
        print(f"step {step}  x={x[0]:.4f}")
```

Output

```
step 0  x=4.9000
step 2  x=4.7002
step 4  x=4.5010
step 6  x=4.3026
step 8  x=4.1056
```

7.5 AdamW: Decoupling Weight Decay

Weight decay (a form of L2 regularization, covered fully in Chapter 9) is meant to shrink weights toward zero independently of the gradient signal. In the original Adam formulation, weight decay was implemented by simply adding $\lambda\theta$ to the gradient before it entered the Adam update, which means it got divided by $\sqrt{\hat{v}_t}$ along with everything else, scaled adaptively and inconsistently rather than applied as a clean, uniform shrinkage.

AdamW

A variant of Adam that decouples weight decay from the gradient-based update entirely: instead of folding $\lambda\theta$ into the gradient, AdamW applies $\theta \leftarrow \theta - \eta\lambda\theta$ as a separate step, alongside the normal Adam update. This restores weight decay to its intended behavior, a uniform shrinkage toward zero, independent of each parameter's gradient history.

★ Key Insight

This sounds like a minor implementation detail, but it isn't. The original Adam-plus-L2 combination effectively under-regularizes parameters with large historical gradients (since their effective learning rate, and therefore their effective decay, is suppressed by the same $\sqrt{\hat{v}_t}$ term) and over-regularizes parameters with small ones. AdamW fixes a genuine, measurable bug, not just a stylistic preference, and has become the practical default over plain Adam for almost all modern training, particularly for transformer-based models.

7.6 Architecture Diagram

Figure 7.1 traces a single Adam step. The raw gradient feeds two exponential moving averages, the first moment m_t (momentum-style direction) and the second moment v_t (RMSProp-style per-parameter scale). Both are bias-corrected, then combined into the adaptive update. AdamW's one change is the separate decoupled decay branch (dashed) applied directly to θ .

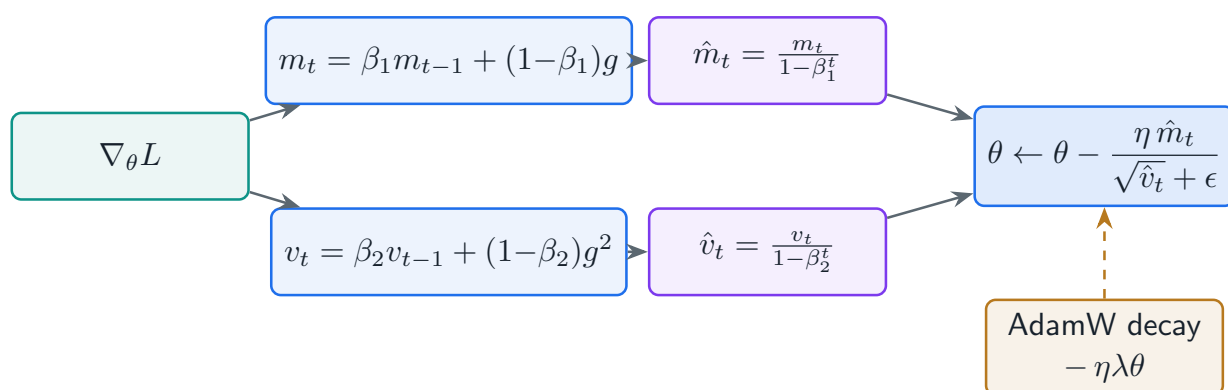


Figure 7.1: One Adam/AdamW update. The gradient drives two moving averages; their bias-corrected values give a per-parameter step. AdamW's decoupled weight decay (dashed) is applied straight to θ , never scaled by $\sqrt{\hat{v}_t}$.

Table 7.1: Optimizers compared: what each tracks, its extra memory, and where it shines.

Optimizer	Tracks	State/param	Best for
SGD	nothing	0	simple, well-understood baselines
SGD+Momentum	velocity	1×	CNNs; best final generalization (tuned)
RMSProp	sq-grad EMA	1×	RNNs, non-stationary objectives
Adam	1st + 2nd moment	2×	low-tuning default; fast prototyping
AdamW	moments + decoupled decay	2×	modern default, transformers

7.7 Practical Optimizer Selection

Situation	Reasonable Default
General default, limited tuning budget	AdamW
Very large-scale transformer training	AdamW, often with learning rate warmup (Chapter 10)
Image classification (CNNs), well-tuned pipeline	SGD with momentum often matches or beats Adam at convergence, with better final generalization
Quick prototyping, small models	Adam or AdamW, fast to get something training
Fine-tuning a pretrained model	AdamW with a small learning rate

From the Field

A real, recurring pattern: switching an optimizer from plain SGD to Adam turned a model that simply wasn't training, flat loss curve after 50 epochs, into one that converged meaningfully within ten. The underlying issue was a learning rate that happened to be reasonable for some parameters and badly miscalibrated for others; Adam's per-parameter adaptive scaling papered over that mismatch automatically. This isn't an argument that Adam is unconditionally superior (the table above has real exceptions), but it is a reasonable first thing to try when a model refuses to train at all and you suspect the optimizer rather than the architecture.

7.8 Chapter Summary

- Mini-batch gradient descent is the practical default, balancing gradient-estimate accuracy against computational cost.
- Momentum accumulates a velocity term from past gradients, accelerating through plateaus and damping oscillation in narrow ravines.
- Adagrad and RMSProp adapt the learning rate per parameter; RMSProp's exponential moving average fixes Adagrad's monotonically shrinking learning rate.

- Adam combines momentum and per-parameter adaptive scaling with bias correction, and is a strong, low-tuning-effort default.
- AdamW decouples weight decay from the adaptive gradient update, fixing a real bug in how Adam originally combined with L2 regularization, and is generally preferred over plain Adam today.

7.9 Chapter Evaluation

Ten questions on batch choice, momentum, adaptive rates, the AdamW fix, and diagnosing optimizer-related training failures. Hints first; answers in section 7.10.

Exercise 7.1

Compare batch, stochastic, and mini-batch gradient descent on the noise/cost/hardware axes.

Hint. Full batch is accurate but slow; SGD is noisy but cheap; mini-batch maps onto GPUs.

Exercise 7.2

What problem does momentum solve, and how, using the velocity equation?

Hint. Oscillation in ravines and slow plateaus. Consistent vs alternating gradients.

Exercise 7.3

Why does Adagrad eventually stop learning, and how does RMSProp fix it?

Hint. G_t only grows. EMA lets old gradients decay.

Exercise 7.4

Write the Adam update and explain the role of bias correction.

Hint. m, v start at 0; early estimates are biased toward 0. Divide by $1 - \beta^t$.

Exercise 7.5

Why does AdamW outperform Adam-with-L2? Be precise about the interaction.

Hint. L2-in-the-gradient gets divided by $\sqrt{\hat{v}_t}$; decoupled decay does not.

Exercise 7.6

When would you pick SGD+momentum over AdamW despite Adam training faster?

Hint. CNNs, flatter minima, better final generalization, if you can afford to tune.

Exercise 7.7

(Debugging) Adam training loss is great but val loss is poor; with SGD both are mediocre. What's likely happening?

Hint. Sharp vs flat minima; over-adaptation. Consider AdamW + decay or longer SGD schedule.

Exercise 7.8

(Production) A model with plain SGD won't train (flat loss, 50 epochs). What's the fastest high-value thing to try and why?

Hint. Per-parameter scale mismatch. Adam papers over it.

Exercise 7.9

Implement Adam from scratch in NumPy and use it to minimize $f(x) = x^2$.

Hint. Track m, v ; bias-correct; update with $\eta \hat{m} / (\sqrt{\hat{v}} + \epsilon)$.

Exercise 7.10

(Technical) Adam stores 2 extra tensors per parameter. What does that imply for memory budgeting a large model, and one mitigation?

Hint. Optimizer state can exceed the weights. Think 8-bit optimizers / sharding.

7.10 Answers

Answer 7.1

Batch vs stochastic vs mini-batch.

Full batch: gradient over the whole set, accurate/low-variance but expensive and can converge to sharp, poorly-generalizing minima (no helpful noise). **SGD** (one example): cheap and noisy; the noise helps escape sharp minima but updates are high-variance. **Mini-batch** (32–few thousand): the practical default, much lower variance than single-example SGD, far cheaper than full batch, and it saturates GPU parallelism. Batch size trades gradient quality against step cost and memory.

Answer 7.2

What momentum solves.

Plain SGD steps on the raw current gradient, so it oscillates across narrow ravines and crawls over flat plateaus. Momentum keeps a velocity $v_t = \beta v_{t-1} + (1 - \beta) \nabla L$ and steps along v_t . Gradients that point consistently the same way reinforce (acceleration through plateaus); gradients that alternate across a ravine partly cancel (damped oscillation). Physically: give the “ball” mass so it coasts through flat spots and small bumps.

Answer 7.3

Adagrad's decay-to-zero and RMSProp.

Adagrad divides by $\sqrt{G_t}$ where $G_t = \sum (\nabla L)^2$ accumulates *all* past squared gradients, so G_t grows monotonically and the effective LR shrinks toward zero, learning halts even if not converged. RMSProp replaces the cumulative sum with an exponential moving average $G_t = \gamma G_{t-1} + (1 - \gamma) (\nabla L)^2$, so old gradients decay and the denominator stabilizes instead of inflating forever.

Answer 7.4

Adam update + bias correction.

$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g$, $v_t = \beta_2 v_{t-1} + (1 - \beta_2)g^2$, $\hat{m}_t = m_t / (1 - \beta_1^t)$, $\hat{v}_t = v_t / (1 - \beta_2^t)$, $\theta \leftarrow \theta - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$. Bias correction exists because m, v are initialized at 0, so for small t the EMAs are biased toward zero (especially v with $\beta_2 = 0.999$). Dividing by $1 - \beta^t$ inflates the early estimates to undo that bias; the factor $\rightarrow 1$ as t grows.

Answer 7.5

Why AdamW beats Adam+L2.

Adding $\lambda\theta$ to the gradient (L2) routes the decay through Adam's adaptive scaling, so it gets divided by $\sqrt{\hat{v}_t}$: parameters with large gradient history are decayed *less*, those with small history *more*, the opposite of uniform decay. AdamW decouples it: do the normal Adam step on the loss gradient, then apply $\theta \leftarrow \theta - \eta\lambda\theta$ separately, untouched by $\sqrt{\hat{v}_t}$. Decay becomes the uniform shrinkage it was meant to be, improving generalization, markedly so for transformers.

Answer 7.6

SGD+momentum over AdamW.

For well-tuned CNN image classification, SGD+momentum with a good LR schedule often reaches slightly *better-generalizing* (flatter) minima than Adam, even though Adam reaches a given *training* loss faster in wall-clock. Adam's per-parameter scaling can steer toward sharp minima that overfit. The catch is tuning cost: SGD is far more sensitive to LR/schedule, so the win only materializes with a tuning budget; otherwise AdamW is the pragmatic default.

Answer 7.7

Debugging: Adam great train / poor val.

Classic over-adaptation, Adam found a sharp minimum that fits train but generalizes worse, while SGD's mediocrity suggests under-tuning, not overfitting. Remedies: switch to **AdamW** with proper weight decay, add regularization (dropout, augmentation, chapter 9), reduce LR / add a schedule (chapter 10), or invest in a tuned SGD+momentum run for the final model. Confirm by watching the train/val gap widen over epochs.

Answer 7.8

Production: SGD won't train at all.

Switch to **Adam/AdamW**. A flat loss under SGD usually means a single global LR is right for some parameters and off by orders of magnitude for others; Adam's per-parameter adaptive scaling auto-corrects that mismatch, often turning a non-training model into a converging one within a few epochs. It's cheap to try and high-information: if Adam trains, the problem was optimizer/LR scaling, not the architecture or data.

Answer 7.9

Adam from scratch.

`adam_min.py`

```

import numpy as np

class Adam:
    def __init__(self, lr=0.1, b1=0.9, b2=0.999, eps=1e-8):
        self.lr, self.b1, self.b2, self.eps = lr, b1, b2, eps
        self.m = self.v = 0.0; self.t = 0
    def step(self, x, g):
        self.t += 1
        self.m = self.b1*self.m + (1-self.b1)*g
        self.v = self.b2*self.v + (1-self.b2)*g*g
        mh = self.m/(1-self.b1**self.t)
        vh = self.v/(1-self.b2**self.t)
        return x - self.lr*mh/(np.sqrt(vh)+self.eps)

x, opt = 5.0, Adam()
for _ in range(200):
    x = opt.step(x, 2*x)          # grad of x^2
print(round(x, 4))               # -> ~0.0

```

The bias correction is what lets the first few steps move meaningfully despite m, v starting at 0.

Answer 7.10

Optimizer-state memory.

Adam keeps m and v per parameter, so optimizer state is $2\times$ the model's parameter memory (plus the gradients themselves), for a 7B-param model in fp32 that's tens of GB beyond the weights, often the binding constraint, not the weights. Mitigations: 8-bit optimizers (`bitsandbytes`), optimizer-state sharding (ZeRO/FSDP) across devices, mixed precision, or memory-lean optimizers (e.g. Adafactor) that don't store the full second moment.

Weight Initialization & Training Stability

Subscribe → aiengineeringinsider.substack.com/subscribe

Why the first few milliseconds of training determine a lot of what follows.

Before a single gradient step happens, a network's weights have to be set to *something*. That initial setting turns out to matter far more than its triviality suggests, a poorly initialized network can fail to train at all, regardless of how good the architecture, loss function, or optimizer choices are.

8.1 Why Naive Initialization Fails

8.1.1 All-Zero Initialization

The most obvious choice, initializing every weight to zero, fails completely, and not for a subtle reason.

! Pitfall / Warning

If every weight in a layer starts at zero, every neuron in that layer computes the exact same output on the forward pass (since they all see the same inputs and the same, zero, weights), and therefore receives the exact same gradient on the backward pass. Every neuron in the layer updates identically, forever. The layer never breaks this symmetry on its own, no matter how many training steps you run, and you end up with what is effectively a layer of width 1, no matter how wide you intended it to be.

8.1.2 Naive Random Initialization

Randomizing the weights breaks the symmetry problem, but naive choices, say, sampling from a standard normal distribution regardless of layer size, introduce a different failure: the scale of activations either explodes or vanishes as signal propagates through the network's depth.

Symmetry Breaking

The requirement that different neurons within the same layer start with different weights, so that they compute different functions of their shared input and can specialize during training. Any initialization scheme that fails to break symmetry collapses a wide layer down to the representational capacity of a much narrower one.

8.2 Signal Propagation: Keeping Variance Stable

Consider a layer's pre-activation $z = \mathbf{w}^\top \mathbf{x}$, with n inputs, weights drawn i.i.d. with variance $\text{Var}(w)$, and inputs with variance $\text{Var}(x)$. Assuming independence, the variance of z is:

$$\text{Var}(z) = n \cdot \text{Var}(w) \cdot \text{Var}(x)$$

If $\text{Var}(w)$ is too large relative to $1/n$, the variance of activations grows by a multiplicative factor at every layer, compounding exponentially with depth, exploding activations, and by extension, exploding gradients on the backward pass. If $\text{Var}(w)$ is too small, the opposite happens: activations and gradients both shrink toward zero with depth.

★ Key Insight

Good initialization schemes are, at their core, just a solution to one equation: choose $\text{Var}(w)$ so that $n \cdot \text{Var}(w) \approx 1$, keeping the variance of activations roughly constant as signal passes through each layer, neither exploding nor vanishing. Everything else in this chapter is detail on top of that one idea.

8.3 Xavier/Glorot Initialization

Proposed by Glorot and Bengio (2010), this scheme accounts for both the forward pass (preserving activation variance) and the backward pass (preserving gradient variance) simultaneously, by averaging the fan-in and fan-out of a layer:

$$\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

In practice this is usually implemented as a uniform or normal distribution scaled to match this target variance. Xavier initialization was derived under the assumption of a linear or near-linear activation (specifically, it was developed with tanh in mind, whose derivative is close to 1 near zero), and it works well for tanh and sigmoid networks.

8.4 He Initialization

ReLU breaks Xavier's underlying assumption: a ReLU unit zeroes out roughly half its inputs (everything negative), which effectively halves the variance contributed by each layer relative to a linear or tanh activation. He et al. (2015) derived a corrected variance that accounts for this:

$$\text{Var}(w) = \frac{2}{n_{\text{in}}}$$

He Initialization

A weight initialization scheme that scales variance as $2/n_{\text{in}}$, specifically derived to compensate for the variance reduction caused by ReLU's zeroing of negative inputs. It is the

standard default for networks using ReLU or its variants (Leaky ReLU, PReLU), and using Xavier initialization with a ReLU network will tend to systematically underestimate the needed weight variance, leading to vanishing activations as depth increases.

init_schemes.py

```
import numpy as np

def xavier_init(n_in, n_out, size):
    std = np.sqrt(2.0 / (n_in + n_out))
    return np.random.normal(0, std, size=size)

def he_init(n_in, size):
    std = np.sqrt(2.0 / n_in)
    return np.random.normal(0, std, size=size)

np.random.seed(0)
n_in, n_out = 256, 256

w_xavier = xavier_init(n_in, n_out, (n_out, n_in))
w_he = he_init(n_in, (n_out, n_in))

print("Xavier std:", w_xavier.std().round(5), " (target:",
      ↪ round((2/(n_in+n_out))*0.5, 5), ")")
print("He std:      ", w_he.std().round(5), " (target:", round((2/n_in)*0.5, 5), ")")
```

Output

```
Xavier std: 0.06221 (target: 0.0625 )
He std:      0.08843 (target: 0.08839 )
```

8.5 Architecture Diagram

Figure 8.1 shows the whole story on one log-scaled axis: how the standard deviation of activations evolves with depth under three schemes. Too-large variance compounds upward and overflows; too-small variance decays to zero (vanishing); a variance-matched scheme ($n \cdot \text{Var}(w) \approx 1$) holds the signal roughly flat all the way down the network.

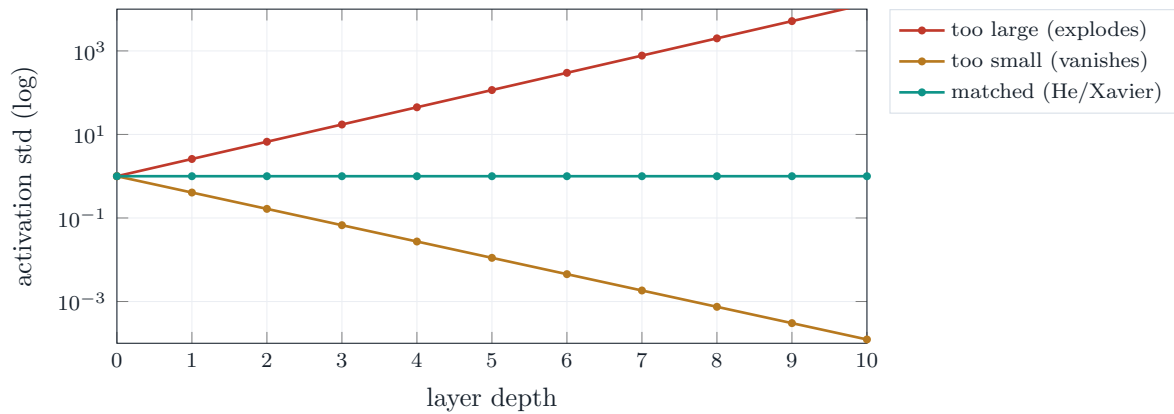


Figure 8.1: Activation standard deviation versus layer depth (log scale). Bad fixed-variance init explodes; under-scaled init vanishes; a matched scheme (Xavier for tanh, He for ReLU) keeps variance stable, the entire goal of initialization.

Table 8.1: Initialization schemes: variance target, the activation they assume, and when to use each.

Scheme	$\text{Var}(w)$	Use when
All-zeros	0	never (no symmetry breaking)
Naive $\mathcal{N}(0, 1)$	1	never (ignores fan-in)
Xavier/Glorot	$2/(n_{in} + n_{out})$	tanh, sigmoid
He (Kaiming)	$2/n_{in}$	ReLU, Leaky ReLU, PReLU
LeCun	$1/n_{in}$	SELU / self-normalizing nets

8.6 Diagnosing Bad Initialization in Practice

✓ Best Practice

Before a single real training step, run a sanity forward and backward pass on a freshly initialized network and plot the distribution (a histogram is enough) of activations and gradients at every layer. A healthy network shows roughly stable variance across layers. A network heading toward vanishing gradients shows activation and gradient magnitudes shrinking visibly with depth; one heading toward exploding gradients shows the opposite, or outright NaN or inf values within the first few layers.

activation_histogram_check.py

```
import numpy as np

def relu(z):
    return np.maximum(0, z)

def forward_with_logging(x, num_layers=10, n_in=256, init_fn=None):
    a = x
    for layer in range(num_layers):
        W = init_fn(n_in, (n_in, n_in))
        a = relu(W @ a)
        print(f"layer {layer:2d} mean={a.mean():.4f} std={a.std():.4f}")
```

```

    return a

def bad_init(n_in, size):
    return np.random.normal(0, 1.0, size=size) # ignores layer size entirely

def he_init(n_in, size):
    return np.random.normal(0, np.sqrt(2.0/n_in), size=size)

np.random.seed(0) # reproducible output below
x = np.random.normal(0, 1, size=256)
print("--- bad (fixed-variance) init ---")
forward_with_logging(x, num_layers=4, init_fn=bad_init)

```

Output

```

--- bad (fixed-variance) init ---
layer 0 mean=6.4500 std=9.0733
layer 1 mean=77.7521 std=114.0040
layer 2 mean=846.4603 std=1196.7585
layer 3 mean=8480.6657 std=12899.9582

```

! Pitfall / Warning

Notice how quickly this diverges, by layer 3, activations are already in the tens of thousands, and a real ten-layer network with this initialization would overflow to **inf** well before reaching its output. This is exactly the signature to look for when diagnosing a model that produces NaN loss within the first few training steps: check activation magnitudes layer by layer before suspecting the learning rate or the data.

8.7 Chapter Summary

- All-zero initialization fails completely: it never breaks symmetry, collapsing every neuron in a layer to identical behavior forever.
- Naive fixed-variance random initialization causes activation variance to explode or vanish with depth, since the right variance scale depends on layer width.
- Xavier/Glorot initialization scales variance as $2/(n_{\text{in}} + n_{\text{out}})$ and is well suited to tanh and sigmoid networks.
- He initialization scales variance as $2/n_{\text{in}}$, correcting for the variance reduction caused by ReLU zeroing roughly half its inputs, and is the standard default for ReLU-family networks.
- Plotting activation and gradient histograms layer by layer, before real training begins, is the fastest way to catch a bad initialization before it wastes a full training run.

8.8 Chapter Evaluation

Ten questions on symmetry breaking, variance preservation, He vs Xavier, and diagnosing init-versus-architecture failures. Hints first; answers in section 8.9.

Exercise 8.1

Why does all-zero initialization fail, and what is the precise term for the failure?

Hint. Identical weights $\Rightarrow$ identical outputs $\Rightarrow$ identical gradients.

Exercise 8.2

Derive $\text{Var}(z) = n \text{Var}(w) \text{Var}(x)$ and state the single condition good init enforces.

Hint. Sum of n independent products; want $n \text{Var}(w) \approx 1$.

Exercise 8.3

Why does He init ($2/n_{in}$) beat Xavier ($2/(n_{in} + n_{out})$) for ReLU networks?

Hint. ReLU zeros half its inputs, halving forwarded variance.

Exercise 8.4

What symptoms tell you initialization (not architecture) is the problem?

Hint. Timing: instability/NaN in the first few steps, before learning could matter.

Exercise 8.5

Why does Xavier average fan-in and fan-out instead of using just fan-in?

Hint. It balances forward (activation) and backward (gradient) variance.

Exercise 8.6

(Debugging) Loss is NaN by step 3. Walk through localizing it to initialization.

Hint. Log per-layer activation std on the first forward pass.

Exercise 8.7

(Production) Normalization layers (BatchNorm) reduce sensitivity to init. Does that make init choice irrelevant? Why not?

Hint. Norm helps mid-network, but very deep nets and the first pass still depend on init.

Exercise 8.8

Implement a layer-by-layer activation-std logger to compare bad vs He init.

Hint. Forward through L random layers; print `a.std()` each layer.

Exercise 8.9

Why is the bias usually initialized to zero even though weights cannot be?

Hint. Symmetry is broken by the weights; the bias has no symmetry problem.

Exercise 8.10

(Technical) How do residual connections change the initialization story for very deep nets?

Hint. Identity skip paths keep signal/gradient variance near 1 regardless of depth.

8.9 Answers**Answer 8.1**

Why all-zeros fails: symmetry.

With all weights zero, every neuron in a layer sees the same input and the same (zero) weights, so they produce identical outputs and, on the backward pass, receive identical gradients, and update identically forever. The layer never differentiates its neurons; a width- m layer behaves like width 1. This is a failure to **break symmetry**, which random init exists to fix.

Answer 8.2

Variance derivation and the key condition.

For $z = \sum_{i=1}^n w_i x_i$ with w_i, x_i independent, zero-mean: $\text{Var}(z) = \sum_i \text{Var}(w_i x_i) = n \text{Var}(w) \text{Var}(x)$. So each layer multiplies activation variance by $n \text{Var}(w)$. To keep variance stable across depth (neither exploding nor vanishing), enforce $n \text{Var}(w) \approx 1$, i.e. $\text{Var}(w) \approx 1/n$. Every named scheme is a particular choice of that constant.

Answer 8.3

He vs Xavier for ReLU.

Xavier assumes a near-linear activation around 0 (true for tanh). ReLU zeros all negative pre-activations, removing roughly half the variance per layer. Xavier's $2/(n_{in} + n_{out})$ doesn't account for that, so ReLU+Xavier vanishes with depth. He builds the halving into the derivation, $\text{Var}(w) = 2/n_{in}$, restoring stable variance. Rule: He for ReLU-family, Xavier for tanh/sigmoid.

Answer 8.4

Init vs architecture symptoms.

Init problems strike *immediately*: NaN/Inf loss in the first few steps, wildly different per-layer activation magnitudes on the very first forward pass, or near-zero early-layer gradients from step one. Architecture/capacity problems appear *later*: stable early training that plateaus, or good train but poor generalization. If the model can't survive the first few steps, suspect init (and LR) first.

Answer 8.5

Why Xavier averages fan-in and fan-out.

Preserving *forward* activation variance wants $\text{Var}(w) = 1/n_{in}$; preserving *backward* gradient variance wants $1/n_{out}$. You generally can't satisfy both unless $n_{in} = n_{out}$, so Glorot compromises with the harmonic-ish average $2/(n_{in} + n_{out})$, keeping both passes approximately stable.

Answer 8.6

Debugging NaN by step 3.

(1) Run one forward pass on a fresh net and log each layer's `activation.std()`. (2) If std grows multiplicatively with depth (e.g. $9 \rightarrow 100 \rightarrow 1000$), it's exploding init, the variance is too large. (3) Confirm by switching to He/Xavier and re-logging: std should flatten. (4) Only after ruling init out do you look at LR or data. Exploding activations on the *first* pass can't be a learning problem, nothing has learned yet.

Answer 8.7

Production: does BatchNorm make init irrelevant?

No. Normalization re-centers/scales activations mid-network and genuinely reduces init sensitivity, but (a) the *first* forward/backward pass and the input to the first norm layer still depend on init; (b) very deep nets can still see gradient pathologies before norm stabilizes them; (c) some architectures (certain transformers, norm-free nets) are quite init-sensitive. Good init and normalization are complementary, not substitutes.

Answer 8.8

Activation-std logger.

`init_logger.py`

```
import numpy as np

def log_stds(x, L=10, n=256, std_w=None):
    a = x
    for l in range(L):
        W = np.random.normal(0, std_w(n), size=(n, n))
        a = np.maximum(0, W @ a)           # ReLU layer
        print(f"layer {l:2d}   std={a.std():.4e}")

x = np.random.normal(0, 1, size=256)
log_stds(x, std_w=lambda n: 1.0)         # bad: explodes
log_stds(x, std_w=lambda n: np.sqrt(2.0/n)) # He: stable
```

The bad run diverges within a few layers; the He run holds std near 1.

Answer 8.9

Why zero bias is fine.

Symmetry breaking is the job of the *weights*; once neurons have distinct random weight vectors they compute distinct functions regardless of the bias. The bias carries no symmetry risk, so initializing it to 0 is standard (occasionally a small positive bias is used with ReLU to keep units active early).

Answer 8.10*Residual connections and init.*

A residual block computes $x + F(x)$. The identity path carries signal and gradient through unchanged, so even if F is initialized to contribute little (e.g. zero-init the last layer of each block), the overall variance stays near 1 and gradients reach early layers regardless of depth. This is much of why ResNets trained to 100+ layers where plain deep nets failed, residuals make the network far more forgiving of init and depth.

Regularization: Dropout, Weight Decay, BatchNorm

Subscribe → aiengineeringinsider.substack.com/subscribe

Preventing a network from memorizing instead of learning.

A network with enough parameters can, in principle, memorize its entire training set, achieving near-zero training loss while learning nothing generalizable about the underlying task. **Regularization** is the family of techniques that fight this tendency, trading some training-set performance for meaningfully better behavior on data the model hasn't seen.

9.1 Overfitting vs. Underfitting, Revisited

Bias-Variance Tradeoff

A model with high **bias** is too simple to capture the underlying pattern in the data (underfitting); it performs poorly on both training and validation data. A model with high **variance** fits the training data's noise as if it were signal (overfitting); it performs well on training data but poorly on validation data. Regularization techniques generally trade some bias for reduced variance, accepting a slightly worse fit to training data in exchange for a fit that generalizes better.

The clearest diagnostic signal: a growing gap between training loss and validation loss, with training loss still decreasing while validation loss flattens or rises, is the textbook signature of overfitting. Chapter 11 covers reading these curves in full detail; this chapter covers what to do once you've identified the problem.

9.2 Architecture Diagram

Figure 9.1 is the single most important picture in this chapter: training versus validation loss over epochs. They fall together at first, then *diverge*, train keeps dropping while validation bottoms out and turns upward. That widening gap is overfitting, and the dashed line marks the early-stopping point (best validation) where you should keep the checkpoint.

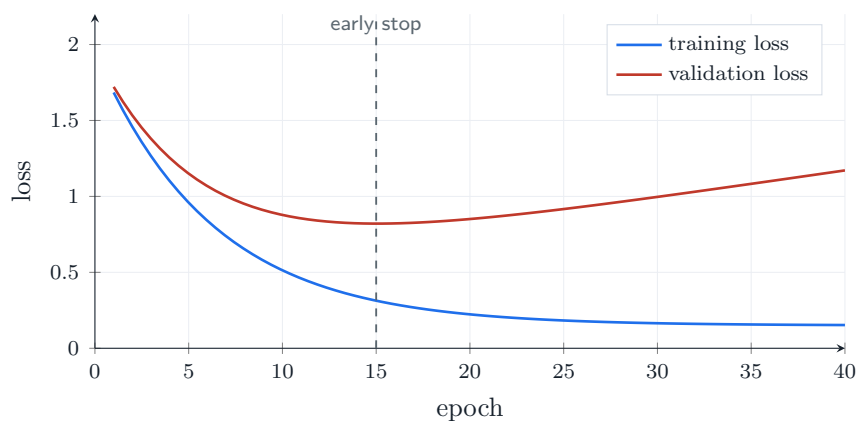


Figure 9.1: Training versus validation loss. The minimum of the validation curve (dashed) is the early-stopping point; beyond it the model memorizes training noise and the train/val gap, the overfitting signal, widens.

9.3 L1 and L2 Weight Regularization

Both techniques add a penalty term to the loss based on the magnitude of the weights, discouraging the model from relying on any single weight too heavily.

$$L_{L2} = L_{\text{original}} + \frac{\lambda}{2} \sum_i w_i^2, \quad L_{L1} = L_{\text{original}} + \lambda \sum_i |w_i|$$

L2 regularization (also called weight decay when implemented directly in the optimizer, see Chapter 7’s AdamW discussion) shrinks every weight proportionally to its current magnitude, pulling all weights toward zero smoothly but rarely making any of them exactly zero. **L1 regularization** has a fundamentally different geometric effect: it tends to push many weights to be *exactly* zero, producing sparse weight matrices.

★ Key Insight

The sparsity difference between L1 and L2 isn’t an accident of implementation, it follows from the shape of their penalty regions. L1’s penalty contour is a diamond (in 2D) with sharp corners on the axes; L2’s is a smooth circle. Gradient descent on a loss surface combined with these penalty shapes is far more likely to land exactly on one of L1’s corners (where one weight is exactly zero) than anywhere on L2’s smooth boundary. This is why L1 is often used explicitly for feature selection, while L2 is the more common default for general-purpose regularization.

9.4 Dropout

Dropout (Srivastava et al., 2014) takes a different approach entirely: during training, randomly zero out a fraction p of a layer’s activations on every forward pass, forcing the network to not rely too heavily on any single neuron.

`dropout_demo.py`

```
import numpy as np

def dropout(a, p, training=True):
    if not training:
        return a # no-op at inference time
    mask = (np.random.rand(*a.shape) > p).astype(float)
    return a * mask / (1 - p) # inverted dropout: scale during training

np.random.seed(3)
a = np.array([1.0, 2.0, 3.0, 4.0, 5.0])
print("original:      ", a)
print("dropout p=0.4:  ", dropout(a, p=0.4))
```

Output

```
original:      [1. 2. 3. 4. 5.]
dropout p=0.4: [1.66666667 3.33333333 0.          6.66666667 8.33333333]
```

! Pitfall / Warning

Notice the division by $(1 - p)$ in the code above, this is **inverted dropout**, the standard modern implementation. Without it, the expected magnitude of a layer's output would differ between training (where roughly p fraction of units are zeroed) and inference (where dropout is disabled and all units are active), requiring an awkward rescaling at test time instead. Inverted dropout rescales during training so that no adjustment is needed at inference, simply turn dropout off.

9.4.1 Why Dropout Works

★ Key Insight

Dropout can be understood as training an enormous ensemble of exponentially many “thinned” sub-networks (each one corresponding to a different random pattern of dropped units) that share weights, and then approximating an average over that ensemble at inference time by simply running the full network once. Ensembles famously generalize better than any single model, and dropout is, in effect, a computationally cheap way to approximate the benefits of ensembling without literally training and storing many separate networks.

9.5 Batch Normalization

Batch normalization (BatchNorm; Ioffe & Szegedy, 2015) normalizes a layer's pre-activations to zero mean and unit variance, computed across the current mini-batch, then applies a learned scale γ and shift β so the network can recover the original representation if that's actually optimal:

$$\hat{z} = \frac{z - \mu_{\text{batch}}}{\sqrt{\sigma_{\text{batch}}^2 + \epsilon}}, \qquad \text{BN}(z) = \gamma \hat{z} + \beta$$

BatchNorm was originally motivated by reducing **internal covariate shift**, the idea that each layer’s input distribution shifts as the parameters of earlier layers change during training, forcing later layers to continuously readapt. Subsequent research has complicated this explanation (some results suggest BatchNorm’s main benefit is actually smoothing the loss landscape, making it more amenable to larger learning rates, rather than reducing covariate shift per se), but its practical effect is consistent regardless of the exact mechanism: faster, more stable training, often with some regularizing side effect from the noise introduced by per-batch statistics.

BatchNorm Train vs. Inference Behavior

During **training**, BatchNorm normalizes using the current mini- batch’s mean and variance, and simultaneously updates a running estimate of the population mean and variance. During **inference**, it uses that stored running estimate instead of any batch statistic, since a single inference example (or a differently-sized batch) shouldn’t have its normalization depend on whatever other examples happen to be processed alongside it. Forgetting to switch a framework’s BatchNorm layer into evaluation mode at inference time (`model.eval()` in PyTorch) is a common, quietly damaging bug, the model will use batch statistics from whatever inference batch happens to be passed in, producing inconsistent and incorrect predictions.

9.6 Layer Normalization vs. Batch Normalization

Layer normalization normalizes across the *feature* dimension for a single example, rather than across the batch dimension for a single feature. This makes it independent of batch size and batch composition entirely, which matters in two common situations: very small batch sizes (where per-batch statistics are noisy and unreliable) and sequence models, including transformers, where examples in a batch may have different lengths, making a consistent per-batch statistic awkward to define.

	BatchNorm	LayerNorm
Normalizes across	The batch, per feature	The features, per example
Depends on batch size	Yes	No
Train/inference behavior differs	Yes (running stats)	No
Typical use	CNNs	Transformers, RNNs

9.7 Data Augmentation

Data augmentation regularizes by expanding the effective size and diversity of the training set, applying label-preserving transformations (rotation, cropping, color jitter for images; synonym replacement or back-translation for text) so the model sees many variations of conceptually the same example rather than memorizing exact pixel or token patterns.

9.8 Early Stopping

Early stopping monitors validation loss during training and halts once it stops improving (or starts degrading) for a set number of epochs, even if training loss is still decreasing. It's less a technique applied *to* the model and more a stopping policy applied *to training itself*, but it's one of the cheapest and most broadly applicable regularizers available: it costs nothing extra to compute and requires no architectural change.

✓ Best Practice

Always monitor validation loss (not just training loss) with a held-out set, and save the model checkpoint that achieves the best validation performance, not necessarily the checkpoint from the final epoch. Combined with early stopping, this protects against the common pattern where validation performance peaks partway through a long training run and then quietly degrades while training loss continues to fall.

Table 9.1: Regularization techniques: mechanism, what it costs, and when to reach for it.

Technique	Mechanism	Reach for it when
L2 / weight decay	smooth shrink toward 0	almost always (default)
L1	sparse, exact-zero weights	feature selection, sparsity
Dropout	random unit masking $\approx$ ensemble	wide FC layers overfit
BatchNorm	per-batch normalization	CNNs; faster, stabler training
LayerNorm	per-example normalization	transformers, RNNs, tiny batches
Data augmentation	more effective data	images/text/audio overfit
Early stopping	halt at best val loss	always (free)

9.9 Chapter Summary

- Overfitting shows up as a growing gap between training and validation loss; regularization trades some training-set fit for better generalization.
- L2 regularization shrinks weights smoothly toward zero; L1 tends to produce exactly-zero, sparse weights, useful for implicit feature selection.
- Dropout randomly zeroes activations during training, approximating an ensemble of thinned sub-networks; inverted dropout rescales during training so inference needs no adjustment.
- BatchNorm normalizes pre-activations per-batch, stabilizing and often accelerating training, but behaves differently at train versus inference time, this distinction matters and is a common bug source.
- LayerNorm normalizes per-example across features instead of across the batch, making it batch-size independent and the standard choice for transformers and RNNs.

9.10 Chapter Evaluation

Ten questions on the bias-variance tradeoff, L1 vs L2, dropout, the BatchNorm train/eval trap, and choosing normalization. Hints first; answers in section 9.11.

Exercise 9.1

Define the bias-variance tradeoff and tie each end to over/underfitting.

Hint. High bias = too simple (underfit); high variance = fits noise (overfit).

Exercise 9.2

Why does L1 produce sparse (exact-zero) weights while L2 does not?

Hint. Diamond vs circle penalty contours, corners on the axes.

Exercise 9.3

Explain inverted dropout and why the $1/(1-p)$ scaling is applied during training.

Hint. Keep expected activation magnitude equal at train and inference so eval needs no rescale.

Exercise 9.4

How is dropout an implicit ensemble, and what happens at inference?

Hint. Each mask = a thinned subnet sharing weights; full net at test $\approx$ averaging them.

Exercise 9.5

Why does BatchNorm behave differently at train vs inference? Name the resulting bug.

Hint. Batch stats vs running stats; forgetting `model.eval()`.

Exercise 9.6

When do you choose LayerNorm over BatchNorm? Give two concrete regimes.

Hint. Tiny/variable batches and sequence models.

Exercise 9.7

(Debugging) Train accuracy 99%, val accuracy 70%, gap widening. List four interventions in priority order.

Hint. More data/augmentation, weight decay, dropout, early stopping; or reduce capacity.

Exercise 9.8

(Production) Inference predictions are inconsistent batch-to-batch for the *same* input. Most likely cause?

Hint. Is the model in eval mode? Which layer depends on batch composition?

Exercise 9.9

Implement inverted dropout and an early-stopping monitor in NumPy/Python.

Hint. Mask + rescale; track best val and a patience counter.

Exercise 9.10

(Technical) Is weight decay the same as L2 regularization? When do they diverge?

Hint. Identical for plain SGD; they differ under Adam (chapter 7).

9.11 Answers

Answer 9.1

Bias-variance tradeoff.

Bias is error from a model too simple to capture the pattern, high bias underfits, poor on *both* train and val. **Variance** is sensitivity to training-set noise, high variance overfits, great on train, poor on val. Regularization deliberately adds a little bias to cut variance, accepting a worse train fit for a better-generalizing one. The diagnostic is the train/val gap (fig. 9.1).

Answer 9.2

L1 sparsity vs L2.

The penalty's geometry decides it. L1's contour $\sum |w_i| = c$ is a diamond with corners *on the axes* (where some $w_i = 0$); L2's $\sum w_i^2 = c$ is a smooth circle. The optimum sits where the loss contour first touches the penalty region, the diamond's sharp corners make exact-zero solutions far more likely, so L1 zeroes weights (feature selection). L2's smooth boundary shrinks weights toward, but rarely to, zero.

Answer 9.3

Inverted dropout and its scaling.

During training, dropout zeroes a fraction p of activations, which would lower a layer's expected output magnitude by $(1 - p)$ versus inference (where all units are on). Inverted dropout divides surviving activations by $(1 - p)$ *during training*, restoring the expected magnitude, so at inference you simply turn dropout off, no rescaling needed. The alternative (scale at test time) is more error-prone.

Answer 9.4

Dropout as ensemble.

Each training forward pass uses a different random mask, i.e. a different thinned sub-network, but all share one weight set. Over training you implicitly fit exponentially many overlapping sub-nets. At inference, dropout is off and the full (rescaled) network runs once, which approximates averaging all those sub-nets' predictions. Ensembles generalize better than single models; dropout gets much of that benefit at the cost of one network.

Answer 9.5*BatchNorm train vs inference.*

In training it normalizes with the *current batch's* mean/variance while accumulating running (EMA) population stats. At inference it uses those running stats, because a prediction shouldn't depend on which other examples share its batch (and a batch of one has zero variance). The bug: forgetting `model.eval()`, so inference uses batch stats, producing inconsistent, often worse predictions.

Answer 9.6*LayerNorm over BatchNorm.*

Choose LayerNorm when (1) **batches are tiny or variable** (per-batch stats are noisy/undefined), e.g. large-model fine-tuning with batch size 1–2; and (2) **sequence models / transformers**, where variable lengths make a consistent per-batch statistic awkward and you want normalization independent of batch composition. LayerNorm normalizes per-example across features, so it's batch-size independent and identical at train and test.

Answer 9.7*Debugging a widening gap.*

Priority order: (1) **more/augmented data**, the highest-leverage fix for overfitting; (2) **weight decay** (AdamW) tuned up; (3) **dropout** on wide FC layers; (4) **early stopping** + keep best-val checkpoint. If those don't close it, **reduce capacity**. Always change one thing at a time and watch the val curve.

Answer 9.8*Production: same input, inconsistent predictions.*

The model is in *train* mode at inference, so BatchNorm normalizes by the current inference batch's statistics, the same input gets different normalization depending on its batch-mates (and dropout is also still active, adding noise). Fix: `model.eval()` (and `torch.no_grad()`). This is the single most common BatchNorm/dropout production bug.

Answer 9.9*Inverted dropout + early stopping.*

```
reg_utils.py
```

```

import numpy as np

def inverted_dropout(a, p, training=True):
    if not training or p == 0:
        return a
    mask = (np.random.rand(*a.shape) >= p) / (1.0 - p)    # scale survivors
    return a * mask

class EarlyStopping:
    def __init__(self, patience=5):
        self.patience, self.best, self.wait, self.stop = patience, np.inf, 0,
        ↪ False
    def step(self, val_loss):
        if val_loss < self.best:
            self.best, self.wait = val_loss, 0                # improved: save ckpt
            ↪ here
        else:
            self.wait += 1
            self.stop = self.wait >= self.patience
        return self.stop

```

Save the checkpoint whenever `val_loss` improves; stop after `patience` epochs without improvement.

Answer 9.10

Weight decay vs L2.

For plain SGD they are *equivalent*: adding $\frac{\lambda}{2}\|w\|^2$ to the loss yields the gradient term λw , identical to decaying w by $\eta\lambda$ each step. They *diverge* under adaptive optimizers: L2-in-the-loss gets scaled by Adam's $1/\sqrt{\hat{v}_t}$ per parameter, whereas true (decoupled) weight decay is applied uniformly, which is exactly the AdamW fix (chapter 7). So “weight decay = L2” is true only for non-adaptive optimizers.

Learning Rate Schedules & Warmup

Subscribe → aiengineeringinsider.substack.com/subscribe

Treating the learning rate as a function of time, not a constant.

Every optimizer in Chapter 7 has a learning rate η , and so far this book has treated it as a single fixed number, chosen once and held constant for the entire training run. In practice, the single most impactful free lunch available to most training runs is to stop doing that.

10.1 Why a Fixed Learning Rate Is Rarely Optimal

★ Key Insight

Early in training, weights are far from any good solution, and a large learning rate lets the optimizer cover that distance quickly. Late in training, weights are close to a good solution, and that same large learning rate now causes the optimizer to overshoot and oscillate around the minimum rather than settling into it. No single fixed value of η is well-suited to both phases simultaneously, which is the entire motivation for scheduling it instead.

10.2 Step Decay and Exponential Decay

Step decay multiplies the learning rate by a fixed factor (commonly 0.1 or 0.5) at predetermined epochs:

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor}$$

where s is the step size (in epochs) and γ is the decay factor. This is simple and was the standard approach for years, but it requires hand-picking the schedule points, often through trial and error specific to each dataset and architecture.

Exponential decay applies continuous, smooth decay instead of discrete drops:

$$\eta_t = \eta_0 \cdot e^{-kt}$$

10.3 Cosine Annealing

Cosine annealing (Loshchilov & Hutter, 2017) decays the learning rate following a cosine curve from an initial value down to (near) zero over the course of training:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \left(\frac{t}{T} \pi \right) \right)$$

where T is the total number of training steps. The shape is deliberate: the learning rate decays slowly at first, drops more steeply through the middle of training, and flattens out again near the end, spending more time at both the high end (covering ground early) and the low end (fine-tuning late) than a linear schedule would.

`cosine_schedule.py`

```
import numpy as np

def cosine_lr(step, total_steps, lr_max=1e-3, lr_min=1e-6):
    progress = step / total_steps
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + np.cos(progress * np.pi))

for step in [0, 250, 500, 750, 1000]:
    print(f"step {step:4d} lr={cosine_lr(step, total_steps=1000):.6f}")
```

Output

```
step    0 lr=0.001000
step  250 lr=0.000854
step  500 lr=0.000501
step  750 lr=0.000147
step 1000 lr=0.000001
```

✓ Best Practice

Cosine annealing is a strong, low-effort default for most training runs where you have a known, fixed training budget (a set number of epochs or steps decided in advance). It requires tuning only $\eta_{\max}$ and the total step count, far less manual schedule design than step decay.

10.4 Warmup

Warmup does the opposite of decay, at the very beginning of training: start the learning rate near zero and ramp it up to its target value over a small number of initial steps, before any decay schedule takes over.

$$\eta_t = \eta_{\max} \cdot \frac{t}{T_{\text{warmup}}} \quad \text{for } t \leq T_{\text{warmup}}$$

! Pitfall / Warning

Without warmup, a large initial learning rate applied to a freshly, randomly initialized network (Chapter 8) can cause an unstable first few steps, since the loss landscape near a

random initialization is often poorly conditioned and the gradient estimates from early mini-batches (before the optimizer’s momentum and adaptive-rate statistics, Chapter 7, have accumulated any meaningful history) can be unreliable. This is especially pronounced in transformer architectures and any training setup using very large batch sizes, both of which are now standard enough that warmup is treated as close to mandatory rather than optional in those settings.

In practice, warmup and a decay schedule are almost always combined: a short linear warmup phase, followed by cosine annealing (or another decay schedule) for the remainder of training.

warmup_plus_cosine.py

```
import numpy as np

def warmup_cosine_lr(step, warmup_steps, total_steps, lr_max=1e-3, lr_min=1e-6):
    if step < warmup_steps:
        return lr_max * step / warmup_steps
    progress = (step - warmup_steps) / (total_steps - warmup_steps)
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + np.cos(progress * np.pi))

for step in [0, 50, 100, 300, 600, 1000]:
    lr = warmup_cosine_lr(step, warmup_steps=100, total_steps=1000)
    print(f"step {step:4d}   lr={lr:.6f}")
```

Output

```
step    0   lr=0.000000
step   50   lr=0.000500
step  100   lr=0.001000
step  300   lr=0.000883
step  600   lr=0.000414
step 1000   lr=0.000001
```

10.5 Cyclical Learning Rates and the LR Range Test

Rather than monotonically decaying, **cyclical learning rates** (CLR) oscillate the learning rate between a lower and upper bound throughout training. The intuition: periodically raising the learning rate again, after it has decayed, can help the optimizer escape sharp local minima or saddle points it might otherwise get stuck near.

A closely related and highly practical tool is the **learning rate range test**: run a short training session (a few hundred to a couple thousand steps) where the learning rate increases linearly or exponentially from a tiny value to a large one, and plot loss against learning rate. The point where loss starts decreasing fastest, and the point just before it starts diverging again, bracket a good range for $\eta_{\max}$ in a real training run.

✓ Best Practice

Run a learning rate range test before committing to a schedule, rather than guessing $\eta_{\max}$ from a default or a value that worked on a different dataset. It costs a small fraction of total training time and directly answers the single most consequential hyperparameter question for the entire training run.

10.6 Architecture Diagram

Figure 10.1 plots three schedules on one axis of step versus learning rate. The warmup-plus-cosine curve, today's default for transformers, ramps from near zero (stabilizing the fragile early steps), peaks, then anneals smoothly toward zero. Cosine-only skips the ramp; step decay drops in discrete stairs.

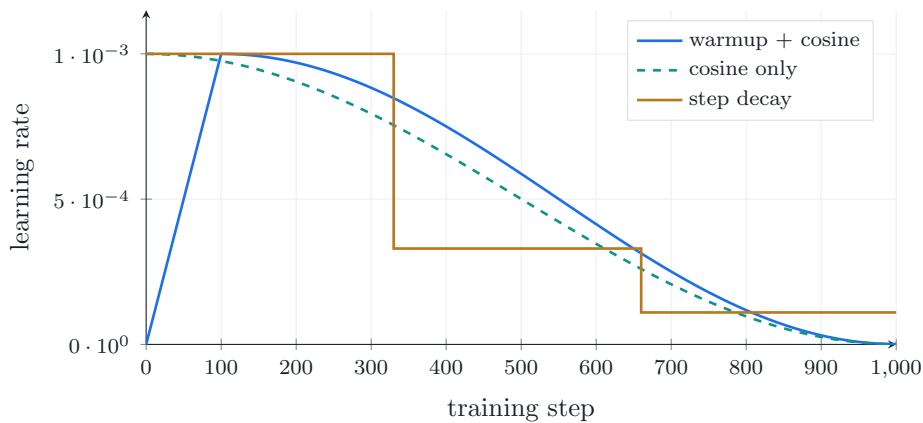


Figure 10.1: Three learning-rate schedules. Warmup+cosine ramps up to protect the early, poorly-conditioned steps, then anneals; cosine-only starts hot; step decay drops in stairs at chosen epochs.

Table 10.1: Learning-rate schedules: shape, what you must tune, and when each is the right default.

Schedule	Tunes	Use when
Constant	η	quick baselines, debugging
Step decay	drop points, γ	replicating older papers; explicit control
Exponential	η_0, k	smooth decay, simple
Cosine annealing	$\eta_{\max}, T$	fixed budget; strong default
Warmup + cosine	warmup len, $\eta_{\max}, T$	transformers, large batches
Cyclical (CLR)	bounds, period	escaping sharp minima; LR range test

10.7 Practical Scheduling Recipes

Setting	Reasonable Default
Transformer pretraining	Linear or cosine warmup (a few thousand steps), then cosine or linear decay
Fine-tuning a pretrained model	Short warmup, then constant or gently decaying small learning rate
CNN training from scratch	Step decay or cosine annealing, no warmup usually necessary at moderate batch sizes
Large-batch training (any architecture)	Warmup is close to essential, regardless of the decay schedule used afterward

10.8 Chapter Summary

- A fixed learning rate is rarely optimal across an entire training run: large early, small late, is the basic shape almost every schedule approximates.
- Step decay drops the learning rate at fixed intervals; cosine annealing decays smoothly along a cosine curve and requires less manual schedule design.
- Warmup starts the learning rate near zero and ramps it up over the first portion of training, stabilizing the unreliable early steps, especially important for transformers and large-batch training.
- Cyclical learning rates oscillate rather than monotonically decay, aiming to help escape sharp minima; the learning rate range test is a cheap, practical way to find a good $\eta_{\max}$ before committing to a full schedule.

10.9 Chapter Evaluation

Ten questions on why schedules exist, warmup, cosine annealing, the LR range test, and debugging schedule-related instability. Hints first; answers in section 10.10.

Exercise 10.1

Why is a single fixed learning rate rarely optimal across a whole run?

Hint. Large early to cover ground; small late to settle. One value can't be both.

Exercise 10.2

Why do transformers and large-batch training “need” warmup? Give two compounding reasons.

Hint. Poorly-conditioned landscape near random init; Adam's moment estimates start unreliable.

Exercise 10.3

Write the cosine annealing formula and explain why its shape is desirable.

Hint. Slow at top and bottom, steep in the middle, spends time covering ground and fine-tuning.

Exercise 10.4

Compare cosine annealing and step decay; when would you pick each?

Hint. Smooth + low-tuning vs explicit staircase control / paper replication.

Exercise 10.5

What is the LR range test and what two points does it identify?

Hint. Sweep η up, plot loss vs η ; find fastest-decrease and pre-divergence.

Exercise 10.6

Why do cyclical learning rates re-raise η instead of monotonically decaying?

Hint. Periodic warm restarts can escape sharp minima / saddle points.

Exercise 10.7

(Debugging) Loss is stable for 90 steps then explodes to NaN right around step 100. With a 100-step warmup, what do you suspect?

Hint. What happens to η exactly when warmup ends and peak LR hits?

Exercise 10.8

(Production) You fine-tune a pretrained model with the from-scratch schedule and it degrades fast. What schedule should you use instead?

Hint. Pretrained weights are already good, don't blast them with a large LR.

Exercise 10.9

Implement a combined linear-warmup-then-cosine scheduler.

Hint. Linear ramp for $t < T_w$; cosine on the remaining steps.

Exercise 10.10

(Technical) Should the LR schedule step per-epoch or per-step, and how does that interact with gradient accumulation?

Hint. Most modern schedules step per optimizer update, not per epoch.

10.10 Answers

Answer 10.1

Why fixed LR is suboptimal.

Early on, weights are far from any good solution and a large η covers distance fast. Late on, weights are near a minimum and that same large η overshoots and oscillates instead of settling. No single value serves both phases, so we schedule η high-then-low. That “large early, small late” shape is what nearly every schedule approximates.

Answer 10.2

Why warmup for transformers / large batch.

(1) Near random init the deep attention/FFN landscape is poorly conditioned, a big first step can shove the model into a bad region it can’t recover from. (2) Adam/AdamW moment estimates start at zero and are noisy for the first steps, yielding erratic effective step sizes. Warmup keeps η tiny through exactly this fragile window so the optimizer’s statistics stabilize before full-size steps begin. Large batches amplify both effects, making warmup near-mandatory.

Answer 10.3

Cosine annealing formula and shape.

$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})(1 + \cos(\frac{t}{T}\pi))$. It decays slowly at the top (more high-LR exploration), steeply through the middle, and flattens near zero at the end (more low-LR fine-tuning), spending time where it’s most useful at both ends, while needing only $\eta_{\max}$ and T , no manual drop points.

Answer 10.4

Cosine vs step decay.

Step decay multiplies η by γ at hand-chosen epochs (a staircase); cosine decays smoothly with only $\eta_{\max}$ and a total step count to set. Prefer **cosine** (often + warmup) when you know the budget and want a strong low-tuning default, most modern runs. Prefer **step decay** when you want explicit, interpretable control over *when* η changes, or to replicate an older paper’s exact schedule.

Answer 10.5

LR range test.

Run a few hundred to a couple thousand steps while increasing η from tiny to large, and plot loss versus η . It identifies (1) where loss starts decreasing fastest (a good lower/working region) and (2) the η just before loss diverges, those bracket a sensible $\eta_{\max}$. It costs a fraction of a run and answers the single most consequential hyperparameter directly.

Answer 10.6

Why cyclical LR re-raises η .

Monotonic decay can leave the optimizer stuck near a sharp minimum or saddle. Periodically raising η again (a “warm restart”) injects energy to jump out of such regions and explore a new basin, then decay refines it. Repeated, this can find flatter, better-generalizing minima and provides cheap snapshot ensembles.

Answer 10.7

Debugging NaN at end of warmup.

At step 100 warmup ends and η hits its *peak*, $\eta_{\max}$. A NaN right there means the peak LR is too high for the model/data, warmup was masking it. Lower $\eta_{\max}$ (and/or lengthen warmup), add gradient clipping, and re-check. The timing, stable through ramp, blow-up at peak, is the tell that it’s the peak LR, not the data.

Answer 10.8

Production: fine-tuning schedule.

Pretrained weights already sit in a good region, so a large from-scratch LR with a long high-LR phase destroys learned features (catastrophic forgetting). Use a *short* warmup then a *small*, gently-decaying LR (often 10–100 $\times$ smaller than pretraining), sometimes with layer-wise LR decay (lower layers slower).

Answer 10.9

Warmup-then-cosine scheduler.

warmup_cosine.py

```
import math

def warmup_cosine(step, warmup, total, lr_max=1e-3, lr_min=1e-6):
    if step < warmup:
        return lr_max * step / warmup                # linear ramp
    prog = (step - warmup) / (total - warmup)        # 0..1
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + math.cos(math.pi * prog))

# Drop-in for a PyTorch optimizer via LambdaLR(lambda s:
↪ warmup_cosine(s,...)/lr_max)
```

This is the canonical transformer schedule; wrap it in `LambdaLR` returning a *multiplier* of the base LR.

Answer 10.10

Per-step vs per-epoch; gradient accumulation.

Modern schedules step *per optimizer update* (per step), because warmup/cosine are defined over total *steps*, not epochs, this keeps the schedule consistent regardless of dataset size. With gradient accumulation over k micro-batches, one optimizer update spans k forward/backward passes, so call `scheduler.step()` once per *optimizer* step (every k micro-batches), not per micro-batch, and compute `total_steps` accordingly. Stepping per micro-batch would advance the schedule $k\times$ too fast.

Evaluation & Diagnosis

Subscribe → aiengineeringinsider.substack.com/subscribe

Knowing whether your model is actually good, and why it's failing if it isn't.

Every technique in Part II so far has been about training a network well. This chapter is about a different, equally essential question: how do you know, with confidence, whether it actually worked, and if it didn't, where to look first.

11.1 Train, Validation, and Test Splits

Train / Validation / Test Split

The **training set** is the data the model directly learns from, weights are updated to fit it. The **validation set** is held out from training and used to tune hyperparameters and make modeling decisions (when to stop training, which architecture to use, which learning rate). The **test set** is touched only once, at the very end, to report a final, unbiased estimate of real-world performance. A common split is roughly 70/15/15 or 80/10/10, though the right proportions depend heavily on total dataset size.

! Pitfall / Warning

Data leakage, information from outside the training set inadvertently influencing the model, is one of the most common and most damaging mistakes in applied machine learning, and it's often invisible until the model is deployed and performs far worse than validation metrics suggested. Common sources: normalizing features using statistics computed over the entire dataset (including validation/test) instead of training data alone; duplicate or near-duplicate examples split across train and test; time-series data split randomly instead of chronologically, letting the model implicitly “see the future”; and tuning hyperparameters repeatedly against the test set instead of the validation set, which slowly turns the test set into a second validation set and invalidates its purpose as an unbiased final check.

11.2 Accuracy, Precision, Recall, F1

For a binary classifier, every prediction falls into one of four categories relative to the ground truth: true positive (TP), true negative (TN), false positive (FP), and false negative (FN).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

Precision answers “of everything the model flagged as positive, how much was actually positive?” Recall answers “of everything that was actually positive, how much did the model catch?” These two metrics trade off against each other, a model that flags everything as positive achieves perfect recall but terrible precision, and vice versa for a model that almost never flags anything.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

F1 score is the harmonic mean of precision and recall, a single number that penalizes models which sacrifice one metric heavily to inflate the other.

Precision/Recall Tradeoff

Most classifiers don’t output a hard label directly, they output a probability or score, which is then thresholded (commonly at 0.5) to produce a final decision. Lowering that threshold flags more examples as positive, increasing recall but typically decreasing precision (more false positives slip through); raising the threshold has the opposite effect. This tradeoff is a property of the threshold choice, not just the underlying model, the same trained model can be tuned toward higher precision or higher recall after the fact, without retraining, simply by moving the decision threshold.

11.3 Confusion Matrices

A **confusion matrix** lays out the full TP/TN/FP/FN breakdown (and its multi-class generalization) as a grid, predicted class against true class, making it easy to spot exactly which classes a model confuses with each other, information a single accuracy number completely hides.

`confusion_matrix.py`

```
import numpy as np

def confusion_matrix(y_true, y_pred, num_classes):
    cm = np.zeros((num_classes, num_classes), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1
    return cm

y_true = [0, 1, 1, 0, 2, 1, 2, 0, 1, 2]
y_pred = [0, 1, 0, 0, 2, 1, 2, 0, 1, 1]

cm = confusion_matrix(y_true, y_pred, num_classes=3)
print(cm)
```

Output

```
[[3 0 0]
 [1 3 0]
 [0 1 2]]
```

Reading this matrix: row 1 (true class 1) has one example predicted as class 0, that's the specific kind of mistake the model is making, visible immediately from the matrix in a way a single 80% accuracy figure would never reveal.

11.4 ROC Curves and AUC

A **ROC curve** (Receiver Operating Characteristic) plots the true positive rate (recall) against the false positive rate across every possible classification threshold, visualizing the full precision/recall tradeoff at once rather than committing to a single threshold. **AUC** (Area Under the Curve) condenses this into a single number between 0 and 1: an AUC of 0.5 corresponds to random guessing, and 1.0 to a perfect classifier across every threshold simultaneously.

! Pitfall / Warning

ROC/AUC can be misleadingly optimistic on heavily imbalanced datasets, since the false positive rate's denominator is the (large) number of true negatives, a model can rack up a large absolute number of false positives while the false positive *rate* still looks small. For imbalanced classification (fraud detection, rare disease screening, and similar), the precision-recall curve is usually a more honest and more actionable diagnostic than ROC/AUC.

11.5 k-Fold Cross-Validation

A single train/validation split gives one estimate of generalization performance, but that estimate has variance: a different random split could give a meaningfully different number, especially on smaller datasets. **k-fold cross-validation** splits the data into k roughly equal folds, trains k separate models (each using $k - 1$ folds for training and the remaining fold for validation), and averages the resulting metric across all k runs, trading $k \times$ the computation for a lower-variance, more trustworthy performance estimate.

✓ Best Practice

Cross-validation is most valuable when your dataset is small enough that a single validation split's results feel noisy or sensitive to which exact examples ended up in which split. For large modern deep learning datasets, a single, well-constructed validation split is usually sufficient, and k -fold's extra computational cost (k full training runs instead of one) is rarely worth paying at scale.

11.6 Reading Loss Curves

Pattern	Likely Diagnosis
Both train and val loss high, both flat	Underfitting: model too simple, learning rate too low, or undertrained
Train loss low, val loss high and diverging	Overfitting: add regularization (Chapter 9), get more data, or reduce model capacity
Both decreasing steadily, gap stays small	Healthy training, likely underfit further if both are still decreasing at training's end
Loss is NaN or inf	Numerical instability, check initialization (Chapter 8), learning rate, or loss/activation mismatch (Chapter 6)
Loss flat from step one	Likely a dead network: check initialization, learning rate, or a frozen/disconnected gradient path
Loss oscillates wildly, doesn't trend down	Learning rate likely too high

11.7 Architecture Diagram

Figure 11.1 turns the loss-curve table into a decision flow, the mental routine to run whenever a model underperforms. Start at the training loss, branch on whether it's low, then on the train/val gap, and arrive at a concrete diagnosis and the chapter that fixes it.

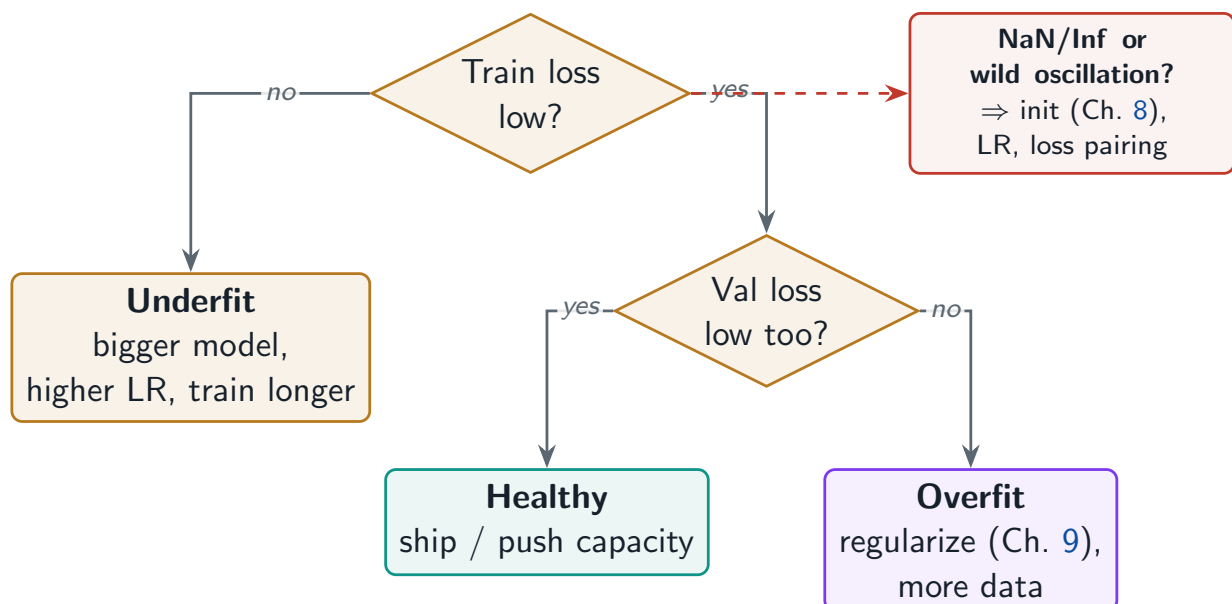


Figure 11.1: A model-diagnosis decision flow. Read top to bottom: the two questions, “is training loss low?” and “is the train/val gap small?”, localize almost every training failure to underfitting, overfitting, instability, or a healthy run.

Table 11.1: Classification metrics: what each answers and when to prefer it.

Metric	Answers	Prefer when
Accuracy	overall correct fraction	balanced classes, symmetric costs
Precision	of flagged, how many right	false positives costly
Recall	of actual, how many caught	false negatives costly
F1	harmonic mean of P/R	imbalance; both errors matter
ROC-AUC	ranking across thresholds	balanced; threshold-free comparison
PR-AUC	precision vs recall area	heavy class imbalance

11.8 Hyperparameter Tuning Workflow

Grid search exhaustively tries every combination of a predefined set of hyperparameter values, simple and complete but combinatorially expensive as the number of hyperparameters grows. **Random search** samples hyperparameter combinations randomly instead, and, somewhat counterintuitively, tends to outperform grid search at a fixed computational budget, because it explores more distinct values along each individual hyperparameter dimension rather than wasting trials on a rigid grid. **Bayesian optimization** goes a step further, using the results of previous trials to intelligently choose the next combination to try, focusing search effort on the most promising regions of hyperparameter space rather than sampling blindly.

✓ Best Practice

For most practical projects, random search over a well-chosen range is a strong default: it's simple to implement, parallelizes trivially, and captures most of grid search's benefit at a fraction of the cost. Reach for a dedicated Bayesian optimization library only once you have a training run expensive enough that squeezing out extra search efficiency is worth the added tooling complexity.

11.9 Chapter Summary

- Train/validation/test splits serve distinct purposes; data leakage between them, especially subtle leakage like dataset-wide normalization or repeated test-set tuning, silently invalidates evaluation results.
- Precision and recall trade off against each other via the decision threshold; F1 combines both into a single number, and a confusion matrix reveals exactly which classes a model confuses.
- ROC/AUC summarizes performance across all thresholds at once, but can be misleading on imbalanced data, where precision-recall curves are usually more informative.
- k-fold cross-validation trades extra compute for a lower-variance performance estimate, most valuable on smaller datasets.

- Reading the shape of training and validation loss curves together is the fastest first diagnostic step for almost any training problem.

11.10 Chapter Evaluation

Ten questions on splits and leakage, the precision/recall tradeoff, ROC vs PR, reading loss curves, and tuning. Hints first; answers in section 11.11.

Exercise 11.1

State the distinct purpose of train, validation, and test sets, and what's violated if you tune on the test set.

Hint. Test is touched once. Repeated tuning turns it into a second val set.

Exercise 11.2

Name four concrete sources of data leakage and how each inflates metrics.

Hint. Dataset-wide normalization, duplicates across splits, random split of time series, test-set tuning.

Exercise 11.3

Define precision and recall and explain how the decision threshold trades them.

Hint. Lower threshold $\rightarrow$ more positives $\rightarrow$ higher recall, lower precision.

Exercise 11.4

When is F1 a better metric than accuracy? Give the 95%-negative example.

Hint. A constant "negative" predictor scores 95% accuracy but recall 0.

Exercise 11.5

Why can ROC-AUC mislead on imbalanced data, and what's the better curve?

Hint. FPR's denominator is the huge TN count. Use PR curves.

Exercise 11.6

(Debugging) Validation loss decreases but training loss is flat. What do you check first, and why?

Hint. This is backwards for a healthy run, suspect the measurement/logging, not the model.

Exercise 11.7

Map four loss-curve shapes to diagnoses: both high/flat; train low + val rising; NaN; flat from step 0.

Hint. Underfit; overfit; instability; dead network.

Exercise 11.8

(Production) A fraud model shows val AUC 0.99 but fails in production. Name the two most likely culprits.

Hint. Leakage (a feature encodes the label) and train/serve skew or imbalance masking.

Exercise 11.9

Implement precision, recall, F1, and a confusion matrix from predictions.

Hint. Count TP/FP/FN; build a $K \times K$ grid.

Exercise 11.10

(Technical) Why does random search usually beat grid search at a fixed budget?

Hint. It explores more distinct values per dimension; few hyperparameters truly matter.

11.11 Answers

Answer 11.1

Purpose of the three splits.

Train: weights are fit to it. **Validation:** held out, used to choose hyperparameters, architecture, and stopping point. **Test:** touched exactly once at the end for an unbiased estimate. Tuning on the test set violates that unbiasedness, each decision you make against it leaks information, slowly turning it into a second validation set, so the final number no longer estimates real-world performance.

Answer 11.2

Four leakage sources.

(1) **Dataset-wide normalization**, computing mean/std over all data lets val/test statistics into training. (2) **Duplicates** split across train/test, the model “memorizes” a test example. (3) **Random split of time series**, the model sees the future. (4) **Test-set tuning**, repeated peeking inflates the reported metric. Each makes validation look better than deployment will.

Answer 11.3

Precision/recall and the threshold.

Precision = $TP/(TP + FP)$ (of flagged, how many correct); recall = $TP/(TP + FN)$ (of actual positives, how many caught). Classifiers output a score thresholded to a label. Lowering the threshold flags more positives, raising recall but admitting more false positives (lower precision); raising it does the reverse. The tradeoff is a property of the threshold, tunable post-hoc without retraining.

Answer 11.4*F1 vs accuracy.*

When classes are imbalanced. With 95% negatives, a model that always predicts “negative” gets 95% accuracy but is useless, $\text{recall} = 0$, so F1 (harmonic mean of precision and recall) collapses to 0, exposing the failure. Use F1 when both error types are costly and you want one number that punishes lopsidedness. Accuracy is fine only with balanced classes and symmetric costs.

Answer 11.5*ROC-AUC on imbalance.*

$\text{FPR} = \text{FP}/(\text{FP} + \text{TN})$ has the large true-negative count in its denominator, so even many false positives barely move FPR, ROC-AUC looks great while precision is awful. The precision-recall curve (and PR-AUC) puts FP against TP directly, so it honestly reflects performance on the rare positive class. Prefer PR for fraud, rare disease, etc.

Answer 11.6*Debugging val-down/train-flat.*

That ordering is abnormal, train loss should fall at least as fast as val since the model optimizes it directly. Suspect *measurement*, not the model: training loss logged on a stale/frozen batch, dropout/augmentation inflating the *reported* train loss, a logging bug not updating the value, or train and val losses computed differently (one includes the reg penalty). Verify the logging pipeline before theorizing about dynamics.

Answer 11.7*Loss-curve shapes.*

Both high and flat → **underfit** (too small, LR too low, undertrained). Train low, val rising → **overfit** (regularize, more data, less capacity). NaN/inf → **instability** (init, LR, loss/activation mismatch). Flat from step 0 → **dead network** (bad init, frozen/ disconnected gradient path, LR effectively zero). See fig. 11.1.

Answer 11.8*Production: AUC 0.99 but fails live.*

(1) **Leakage**: a feature available at train time encodes the label or the future (e.g. a post-event field), inflating offline AUC but absent/invalid at serve time. (2) **Train/serve skew or imbalance masking**: feature distributions or preprocessing differ in production, or AUC hides poor precision at the operating threshold on a heavily imbalanced live stream. Audit features for leakage and compare offline vs online feature distributions.

Answer 11.9*Metrics from scratch.*

```
metrics.py
```

```

import numpy as np

def binary_metrics(y_true, y_pred):
    y_true, y_pred = np.asarray(y_true), np.asarray(y_pred)
    tp = int(((y_pred == 1) & (y_true == 1)).sum())
    fp = int(((y_pred == 1) & (y_true == 0)).sum())
    fn = int(((y_pred == 0) & (y_true == 1)).sum())
    prec = tp / (tp + fp + 1e-12)
    rec = tp / (tp + fn + 1e-12)
    f1 = 2 * prec * rec / (prec + rec + 1e-12)
    return prec, rec, f1

def confusion_matrix(y_true, y_pred, k):
    cm = np.zeros((k, k), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1
    return cm

```

The confusion matrix's off-diagonal cells show *which* classes get confused, information a single accuracy number hides.

Answer 11.10

Random vs grid search.

At a fixed trial budget, grid search wastes trials on a rigid lattice, so it tests few distinct values along each axis. Random search samples each dimension independently, exploring many more distinct values per hyperparameter. Since typically only a few hyperparameters strongly affect the result, random search is far likelier to hit good values on those important axes, hence its edge at equal cost. Bayesian optimization improves further by using past trials to guide the next.

PART III

Real Frameworks

Your mini-framework proved you understand the mechanics. Now you trade it in for tools built for scale: PyTorch's imperative autograd and JAX's functional transforms. This part also covers the single highest-leverage skill in applied deep learning, debugging a model that silently isn't working.

Introduction to PyTorch

Subscribe → aiengineeringinsider.substack.com/subscribe

Moving from your toy framework to the industry-standard tool.

Chapter 4’s **Value** engine proved you understand what automatic differentiation actually does. PyTorch (Paszke et al., 2019) does the same thing, conceptually, at a scale and speed your toy engine never could: GPU acceleration, highly optimized tensor operations, and a vast ecosystem of pretrained models and tooling built on top of it.

12.1 Tensors: PyTorch’s Core Data Structure

A PyTorch **Tensor** is, at its simplest, a NumPy-like multidimensional array with two extra capabilities: it can live on a GPU, and it can track gradients automatically.

tensors_basics.py

```
import torch

x = torch.tensor([1.0, 2.0, 3.0])
y = torch.zeros(2, 3)
z = torch.randn(3, 3) # standard normal random values

print(x.shape, x.dtype)
print(y)
print(z.shape)
```

Output

```
torch.Size([3]) torch.float32
tensor([[0., 0., 0.],
        [0., 0., 0.]])
torch.Size([3, 3])
```

12.2 Autograd: Comparing to Your Chapter 4 Engine

The single most important flag on a tensor is `requires_grad`. Setting it to `True` tells PyTorch to track every operation applied to that tensor in a computational graph, exactly the graph your **Value** class built manually in Chapter 4.

autograd_basics.py

```
import torch

w = torch.tensor(0.5, requires_grad=True)
b = torch.tensor(0.1, requires_grad=True)
x = torch.tensor(2.0)

z = w * x + b
loss = z ** 2

loss.backward()  # exactly your Value.backward() from Chapter 4

print("loss:", loss.item())
print("dloss/dw:", w.grad.item())
print("dloss/db:", b.grad.item())
```

Output

```
loss: 1.2100000381469727
dloss/dw: 4.400000095367432
dloss/db: 2.200000095367432
```

★ Key Insight

`loss.backward()` is doing precisely what your hand-built `backward()` method did in Chapter 4: a topological sort of the computational graph, followed by a reverse traversal applying the chain rule at every node, accumulating gradients into each leaf tensor's `.grad` attribute. The only real differences are implementation quality: PyTorch's version is written in optimized C++, supports arbitrarily shaped tensors instead of scalars, and can run on a GPU. The algorithm is the one you already implemented yourself.

12.3 nn.Module: Building Networks Declaratively

Rather than manually wiring up weight tensors, PyTorch's `nn.Module` provides a structured way to define layers and compose them, handling parameter registration and the forward pass in an organized, reusable way.

nn_module_example.py

```
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, n_in, n_hidden, n_out):
        super().__init__()
        self.fc1 = nn.Linear(n_in, n_hidden)
        self.relu = nn.ReLU()
```

```

        self.fc2 = nn.Linear(n_hidden, n_out)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x

model = MLP(n_in=3, n_hidden=8, n_out=1)
x = torch.randn(4, 3)  # batch of 4 examples, 3 features each
out = model(x)
print(out.shape)
print(sum(p.numel() for p in model.parameters()), "total parameters")

```

Output

```

torch.Size([4, 1])
41 total parameters

```

nn.Module

PyTorch's base class for any layer or full network. Subclassing it and defining `__init__` (to declare sub-layers) and `forward` (to define the computation) is the standard pattern for building models. `nn.Module` automatically tracks every parameter declared inside it, which is what makes `model.parameters()` work without any manual bookkeeping, directly analogous to the `.parameters()` method you hand-wrote for your MLP class in Chapter 4.

12.4 Optimizers and the Training Loop

PyTorch's `torch.optim` module implements every optimizer from Chapter 7. The overall training loop shape is identical to the one you wrote by hand in Chapter 4, just using PyTorch's tensors and built-in optimizer instead of your own.

pytorch_training_loop.py

```

import torch
import torch.nn as nn

torch.manual_seed(0)  # reproducible run

model = nn.Sequential(
    nn.Linear(3, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
)
optimizer = torch.optim.AdamW(model.parameters(), lr=0.01)
loss_fn = nn.MSELoss()

```

```

X = torch.randn(16, 3)
y = torch.randn(16, 1)

for epoch in range(5):
    optimizer.zero_grad()           # clear old gradients
    pred = model(X)                 # forward pass
    loss = loss_fn(pred, y)         # compute loss
    loss.backward()                 # backward pass (autograd)
    optimizer.step()                # update parameters
    print(f"epoch {epoch}  loss {loss.item():.4f}")

```

This is the one example in the book whose output depends on a PyTorch install, so the exact figures vary slightly by version and hardware; what matters is the *shape* of the run, a loss that decreases monotonically. A representative seeded CPU run ([code/ch12_pytorch.py](#)) prints:

Output (representative seeded run)

```

epoch 0  loss 1.0664
epoch 1  loss 1.0421
epoch 2  loss 1.0183
epoch 3  loss 0.9951
epoch 4  loss 0.9724

```

! Pitfall / Warning

`optimizer.zero_grad()` at the start of every step is not optional. Recall from Chapter 4: gradients *accumulate* by addition, not overwrite, exactly the behavior you implemented in your own `Value` class. If you forget to zero gradients between steps, PyTorch will keep adding each new step's gradient on top of every previous step's, silently corrupting training in a way that often doesn't crash, it just trains badly and confusingly.

12.5 DataLoader and Dataset

Real training rarely fits an entire dataset into a single forward pass. `Dataset` defines how to access individual examples, and `DataLoader` wraps it to handle batching, shuffling, and (optionally) parallel data loading.

`dataloader_example.py`

```

import torch
from torch.utils.data import Dataset, DataLoader

class ToyDataset(Dataset):
    def __init__(self, n_samples=100):
        self.X = torch.randn(n_samples, 3)
        self.y = torch.randn(n_samples, 1)

    def __len__(self):

```

```

        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

dataset = ToyDataset()
loader = DataLoader(dataset, batch_size=16, shuffle=True)

for batch_X, batch_y in loader:
    print(batch_X.shape, batch_y.shape)
    break  # just show the first batch

```

Output

```
torch.Size([16, 3]) torch.Size([16, 1])
```

12.6 GPU Placement

Moving computation to a GPU is, in the common case, a single method call, but every tensor involved in an operation must live on the same device.

gpu_placement.py

```

import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = torch.nn.Linear(10, 1).to(device)
x = torch.randn(4, 10).to(device)

out = model(x)
print(out.device)

```

! Pitfall / Warning

A device mismatch, attempting an operation between a tensor on the GPU and a tensor still on the CPU, is one of the single most common errors for engineers new to PyTorch, and the resulting error message (`RuntimeError: Expected all tensors to be on the same device`) is usually clear once you know to look for it. The fix is almost always the same: find the tensor that wasn't moved with `.to(device)` (often a freshly created tensor inside the forward pass, or a batch of data that bypassed the standard data-loading path) and move it explicitly.

12.7 Saving, Loading, and Checkpointing

checkpointing.py

```
import torch

# Save just the learned parameters (the recommended approach)
torch.save(model.state_dict(), "model_checkpoint.pt")

# Later, or in a different process: rebuild the architecture, then load weights
model2 = torch.nn.Linear(10, 1)
model2.load_state_dict(torch.load("model_checkpoint.pt"))
model2.eval() # switch to inference mode (Chapter 9's BatchNorm note applies here
↪ too)
```

✓ Best Practice

Save `model.state_dict()` (the parameter values), not the entire model object via `torch.save(model, ...)`. The state-dict approach is more portable across PyTorch versions and codebase refactors, since loading it only requires the architecture definition to still exist and match, rather than depending on the exact class definition being unpickle-able byte-for-byte from when it was saved.

12.8 A Minimal, Correct Training Loop Template

full_template.py

```
import torch
import torch.nn as nn

model = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 1))
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
loss_fn = nn.MSELoss()

model.train() # set training mode (matters for dropout/batchnorm)
for epoch in range(num_epochs):
    for batch_X, batch_y in train_loader:
        batch_X, batch_y = batch_X.to(device), batch_y.to(device)

        optimizer.zero_grad()
        pred = model(batch_X)
        loss = loss_fn(pred, batch_y)
        loss.backward()
        optimizer.step()

    # Validation pass: no gradient tracking needed
    model.eval()
    with torch.no_grad():
```

```
val_loss = sum(loss_fn(model(vx), vy).item() for vx, vy in val_loader)
model.train()
```

★ Key Insight

This nine-line core loop, zero-grad, forward, loss, backward, step, is the single most repeated pattern in applied deep learning. Every variation you’ll encounter (gradient accumulation, mixed precision, distributed training) is an elaboration on this exact skeleton, not a replacement for it. Internalizing this loop, and specifically why each line is in the order it’s in, pays off across nearly everything else in this book from here forward.

12.9 Architecture Diagram

Figure 12.1 is the canonical PyTorch training step as a cycle. The five operations, `zero_grad`, forward, loss, `backward`, `step`, run once per mini-batch and repeat. Every advanced technique (mixed precision, gradient accumulation, distributed training) is an elaboration on this exact loop, not a replacement.

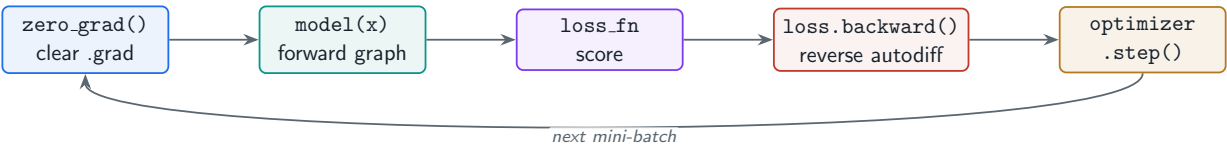


Figure 12.1: The PyTorch training step as a cycle. Order matters: clear stale gradients, compute the forward graph, score it, backpropagate, then update. Skipping `zero_grad` silently accumulates gradients across steps.

Table 12.1: Your Chapter 4 toy engine mapped to its PyTorch equivalent: same concepts, production implementation.

Chapter 4 (Value)	PyTorch
Value(data, _children)	<code>torch.tensor(..., requires_grad=True)</code>
manual <code>_backward</code> closures	autograd-recorded <code>grad_fn</code>
<code>Value.backward()</code>	<code>loss.backward()</code>
<code>p.grad = 0.0</code> loop	<code>optimizer.zero_grad()</code>
<code>p.data += -lr*p.grad</code>	<code>optimizer.step()</code>
<code>MLP.parameters()</code>	<code>nn.Module.parameters()</code>
CPU scalars	GPU tensors, fused kernels

12.10 Chapter Summary

- PyTorch tensors with `requires_grad=True` build a computational graph automatically, and `.backward()` performs exactly the reverse-mode autodiff you implemented by hand in Chapter 4.

- `nn.Module` provides a structured way to declare layers and define a forward pass, with automatic parameter tracking.
- The standard training loop, zero-grad, forward, loss, backward, step, is the same pattern across virtually all PyTorch training code.
- `Dataset` and `DataLoader` handle batching, shuffling, and data access in a consistent, reusable interface.
- Saving `model.state_dict()` rather than the full model object is the recommended, more portable checkpointing approach.

12.11 Chapter Evaluation

Ten questions on tensors and autograd, `nn.Module`, the training-loop order, devices, and the train/eval and checkpointing traps. Hints first; answers in section 12.12.

Exercise 12.1

What does `loss.backward()` actually do, and how does it map to your Chapter 4 engine?

Hint. Topological sort + reverse traversal + grad accumulation into leaf `.grad`.

Exercise 12.2

Why is `optimizer.zero_grad()` mandatory every step, and when would you deliberately skip it?

Hint. Grads accumulate by `+=`. Gradient accumulation exploits that.

Exercise 12.3

What does `nn.Module` give you over manually managing weight tensors?

Hint. Automatic parameter registration $\Rightarrow$ `.parameters()`, `.to()`, `state_dict()`.

Exercise 12.4

State the five lines of the core training loop in order and justify the ordering.

Hint. `zero_grad` $\rightarrow$ `forward` $\rightarrow$ `loss` $\rightarrow$ `backward` $\rightarrow$ `step`.

Exercise 12.5

Why use `torch.no_grad()` and `model.eval()` during validation? Are they the same thing?

Hint. One disables grad tracking (memory/speed); the other switches layer behavior.

Exercise 12.6

(Debugging) `RuntimeError: Expected all tensors to be on the same device.` Procedure to fix?

Hint. Find the tensor not moved with `.to(device)`, often created inside `forward`.

Exercise 12.7**Why save `state_dict()` instead of the whole model object?***Hint. Portability across versions/refactors; pickling the class is brittle.***Exercise 12.8****(Production) After loading a checkpoint, your model gives different outputs each call for the same input. Cause?***Hint. Did you call `model.eval()`? BatchNorm/dropout (chapter 9).***Exercise 12.9****Implement gradient accumulation to simulate a 4× larger batch.***Hint. Backward every micro-batch; step + zero every 4th; scale the loss by 1/4.***Exercise 12.10****(Technical) When should you call `.item()` / `.detach()` and why does it matter for memory?***Hint. Keeping a tensor with `grad_fn` in a Python list retains the whole graph.*

12.12 Answers

Answer 12.1*What `loss.backward()` does.*

It runs reverse-mode autodiff over the graph PyTorch recorded during the forward pass (each op on a `requires_grad` tensor became a node with a `grad_fn`). It seeds the scalar loss's gradient at 1.0, walks the graph in reverse topological order applying the chain rule, and accumulates each leaf's gradient into its `.grad`. This is exactly your Chapter 4 `Value.backward()`, just on GPU tensors of arbitrary shape via optimized C++ (table 12.1).

Answer 12.2*Why `zero_grad()`.*

Autograd *adds* new gradients to `.grad` rather than overwriting (same += as your `Value` class). In the standard loop each step should reflect only the current batch, so you must clear first, otherwise stale gradients accumulate and silently corrupt training (no crash, just worse). You *deliberately* skip it for **gradient accumulation**: backward several micro-batches, then step+zero once, simulating a larger batch.

Answer 12.3*Why `nn.Module`.*

It auto-registers every `Parameter` and sub-module declared in `__init__`, so `model.parameters()`, `model.to(device)`, `state_dict()`, and train/eval mode all work without manual bookkeeping, the production version of the `.parameters()` method

you hand-wrote in Chapter 4. It also gives a clean, composable `forward` and hooks for debugging.

Answer 12.4

Core loop ordering.

`optimizer.zero_grad()` (clear stale grads) → `pred = model(x)` (build forward graph) → `loss = loss_fn(pred, y)` (scalar to differentiate) → `loss.backward()` (fill `.grad`) → `optimizer.step()` (apply update). The order is forced: you can't backward before a loss exists, can't step before grads are computed, and must clear before accumulating fresh ones.

Answer 12.5

no_grad vs eval.

Different and both needed. `torch.no_grad()` stops the autograd graph from being built, saving memory and time during inference/validation (no backward will run). `model.eval()` switches *layer behavior*, BatchNorm uses running stats, dropout turns off. Forgetting `eval()` gives wrong outputs; forgetting `no_grad()` just wastes memory. Use both for validation.

Answer 12.6

Device-mismatch fix.

The error names a CPU/GPU clash. Procedure: (1) confirm the model is on the device (`model.to(device)`); (2) move every batch (`x.to(device)`); (3) the usual offender is a tensor *created inside forward* (a mask, positional index, constant), construct it with `device=x.device`; (4) re-run. A one-line `assert x.device == next(self.parameters()).device` pinpoints it fast.

Answer 12.7

Why state_dict().

`state_dict()` stores just the parameter tensors as a plain dict, portable across PyTorch versions and code refactors, you only need the architecture class to still exist to load it. `torch.save(model)` pickles the whole object, binding the checkpoint to the exact class definition and import paths, which breaks easily on refactor or version change.

Answer 12.8

Production: nondeterministic outputs after load.

The model is still in *train* mode. Dropout is active (random per call) and BatchNorm uses batch statistics instead of the running estimates, so the same input yields different outputs. Call `model.eval()` (and wrap inference in `torch.no_grad()`). This is the canonical post-load inference bug (chapter 9).

Answer 12.9

Gradient accumulation.

`grad_accum.py`

```
accum = 4
optimizer.zero_grad()
for i, (x, y) in enumerate(train_loader):
    x, y = x.to(device), y.to(device)
    loss = loss_fn(model(x), y) / accum      # scale so the sum matches a big
    ↪ batch
    loss.backward()                          # accumulates into .grad
    if (i + 1) % accum == 0:
        optimizer.step()
        optimizer.zero_grad()
```

Dividing by `accum` makes the accumulated gradient equal the average over the effective $4\times$ batch; step and zero only every 4th micro-batch.

Answer 12.10

.item() / .detach() and memory.

A tensor that still carries a `grad_fn` keeps the *entire* computational graph that produced it alive. If you append `loss` (not `loss.item()`) to a list for logging, you retain every step's graph and leak memory until OOM. Use `.item()` for scalars you only display, and `.detach()` when you need the tensor's values but not its gradient history (e.g. caching activations, metrics, or feeding to non-differentiable code).

Introduction to JAX

Subscribe → aiengineeringinsider.substack.com/subscribe

A functional, composable take on the same ideas, and why research labs lean on it.

PyTorch’s autograd, covered in Chapter 12, tracks operations imperatively: you run code, and a graph gets built as a side effect. JAX (Bradbury et al., 2018) takes a different philosophical stance entirely, one rooted in functional programming, and that difference in philosophy is the entire reason this chapter exists alongside, rather than instead of, the previous one.

13.1 JAX’s Philosophy: Pure Functions, No Hidden State

Pure Function

A function whose output depends *only* on its inputs, with no reliance on or modification of external state. Calling a pure function twice with the same inputs always produces exactly the same outputs, with no side effects (no modified global variables, no in-place mutation of inputs, no hidden internal counters).

JAX is built around the idea that if your numerical code is written as pure functions, an entire family of powerful transformations becomes possible: automatic differentiation, just-in-time compilation, and automatic vectorization, all composable with each other. PyTorch’s imperative, object-oriented style (mutable tensors, classes with internal state like `nn.Module`) makes some of these transformations harder to apply safely and predictably.

! Pitfall / Warning

JAX arrays are immutable by design, you cannot do `x[0] = 5` the way you would with a NumPy array or a PyTorch tensor. Instead, JAX provides an explicit, functional update syntax: `x = x.at[0].set(5)`, which returns a *new* array rather than mutating the original in place. This is a deliberate consequence of the pure-function philosophy, and it’s the single most common adjustment engineers coming from NumPy or PyTorch need to make.

13.2 grad: Automatic Differentiation as a Function Transformation

Where PyTorch attaches gradient tracking to tensors and calls `.backward()` as a method, JAX treats differentiation as a transformation applied directly to a function: `grad(f)` returns a *new function* that computes f ’s gradient.

jax_grad_basics.py

```
import jax
import jax.numpy as jnp

def f(x):
    return x ** 2 + 3 * x + 1

df_dx = jax.grad(f)

print("f(2.0) =", f(2.0))
print("f'(2.0) =", df_dx(2.0))    # analytical: 2x + 3 = 7.0
```

Output

```
f(2.0) = 11.0
f'(2.0) = 7.0
```

★ Key Insight

This is the same chain-rule machinery from Chapter 3 and the same reverse-mode autodiff implemented by hand in Chapter 4, expressed through a different interface. PyTorch asks “given this tensor and its history, compute its gradient.” JAX asks “given this function, give me back its derivative as another function.” The second framing is what unlocks `jit` and `vmap` composing cleanly with `grad`, since all three operate uniformly on functions rather than on stateful objects.

For a function with multiple inputs, `grad` differentiates with respect to the first argument by default, controllable with the `argnums` parameter, and `jax.value_and_grad` returns both the function’s output and its gradient in a single call, avoiding redundant computation when you need both (as you almost always do during training).

jax_multi_arg_grad.py

```
import jax
import jax.numpy as jnp

def loss_fn(w, b, x, y):
    pred = w * x + b
    return jnp.mean((pred - y) ** 2)

grad_fn = jax.grad(loss_fn, argnums=(0, 1))    # gradient w.r.t. w and b

w, b = 0.5, 0.1
x = jnp.array([1.0, 2.0, 3.0])
y = jnp.array([2.0, 4.0, 6.0])

dw, db = grad_fn(w, b, x, y)
print("dL/dw:", dw, " dL/db:", db)
```

Output

dL/dw: -13.6 dL/db: -5.8

13.3 jit: Just-In-Time Compilation

`jax.jit` compiles a function down to optimized, fused machine code (via XLA, the same compiler backend used internally by TensorFlow) the first time it's called with a given input shape, and reuses that compiled version on subsequent calls with matching shapes.

jax_jit_basics.py

```
import jax
import jax.numpy as jnp
import time

def slow_fn(x):
    for _ in range(50):
        x = jnp.sin(x) + jnp.cos(x)
    return x

fast_fn = jax.jit(slow_fn)

x = jnp.ones(1000)
_ = fast_fn(x)  # first call triggers compilation, excluded from timing below

start = time.time()
for _ in range(100):
    fast_fn(x).block_until_ready()
print("jit: ", time.time() - start)

start = time.time()
for _ in range(100):
    slow_fn(x)
print("no jit:", time.time() - start)
```

! Pitfall / Warning

`jit` retraces (recompiles) a function whenever it's called with a new input *shape* or a changed Python-level control-flow path, not just new values. A function with a Python `if` statement that branches on a traced array's value (rather than a static, compile-time known value) will generally fail to `jit`-compile at all, since JAX needs to trace a single, fixed computational graph ahead of time. JAX provides `jax.lax.cond` and `jax.lax.while_loop` as `jit`-compatible alternatives to Python's native control flow specifically for this reason.

13.4 vmap: Automatic Vectorization

`jax.vmap` transforms a function written for a single example into one that operates on a batch, automatically, without you manually adding a batch dimension or writing an explicit loop.

`jax_vmap_basics.py`

```
import jax
import jax.numpy as jnp

def predict_single(w, x):
    return jnp.dot(w, x)

w = jnp.array([1.0, 2.0, 3.0])
batch_x = jnp.array([[1.0, 0.0, 0.0],
                     [0.0, 1.0, 0.0],
                     [1.0, 1.0, 1.0]])

# Without vmap, you'd need an explicit loop or a manually batched dot product.
batched_predict = jax.vmap(predict_single, in_axes=(None, 0))
print(batched_predict(w, batch_x))
```

Output

```
[1.  2.  6.]
```

★ Key Insight

The `in_axes=(None, 0)` argument tells `vmap` exactly how to treat each input: `None` for `w` means “don’t batch this, use the same value for every example,” and `0` for `x` means “batch along axis 0.” This is precisely what saves you from writing manual batching logic by hand, the most common source of subtle indexing bugs when implementing a new architecture component, write the function for one example, then let `vmap` handle the rest.

13.5 PyTrees

A **PyTree** is JAX’s name for any nested Python structure of lists, tuples, and dictionaries, with arrays at the leaves. This matters because a neural network’s full set of parameters is rarely a single array, it’s typically a nested structure (one array per layer, organized into a dictionary or a custom structure), and JAX’s transformations (`grad`, `jit`, `vmap`) all operate transparently across arbitrarily nested PyTrees, not just flat arrays.

`jax_pytree_example.py`

```
import jax
import jax.numpy as jnp

params = {
```

```

    "layer1": {"w": jnp.ones((3, 4)), "b": jnp.zeros(4)},
    "layer2": {"w": jnp.ones((4, 1)), "b": jnp.zeros(1)},
}

def loss_fn(params, x, y):
    h = jnp.tanh(x @ params["layer1"]["w"] + params["layer1"]["b"])
    pred = h @ params["layer2"]["w"] + params["layer2"]["b"]
    return jnp.mean((pred - y) ** 2)

x = jnp.ones((2, 3))
y = jnp.ones((2, 1))

grads = jax.grad(loss_fn)(params, x, y)
print(grads["layer1"]["w"].shape, grads["layer2"]["b"].shape)

```

Output

```
(3, 4) (1,)
```

Notice that `jax.grad` returned a gradient with *exactly the same nested structure* as the input `params` dictionary, this structural mirroring is the PyTree abstraction at work, and it's what lets optimizer libraries like Optax apply updates to deeply nested parameter structures generically, without needing to know the specific architecture in advance.

13.6 A Minimal Training Loop in JAX

jax_training_loop.py

```

import jax
import jax.numpy as jnp

def loss_fn(params, x, y):
    pred = x @ params["w"] + params["b"]
    return jnp.mean((pred - y) ** 2)

params = {"w": jnp.zeros((3, 1)), "b": jnp.zeros(1)}
x = jnp.array(jax.random.normal(jax.random.PRNGKey(0), (16, 3)))
y = jnp.ones((16, 1))

lr = 0.1
grad_fn = jax.jit(jax.grad(loss_fn))

for step in range(5):
    grads = grad_fn(params, x, y)
    # Functional update: build a *new* params dict, nothing mutated in place
    params = jax.tree_util.tree_map(lambda p, g: p - lr * g, params, grads)
    print(f"step {step}   loss {loss_fn(params, x, y):.4f}")

```

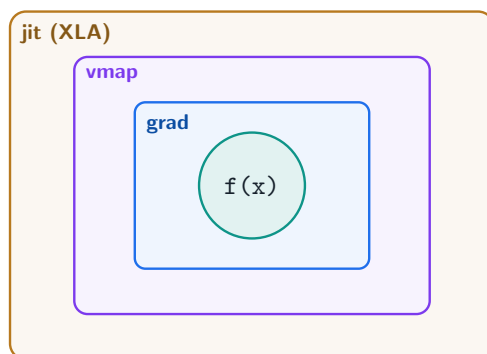
! Pitfall / Warning

Notice there is no `optimizer.zero_grad()` anywhere in this loop, and that's not an oversight. Because `jax.grad` returns a fresh gradient computed purely from the current `params` and current batch, with no hidden accumulator state to reset, there's nothing to zero. This is a direct, practical consequence of JAX's pure-function philosophy: an entire category of bugs that exists in PyTorch (forgetting `zero_grad()`, covered in Chapter 12) simply has no equivalent failure mode here.

In practice, most real JAX training code uses **Optax** for optimizers (Adam, AdamW, and the rest of Chapter 7's catalog, implemented as composable functional transformations) and **Flax** or **Equinox** for neural network layer definitions, rather than writing parameter dictionaries and update rules entirely by hand as shown above.

13.7 Architecture Diagram

Figure 13.1 captures JAX's central idea: a single *pure function* sits at the core, and the transformations `grad`, `jit`, and `vmap` are composable wrappers around it. Because each takes a function and returns a function, you can nest them in any order, e.g. `jit(vmap(grad(f)))`, a batched, compiled gradient, with no special-casing.



`jit(vmap(grad(f)))` — a batched, compiled gradient

Figure 13.1: JAX transformations compose around a pure function. `grad` returns the derivative function, `vmap` batches it, `jit` XLA-compile the result. The purity of `f` is what makes the nesting safe.

13.8 JAX vs. PyTorch: When Each Makes Sense

	PyTorch	JAX
Programming style	Imperative, object-oriented	Functional, pure functions
Ecosystem maturity	Larger, more production tooling	Smaller but strong in research
Debugging	Easier to step through interactively	Tracing/compilation can obscure intermediate values
Multi-device scaling	Well supported, more manual setup	<code>vmap</code> / <code>pmap</code> composition is a notable strength
Typical strongest fit	Production deployment, broad industry use	Research requiring heavy custom transformations, large-scale TPU work

✓ Best Practice

Don't treat this as a permanent allegiance. The underlying mathematics in this entire book, backpropagation, optimizers, architectures, is identical regardless of which framework expresses it. Many practitioners use PyTorch for the bulk of applied work and reach for JAX specifically when a project benefits from its composable transformations, custom differentiation through complex control flow, or large-scale TPU-based training.

13.9 Chapter Summary

- JAX is built on pure functions and immutable arrays, which is what enables its transformations (`grad`, `jit`, `vmap`) to compose cleanly with each other.
- `jax.grad` transforms a function into its derivative function, the same reverse-mode autodiff from Chapters 3 and 4, exposed through a functional rather than object-oriented interface.
- `jax.jit` compiles functions via XLA for speed, but requires static shapes and jit-compatible control flow.
- `jax.vmap` automatically vectorizes a function written for a single example into one that handles a batch, eliminating manual batching logic.
- PyTrees let JAX's transformations operate transparently over arbitrarily nested parameter structures, not just flat arrays.

13.10 Chapter Evaluation

Ten questions on pure functions and immutability, the three transforms, PyTrees, and the practical PyTorch/JAX tradeoff. Hints first; answers in section 13.11.

Exercise 13.1

What is a pure function, and why is purity the foundation of JAX’s design?

Hint. Output depends only on inputs, no side effects. That’s what makes transforms composable.

Exercise 13.2

JAX arrays are immutable. How do you “update” one, and why is it designed this way?

Hint. `x = x.at[i].set(v)` returns a new array.

Exercise 13.3

Contrast PyTorch’s `.backward()` with `jax.grad(f)` conceptually.

Hint. Stateful tensor graph vs a function→function transform; same chain rule.

Exercise 13.4

What does `jit` compile, when does it recompile, and what control flow breaks it?

Hint. Traces per input shape; Python `if` on a traced value fails; use `lax.cond`.

Exercise 13.5

What does `vmap` save you from, and what does `in_axes` control?

Hint. Manual batching/broadcasting; `None` = shared, `0` = batch this arg.

Exercise 13.6

What is a PyTree and why does it matter for neural-net parameters?

Hint. Nested dict/list/tuple of arrays; `grad` mirrors the structure; `Optax` updates it generically.

Exercise 13.7

Why is there no `zero_grad()` in a JAX training loop?

Hint. `grad` returns a fresh gradient from current params; no accumulator state.

Exercise 13.8

(Debugging) Your jitted function gives a `TracerError` / won’t compile. Likely cause and fix?

Hint. Branching/looping on a traced array value, or using a concrete shape from data.

Exercise 13.9

Implement a single jitted SGD step over a PyTree of params using `tree_map`.

Hint. `grad` then `tree_map(lambda p, g: p - lr*g, params, grads)`.

Exercise 13.10**(Production) When would you pick JAX over PyTorch, and when not?***Hint. Heavy custom transforms / TPU scale vs broad ecosystem + easy debugging.*

13.11 Answers

Answer 13.1*Pure functions and why they matter.*

A pure function's output depends only on its inputs, with no side effects (no global mutation, no in-place changes, no hidden counters), so it always returns the same output for the same input. Purity is what lets JAX *trace* a function into a clean computational representation and apply transformations (`grad`, `jit`, `vmap`) that compose predictably, hidden state would make the result of a transform depend on execution history.

Answer 13.2*Immutable arrays.*

You don't mutate; you produce a new array: `x = x.at[i].set(v)` returns a copy with index *i* changed. This enforces purity (no operation silently alters an array another transform relies on), which is what keeps `grad/jit/vmap` safe to compose. XLA optimizes away the apparent copy when it can, so it's not as expensive as it looks.

Answer 13.3*.backward() vs grad.*

PyTorch builds a graph as a side effect of running imperative code and `.backward()` walks that stateful, execution-tied graph. JAX's `grad(f)` *returns a new function* by tracing *f* once and applying the chain rule across the trace, no mutable state. Same reverse-mode autodiff (Chapters 3–4), different interface: object-and-method vs function-to-function.

Answer 13.4*jit compilation and control flow.*

`jit` traces the function and hands it to XLA, compiling fused machine code keyed by input *shape/dtype*; it recompiles on a new shape or a changed static control-flow path. A Python `if/for` that branches on a *traced array value* can't be baked into one static graph and fails, use `jax.lax.cond` and `jax.lax.while_loop` (or `scan`) for data-dependent control flow.

Answer 13.5*vmap and in_axes.*

`vmap` turns a single-example function into a batched one without manual broadcasting or a Python loop, you write/test the clean per-example version and get vectorization for free. `in_axes` says how to map each argument: `None` means “don't batch, share this value” (e.g. weights), an integer *k* means “batch along axis *k*” (e.g. inputs). It removes a major source

of indexing bugs.

Answer 13.6

PyTrees.

A PyTree is any nested structure of lists/tuples/dicts with arrays at the leaves, exactly how a network's parameters are organized (per-layer dicts of weights/ biases). JAX transforms operate transparently over PyTrees: `grad` returns a gradient with the *same nested shape* as the params, and `tree_map`/ `Optax` apply updates generically, without hardcoding the architecture.

Answer 13.7

No `zero_grad()` in JAX.

`jax.grad` computes a fresh gradient purely from the current params and batch, there is no mutable `.grad` accumulator to reset. The entire class of PyTorch “forgot `zero_grad()`” bugs (chapter 12) simply cannot occur, a direct payoff of the pure-function design.

Answer 13.8

Debugging a `jit` failure.

The usual cause is data-dependent Python control flow inside the jitted function, an `if/while` that inspects a traced array's *value*, or using `int(x)/x.shape[0]` from a traced array as a concrete number. Fixes: replace with `lax.cond/lax.while_loop/lax.scan`, mark truly-static args with `static_argnums`, or keep shapes fixed. Use `jax.debug.print` (not Python `print`) inside jitted code.

Answer 13.9

Jitted SGD step over a PyTree.

`jax_sgd_step.py`

```
import jax, jax.numpy as jnp

def loss_fn(params, x, y):
    pred = x @ params["w"] + params["b"]
    return jnp.mean((pred - y) ** 2)

@jax.jit
def sgd_step(params, x, y, lr=0.1):
    grads = jax.grad(loss_fn)(params, x, y)
    return jax.tree_util.tree_map(lambda p, g: p - lr * g, params, grads)
```

`tree_map` applies the update to every leaf of the params PyTree, mirroring the gradient's matching structure; `@jax.jit` compiles the whole step.

Answer 13.10

Production: JAX vs PyTorch.

Pick **JAX** when you need heavy, composable custom transformations (custom `grad`, per-sample gradients via `vmap`, `pmap`/`shard_map` for large-scale TPU training) or research that bends autodiff. Pick **PyTorch** for the broader ecosystem, pretrained models, deployment tooling, and easier interactive debugging, the default for most applied/industry work. The underlying math is identical; choose by ecosystem and transformation needs, not allegiance.

Debugging Neural Networks

Subscribe → aiengineeringinsider.substack.com/subscribe

A systematic playbook for when training goes wrong, in a real framework, not a toy one.

Every chapter so far has mentioned debugging in passing, dying ReLUs, bad initialization, mismatched loss and activation functions, forgotten `zero_grad()` calls. This chapter pulls all of it together into a single, ordered playbook, because the actual hard part of applied deep learning is rarely the math, it's figuring out which of two dozen plausible causes is actually responsible for a model that won't train.

14.1 The Debugging Mindset: Isolate Before You Tune

★ Key Insight

The single most valuable habit in debugging a neural network is resisting the urge to change multiple things at once. A model that isn't training has, somewhere, exactly one or two root causes, but a tempting first reaction is to simultaneously adjust the learning rate, swap the optimizer, add regularization, and tweak the architecture, after which, if something improves, you have no idea which change actually mattered. Change one variable, rerun a fast, small-scale test, observe the result, then decide the next move.

14.2 Shape Errors and How to Localize Them Fast

Shape mismatches are the most common error of all, and usually the easiest to fix once located.

✓ Best Practice

The fastest way to localize a shape error is not to stare at the stack trace first, it's to add `print(x.shape)` (or, better, a debugger breakpoint) immediately before the line the error points to, and walk backward one operation at a time until the shapes stop making sense. The computational graph structure from Chapter 2 is exactly what makes this tractable: every operation has a well-defined expected input shape, so the mismatch always has a precise, findable origin, it's never genuinely mysterious, just occasionally tedious to trace.

`shape_debug_example.py`

```
import torch
import torch.nn as nn
```

```
model = nn.Sequential(
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 10),
)

x = torch.randn(32, 256)  # bug: 256 features, but first layer expects 128

try:
    out = model(x)
except RuntimeError as e:
    print("Error:", e)
```

Output

Error: mat1 and mat2 shapes cannot be multiplied (32x256 and 128x64)

The error message itself usually tells you exactly what mismatched (32x256 versus 128x64, here), the remaining work is just figuring out *which* of the two shapes is the actual bug: in this example, it's the input data's feature dimension, not the layer definition, that's wrong.

14.3 NaN/Inf Losses: Common Causes and Fixes

Cause	Fix
Learning rate too high	Lower it; run a learning rate range test (Chapter 10)
Bad weight initialization	Switch to He (ReLU networks) or Xavier (tanh/sigmoid), Chapter 8
Division by zero / log of zero	Add a small epsilon (e.g. inside custom loss functions, normalization layers)
Unstable softmax/exp computation	Always subtract the max before exponentiating, Chapter 5
Exploding gradients in deep or recurrent networks	Apply gradient clipping; revisit normalization layers, Chapter 9
Mismatched loss and activation	Don't apply softmax <i>and</i> use a loss that expects raw logits, Chapter 6

gradient_clipping.py

import torch
import torch.nn as nn

model = nn.Linear(10, 1)

```
optimizer = torch.optim.AdamW(model.parameters(), lr=0.01)

x = torch.randn(8, 10)
y = torch.randn(8, 1)

loss = nn.functional.mse_loss(model(x), y)
loss.backward()

# Clip gradients to a maximum norm before stepping, common for RNNs (Chapter 16)
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

! Pitfall / Warning

Once a single weight becomes NaN, it propagates: any subsequent computation involving that weight also becomes NaN, and within a few steps the entire model is poisoned. This is why catching the *first* appearance of NaN matters far more than investigating it after a hundred more training steps have passed, by then, every weight in the network may already be contaminated, and you’ve lost the specific signal of which operation introduced it. Logging loss every step, and stopping immediately at the first NaN, is worth the minor overhead.

14.4 Loss Not Decreasing: A Decision Tree

When loss is flat from the very first step, work through these checks in order, since each one’s symptom can look identical from the outside:

1. **Is the gradient actually flowing?** Print `param.grad` for a few parameters after `backward()`. If every gradient is exactly zero, suspect a detached computational graph (an accidental `.detach()` call, or a `with torch.no_grad()` block wrapping code that should be tracked), a dead activation (all-zero ReLU outputs, Chapter 5), or a frozen parameter (`requires_grad=False` set unintentionally).
2. **Is the learning rate effectively zero?** A learning rate that’s many orders of magnitude too small produces updates too tiny to visibly move the loss within a reasonable number of steps, an easy configuration typo to miss (e.g., `1e-5` instead of the intended `1e-3`).
3. **Is the loss function actually connected to the model’s output?** A surprisingly common bug: computing loss against the wrong tensor (a stale prediction from a previous step, or a tensor that never actually flowed through the model at all).
4. **Is the data correct?** Verify labels are the expected shape, range, and encoding (Chapter 11’s data leakage discussion is adjacent but distinct, this is simply “is the data wired up correctly”), print and manually inspect a handful of raw examples and their labels before assuming the model architecture is at fault.

14.5 Loss Decreasing but Model Not Generalizing

This is a different problem entirely, and it's Chapter 9 and Chapter 11's territory more than this chapter's: training loss decreasing while validation loss stagnates or rises is the classic overfitting signature, the fix is regularization, more data, or reduced model capacity, not a training-mechanics bug. The decision tree above is for the case where the model fails to learn anything at all; this is the case where it learns something, just not something that generalizes.

14.6 Gradient Checking and Gradient Flow Visualization

Chapter 3 introduced numerical gradient checking for hand-derived gradients; the same technique is valuable for diagnosing real models, especially any model with a custom loss function or custom backward logic.

gradient_flow_check.py

```
import torch
import torch.nn as nn

def print_grad_norms(model):
    for name, param in model.named_parameters():
        if param.grad is not None:
            print(f"{name:20s} grad norm: {param.grad.norm().item():.6f}")
        else:
            print(f"{name:20s} grad is None (no gradient flowed here!)")

model = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 1))
x = torch.randn(8, 10)
y = torch.randn(8, 1)

loss = nn.functional.mse_loss(model(x), y)
loss.backward()
print_grad_norms(model)
```

Output

```
0.weight          grad norm: 0.183421
0.bias            grad norm: 0.091847
2.weight          grad norm: 0.402913
2.bias            grad norm: 0.512004
```

✓ Best Practice

Run a gradient-norm check like this periodically (not necessarily every step, but at least once early in a new training setup) and watch for two failure signatures: norms that are exactly zero or `None` (disconnected gradient, see the decision tree above) at any layer, and norms that systematically shrink toward zero at earlier layers, the vanishing gradient signature from Chapter 3, most visible in deep or recurrent networks before normalization and skip

connections are applied.

14.7 Overfitting a Single Batch as a Sanity Test

★ Key Insight

Before trusting a new architecture or training pipeline on a real dataset, try to make it overfit a single, tiny batch (a handful of examples, sometimes just one) to near-zero loss. A correctly implemented model with correctly flowing gradients should be able to memorize a tiny batch almost perfectly within a few hundred steps, if it can't, the bug is in your implementation, not in your hyperparameters or your real dataset, and it's far cheaper to find that bug against eight examples than against an eight-hour training run on the full dataset.

overfit_single_batch.py

```
import torch
import torch.nn as nn

model = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 1))
optimizer = torch.optim.AdamW(model.parameters(), lr=0.01)

# A single fixed tiny batch, reused every step
x = torch.randn(4, 10)
y = torch.randn(4, 1)

for step in range(300):
    optimizer.zero_grad()
    loss = nn.functional.mse_loss(model(x), y)
    loss.backward()
    optimizer.step()
    if step % 100 == 0:
        print(f"step {step:3d}  loss {loss.item():.6f}")
```

Output

```
step   0  loss 1.142387
step 100  loss 0.002841
step 200  loss 0.000019
```

If this loss does *not* approach zero, every previous section in this chapter applies, but you now know with certainty the bug is in the model or training code itself, not in the scale or difficulty of the real dataset.

14.8 Reproducibility: Seeding and Determinism

! Pitfall / Warning

“It worked yesterday” is frequently a seeding problem rather than a code change. Random weight initialization (Chapter 8), data shuffling, and dropout masks (Chapter 9) are all sources of run-to-run variance unless explicitly seeded. For genuine debugging (isolating whether a code change caused a result to differ, versus normal training variance), fix every relevant seed, Python’s `random`, NumPy’s, and the framework’s own (`torch.manual_seed` or JAX’s explicit `PRNGKey` system from Chapter 13), and be aware that even with seeds fixed, some GPU operations remain non-deterministic by default for performance reasons unless additional determinism flags are set.

14.9 Architecture Diagram

Figure 14.1 is the whole chapter as one playbook. Triage by the *symptom*, crash, NaN, flat loss, or a train/val gap, and each branch routes to a small, ordered set of suspects and the chapter that addresses them. The single-batch overfit test sits in the middle because it cleanly separates a model bug from a data bug.

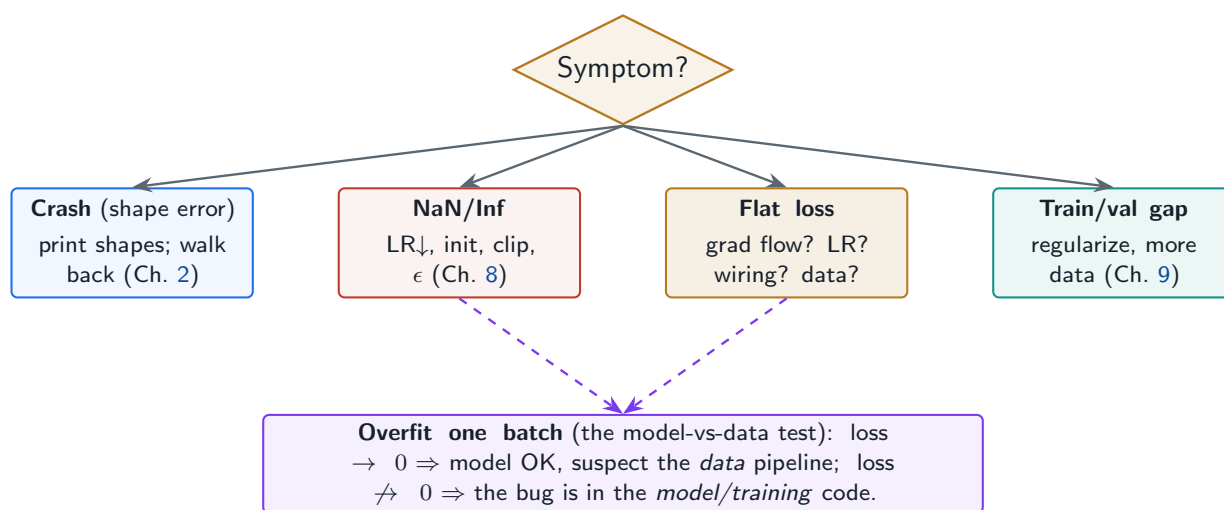


Figure 14.1: The neural-network debugging playbook. Route by symptom, then apply the ordered checks inside each box. The “overfit one batch” test is the cheapest way to decide whether the bug lives in the model/training code or the data pipeline.

14.10 Chapter Summary

- Change one variable at a time when debugging; simultaneous changes destroy your ability to attribute cause and effect.
- Shape errors are usually fast to localize: print shapes immediately before the failing operation and work backward.

- NaN/inf losses have a short, checkable list of usual suspects: learning rate, initialization, numerical stability in custom ops, and loss/activation mismatches.
- A flat loss from step one has a specific decision tree: check gradient flow first, then learning rate, then loss-output wiring, then the data itself.
- Overfitting a single tiny batch to near-zero loss is the fastest, cheapest sanity check for a new model or pipeline, before ever running it on the full dataset.

14.11 Chapter Evaluation

Ten questions on the debugging mindset, the NaN and flat-loss playbooks, the single-batch test, gradient-flow checks, and reproducibility. Hints first; answers in section 14.12.

Exercise 14.1

Why is “change one variable at a time” the core debugging discipline?

Hint. Simultaneous changes destroy cause attribution.

Exercise 14.2

Give your full process for a loss that goes NaN after 200 steps.

Hint. Find the first NaN step; then LR, init, numerical ops, clipping, in order.

Exercise 14.3

Why does catching the *first* NaN matter so much?

Hint. NaN propagates; after 100 steps every weight may be poisoned.

Exercise 14.4

Walk the decision tree for a loss that’s flat from step 1.

Hint. Gradient flow $\rightarrow$ LR $\rightarrow$ loss/output wiring $\rightarrow$ data.

Exercise 14.5

How do you distinguish a model/training bug from a data-pipeline bug?

Hint. Try to overfit one tiny batch to 0 loss.

Exercise 14.6

What two failure signatures do you look for in a per-layer gradient-norm check?

Hint. Zero/None norms (disconnected) and shrinking-toward-input (vanishing).

Exercise 14.7

(Debugging) After backward(), a layer’s param.grad is None. List three causes.

Hint. detach/no_grad, requires_grad=False, output not on the loss path.

Exercise 14.8

Localize a shape error (32x256 and 128x64) in a 3-layer net, fast.

Hint. Read the two shapes; print `x.shape` before the failing `matmul`; which is the bug?

Exercise 14.9

Implement the single-batch overfit sanity test and state the pass criterion.

Hint. Reuse one fixed tiny batch each step; loss should reach 0 in a few hundred steps.

Exercise 14.10

(Production) “It worked yesterday, same code.” What non-code cause tops your list, and how do you make runs comparable?

Hint. Seeding/nondeterminism; fix all seeds + determinism flags.

14.12 Answers

Answer 14.1

One variable at a time.

A non-training model usually has one or two root causes. If you change the LR, optimizer, regularization, and architecture at once and something improves, you can’t attribute the win, and you may have introduced a second bug that cancels the first. Change one thing, rerun a fast small test, observe, decide. Slower per step, far faster to the root cause.

Answer 14.2

NaN-after-200-steps process.

(1) Log loss *every* step and find the *first* NaN. (2) Check LR (run an LR range test, chapter 10), the most common cause. (3) Check init matches the activation (He for ReLU, Xavier for tanh/sigmoid, chapter 8). (4) Audit custom ops for `log/div/exp` hitting 0/overflow, and confirm softmax uses max-subtraction (chapter 5). (5) Add gradient clipping, especially for depth/recurrence. LR and init explain most real cases.

Answer 14.3

Why the first NaN.

NaN is contagious: any operation touching a NaN weight produces NaN, so within a few steps the whole network is poisoned and every weight looks corrupted, erasing the signal of *which* op introduced it. Catching the first occurrence (log every step, stop on first NaN) preserves that signal and makes the cause findable.

Answer 14.4

Flat-loss decision tree.

(1) **Gradient flow**: print `param.grad`; all-zero/None $\Rightarrow$ detached graph, dead ReLU, or frozen params. (2) **LR**: is it effectively zero (e.g. `1e-5` typo)? (3) **Wiring**: is the loss

computed against the model's *actual* current output, not a stale/unconnected tensor? (4) **Data**: are labels the right shape/range/encoding? Inspect a few raw examples. Check in that order, cheapest and most common first.

Answer 14.5

Model bug vs data bug.

Overfit a single tiny batch (a few examples) to near-zero loss, ignoring generalization. If it *can*, the forward pass, loss, and gradient flow work, so suspect the **data pipeline** (label encoding, misaligned features/labels in the **DataLoader**, bad preprocessing). If it *can't* memorize a trivial batch, the bug is in the **model/training code**, no real dataset should stop a correct net from memorizing a handful of points.

Answer 14.6

Gradient-norm signatures.

(1) **Zero or None** norms at some layer: a disconnected/frozen gradient path, nothing is learning there. (2) **Norms systematically shrinking** toward the *earlier* layers: the vanishing-gradient signature (chapter 3), common in deep/recurrent nets before normalization and skip connections. Both are visible at a glance from a per-layer norm printout.

Answer 14.7

grad is None: three causes.

(1) The output was **detached** from the graph (an accidental `.detach()` or a `torch.no_grad()` block wrapping tracked code). (2) The parameter has `requires_grad=False` (frozen, e.g. a mis-configured fine-tune). (3) The parameter **didn't participate** in producing the loss (a layer whose output never reaches the loss, or a wiring bug). Print `named_parameters()` grad norms to spot exactly which.

Answer 14.8

Localizing a shape error.

(32x256 and 128x64) means a 32×256 activation hit a 128×64 weight: the contraction dims (256 vs 128) disagree. Print `x.shape` right before that `matmul`. Here the input carries 256 features but the first **Linear** expects 128, the *data* feature dim is the bug, not the layer. Fix the input width (or the layer's `in_features`) so they match.

Answer 14.9

Single-batch overfit test.

```
overfit_one_batch.py
```

```
import torch, torch.nn as nn

model = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 1))
opt = torch.optim.AdamW(model.parameters(), lr=1e-2)
x, y = torch.randn(4, 10), torch.randn(4, 1)      # ONE fixed tiny batch

for step in range(300):
    opt.zero_grad()
    loss = nn.functional.mse_loss(model(x), y)
    loss.backward(); opt.step()
print(loss.item())                                # pass if ~0 (e.g. < 1e-4)
```

Pass criterion: loss drives to 0 within a few hundred steps. If it doesn't, the bug is in the model/training code, not the dataset.

Answer 14.10

Production: “worked yesterday.”

Top non-code cause: **unseeded randomness**, weight init, data shuffling, and dropout masks vary run to run, so results differ without any code change. Make runs comparable by fixing every seed (Python `random`, NumPy, `torch.manual_seed` or JAX `PRNGKey`) and enabling determinism flags (`torch.use_deterministic_algorithms(True)`, `cuda.nn.deterministic`), noting some GPU ops stay nondeterministic for speed unless forced.

PART IV

Architectures

Everything before this part is domain-agnostic. CNNs and RNNs are where the foundations get applied to real data with real structure, spatial in the case of images, sequential in the case of text, audio, and time series. They are also the conceptual ancestors of the attention-based Transformer, the architecture that now powers nearly everything, which this part builds from first principles and connects back to every system in the bonus chapters.

Convolutional Neural Networks

Subscribe → aiengineeringinsider.substack.com/subscribe

Exploiting spatial structure instead of treating images as flat vectors.

Everything in Parts I through III applies to any feedforward network, regardless of what the input represents. This chapter is the first place this book commits to a specific kind of input, images, and asks what changes when the data has spatial structure that a flat vector representation throws away.

15.1 Why Fully Connected Layers Fail on Images

A small 224×224 RGB image has $224 \times 224 \times 3 = 150,528$ input values. A single fully connected layer mapping this to even a modest 1,000 hidden units requires over 150 million weights, in that one layer alone. Beyond the sheer parameter count, a fully connected layer treats every pixel as independent and unrelated to its neighbors, discarding the fact that nearby pixels are highly correlated and that the same visual pattern (an edge, a texture, an eye) can appear anywhere in the image.

★ Key Insight

Convolutional layers encode two assumptions directly into their structure, assumptions that happen to be true for almost all natural images: **locality** (a pixel's relevant context is nearby pixels, not the whole image at once) and **translation invariance** (a useful pattern, like an edge or a corner, is useful no matter where in the image it appears, so the same detector should be applied everywhere). Baking these assumptions into the architecture, rather than hoping a fully connected layer learns them from data, is what makes CNNs dramatically more parameter-efficient on image data.

15.2 The Convolution Operation

A convolutional layer slides a small learnable filter (also called a kernel) across the input, computing a weighted sum at each position.

Kernel / Filter

A small matrix of learnable weights (commonly 3×3 or 5×5 for images) that is applied repeatedly across every spatial position of the input, producing one output value per position. Unlike a fully connected layer, where every output unit has its own independent set of weights, every position in a convolutional layer's output is produced by the exact same

shared kernel, this weight sharing is the direct architectural implementation of translation invariance.

conv2d_from_scratch.py

```
import numpy as np

def conv2d(image, kernel):
    ih, iw = image.shape
    kh, kw = kernel.shape
    oh, ow = ih - kh + 1, iw - kw + 1
    output = np.zeros((oh, ow))
    for i in range(oh):
        for j in range(ow):
            patch = image[i:i+kh, j:j+kw]
            output[i, j] = np.sum(patch * kernel)
    return output

# A simple vertical-edge-detecting kernel
image = np.array([
    [0, 0, 1, 1, 1],
    [0, 0, 1, 1, 1],
    [0, 0, 1, 1, 1],
    [0, 0, 1, 1, 1],
])
vertical_edge_kernel = np.array([
    [1, 0, -1],
    [1, 0, -1],
    [1, 0, -1],
])

print(conv2d(image, vertical_edge_kernel))
```

Output

```
[[ -3. -3.  0.]
 [ -3. -3.  0.]]
```

The output has magnitude 3 (negative here, because this kernel weights the left column +1 and the right column -1) exactly where the 3×3 window straddles the boundary between the dark (0) and light (1) regions, the first two output columns, and is 0 in the right-most column where the window sees only the uniform light region. Note the output is 2×3 , not 3×3 : a 4×5 input with a 3×3 valid convolution yields $(4-3+1) \times (5-3+1)$. A real CNN doesn't hand-design kernels like this one, it learns them via backpropagation, but this hand-built example shows precisely what a single learned kernel is mechanically computing: a local pattern detector applied identically across the whole image.

15.3 Stride, Padding, and Output Size

Stride controls how far the kernel moves between applications; stride 1 (used above) slides one pixel at a time, while a larger stride skips positions, producing a smaller output and reducing computation.

Padding adds extra pixels (usually zeros) around the input's border, controlling how much the output shrinks relative to the input, and, in the case of **“same” padding**, can keep output size equal to input size.

$$O = \left\lfloor \frac{I - K + 2P}{S} \right\rfloor + 1$$

where I is input size, K is kernel size, P is padding, S is stride, and O is output size (applied independently to height and width).

output_size_check.py

```
def conv_output_size(I, K, P, S):
    return (I - K + 2 * P) // S + 1

# 32x32 input, 3x3 kernel, no padding, stride 1
print(conv_output_size(32, 3, 0, 1))    # -> 30

# Same input/kernel, with padding=1 to preserve spatial size ("same" padding)
print(conv_output_size(32, 3, 1, 1))    # -> 32

# Larger stride for downsampling
print(conv_output_size(32, 3, 1, 2))    # -> 16
```

Output

```
30
32
16
```

15.4 Pooling: Max and Average

Pooling layers downsample spatial dimensions by summarizing small regions with a single value, most commonly **max pooling** (take the maximum value in each region) or **average pooling** (take the mean).

max_pooling.py

```
import numpy as np

def max_pool2d(x, pool_size=2, stride=2):
    h, w = x.shape
```

```

oh, ow = (h - pool_size) // stride + 1, (w - pool_size) // stride + 1
output = np.zeros((oh, ow))
for i in range(oh):
    for j in range(ow):
        region = x[i*stride:i*stride+pool_size, j*stride:j*stride+pool_size]
        output[i, j] = region.max()
return output

x = np.array([
    [1, 3, 2, 4],
    [5, 6, 1, 0],
    [2, 1, 9, 8],
    [3, 0, 7, 6],
])
print(max_pool2d(x))

```

Output

```

[[6. 4.]
 [3. 9.]]

```

✓ Best Practice

Max pooling is the more common default in practice: it tends to preserve the strongest activating feature in each region (useful, since a strong activation typically indicates the presence of whatever pattern the preceding kernel was detecting), while average pooling smooths features out, sometimes losing exactly the sharp signal you want to preserve. Several modern architectures replace pooling with strided convolutions instead (a convolution with stride > 1 accomplishes downsampling and feature extraction in a single learned operation), worth knowing as an alternative if you encounter it in an architecture you're reading.

15.5 Receptive Fields

Receptive Field

The region of the original input image that a given unit's output value is actually influenced by. A unit in the first convolutional layer (with a 3×3 kernel) has a receptive field of exactly 3×3 pixels. A unit in the second layer, built from 3×3 kernels applied to the first layer's output, has a receptive field of 5×5 in the original image, since each of the 3×3 positions it looks at already aggregated a 3×3 region of the input.

receptive_field_calc.py

```

def receptive_field(kernel_sizes, strides):
    rf = 1
    jump = 1
    for k, s in zip(kernel_sizes, strides):

```

```

    rf += (k - 1) * jump
    jump *= s
    return rf

# Three 3x3 conv layers, all stride 1
print(receptive_field([3, 3, 3], [1, 1, 1])) # -> 7

```

Output

7

★ Key Insight

This is the precise, computable answer to “why does depth matter for CNNs”: stacking convolutional layers grows the receptive field, letting later layers respond to progressively larger regions of the original image, without ever needing a single enormous kernel. A network needs sufficient depth (or sufficiently large/strided kernels) for its final layers’ receptive field to cover enough of the image to recognize large-scale objects, not just small local textures.

15.6 The Canonical CNN Pattern

Classic CNN architectures repeat a simple block: convolution, activation, pooling, with the number of filters (channels) typically increasing while spatial dimensions shrink at each stage.

simple_cnn_pytorch.py

```

import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2), # 32x32 -> 16x16
            nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2), # 16x16 -> 8x8
            nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2), # 8x8 -> 4x4
        )
        self.classifier = nn.Linear(128 * 4 * 4, num_classes)

    def forward(self, x):
        x = self.features(x)
        x = x.flatten(1) # flatten everything except the batch dim
        return self.classifier(x)

```

From the Field

The single most common bug when first writing a CNN: getting the flattened size feeding into the first fully connected layer wrong, exactly the $128 * 4 * 4$ above, computed from the spatial size after three stride-2 poolings applied to a 32×32 input. Get this wrong and you'll see precisely the shape-mismatch error from Chapter 14. The fix is always the same: work through the output-size formula from this chapter layer by layer, or simply run a dummy forward pass with `print(x.shape)` right before the flatten, and read off the correct number.

15.7 Architecture Diagram

Figure 15.1 traces the canonical CNN from chapter 15’s SimpleCNN: alternating conv+ReLU and pooling stages *shrink* the spatial size while the channel count *grows*, then a flatten feeds a fully connected classifier. The shape annotations are exactly the numbers you must get right to avoid the flatten-size bug.

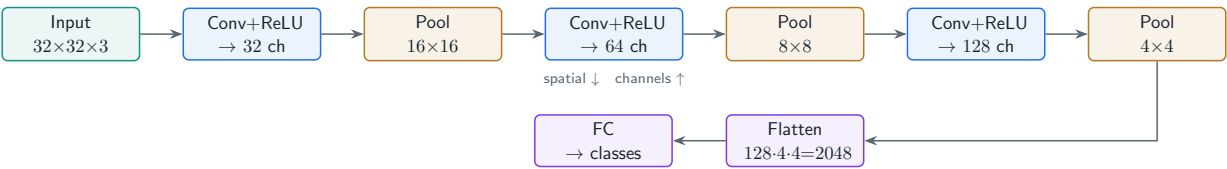


Figure 15.1: The canonical CNN pipeline for a $32 \times 32 \times 3$ input. Spatial dimensions halve at each pool while channels increase, the network trades spatial resolution for semantic depth, then flattens to a classifier.

15.8 A Brief Tour of Milestone Architectures

Architecture	Key Idea
LeNet (LeCun et al., 1998)	The original conv-pool-conv-pool pattern, applied to digit recognition
AlexNet (Krizhevsky et al., 2012)	Proved deep CNNs could dramatically outperform prior methods at ImageNet scale, the result that reignited deep learning research broadly
VGG (2014)	Demonstrated that stacking many small 3×3 kernels outperforms fewer large kernels, at lower parameter cost for the same receptive field
ResNet (He et al., 2016)	Introduced skip (residual) connections, allowing gradients to bypass layers directly, enabling stable training of networks with over 100 layers

Residual / Skip Connection

A connection that adds a layer's input directly to its output, $\text{output} = F(x) + x$, rather than relying purely on the layer's transformation $F(x)$ alone. This gives gradients a direct, untransformed path backward through the network (the identity term contributes a gradient of exactly 1, regardless of depth), directly counteracting the vanishing gradient problem from Chapter 3 and making very deep networks practically trainable.

15.9 Transfer Learning

★ Key Insight

Training a CNN from scratch on a small dataset is rarely the right default choice today. **Transfer learning**, taking a model already trained on a large dataset (commonly ImageNet) and either fine-tuning it on your specific task or using its learned features as a fixed extractor for a new, smaller classifier, almost always outperforms training from scratch when your own dataset is modest in size. The intuition: early convolutional layers learn fairly generic features (edges, textures, simple shapes) that transfer well across almost any image task, only the later layers need to specialize to your specific classes.

15.10 Chapter Summary

- CNNs exploit locality and translation invariance, assumptions true for most natural images, encoding them directly into the architecture via shared, spatially-applied kernels.
- Convolution output size depends on kernel size, padding, and stride via a single, computable formula.
- Pooling downsamples spatial dimensions, with max pooling the more common default; some modern architectures use strided convolutions instead.
- Receptive field, the input region a given unit actually responds to, grows with network depth, which is the precise reason depth matters for recognizing large-scale visual patterns.
- Transfer learning from a model pretrained on a large dataset is the practical default over training from scratch, especially with limited task-specific data.

15.11 Chapter Evaluation

Ten questions on the convolution inductive biases, output-size arithmetic, pooling, receptive fields, residuals, and transfer learning. Hints first; answers in section 15.12.

Exercise 15.1

Why do CNNs use far fewer parameters than an MLP on images, and what inductive biases does that encode?

Hint. *Weight sharing; locality + translation invariance.*

Exercise 15.2

Write the conv output-size formula and compute it for $I=32, K=3, P=1, S=2$.

Hint. $O = \lfloor (I - K + 2P)/S \rfloor + 1$.

Exercise 15.3

Compare max and average pooling; when does each lose information you care about?

Hint. *Max keeps the strongest activation; average smooths/dilutes it.*

Exercise 15.4

Define receptive field and explain why it makes depth matter for CNNs.

Hint. *Stacked 3×3 s grow the input region a unit sees, 7×7 after three layers.*

Exercise 15.5

Why does VGG prefer stacked 3×3 kernels over a single large kernel?

Hint. *Same receptive field, fewer parameters, more nonlinearity.*

Exercise 15.6

What problem do residual/skip connections solve, and how, mechanically?

Hint. $F(x) + x$: the identity path gives gradient 1 regardless of depth.

Exercise 15.7

(Debugging) Your first FC layer throws a shape error after the conv stack. Fastest fix?

Hint. *The flatten size is wrong; print $x.shape$ before flatten or use the formula.*

Exercise 15.8

(Production) Small labeled dataset, image task. Train from scratch or transfer-learn? Why?

Hint. *Early conv features are generic; fine-tune a pretrained backbone.*

Exercise 15.9

Implement a 2D convolution and the output-size check in NumPy.

Hint. *Slide the kernel; sum elementwise products; use the formula for shape.*

Exercise 15.10

(Technical) When would you replace max-pooling with a strided convolution?

Hint. *Strided conv learns the downsampling + features jointly.*

15.12 Answers

Answer 15.1

Why CNNs are parameter-efficient.

A conv layer shares one small kernel across every spatial position, instead of a unique weight per input-output pixel pair. A 3×3 kernel has 9 weights regardless of image size, while an FC layer on a 32×32 image needs millions per layer. The shared kernel encodes **translation invariance** (a detector is useful anywhere) and **locality** (context is nearby pixels), strong, correct biases for natural images that also let one detector learn from all positions' signal.

Answer 15.2

Output-size formula.

$O = \lfloor (I - K + 2P) / S \rfloor + 1$. For $I=32, K=3, P=1, S=2$: $\lfloor (32 - 3 + 2) / 2 \rfloor + 1 = \lfloor 31 / 2 \rfloor + 1 = 15 + 1 = 16$. So a 32×32 input with a padded, stride-2 3×3 conv produces a 16×16 output, the standard halving used for downsampling.

Answer 15.3

Max vs average pooling.

Max pooling keeps the strongest activation in each region, preserving the “feature present here” signal a kernel just detected; it's the common default. Average pooling means over the region, which *smooths* and can dilute a sharp, localized activation (bad when the peak is the signal). Average pooling is fine for global summaries (e.g. global average pooling before a classifier); max is better mid-network where sharp features matter.

Answer 15.4

Receptive field and depth.

The receptive field is the region of the original input that influences a unit. One 3×3 layer sees 3×3 ; stacking three stride-1 3×3 layers yields 7×7 , because each layer's view aggregates an already-aggregated region. Recognizing large objects needs a receptive field spanning much of the image, and depth grows it cheaply, far cheaper than one giant kernel. That's the mechanical reason CNN depth matters.

Answer 15.5

Stacked 3×3 vs one large kernel.

Two stacked 3×3 convs have a 5×5 receptive field; three have 7×7 , matching a single 7×7 kernel but with fewer parameters ($3 \cdot (3^2 C^2) = 27C^2$ vs $49C^2$) and *two extra nonlinearities* in between, increasing expressive power. VGG's insight: deep stacks of small kernels beat shallow large ones at equal receptive field.

Answer 15.6

Residual connections.

They compute output $= F(x) + x$, adding the block's input to its output. The identity term gives gradients a direct path backward, contributing a gradient of exactly 1 regardless of

depth, so signal reaches early layers and vanishing gradients (chapter 3) are countered. This let ResNet train 100+ layer networks where plain deep stacks failed, and makes blocks easy to initialize near identity.

Answer 15.7

Debugging the flatten-size error.

The number feeding the first FC layer is wrong. Fastest fix: run a dummy forward pass and `print(x.shape)` right before `flatten`, then set the FC `in_features` to $C \times H \times W$. Or compute it with the output-size formula stage by stage ($32 \rightarrow 16 \rightarrow 8 \rightarrow 4$, 128 channels $\Rightarrow 128 \cdot 4 \cdot 4$). Using `nn.LazyLinear` or adaptive pooling avoids the hardcoded number entirely.

Answer 15.8

Production: small dataset.

Transfer-learn. Train-from-scratch needs lots of data to relearn generic early features (edges, textures) that an ImageNet-pretrained backbone already has. Take a pretrained model, freeze (or lightly fine-tune) early layers, and train a new head on your classes; fine-tune more layers if you have more data. It almost always beats from-scratch on modest datasets and trains far faster.

Answer 15.9

Conv2d + output size in NumPy.

`conv2d.py`

```
import numpy as np

def conv2d(image, kernel, stride=1, pad=0):
    if pad: image = np.pad(image, pad)
    kh, kw = kernel.shape
    oh = (image.shape[0] - kh) // stride + 1
    ow = (image.shape[1] - kw) // stride + 1
    out = np.zeros((oh, ow))
    for i in range(oh):
        for j in range(ow):
            r, c = i * stride, j * stride
            out[i, j] = np.sum(image[r:r+kh, c:c+kw] * kernel)
    return out

out_size = lambda I, K, P, S: (I - K + 2 * P) // S + 1
```

The loop is the literal definition; production frameworks vectorize it via `im2col` or `cuDNN`, but the math is identical.

Answer 15.10

Strided conv vs max-pool.

Use a strided convolution when you want the downsampling to be *learned* jointly with feature extraction rather than a fixed max operation, it can preserve information max-pooling discards and adds parameters/flexibility. Many modern nets (all-convolutional architectures, GANs, some ResNet variants) downsample with stride-2 convs. Max-pool remains simpler, parameter-free, and a strong default when you want translation robustness without extra weights.

Recurrent Neural Networks, LSTM & GRU

Subscribe → aiengineeringinsider.substack.com/subscribe

Giving a network memory, and fixing the memory it forgets too fast.

Chapter 15 gave a network spatial structure. This final chapter gives it *temporal* structure: a way to process sequences, text, audio, time series, where the order of inputs matters and where understanding the current input often depends on everything that came before it.

16.1 Sequential Data and the Limits of Feedforward Networks

A feedforward network (everything covered through Chapter 15) has no notion of order or memory: each input is processed independently, with no mechanism to carry information from one input in a sequence to the next. You could feed a sentence’s words into a feedforward network one at a time, but it would have no way to remember word three when processing word seven.

16.2 The Vanilla RNN: Hidden State as Memory

A **recurrent neural network** introduces a **hidden state** $\mathbf{h}_t$ that’s updated at every time step and carried forward to the next, functioning as the network’s memory of everything it has processed so far in the sequence.

$$\mathbf{h}_t = \tanh(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

Hidden State

A vector, updated at every time step, that summarizes everything the network has processed in the sequence so far. It plays the same role biologically intuitive “memory” plays in a human reading a sentence, each new word is interpreted in the context of everything read previously, encoded compactly in this fixed-size vector rather than by literally storing the entire sequence history.

The same weight matrices W_{xh} and W_{hh} are reused at every time step, exactly the weight-sharing principle from Chapter 15’s convolutions, applied along time instead of space.

`vanilla_rnn_step.py`

```
import numpy as np

def rnn_step(x_t, h_prev, Wxh, Whh, bh):
    return np.tanh(Wxh @ x_t + Whh @ h_prev + bh)

# Tiny example: 3-dim input, 4-dim hidden state
np.random.seed(0)
Wxh = np.random.randn(4, 3) * 0.1
Whh = np.random.randn(4, 4) * 0.1
bh = np.zeros(4)

h = np.zeros(4) # initial hidden state
sequence = [np.random.randn(3) for _ in range(3)]

for t, x_t in enumerate(sequence):
    h = rnn_step(x_t, h, Wxh, Whh, bh)
    print(f"t={t}  h={h.round(4)}")
```

Output

```
t=0  h=[0.3314  0.539   0.1212  0.1062]
t=1  h=[-0.1214  0.1447  0.023   -0.2821]
t=2  h=[ 0.0494 -0.1638  0.0152  0.1231]
```

16.3 Backpropagation Through Time

Training an RNN requires **backpropagation through time** (BPTT): conceptually, **unroll** the recurrence into a long, explicit chain, one copy of the network per time step, each feeding its hidden state to the next, exactly the directed acyclic graph that Chapter 4’s interview question described as the resolution to “can a computational graph have a cycle.” Backpropagation then proceeds through this unrolled chain exactly as in Chapter 3, just with the same shared weights receiving gradient contributions from every time step simultaneously.

16.4 Vanishing Gradients in RNNs, Revisited

! Pitfall / Warning

RNNs suffer from vanishing (and exploding) gradients far more severely than typical feedforward networks, because the same weight matrix W_{hh} is multiplied into the gradient repeatedly, once per time step, through backpropagation through time. For a sequence of length T , this is structurally identical to Chapter 3’s depth- T vanishing gradient analysis, except here, “depth” is sequence length, and a long sequence (hundreds or thousands of time steps) produces an effective depth far greater than almost any feedforward network in this book. A vanilla RNN trained on a long sequence will, in practice, struggle to learn dependencies between events that are far apart in time, exactly the failure mode LSTM was designed to

fix.

16.5 LSTM: Forget, Input, and Output Gates

Long Short-Term Memory (LSTM; Hochreiter & Schmidhuber, 1997) networks fix the vanishing gradient problem with a deliberately engineered architecture: a separate **cell state** $\mathbf{c}_t$ that information can flow through largely unchanged across many time steps, regulated by three learned gates.

The Three LSTM Gates

The **forget gate** decides what fraction of the previous cell state to keep versus discard. The **input gate** decides what new information from the current input to write into the cell state. The **output gate** decides what part of the (updated) cell state to expose as the hidden state used for predictions and passed to the next time step. All three gates are themselves small neural network layers, typically a sigmoid applied to a linear combination of the current input and previous hidden state, producing values between 0 (block everything) and 1 (pass everything through).

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{forget gate})$$

$$\mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{input gate})$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad (\text{candidate cell update})$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (\text{new cell state})$$

$$\mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (\text{output gate})$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (\text{new hidden state})$$

★ Key Insight

The critical design choice is the cell-state update, $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$, which is additive, not a repeated matrix multiplication the way the vanilla RNN's hidden-state update was. When the forget gate is close to 1, the gradient of $\mathbf{c}_t$ with respect to $\mathbf{c}_{t-1}$ is close to 1 as well, allowing gradient to flow backward through many time steps largely unattenuated. This is structurally the same fix as ResNet's skip connections from Chapter 15, an additive path that lets gradient bypass repeated multiplicative shrinkage, applied here along the time dimension instead of network depth.

lstm_pytorch.py

```
import torch
import torch.nn as nn

lstm = nn.LSTM(input_size=10, hidden_size=20, batch_first=True)

x = torch.randn(4, 15, 10)  # batch=4, sequence_length=15, input_dim=10
output, (h_n, c_n) = lstm(x)
```

```
print("output shape:", output.shape)  # hidden state at every time step
print("final hidden:", h_n.shape)     # hidden state at the last time step
print("final cell:  ", c_n.shape)     # cell state at the last time step
```

Output

```
output shape: torch.Size([4, 15, 20])
final hidden: torch.Size([1, 4, 20])
final cell:  torch.Size([1, 4, 20])
```

16.6 Architecture Diagram

Figure 16.1 shows one LSTM cell. The key feature is the *cell-state highway* along the top: $\mathbf{c}_{t-1}$ flows to $\mathbf{c}_t$ through only a multiply (forget gate) and an add (input gate $\times$ candidate), no repeated matrix multiply or squashing. That additive path is the gradient highway that defeats vanishing gradients, the same trick as a residual connection, applied along time.

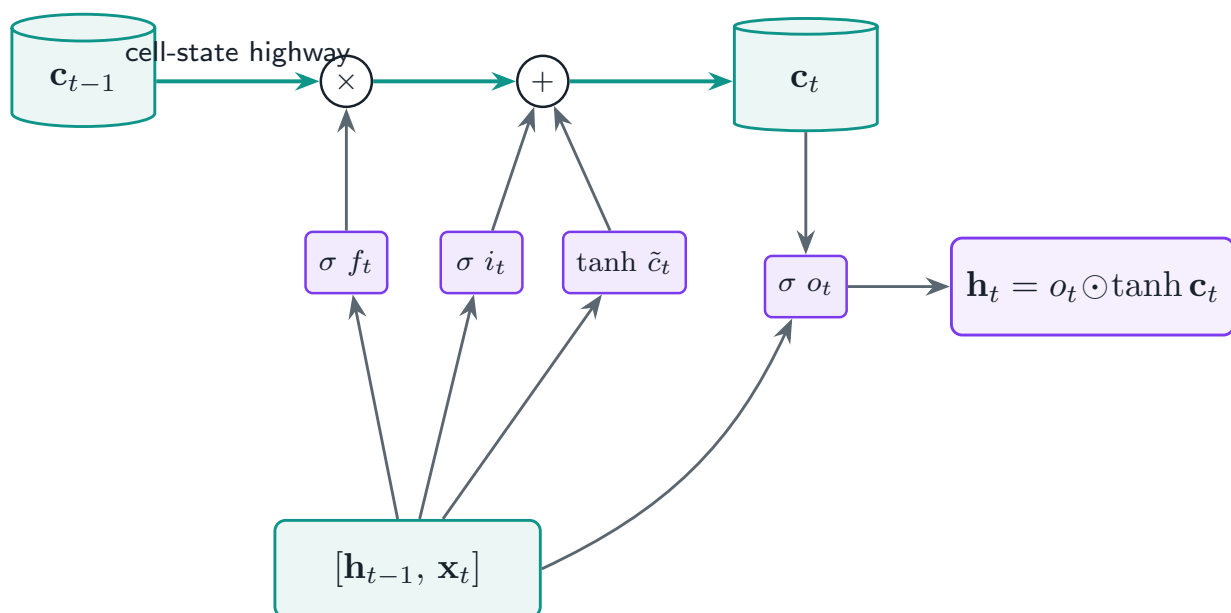


Figure 16.1: One LSTM cell. The top line is the cell-state highway: $\mathbf{c}_{t-1} \rightarrow \mathbf{c}_t$ via an elementwise forget-multiply and an input-add only. The forget (f), input (i), and output (o) gates are sigmoids of $[\mathbf{h}_{t-1}, \mathbf{x}_t]$.

16.7 GRU: A Simplified Gating Alternative

The **Gated Recurrent Unit** (GRU; Cho et al., 2014) simplifies LSTM's three gates down to two (an update gate and a reset gate) and merges the cell state and hidden state into a single vector, reducing parameter count while retaining the additive, gated update that solves the vanishing gradient problem.

$$\mathbf{z}_t = \sigma(W_z[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (\text{update gate})$$
$$\mathbf{r}_t = \sigma(W_r[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (\text{reset gate})$$
$$\tilde{\mathbf{h}}_t = \tanh(W_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t])$$
$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

16.8 LSTM vs. GRU: When Each Is Preferable

	LSTM	GRU
Gates	3 (forget, input, output)	2 (update, reset)
Separate cell state	Yes	No (merged into hidden state)
Parameter count	Higher	Lower (roughly 25% fewer)
Typical performance	Slight edge on some long-sequence, large-dataset tasks	Comparable, often matches LSTM with faster training

✓ Best Practice

Empirically, LSTM and GRU perform comparably on most tasks, neither dominates the other universally, so the practical decision usually comes down to computational budget. GRU’s smaller parameter count makes it faster to train and cheaper to deploy under latency constraints; LSTM’s extra gate and separate cell state occasionally show a measurable edge on tasks with very long-range dependencies or very large datasets, where the added capacity has more signal to be useful for. When in doubt and compute isn’t tightly constrained, starting with GRU and only moving to LSTM if a specific performance gap shows up empirically is a reasonable, low-cost default.

16.9 Where RNNs Still Matter Today

★ Key Insight

Transformer architectures (attention-based, with no recurrence at all) have displaced RNNs as the default for most large-scale language and sequence modeling tasks, a complete treatment is intentionally left for a dedicated volume in this series. But RNNs, and LSTM in particular, remain genuinely relevant: in resource-constrained or streaming settings where processing a sequence strictly one step at a time (without needing the entire sequence available upfront, the way attention does) is a hard requirement, in smaller-scale time-series and signal-processing applications where a transformer’s data and compute appetite isn’t justified, and as a foundational concept, the gating mechanisms introduced here (additive, gated updates that preserve gradient flow) echo directly into the residual and gating ideas used throughout modern architectures, including transformers themselves.

16.10 Chapter Summary

- RNNs introduce a hidden state, updated and carried forward at every time step, giving the network a form of memory across a sequence.
- Backpropagation through time unrolls the recurrence into an explicit chain and applies standard backpropagation across it, with shared weights accumulating gradient from every time step.
- Vanilla RNNs suffer severe vanishing gradients on long sequences, the same Chapter 3 mechanism, but with sequence length playing the role of depth.
- LSTM fixes this with an additive cell-state update regulated by forget, input, and output gates, letting gradient flow across many time steps largely unattenuated, structurally similar to ResNet's skip connections.
- GRU simplifies LSTM to two gates and a merged cell/hidden state, trading some capacity for fewer parameters and faster training, with comparable empirical performance on most tasks.

16.11 Chapter Evaluation

Ten questions on hidden state, BPTT, the RNN vanishing-gradient mechanism, LSTM/GRU gating, and where recurrence still fits. Hints first; answers in section 16.12.

Exercise 16.1

Why can't a feedforward network model sequences, and what does the hidden state add?

Hint. No memory/order. $\mathbf{h}_t$ carries a summary of the past.

Exercise 16.2

What is backpropagation through time, and how does it relate to the DAG requirement from Chapter 4?

Hint. Unroll the recurrence into an acyclic chain; shared weights accumulate gradient over steps.

Exercise 16.3

Why do vanilla RNNs suffer vanishing gradients more severely than typical feedforward nets?

Hint. W_{hh} and $\tanh'$ multiply once per time step; sequence length is the depth.

Exercise 16.4

Explain the LSTM cell-state update and why its additivity defeats vanishing gradients.

Hint. $\mathbf{c}_t = f_t \odot \mathbf{c}_{t-1} + i_t \odot \tilde{\mathbf{c}}_t$; $\text{forget} \approx 1 \Rightarrow \partial c_t / \partial c_{t-1} \approx 1$.

Exercise 16.5**State the role of each of the three LSTM gates.***Hint. Forget: keep/discard old cell; input: write new; output: expose as hidden.***Exercise 16.6****How does GRU simplify LSTM, and what does it trade?***Hint. Two gates, merged cell/hidden, 25% fewer params; capacity vs speed.***Exercise 16.7****Why is the LSTM cell-state highway “the same trick” as a ResNet skip connection?***Hint. An additive path lets gradient bypass repeated multiplicative shrinkage.***Exercise 16.8****(Production) Under a tight latency budget, LSTM or GRU? Justify and caveat.***Hint. GRU default for fewer params; still validate empirically.***Exercise 16.9****(Debugging) An RNN’s loss explodes to NaN on long sequences. First two fixes?***Hint. Gradient clipping; truncated BPTT / shorter sequences.***Exercise 16.10****(Technical) Transformers displaced RNNs for large-scale NLP. Where do RNNs/LSTMs still make sense?***Hint. Streaming/online, resource-constrained, small time-series; gating ideas persist.*

16.12 Answers

Answer 16.1*Feedforward limits and the hidden state.*

A feedforward net processes each input independently, with no memory of order, so it can’t relate word 7 to word 3. An RNN adds a hidden state $\mathbf{h}_t$ updated each step, $\mathbf{h}_t = \tanh(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$, a fixed-size vector summarizing everything seen so far, so the current input is interpreted in context. The same weights are reused every step (weight sharing along time).

Answer 16.2*BPTT and the DAG.*

BPTT trains an RNN by *unrolling* the recurrence into an explicit chain, one network copy per time step, each passing its hidden state forward. That unrolled chain is a strictly

acyclic graph (resolving Chapter 4’s “no cycles” rule), so standard backprop applies across it. Because the weights are shared, each weight receives summed gradient contributions from every time step.

Answer 16.3

Why RNN vanishing is worse.

Backprop through T steps multiplies $\sim T$ copies of W_{hh} and T copies of $\tanh'$, the same repeated-product mechanism as depth (chapter 3), but here “depth” is the *sequence length*, which can be hundreds or thousands, far more than any feedforward net. So gradients to early time steps vanish (or explode), and vanilla RNNs fail to learn long-range dependencies.

Answer 16.4

LSTM cell update.

$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$ is *additive*, not a matrix multiply through a squashing nonlinearity. When the forget gate $\mathbf{f}_t \approx 1$, $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} \approx 1$, so gradient flows backward across many steps essentially unattenuated, an open gradient highway whenever the gates keep it open. That’s the structural fix for vanishing gradients.

Answer 16.5

The three gates.

Forget ($\mathbf{f}_t$): what fraction of the previous cell state to keep. **Input** ($\mathbf{i}_t$): how much of the new candidate $\tilde{\mathbf{c}}_t$ to write into the cell. **Output** ($\mathbf{o}_t$): what part of the updated cell state to expose as the hidden state $\mathbf{h}_t = \mathbf{o}_t \odot \tanh \mathbf{c}_t$. Each gate is a sigmoid of $[\mathbf{h}_{t-1}, \mathbf{x}_t]$ giving values in $(0, 1)$.

Answer 16.6

GRU’s simplification.

GRU uses two gates (update $\mathbf{z}_t$, reset $\mathbf{r}_t$) and merges cell and hidden state into one vector, keeping the additive gated update $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$. It has 25% fewer parameters than a same-size LSTM, faster to train and deploy, trading a little capacity (LSTM’s separate cell + extra gate) for speed. Empirically they’re comparable on most tasks.

Answer 16.7

Cell highway = skip connection.

Both insert an *additive* identity-like path so gradient can bypass repeated multiplicative shrinkage. ResNet adds the block input to its output ($F(x) + x$) along network *depth*; LSTM carries $\mathbf{c}_{t-1}$ to $\mathbf{c}_t$ by add-and- gate along *time*. In both, the additive term yields a near-unit local gradient, keeping signal alive across many layers/steps (chapter 15).

Answer 16.8

Production: LSTM vs GRU under latency.

Default to **GRU**: 25% fewer parameters means faster training and lower-latency inference, and on most tasks it matches LSTM. LSTM’s edge appears mainly on very long-range

dependencies or very large datasets where the extra capacity pays off. Caveat: validate on the actual task, the gap is small and task-dependent, so confirm empirically rather than assuming.

Answer 16.9

Debugging exploding RNN loss.

(1) **Gradient clipping** (`clip_grad_norm_`), the standard, near- mandatory fix for RNN training; exploding gradients are common because W_{hh} compounds over steps. (2) **Truncated BPTT / shorter sequences**: cap the number of steps you backpropagate through, reducing effective depth. Also check init and LR (chapters 8 and 10); consider switching vanilla RNN $\rightarrow$ LSTM/GRU.

Answer 16.10

Where RNNs still matter.

Transformers dominate large-scale NLP, but RNNs/LSTMs remain apt for: **streaming/online** inference where you must process one step at a time without the whole sequence (attention needs it all); **resource-constrained** edge/embedded settings; **small-scale time-series and signal processing** where a transformer's data/compute appetite isn't justified. Conceptually, their additive gated updates live on in residuals and gating throughout modern architectures.

Transformers & Attention

Subscribe → aiengineeringinsider.substack.com/subscribe

The architecture that replaced recurrence with attention, and now powers nearly everything.

Chapter 16 ended on a cliffhanger: the LSTM's gated cell state is a clever fix for vanishing gradients, but it still processes a sequence *one step at a time*, and even that fix degrades over very long ranges. In 2017 the paper *Attention Is All You Need* (Vaswani et al., 2017) proposed a radical alternative, throw out recurrence entirely and let every token attend directly to every other token in a single, fully parallel operation. That architecture, the **Transformer**, is the conceptual and practical foundation of modern deep learning: BERT, the GPT family, ViT, AlphaFold's Evoformer, and almost every system in chapter 18 are Transformers underneath.

This chapter is built as a guided interview drill. It teaches the architecture section by section, foundations, self-attention, multi-head attention, encoder-decoder, Vision Transformers, and the CNN-versus-ViT debate, and within each section poses the questions an interviewer actually asks, with answers written the way a strong candidate would say them out loud. The from-scratch code is real and runnable (`code/ch17_transformers.py`); the PyTorch is the production form (`code/ch17_transformers_torch.py`).

★ Key Insight

The single sentence to remember: a Transformer replaces the RNN's sequential hidden-state recurrence with **self-attention**, giving every token an $O(1)$ -length path to every other token and making the whole sequence processable in parallel. Everything else, multi-head projections, positional encodings, the feed-forward block, residuals and LayerNorm, is machinery around that one idea.

17.1 Transformer Foundations

Figure 17.1 shows one encoder block, the unit that stacks N times (6 in the original, 96 in GPT-3). Read it bottom to top: token embeddings get a positional signal added, flow through multi-head self-attention wrapped in a residual + LayerNorm, then through a position-wise feed-forward network wrapped the same way. The decoder adds one extra sub-layer (cross-attention) covered in section 17.4.

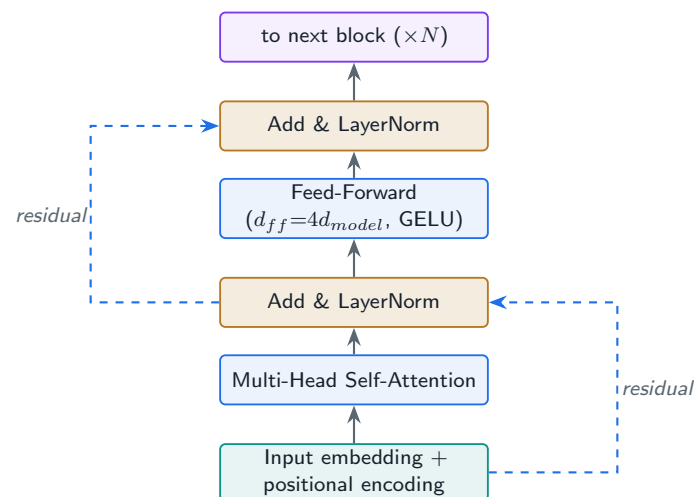


Figure 17.1: One Transformer encoder block. Each sub-layer (multi-head attention, then the position-wise feed-forward network) is wrapped as $\text{LayerNorm}(x + \text{Sublayer}(x))$, the residual path (chapter 15) that lets gradient flow through deep stacks.

Q17.1

What is a Transformer and why was it introduced to replace RNN/LSTM?

A **Transformer** is a deep-learning architecture based entirely on attention, with no recurrence and no convolution. It was introduced in *Attention Is All You Need* (2017) to overcome two crippling RNN/LSTM limitations. First, **sequential processing**: an RNN computes the hidden state at step t from step $t-1$, so it cannot be parallelized across time and is slow on long sequences. Second, **long-range dependency**: information from distant tokens is attenuated by the repeated multiplications of backprop-through-time (chapter 16). Transformers process *all* tokens simultaneously through self-attention, giving every token direct access to every other token regardless of distance.

`transformer_encoder.py`

```
import torch, torch.nn as nn

encoder_layer = nn.TransformerEncoderLayer(
    d_model=512, nhead=8, dim_feedforward=2048,
    dropout=0.1, batch_first=True)
encoder = nn.TransformerEncoder(encoder_layer, num_layers=6)

src = torch.rand(2, 10, 512)          # (batch, seq_len, d_model)
out = encoder(src)                     # (2, 10, 512)
print("Encoder output shape:", out.shape)
```

Q17.2

List and explain the four core components of a Transformer block.

- (1) **Input embedding**, maps discrete tokens to continuous d_{model} -dimensional vectors.
- (2) **Positional encoding**, injects order information since attention is permutation-invariant.
- (3) **Multi-head self-attention**, lets each token attend to all others in parallel.
- (4) **Position-wise feed-forward network (FFN)**, two linear layers with a GELU/ReLU be-

tween them, applied identically at each position. Each sub-layer is wrapped with a residual connection and LayerNorm: $\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$.

`transformer_block.py`

```
import torch, torch.nn as nn

class TransformerBlock(nn.Module):
    def __init__(self, d_model=512, nhead=8, ffn_dim=2048):
        super().__init__()
        self.attn = nn.MultiheadAttention(d_model, nhead, batch_first=True)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, ffn_dim), nn.GELU(),
            nn.Linear(ffn_dim, d_model))
        self.ln1, self.ln2 = nn.LayerNorm(d_model), nn.LayerNorm(d_model)

    def forward(self, x):
        attn_out, _ = self.attn(x, x, x)      # self-attention
        x = self.ln1(x + attn_out)           # residual + norm
        x = self.ln2(x + self.ffn(x))        # FFN + residual + norm
        return x
```

Q17.3

What is positional encoding and why is it necessary?

Self-attention is *permutation-invariant*: shuffle the input tokens and the attention scores are identical, so the model cannot tell “dog bites man” from “man bites dog.” Positional encoding restores order. The original paper uses a fixed sinusoidal scheme, $PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$ and $PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d})$, added (not concatenated) to the token embeddings. Sinusoids extrapolate to unseen sequence lengths; learned positional embeddings perform similarly but are capped at the training-set length.

`positional_encoding.py`

```
import torch, math

def positional_encoding(seq_len, d_model):
    pe = torch.zeros(seq_len, d_model)
    pos = torch.arange(0, seq_len).unsqueeze(1).float()
    div = torch.exp(torch.arange(0, d_model, 2).float()
                    * (-math.log(10000) / d_model))
    pe[:, 0::2] = torch.sin(pos * div)      # even dims
    pe[:, 1::2] = torch.cos(pos * div)      # odd dims
    return pe                              # (seq_len, d_model)
```

Q17.4

How does parallel processing in Transformers improve training versus RNNs?

An RNN’s hidden state at step t depends on step $t-1$, a strict sequential dependency that blocks parallelism: GPU utilization is poor and training time scales with the $O(n)$ serial steps. A Transformer computes attention for all positions at once as a few large matrix

multiplications, so a GPU/TPU processes the entire sequence in one forward pass. The cost is $O(n^2d)$ (quadratic in sequence length) but it is *highly parallel*, which on modern accelerators is far faster in wall-clock time than n sequential RNN steps.

Q17.5

What is Layer Normalization and how does it differ from BatchNorm in Transformers?

BatchNorm normalizes across the *batch* dimension per feature, which is fragile for the variable-length sequences and small batches common in NLP (chapter 9). **LayerNorm** normalizes across the *feature* dimension for each sample independently: $\hat{x}_i = \gamma \frac{x_i - \mu_i}{\sigma_i + \epsilon} + \beta$, where μ_i, σ_i are computed over the d_{model} features of token i . This makes it batch-size independent and identical at train and inference time, the right choice for sequence models.

Q17.6

Explain the residual (skip) connections in the Transformer and their purpose.

Each sub-layer computes $y = \text{LayerNorm}(x + F(x))$ where F is attention or the FFN. The residual path (1) gives gradients a direct identity route backward, defeating vanishing gradients in deep stacks, exactly the ResNet trick of chapter 15 applied to depth; (2) lets each sub-layer learn a small *refinement* of its input rather than a full transformation, easing optimization; and (3) makes very deep stacks trainable (6 layers originally, 96 in GPT-3).

Q17.7

What is the position-wise Feed-Forward Network (FFN) inside a block?

The FFN is applied identically and independently to each token after attention: $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$, typically with $d_{ff} = 4d_{model}$ (e.g. 2048 for $d_{model} = 512$). The expand-then-contract shape projects each token into a higher-dimensional space for non-linear feature mixing before returning to the residual stream. Modern models (BERT, GPT) use GELU (chapter 5) rather than ReLU.

`positionwise_ffn.py`

```
import torch.nn as nn, torch.nn.functional as F

class PositionwiseFFN(nn.Module):
    def __init__(self, d_model=512, d_ff=2048, dropout=0.1):
        super().__init__()
        self.w1, self.w2 = nn.Linear(d_model, d_ff), nn.Linear(d_ff, d_model)
        self.drop = nn.Dropout(dropout)

    def forward(self, x):
        return self.w2(self.drop(F.gelu(self.w1(x))))
```

Q17.8

Why does self-attention have $O(n^2)$ complexity and what methods reduce it?

Every token attends to every other token, forming an $n \times n$ score matrix that costs $O(n^2)$ in both time and memory. Efficient variants: **sparse attention** (Longformer, BigBird) restricts each token to a local window plus a few global tokens; **Linformer** projects K, V to a low rank for $O(n)$ cost; **FlashAttention** keeps the $O(n^2)$ math but is IO-aware, drastically cutting HBM reads/writes (the memory-bandwidth bottleneck of chapter 18); and **Performer** approximates the softmax with random features. The NumPy snippet below prints the FLOP scaling and is run for real in section 17.2.

Q17.9

What applications leverage the Transformer architecture?

Transformers now underpin state-of-the-art systems across every modality, as table 17.1 summarizes; chapter 18 dissects several at the system-design level.

Table 17.1: Transformers across modalities.

Domain	Task	Model examples
NLP	translation, QA, summarization	BERT, GPT-4, T5
Vision	classification, detection	ViT, DINO, CLIP
Speech	ASR, TTS	Whisper, SpeechT5
Biology	protein folding	AlphaFold 2
Multimodal	image + text	GPT-4V, Flamingo

Q17.10

What is the difference between Pre-LN and Post-LN Transformer variants?

Post-LN (original paper): $y = \text{LN}(x + \text{Sublayer}(x))$, with LayerNorm *after* the residual add. It reaches strong peak performance but has unstable gradients early on and needs careful learning-rate warmup (chapter 10). **Pre-LN** (GPT-2 and most modern LLMs): $y = x + \text{Sublayer}(\text{LN}(x))$, with LayerNorm *before* the sub-layer. It trains stably without warmup at a slight cost in peak quality, and is the default for large models.

`pre_ln_block.py`

```
import torch.nn as nn

class PreLNBlock(nn.Module):
    def __init__(self, d=512, h=8, ff=2048):
        super().__init__()
        self.ln1, self.ln2 = nn.LayerNorm(d), nn.LayerNorm(d)
        self.attn = nn.MultiheadAttention(d, h, batch_first=True)
        self.ffn = nn.Sequential(nn.Linear(d, ff), nn.GELU(), nn.Linear(ff, d))

    def forward(self, x):
        x2, _ = self.attn(self.ln1(x), self.ln1(x), self.ln1(x))
        x = x + x2 # residual
        x = x + self.ffn(self.ln2(x)) # residual
        return x
```

17.2 The Self-Attention Mechanism

This is the mathematical heart of the Transformer. Every token is projected into a **query**, a **key**, and a **value**; the query of each token is compared against the keys of all tokens to produce attention weights, which then read out a blend of the values. We implement it from scratch in NumPy and run it, the output below is captured verbatim from `code/ch17_transformers.py`, including a real attention heatmap over a 7-token sentence.

`ch17_transformers.py` (scaled dot-product attention)

```
import numpy as np

def softmax(z, axis=-1):
    z = z - z.max(axis=axis, keepdims=True)      # numerical stability
    e = np.exp(z)
    return e / e.sum(axis=axis, keepdims=True)

def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = (Q @ K.transpose(0, 2, 1)) / np.sqrt(d_k)  # (B,T,T)
    if mask is not None:
        scores = np.where(mask, -1e9, scores)            # mask -> -inf
    weights = softmax(scores, axis=-1)                  # rows sum to 1
    return weights @ V, weights
```

`python ch17\_transformers.py`

```
# input: 7 tokens, d_model=16, heads=4
x shape           : (1, 7, 16)
single-head out   : (1, 7, 16)  attn weights: (1, 7, 7)
attn row sums to 1: True
multi-head out    : (1, 7, 16)  per-head weights: (1, 4, 7, 7)

# self-attention FLOPs vs sequence length (d=64)
n= 128 ~ 2.1 MFLOPs
n= 512 ~ 33.8 MFLOPs
n= 1024 ~ 135.3 MFLOPs
n= 4096 ~ 2164.3 MFLOPs
```

Figure 17.2 is the actual 7×7 attention matrix this code produces for head 0 (the projections are randomly initialized, so this is the raw geometry of attention before any training). Each row is one query token's distribution over all key tokens, and every row sums to 1. The quadratic FLOP growth in the console output, $128 \rightarrow 4096$ tokens raises cost roughly $256\times$, is exactly the $O(n^2)$ wall the efficient-attention methods of chapter 18 were built to climb.

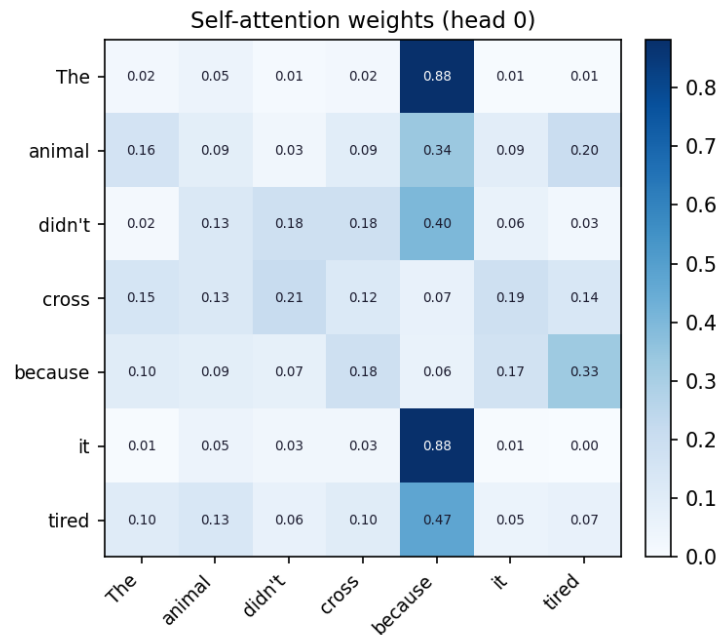


Figure 17.2: A real self-attention weight matrix (head 0) from the from-scratch NumPy code. Row i is how strongly query token i attends to each key token; rows sum to 1. After training, such maps become interpretable (verbs attending to their subjects, pronouns to their referents).

Q17.11

Explain the Query, Key, Value (Q, K, V) abstraction in self-attention.

Each token embedding is linearly projected into three vectors via learned matrices $W^Q, W^K, W^V \in \mathbb{R}^{d \times d_k}$. The **query** asks “what information am I looking for?”; the **key** advertises “what do I have to offer?”; the **value** carries “what content do I actually contribute?” The dot product between a query and every key decides how much of each value is retrieved. In *self*-attention all three come from the same input sequence (in cross-attention, section 17.4, the queries come from the decoder and the keys/values from the encoder).

qkv_projection.py

```
import torch, torch.nn as nn

d_model, d_k = 512, 64
Wq = nn.Linear(d_model, d_k, bias=False)
Wk = nn.Linear(d_model, d_k, bias=False)
Wv = nn.Linear(d_model, d_k, bias=False)

x = torch.rand(2, 10, d_model)      # (batch, seq, d_model)
Q, K, V = Wq(x), Wk(x), Wv(x)      # each (2, 10, 64)
print(Q.shape, K.shape, V.shape)
```

Q17.12

Write out the scaled dot-product attention formula and explain each term.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

$QK^\top$ are the dot-product similarities of every query with every key (an $n \times n$ score matrix). Dividing by $\sqrt{d_k}$ is the **scaling factor** that keeps the scores' variance near 1 (next question). The **softmax** turns each row into a probability distribution summing to 1. Multiplying by V takes the weighted sum of values, so each query retrieves a blend of all values proportional to its attention weights.

Q17.13

Why do we divide by $\sqrt{d_k}$ in the attention formula?

For queries and keys with zero mean and unit variance, the dot product $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ has variance d_k . With large d_k (say 64), the scores grow large in magnitude and push the softmax into a saturated region where one weight ≈ 1 and the rest ≈ 0 , so the gradient nearly vanishes (the same saturation problem as sigmoid in chapter 5). Dividing by $\sqrt{d_k}$ rescales the variance back to 1, keeping the softmax in its responsive, high-gradient regime.

Q17.14

What is an attention mask and when is it used?

A mask sets certain score entries to $-\infty$ *before* the softmax so those positions get exactly zero weight. Two kinds: a **padding mask** zeroes out the pad tokens added to equalize sequence lengths in a batch (the same idea you ran in chapter 19's seq2seq attention); a **causal (look-ahead) mask** makes position i attend only to positions $\leq i$, so a decoder cannot "see the future" it is supposed to predict.

`attention_masks.py`

```
import torch

def causal_mask(seq_len):
    # upper-triangular True = masked out (cannot attend to future)
    return torch.triu(torch.ones(seq_len, seq_len, dtype=torch.bool), diagonal=1)

def padding_mask(lengths, max_len):
    # True where the position is PAD
    return torch.arange(max_len).unsqueeze(0) >= lengths.unsqueeze(1)

print(causal_mask(5).int())
```

Q17.15

How does self-attention capture long-range dependencies?

The path length between any two tokens is constant, $O(1)$, regardless of their distance: a single attention layer connects every token to every other token directly. An RNN must instead propagate information through $O(n)$ hidden-state transitions, so gradients weaken over distance (chapter 16). This direct any-to-any connectivity is why Transformers excel at coreference resolution, long-document summarization, and any task needing distant context.

Q17.16

What is cross-attention (encoder-decoder attention)?

In the decoder, the cross-attention sub-layer draws its *queries* from the decoder's own previous sub-layer but its *keys and values* from the encoder's final output. Every decoder position can therefore attend over the entire encoded source while generating each output token, the mechanism that powers sequence-to-sequence tasks such as translation.

`cross_attention.py`

```
import torch, torch.nn as nn

class CrossAttention(nn.Module):
    def __init__(self, d_model=512, nhead=8):
        super().__init__()
        self.attn = nn.MultiheadAttention(d_model, nhead, batch_first=True)
        self.norm = nn.LayerNorm(d_model)

    def forward(self, decoder_x, encoder_out):
        out, _ = self.attn(query=decoder_x,          # Q from decoder
                           key=encoder_out,          # K, V from encoder
                           value=encoder_out)
        return self.norm(decoder_x + out)
```

Q17.17

What are the advantages and limitations of self-attention?

Table 17.2: Self-attention: what you gain and what it costs.

Advantages	Limitations
Long-range dependencies at $O(1)$ path length	$O(n^2)$ time and memory in sequence length
Fully parallel, no sequential bottleneck	Position must be injected explicitly
Direct token-to-token context	Memory-heavy for very long inputs (documents, genomes)

Q17.18

Implement self-attention from scratch.

A complete single-head module: project to Q, K, V , score, scale, softmax, read out the values, and also return the attention map for inspection.

`self_attention.py`

```

import torch, torch.nn as nn, math, torch.nn.functional as F

class SelfAttention(nn.Module):
    def __init__(self, d_model=512, d_k=64):
        super().__init__()
        self.Wq = nn.Linear(d_model, d_k, bias=False)
        self.Wk = nn.Linear(d_model, d_k, bias=False)
        self.Wv = nn.Linear(d_model, d_k, bias=False)
        self.scale = math.sqrt(d_k)

    def forward(self, x, mask=None):
        Q, K, V = self.Wq(x), self.Wk(x), self.Wv(x)
        scores = torch.bmm(Q, K.transpose(1, 2)) / self.scale
        if mask is not None:
            scores = scores.masked_fill(mask, float('-inf'))
        weights = F.softmax(scores, dim=-1)
        return torch.bmm(weights, V), weights      # output, attention map

```

Q17.19

What is the role of softmax in attention and can it be replaced?

Softmax turns raw scores into a non-negative distribution summing to 1, keeping the weighted sum of values well-scaled. Its downside is that it is *dense*, every position gets some non-zero weight. Alternatives: **sparsemax** produces genuinely sparse weights (many exactly zero); **α -entmax** interpolates between softmax and sparsemax; and some efficient Transformers use **ReLU-based attention** to skip the normalization bottleneck.

Q17.20

How do you visualize attention weights for interpretability?

Plot the $n \times n$ weight matrix of each head as a heatmap (fig. 17.2 is exactly this, generated by the from-scratch code). A bright cell (i, j) means token i attends strongly to token j ; after training, recognizable patterns emerge, verbs attending to subjects, pronouns to their referents. Tools like BertViz give interactive per-head views for BERT-style models.

17.3 Multi-Head Attention

A single attention operation forces every token to blend information through one shared similarity function. Multi-head attention runs h such operations in parallel, each in its own learned subspace, so different heads can specialize, one tracks syntax, another coreference, another local position. Because $d_k = d_{\text{model}}/h$, the total cost matches a single large head.

Q17.21

What is Multi-Head Attention (MHA) and why is it better than single-head?

MHA runs h independent attention “heads” in parallel, each with its own W_i^Q, W_i^K, W_i^V : $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$ with $\text{head}_i =$

$\text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$. Each head can attend to a *different* aspect of the input (syntax, coreference, semantics, position), whereas one head must average over all of them. Total parameters are similar to a single large head because $d_k = d_{\text{model}}/h$.

mha_pytorch.py

```
import torch, torch.nn as nn

mha = nn.MultiheadAttention(embed_dim=512, num_heads=8, batch_first=True)
x = torch.rand(2, 10, 512)
out, attn = mha(x, x, x)                                # self-attention
print("output :", out.shape)                             # (2, 10, 512)
print("weights:", attn.shape)                             # (2, 10, 10)
```

Q17.22

Walk through the step-by-step computation of Multi-Head Attention.

(1) **Project** $X \in \mathbb{R}^{n \times d}$ to $Q_i, K_i, V_i \in \mathbb{R}^{n \times d_k}$ for each head ($d_k = d/h$). (2) **Attend** per head: $\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$. (3) **Concatenate** the head outputs along the feature axis back to $\mathbb{R}^{n \times d}$. (4) **Project** through $W^O \in \mathbb{R}^{d \times d}$.

multi_head_attention.py

```
import torch, torch.nn as nn, math, torch.nn.functional as F

class MultiHeadAttention(nn.Module):
    def __init__(self, d=512, h=8):
        super().__init__()
        self.h, self.dk = h, d // h
        self.Wq = nn.Linear(d, d, bias=False)
        self.Wk = nn.Linear(d, d, bias=False)
        self.Wv = nn.Linear(d, d, bias=False)
        self.Wo = nn.Linear(d, d, bias=False)

    def split(self, x, B, T):
        return x.view(B, T, self.h, self.dk).transpose(1, 2)  # (B, h, T, dk)

    def forward(self, x):
        B, T, _ = x.shape
        Q = self.split(self.Wq(x), B, T)
        K = self.split(self.Wk(x), B, T)
        V = self.split(self.Wv(x), B, T)
        scores = Q @ K.transpose(-2, -1) / math.sqrt(self.dk)
        weights = F.softmax(scores, dim=-1)
        heads = (weights @ V).transpose(1, 2).contiguous().view(B, T, -1)
        return self.Wo(heads)  # concat + project
```

Q17.23

How do you choose the number of attention heads h ?

The constraint is $d_k = d_{model}/h$ (integer). Table 17.3 lists standard choices. More heads add expressiveness but cost compute linearly, and head-pruning research shows many heads end up redundant, so beyond a point the returns flatten.

Table 17.3: Head-count choices across well-known models.

Model	d_{model}	h	d_k	Layers
Transformer-base	512	8	64	6
BERT-base	768	12	64	12
GPT-3 175B	12288	96	128	96
ViT-B/16	768	12	64	12

Q17.24

What does each attention head typically learn?

Probing studies (Clark et al., 2019, on BERT) show heads specialize: **syntactic** heads attend to direct objects/subjects or dependency arcs; **positional** heads attend to adjacent tokens; **coreference** heads link pronouns to antecedents (“it” → “animal”); **rare-token** heads focus on infrequent but informative tokens. This emergent division of labor is the core reason MHA beats single-head attention.

Q17.25

What is the output projection W^O and why is it needed?

After concatenating the h head outputs (each d_k -dimensional) you get a d_{model} -vector, but the heads are still independent “views.” $W^O \in \mathbb{R}^{d \times d}$ mixes information *across* heads, letting the model learn how to combine each head’s specialized representation. Without it, heads would never interact.

Q17.26

Explain Grouped-Query Attention (GQA) used in LLaMA-2/3.

GQA shrinks the inference KV-cache by sharing K/V projections across groups of query heads. **MHA**: h Q heads, h K/V heads. **MQA**: h Q heads, *one* K/V head. **GQA**: h Q heads, g K/V heads ($1 < g < h$), each group of h/g queries sharing one K/V pair. It cuts the KV-cache (critical for long-context decoding, chapter 18) while staying close to full-MHA quality.

```
grouped_query_attention.py
```

```

import torch, torch.nn as nn

H_Q, H_KV, d = 8, 2, 512                                # 8 query heads, 2 KV groups
dk = d // H_Q
Wq = nn.Linear(d, H_Q * dk, bias=False)
Wk = nn.Linear(d, H_KV * dk, bias=False)
Wv = nn.Linear(d, H_KV * dk, bias=False)

x = torch.rand(1, 10, d)
Q = Wq(x).view(1, 10, H_Q, dk).transpose(1, 2)          # (1,8,10,dk)
K = Wk(x).view(1, 10, H_KV, dk).transpose(1, 2)          # (1,2,10,dk)
V = Wv(x).view(1, 10, H_KV, dk).transpose(1, 2)
K = K.repeat_interleave(H_Q // H_KV, dim=1)              # share K/V across groups
V = V.repeat_interleave(H_Q // H_KV, dim=1)              # -> (1,8,10,dk)

```

Q17.27

What is attention dropout and why is it used?

Attention dropout applies dropout (chapter 9) to the attention *weight matrix*, after softmax and before the weighted sum over values. It randomly zeroes some (query, key) connections during training, preventing over-reliance on specific pairs and improving generalization. Typical rate 0.1 for base models, often 0 during large-scale fine-tuning.

`attention_dropout.py`

```

import torch, math, torch.nn.functional as F

def attention_with_dropout(Q, K, V, p=0.1, training=True):
    d_k = Q.size(-1)
    scores = Q @ K.transpose(-2, -1) / math.sqrt(d_k)
    weights = F.softmax(scores, dim=-1)
    weights = F.dropout(weights, p=p, training=training)    # drop connections
    return weights @ V

```

Q17.28

Compare MHA parameter counts: 1 head vs 8 heads for $d_{\text{model}} = 512$.

They are *identical*. One head with $d_k = 512$ uses $4 \times 512^2 = 1,048,576$ parameters (W^Q, W^K, W^V, W^O). Eight heads with $d_k = 64$ use $8 \times 3 \times 512 \times 64 + 512^2 = 786,432 + 262,144 = 1,048,576$. Same cost, but eight heads give richer subspace representations.

`mha_param_count.py`

```
import torch.nn as nn

def count(m): return sum(p.numel() for p in m.parameters())

d = 512
single = nn.MultiheadAttention(d, num_heads=1, bias=False, batch_first=True)
multi = nn.MultiheadAttention(d, num_heads=8, bias=False, batch_first=True)
print(f"1-head : {count(single):,}")      # 1,048,576
print(f"8-heads: {count(multi):,}")      # 1,048,576
```

17.4 Encoder–Decoder Architecture

The original Transformer is an encoder-decoder. The encoder reads the whole source in parallel into contextualized embeddings; the decoder generates the target left to right, attending both to its own past (masked self-attention) and to the encoder output (cross-attention). Modern LLMs often keep only one half, BERT is encoder-only, GPT is decoder-only.

Q17.29

What does the encoder do and what is its output?

The encoder processes the entire source in parallel through N identical blocks (multi-head self-attention + FFN, each with residual and LayerNorm). Its output is a sequence of **contextualized embeddings** $Z \in \mathbb{R}^{n \times d}$, one vector per source token enriched with global context, which is handed to every decoder layer via cross-attention.

encoder_stack.py

```
import torch, torch.nn as nn

encoder_layer = nn.TransformerEncoderLayer(d_model=512, nhead=8, batch_first=True)
encoder = nn.TransformerEncoder(encoder_layer, num_layers=6)
src = torch.rand(2, 15, 512)      # (batch, src_len, d)
memory = encoder(src)             # (2, 15, 512)
print("Memory shape:", memory.shape)
```

Q17.30

Describe the three attention sub-layers inside a decoder block.

(1) **Masked multi-head self-attention**, the decoder attends to its own previous outputs, with a causal mask blocking positions $> t$. (2) **Encoder-decoder cross-attention**, Q from the decoder, K, V from the encoder memory Z , letting each decoder position read the full source. (3) **Position-wise FFN**, identical to the encoder's. Each is wrapped with residual + LayerNorm.

Q17.31

Why is masked self-attention used in the decoder?

In training the whole target is fed at once (teacher forcing, chapter 19). Without masking, position t could attend to ground-truth tokens at $t+1, t+2, \dots$, i.e. read the answer it is meant to predict. The causal mask forces the prediction at t to depend only on positions $\leq t$, matching the left-to-right generation used at inference.

```
decoder_stack.py
```

```
import torch, torch.nn as nn

dec_layer = nn.TransformerDecoderLayer(d_model=512, nhead=8, batch_first=True)
decoder = nn.TransformerDecoder(dec_layer, num_layers=6)
tgt = torch.rand(2, 10, 512)
memory = torch.rand(2, 15, 512) # encoder output
tgt_mask = nn.Transformer.generate_square_subsequent_mask(10) # causal
out = decoder(tgt, memory, tgt_mask=tgt_mask)
print(out.shape) # (2, 10, 512)
```

Q17.32

How does the Transformer generate output sequences at inference?

Autoregressively: (1) start from a [BOS] token; (2) feed the (once-computed) encoder output plus the current decoder sequence; (3) project the last decoder hidden state and softmax to a vocabulary distribution; (4) take the argmax (or sample); (5) append and repeat until [EOS] or a length cap. The **KV-cache** (Q17.35) stores past K/V so each step only computes the new token's projections.

Q17.33

What is the output layer and how are logits produced?

The final decoder hidden state $h_t \in \mathbb{R}^d$ is mapped to vocabulary logits: $\text{logits} = h_t W_{out} + b$ with $W_{out} \in \mathbb{R}^{d \times |V|}$, then softmax (the clean logits $\rightarrow$ cross-entropy pairing of chapter 6). Many models (GPT-2) **tie** W_{out} to the input embedding matrix to save parameters and improve generalization.

```
tied_output_projection.py
```

```
import torch, torch.nn as nn, torch.nn.functional as F

vocab, d = 32000, 512
embedding = nn.Embedding(vocab, d)
output_proj = lambda h: h @ embedding.weight.T # weight tying

h = torch.rand(2, 10, d) # decoder output
logits = output_proj(h) # (2, 10, 32000)
next_token = F.softmax(logits, -1)[: , -1].argmax(-1)
print("next token ids:", next_token)
```

Q17.34

Compare encoder-only, decoder-only, and encoder-decoder architectures.
Table 17.4 summarizes the three families and what each is for.

Table 17.4: The three Transformer families.

Type	Examples	Attention	Best for
Encoder-only	BERT, RoBERTa	bidirectional	classification, NER, QA
Decoder-only	GPT, LLaMA	causal (masked)	generation, language modeling
Encoder-decoder	T5, BART, original	bi + cross	seq2seq, translation

Q17.35

What is the KV-cache and how does it accelerate autoregressive decoding?
During decoding the K and V projections of all *past* tokens never change, so recomputing them each step is wasteful. The KV-cache stores them; at step t only the new token's K/V is computed and appended, cutting per-step work from $O(td)$ to $O(d)$. The cost is memory: $O(nLd)$ for L layers, which is exactly why the cache dominates LLM serving memory (chapter 18) and why GQA (Q17.26) and paged attention exist.

kv_cache.py

```
import torch

def decode_step(x_new, past_k, past_v, attn):
    k_new = attn.key_proj(x_new) # only the NEW token
    v_new = attn.val_proj(x_new)
    K_all = torch.cat(past_k + [k_new], dim=1) # reuse cached K
    V_all = torch.cat(past_v + [v_new], dim=1) # reuse cached V
    q = attn.query_proj(x_new)
    out = scaled_dot_product_attention(q, K_all, V_all)
    return out, K_all, V_all # return updated cache
```

Q17.36

What is beam search and how is it used in Transformer generation?
Beam search keeps the top- k (beam width) most probable partial sequences at each step, expands each, and retains the k best, until all beams hit [EOS]. It approximates the globally most probable sequence without the intractable exhaustive search that greedy decoding (top-1) skips entirely. Wider beams trade compute for quality.

beam_search.py

```

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

tok = AutoTokenizer.from_pretrained("t5-small")
model = AutoModelForSeq2SeqLM.from_pretrained("t5-small")
inputs = tok("translate English to French: Hello world", return_tensors="pt")
out = model.generate(**inputs, num_beams=4, max_new_tokens=20)
print(tok.decode(out[0], skip_special_tokens=True))    # "Bonjour monde"

```

17.5 Vision Transformers (ViT)

The Transformer was built for text, but its only assumption is “a sequence of tokens.” The Vision Transformer (Dosovitskiy et al., 2021) cuts an image into fixed patches, treats each as a token, and feeds the sequence to a standard encoder, demonstrating that with enough data, the convolution’s hand-built spatial priors are not strictly necessary.

Q17.37

What is a Vision Transformer (ViT) and how does it process an image?

ViT applies a standard Transformer encoder to images by tokenizing them: (1) **split** an $H \times W \times C$ image into $N = HW/P^2$ non-overlapping $P \times P$ patches (usually $P = 16$); (2) **flatten and linearly project** each patch to d_{model} ; (3) **prepend a learnable [CLS] token** whose final state is used for classification; (4) **add positional embeddings**; (5) **run the encoder** over the $N+1$ tokens.

patch_embedding.py

```

import torch, torch.nn as nn

class PatchEmbedding(nn.Module):
    def __init__(self, img=224, patch=16, in_ch=3, d_model=768):
        super().__init__()
        n = (img // patch) ** 2
        self.proj = nn.Conv2d(in_ch, d_model, kernel_size=patch, stride=patch)
        self.cls = nn.Parameter(torch.zeros(1, 1, d_model))
        self.pos = nn.Parameter(torch.zeros(1, n + 1, d_model))

    def forward(self, x):
        # (B, 3, 224, 224)
        B = x.size(0)
        x = self.proj(x).flatten(2).transpose(1, 2) # (B, N, d)
        cls = self.cls.expand(B, -1, -1)
        return torch.cat([cls, x], dim=1) + self.pos # (B, N+1, d)

```

Q17.38

Why does ViT need positional embeddings for image patches?

ViT’s self-attention is permutation-invariant, so without position information patch 5 and patch 37 are indistinguishable and the model loses all spatial structure. ViT adds *learnable*

1-D positional embeddings (one per patch index); surprisingly, 2-D embeddings add little, the model learns 2-D structure internally. At a different test resolution, the positional embeddings are 2-D interpolated.

Q17.39

What is the CLS token in ViT and how is it used for classification?

The [CLS] token is a learnable vector prepended to the patch sequence. Because every token (including CLS) attends to all others, by the final layer the CLS token has aggregated information from all patches. Its output z_0^L is fed to a small head: $\hat{y} = \text{MLP}(\text{LN}(z_0^L))$. It is the image analog of BERT's sentence-level [CLS].

`vit_classifier.py`

```
import torch, torch.nn as nn

class ViTClassifier(nn.Module):
    def __init__(self, d_model=768, num_classes=1000):
        super().__init__()
        layer = nn.TransformerEncoderLayer(d_model, nhead=12,
                                           batch_first=True, norm_first=True)
        self.enc = nn.TransformerEncoder(layer, 12)
        self.norm = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, num_classes)

    def forward(self, tokens):          # (B, N+1, d)
        z = self.enc(tokens)
        return self.head(self.norm(z[:, 0])) # classify from CLS token
```

Q17.40

What are the advantages of ViT over CNNs?

Global receptive field from layer one (a CNN builds global context only by stacking local convolutions, chapter 15); **scalability**, ViT quality keeps improving predictably with data and model size; **cross-modal transfer**, the same architecture serves text, image, audio, and video with minimal change; and **interpretability**, attention maps show what the model looks at.

Q17.41

What are the limitations of ViT compared to CNNs?

Data-hungry, lacking the CNN's translation-equivariance and locality biases, ViT needs large datasets (ImageNet-21k, JFT-300M) to win; on small data, CNNs usually beat it. **Quadratic complexity** in the number of patches makes high-resolution images expensive. **High pre-training compute**. And **positional-encoding sensitivity**, changing resolution requires interpolating the position embeddings.

Q17.42

How does the Swin Transformer address ViT's scalability issues?

Two ideas. **Shifted-window attention** computes attention within local non-overlapping windows, dropping cost from $O(N^2)$ to $O(N)$, and shifts the windows between layers so adjacent windows exchange information. **Hierarchical feature maps** merge patches as depth grows, producing the multi-scale pyramid CNNs use, which is essential for detection and segmentation. Swin is a common backbone (e.g. Mask R-CNN + Swin).

Q17.43

Implement a minimal ViT forward pass.

```
mini_vit.py
```

```
import torch, torch.nn as nn

class MiniViT(nn.Module):
    def __init__(self, img=224, patch=16, c=3, d=768, heads=12,
                  layers=12, classes=1000):
        super().__init__()
        n = (img // patch) ** 2
        self.patch_emb = nn.Conv2d(c, d, patch, stride=patch)
        self.cls = nn.Parameter(torch.zeros(1, 1, d))
        self.pos = nn.Parameter(torch.zeros(1, n + 1, d))
        enc = nn.TransformerEncoderLayer(d, heads, dim_feedforward=d * 4,
                                         batch_first=True, norm_first=True)
        self.enc = nn.TransformerEncoder(enc, layers)
        self.norm = nn.LayerNorm(d)
        self.head = nn.Linear(d, classes)

    def forward(self, x):
        B = x.shape[0]
        x = self.patch_emb(x).flatten(2).transpose(1, 2)
        x = torch.cat([self.cls.expand(B, -1, -1), x], 1) + self.pos
        return self.head(self.norm(self.enc(x)[: , 0]))

model = MiniViT()
print(model(torch.rand(2, 3, 224, 224)).shape)  # (2, 1000)
```

17.6 CNN vs. ViT, Hybrids, and Modern Techniques

ViT did not retire the CNN; it reframed the trade-off as one of **inductive bias** versus **data**. This final section covers when to pick each, the hybrid models that blend them, and four techniques (DINO, FlashAttention, LoRA, and the BERT-vs-GPT split) every transformer engineer is expected to know.

Q17.44

What is inductive bias and why do CNNs have more of it than ViTs?

Inductive bias is the set of assumptions baked into an architecture that constrains the hypothesis space and helps generalization when data is scarce. CNNs have two strong ones: **translation equivariance** (the same filter slides everywhere, so a feature found top-left is found anywhere) and **locality** (each neuron sees a small receptive field, assuming nearby pixels matter most). ViT has neither by default, it treats patches as unordered tokens and must learn spatial structure from data, which is why it needs more of it.

Q17.45

When should you choose a CNN over a ViT and vice versa?

Table 17.5 gives the decision grid. The short rule: use a pretrained ViT when data is plentiful; otherwise fine-tune a CNN backbone with ImageNet pretraining.

Table 17.5: CNN versus ViT selection grid.

Scenario	Prefer CNN	Prefer ViT
Dataset size	small/medium (<100k)	large (>1M)
Compute budget	constrained	high
Task	dense prediction (seg, det)	classification, retrieval
Domain shift	low	high (with pretraining)
Edge deployment	efficient CNNs (MobileNet)	ViT-Tiny

Q17.46

What are hybrid CNN-Transformer models? Give examples.

Hybrids combine the CNN’s local feature extraction with the Transformer’s global context. **Sequential CNN→Transformer**: a CNN produces feature maps that are flattened into tokens for an encoder (ViT with a ResNet stem; TransUNet for medical segmentation). **ConvNeXt**: a pure CNN modernized with Transformer design choices (large kernels, LayerNorm, GELU, inverted bottleneck). **Swin**: windowed attention inside a CNN-like pyramid.

```
hybrid_vit.py
```

```
import torch, torch.nn as nn, torchvision.models as tv

class HybridViT(nn.Module):
    def __init__(self, num_classes=1000):
        super().__init__()
        resnet = tv.resnet50(weights="IMAGENET1K_V1")
        self.cnn_stem = nn.Sequential(*list(resnet.children())[:-2]) # ->
        # (B, 2048, 7, 7)
        enc = nn.TransformerEncoderLayer(2048, 8, dim_feedforward=4096,
        # batch_first=True)
        self.tfm = nn.TransformerEncoder(enc, 2)
        self.pool = nn.AdaptiveAvgPool1d(1)
        self.fc = nn.Linear(2048, num_classes)

    def forward(self, x):
        f = self.cnn_stem(x) # local features
        tok = f.flatten(2).transpose(1, 2) # (B, 49, 2048) tokens
        z = self.tfm(tok) # global context
        return self.fc(self.pool(z.transpose(1, 2))[:, :, 0])
```

Q17.47

What is DINO and how does it enable self-supervised ViT training?
DINO (self-distillation with no labels; Caron et al., 2021) trains a student ViT to match the output distribution of an EMA-updated teacher ViT across different augmented crops of the same image, no labels needed. The CLS token learns semantically meaningful features, and the attention maps yield strikingly clean object segmentation (“emergent segmentation”). DINOv2 scales this into a general-purpose visual feature extractor, the contrastive/self-supervised idea of chapter 6 applied to vision transformers.

Q17.48

Compare BERT (encoder-only) and GPT (decoder-only).

Table 17.6: BERT versus GPT at a glance.

Property	BERT	GPT
Architecture	encoder-only	decoder-only
Attention	bidirectional	causal (left context)
Pretraining	masked LM (+ NSP)	autoregressive LM
Strength	understanding (NLU)	generation (NLG)
Adapting	add a classification head	prompt / instruction tuning
Examples	BERT, RoBERTa, ALBERT	GPT-2/3/4, LLaMA, Mistral

Q17.49

What is FlashAttention and how does it speed up training?
FlashAttention (Dao et al., 2022) is an IO-aware *exact* attention algorithm that never materializes the full $n \times n$ matrix in high-bandwidth memory. It tiles Q, K, V into blocks

that fit in fast SRAM, fuses the softmax and matmuls into one GPU kernel, and uses $O(n)$ memory while computing the identical result. The payoff is a 2–4× wall-clock speedup and feasible long contexts (32k–128k tokens), the memory-bandwidth optimization that recurs throughout chapter 18.

`flash_attention.py`

```
import torch, torch.nn.functional as F

# PyTorch 2.0+ dispatches to FlashAttention automatically on supported GPUs.
Q = torch.rand(2, 8, 1024, 64, device="cuda", dtype=torch.float16)
K = torch.rand(2, 8, 1024, 64, device="cuda", dtype=torch.float16)
V = torch.rand(2, 8, 1024, 64, device="cuda", dtype=torch.float16)
out = F.scaled_dot_product_attention(Q, K, V, is_causal=True)
print(out.shape)                    # (2, 8, 1024, 64)
```

Q17.50

What is LoRA and how does it enable efficient fine-tuning of large Transformers?

LoRA (low-rank adaptation; Hu et al., 2021) fine-tunes a frozen pretrained model by injecting trainable low-rank updates into its weight matrices instead of updating all parameters: $W' = W + \Delta W = W + AB$ with $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$. Only A, B train. With $r = 8$, a 768×768 matrix drops from 590k to 12k trainable parameters per layer (a 98% cut) while matching or approaching full fine-tuning, the practical way most teams adapt LLMs.

`lora_linear.py`

```
import torch, torch.nn as nn

class LoRALinear(nn.Module):
    def __init__(self, in_f, out_f, rank=8, alpha=16):
        super().__init__()
        self.linear = nn.Linear(in_f, out_f, bias=False)
        self.linear.weight.requires_grad = False           # frozen base
        self.A = nn.Parameter(torch.randn(rank, in_f) * 0.01)
        self.B = nn.Parameter(torch.zeros(out_f, rank))
        self.scale = alpha / rank

    def forward(self, x):
        return self.linear(x) + self.scale * (x @ self.A.T) @ self.B.T

layer = LoRALinear(768, 768, rank=8)
print(layer(torch.rand(2, 10, 768)).shape)               # (2, 10, 768)
print("trainable:", sum(p.numel() for p in layer.parameters() if
    ↪ p.requires_grad))  # 12,288
```

17.7 Quick Reference

Table 17.7 collects the formulas and facts worth memorizing before an interview.

Table 17.7: Transformer quick reference: the formulas and facts to know cold.

Concept	Formula / key fact
Scaled dot-product	$\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V$
Multi-head	$\text{MH} = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$
Positional encoding	$PE_{(p,2i)} = \sin(p/10000^{2i/d})$
FFN	$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2, d_{ff} = 4d_{model}$
Residual sub-layer	$y = \text{LayerNorm}(x + \text{Sublayer}(x))$
ViT patches	$N = HW/P^2$, typically $P = 16$
Self-attn complexity	$O(n^2d)$ time, $O(n^2)$ memory
LoRA update	$W' = W + AB, r \ll \min(d, k)$
BERT / GPT / T5	encoder-only / decoder-only / encoder-decoder
ViT-B/16	$d=768$, 12 heads, 12 layers, 86M params

★ Key Insight

Drop the recurrence from the encoder-decoder of chapter 16, keep only attention, add positional encodings and the residual/LayerNorm scaffolding, and you have the architecture behind essentially every system in chapter 18. The same block classifies images (ViT), folds proteins (Evoformer), and writes code (Codex). If you understand the $\text{softmax}(QK^\top / \sqrt{d_k})V$ at the center of this chapter, you understand the engine of modern AI.

PART V

Bonus: Systems at Scale

The foundations are domain-agnostic; this bonus part shows them deployed. Twenty of the most-used products on earth, from Tesla Autopilot to ChatGPT to TikTok, are dissected at the system-design level a staff engineer would use in an interview: architecture, components, data flow, and the trade-offs that actually decided each design. The recurring lesson is that, at scale, the neural network is a component and the system around it is the real engineering.

Bonus: 20 Deep Learning Systems in Production

Subscribe → aiengineeringinsider.substack.com/subscribe

How twenty of the world's most-used products actually deploy deep learning at system-design scale.

Everything in the previous sixteen chapters is the vocabulary. This chapter is the literature: twenty real systems, each built by a tech giant or category leader, examined at the level a staff or principal engineer would discuss in a system-design interview. For each case the focus is the same four questions: what is the **system-design level** problem being solved, what is the **architecture**, what are the **components** and how does **data flow** through them, and what **trade-offs** were made. The deep-learning model is almost never the hard part at this scale; the hard part is the data pipeline, the serving latency budget, the feedback loop, and the cost.

★ Key Insight

A recurring pattern across all twenty systems: the model is wrapped in a **two-stage (or multi-stage) funnel**, a cheap **candidate generation / retrieval** stage that narrows millions of items to hundreds, followed by an expensive **ranking** stage that scores those hundreds precisely. This candidate-then-rank shape appears in search, recommendation, ads, and feed systems alike, because no production system can afford to run a heavy model over its entire catalog per request.

18.1 The Twenty Systems at a Glance

Table 18.1 maps the twenty case studies to their domain, the dominant architecture family, and the single hardest system-design constraint each one is organized around.

Table 18.1: The twenty production deep-learning systems in this chapter.

System	Domain	Core architecture	Hardest constraint
Tesla Autopilot	Autonomous driving	Multi-task CNN / BEV transformer	Real-time, safety-critical latency
OpenAI ChatGPT	Conversational AI	Decoder transformer + RLHF	Inference cost at scale

System	Domain	Core architecture	Hardest constraint
Google Search	Web ranking	BERT / MUM + funnel	Sub-second over billions of docs
YouTube Recsys	Video recommendation	Two-tower + deep ranker	Billions of candidates per user
Netflix Recsys	Video recommendation	Matrix factorization + DL	Cold start, personalization
Meta Feed & Ads	Feed / ads ranking	DLRM (embeddings)	Embedding table memory (TB)
Spotify	Music recommendation	CF + audio CNN + bandits	Exploration vs exploitation
Uber DeepETA	ETA prediction	Transformer over routes	Tight latency, freshness
Amazon	E-commerce / Alexa	DSSM + seq models	Catalog scale, latency
TikTok Monolith	Short-video recsys	Collisionless embeddings	Real-time online learning
AlphaFold 2	Protein structure	Evoformer + structure module	Throughput, scientific accuracy
Google Translate	Machine translation	Transformer (seq2seq)	Quality across 100+ languages
Pinterest PinSage	Visual recsys	Graph neural network	3B-node graph training
Waymo	Autonomous driving	Perception + prediction stack	Long-tail safety
Stable Diffusion	Image generation	Latent diffusion + U-Net	Compute per image, control
GitHub Copilot	Code generation	Decoder transformer (Codex)	IDE latency, context
Stripe Radar	Fraud detection	Gradient-boosted + DL	Adversarial drift, <100 ms
LinkedIn	Professional network	GNN + GBDT ranking	Graph freshness, fairness
DoorDash / Instacart	Logistics ETA	GBDT + DL ensembles	Supply/demand dynamics
NVIDIA DLSS	Real-time graphics	Temporal CNN (super-res)	Per-frame <2 ms on GPU

The cases that follow are ordered roughly by how directly they map onto the foundational chapters: vision systems (CNNs, chapter 15) first, then language and sequence systems (chapter 16 and its transformer successors), then the large recommendation and ranking systems that combine both.

18.2 Case 1: Tesla Autopilot & Full Self-Driving

Tesla Autopilot — vision-only autonomous driving

A camera-only perception stack that must convert eight surround cameras into a unified, metric, top-down representation of the world, and do it within a fixed per-frame latency budget on an embedded automotive chip.

System-design level. Tesla’s defining bet is **vision-only** perception: no lidar, no high-definition prior maps. The system must therefore infer 3D geometry, occupancy, lane topology, and the motion of every agent purely from 2D camera pixels, in real time, on the in-car FSD computer (two redundant SoCs), under a hard latency budget measured in tens of milliseconds. At this level the problem is not “train a good detector” but “produce a single, temporally-consistent, vector-space model of the scene that a planner can act on.”

Architecture & components. The stack evolved from per-camera 2D CNNs (chapter 15) into a shared **multi-task “HydraNet”**: a common convolutional backbone (a RegNet/BiFPN-style feature pyramid) feeds dozens of task-specific heads (objects, lanes, traffic lights, drivable space). Crucially, features from all eight cameras are fused into a single **bird’s-eye-view (BEV)** representation by a transformer that attends from a fixed BEV output grid back into each camera’s image features, learning the camera-to-world projection implicitly. A video module adds a feature queue (spatial and temporal) so the network has memory of recently-occluded objects, the same memory motivation as chapter 16, realized with attention over a cache rather than recurrence.

Data flow. (1) Eight cameras stream rectified frames. (2) The shared backbone extracts per-image features. (3) The BEV transformer fuses them into a single top-down feature volume. (4) The temporal module integrates this volume with a queue of past features. (5) Task heads emit objects, occupancy, lanes, and their kinematics in vector space. (6) The planner consumes this vector scene, no neural-network pixels reach the planner, only structured geometry. Offline, a massive **auto-labeling** pipeline reconstructs each clip in 3D so human labelers correct geometry rather than draw boxes frame by frame, and a data-engine actively mines fleet vehicles for rare scenarios (the long tail).

Table 18.2: Tesla Autopilot: the central trade-offs.

Decision	Trade-off
Vision-only (no lidar)	Lower cost, fleet-scalable, but must solve 3D from 2D, harder perception
Multi-task HydraNet	Shared compute and features vs. task interference; heads can hurt each other
BEV vector space	Clean planner interface vs. a hard learned projection
On-device inference	Privacy, latency, no connectivity dependence vs. fixed compute ceiling
Fleet data engine	Unmatched long-tail coverage vs. enormous labeling/infra cost

From the Field

The system-design lesson here generalizes far beyond cars: the highest-leverage investment was not the model architecture but the *data engine*, the closed loop that detects where the model fails in the fleet, mines those exact situations, auto-labels them, and retrains. At scale, the pipeline that decides *what to learn from* matters more than the network that learns it.

End-to-end recommendation. Tesla’s leverage is in the *data* and *monitoring* stages: an auto-labeling pipeline plus a fleet data-engine that mines the exact scenarios where the deployed model fails. Build the closed loop that decides *what to learn from* before tuning the model, at scale the data engine outperforms the architecture.

18.3 Case 2: OpenAI ChatGPT

OpenAI ChatGPT — aligned large language model serving

A decoder-only transformer trained on internet-scale text, aligned to human preferences with RLHF, and served to hundreds of millions of users under an inference-cost constraint that dominates the entire economics of the product.

System-design level. Two distinct problems sit behind ChatGPT. The first is **training**: pre-training a decoder transformer with next-token prediction (the cross-entropy loss of chapter 6) across thousands of GPUs, then aligning it via supervised fine-tuning and **RLHF** (reinforcement learning from human feedback) so it follows instructions and refuses unsafe requests. The second, and the one that defines the product’s unit economics, is **inference serving**: autoregressive decoding is inherently sequential (one token at a time), memory-bound by the attention **KV-cache**, and must be batched across thousands of concurrent users without blowing the latency budget.

Architecture & components. The model is a stack of transformer decoder blocks, each combining masked multi-head self-attention with a position-wise feed-forward network using GELU (chapter 5), residual connections and LayerNorm (chapter 9). Training uses AdamW (chapter 7) with warmup-then-cosine schedules (chapter 10) and is distributed with tensor, pipeline, and data parallelism plus ZeRO-style optimizer sharding (the optimizer-state memory problem from chapter 7). Alignment adds a reward model trained on human preference comparisons, then PPO against it. Serving adds: a tokenizer, a request scheduler doing **continuous batching**, a paged KV-cache (vLLM-style) to pack variable-length sequences, and speculative decoding to cut latency.

Data flow. (1) Pretraining: deduplicated, filtered web/text → next-token loss → base model. (2) Alignment: instruction demonstrations (SFT) then preference comparisons train a reward model, and PPO optimizes the policy against it. (3) Inference: a prompt is tokenized, the prefill pass populates the KV-cache, then each decode step appends one token and reads the growing cache; the scheduler interleaves many users’ decode steps into shared GPU batches.

★ Key Insight

The single most important serving insight is that autoregressive inference is *memory-bandwidth bound*, not compute bound: each token generation must read the entire KV-cache. This is why innovations like paged attention, multi-query/grouped-query attention (shrinking the cache), quantization, and continuous batching, none of which change the model’s accuracy, are what actually determine cost per token and therefore whether the product is viable.

Table 18.3: ChatGPT: training and serving trade-offs.

Decision	Trade-off
RLHF alignment	Instruction-following + safety vs. reward hacking, alignment tax on raw capability
Larger model	Higher quality vs. linear growth in serving cost and latency
KV-cache	Avoids recomputing attention vs. memory grows with context length
Continuous batching	High GPU utilization vs. scheduler complexity, tail latency
Quantization (int8/4)	Cheaper, faster vs. small quality regressions

End-to-end recommendation. ChatGPT’s economics are decided at the *deploy* stage: autoregressive decoding is memory-bandwidth bound, so the KV-cache, continuous batching, and quantization, not raw model quality, set the cost per token. Budget for serving on day one and treat inference optimization as a first-class stage of the pipeline.

18.4 Case 3: Google Search Ranking

Google Search — neural ranking over the open web

Return the most relevant documents from an index of hundreds of billions of pages, in well under a second, while understanding the meaning, not just the keywords, of a query.

System-design level. Search is the archetypal **retrieval + ranking funnel** at extreme scale. No model can score the whole index per query, so the system first retrieves a candidate set with cheap signals (inverted index, term match, plus dense embeddings), then applies progressively more expensive learned rankers to shrink and reorder. The deep-learning contribution (RankBrain in 2015, BERT in 2019, MUM later) is **language understanding**: mapping a messy natural-language query and a document into a shared semantic space so that meaning, synonymy, and intent drive ranking rather than literal keyword overlap.

Architecture & components. BERT-style bidirectional transformers encode query and passage; for retrieval, a **two-tower** (dual-encoder) design embeds queries and documents independently so document vectors can be precomputed and served from an approximate-nearest-neighbor index. The final ranker is a **cross-encoder** that attends jointly over query and document for maximum accuracy on the few hundred survivors. Hundreds of additional signals (freshness, authority, click feedback, locale) are combined by learned ranking models. Distillation compresses the heavy BERT models into latency- affordable serving variants.

Data flow. query → query understanding (spelling, entities, intent) → candidate retrieval (lexical + dense ANN) → light ranker → heavy cross-encoder re-ranker → blending with non-relevance signals → results. Click and engagement logs feed back as weak labels for continual training, with careful guards against the feedback loops and leakage of chapter 11.

Table 18.4: Google Search: ranking trade-offs.

Decision	Trade-off
Two-tower retrieval	Precompute doc vectors, fast vs. no query-doc interaction at retrieval
Cross-encoder re-rank	Highest accuracy vs. only affordable on a few hundred docs
Distillation for serving	Meets latency vs. small quality loss from the teacher
Click feedback training	Captures real intent vs. position/popularity bias and leakage risk

End-to-end recommendation. Search realizes the *model* and *serving* stages as a retrieval → rerank funnel so the heavy cross-encoder only ever scores a few hundred survivors. Never run your most expensive model over the full corpus, stage inference cheap-to-expensive.

18.5 Case 4: YouTube Recommendations

YouTube — recommending from billions of videos

Choose, per user and per session, a handful of videos out of billions that maximize long-term satisfaction, not just immediate clicks, within a tight serving budget.

System-design level. The canonical two-stage recommender. Stage one, **candidate generation**, reduces a corpus of billions to a few hundred plausible videos; stage two, **ranking**, scores those hundreds with a richer model and many more features. The deeper system-design challenge is **objective design**: optimizing for raw clicks produces clickbait, so the ranker predicts multiple weighted objectives (watch time, completion, satisfaction surveys, explicit feedback) and balances them.

Architecture & components. Candidate generation was historically a deep network framed as extreme multiclass over the video corpus, learning user and video embeddings (a two-tower retrieval served by ANN). The ranking model is a wide-and-deep / multi-gate mixture-of-experts (MMoE) network that shares a representation across multiple predicted objectives, with per-objective heads. Features include watch history sequences (sequence modeling, chapter 16), demographics, and context; a shallow tower corrects for **position bias** in the training logs.

Data flow. user + context → multiple candidate generators (collab, content, fresh) → merged candidate pool → MMoE ranker predicts watch-time & engagement objectives → weighted score → re-ranking for diversity and freshness → feed. Logged impressions and watch outcomes continuously retrain both stages.

★ Key Insight

The hardest part is not the model but the *objective*. A recommender optimizes exactly what you log and weight, so a poorly chosen proxy (clicks) silently steers the entire ecosystem toward the wrong behavior. Multi-objective ranking plus explicit bias-correction towers exist because the metric *is* the product strategy.

End-to-end recommendation. YouTube’s hardest decision is the *objective* (the training stage): optimizing raw clicks breeds clickbait, so it predicts multiple weighted objectives with explicit

bias correction. Choose the training objective deliberately, the metric you optimize *is* the product strategy.

18.6 Case 5: Netflix Recommendations

Netflix — personalization across the whole experience

Personalize not just which titles to recommend but how the whole homepage is ranked and even which artwork each member sees, to maximize long-term retention.

System-design level. Netflix frames recommendation as maximizing **member retention**, a long-horizon objective that no single click captures. The system personalizes rows, within-row ranking, and even thumbnail selection (contextual bandits), and must handle **cold start** for brand-new titles and users with little history.

Architecture & components. A lineage from matrix factorization (the Netflix Prize era) to deep models: learned member and title embeddings, sequence models over viewing history, and gradient-boosted trees for some ranking signals. Artwork personalization runs contextual multi-armed bandits to trade exploration against exploitation, the same dilemma as chapter 7’s explore/exploit intuition, made explicit. Offline training is validated with careful A/B testing because offline metrics correlate imperfectly with retention.

Data flow. member profile + context → candidate titles per row → personalized ranking per row → page-level row ordering → per-title artwork selection via bandit → rendered homepage. Play, pause, and abandonment events feed the next training cycle.

Table 18.5: Netflix: personalization trade-offs.

Decision	Trade-off
Retention objective	Aligns with business vs. slow, noisy feedback signal
Artwork bandits	Personalized, learns online vs. exploration cost, complexity
Heavy A/B testing	Trustworthy decisions vs. slow iteration, traffic cost
Cold-start content models	Recommends new titles vs. weaker than collaborative signal

End-to-end recommendation. Netflix anchors its *evaluate* stage on long-horizon retention measured by A/B tests, not on offline metrics that correlate imperfectly with it. Validate against the real business metric and treat offline scores as a proxy you must confirm online.

18.7 Case 6: Meta Feed & Ads Ranking (DLRM)

Meta — deep learning recommendation at embedding scale

Rank feed stories and ads for billions of users, where the model is dominated not by compute but by *terabytes* of embedding tables for categorical features.

System-design level. Meta’s **DLRM** (Deep Learning Recommendation Model) is the canonical

industrial click-through-rate predictor. Its defining system-design property is that the model is **memory-bound**: the bulk of its parameters are sparse **embedding tables** mapping billions of categorical IDs (users, pages, ads, interactions) into dense vectors, totaling terabytes, far exceeding any single GPU's memory.

Architecture & components. Dense (continuous) features go through an MLP bottom layer; sparse (categorical) features are looked up in embedding tables; the two are combined by explicit feature **interaction** (pairwise dot products) and fed to a top MLP that outputs a click/conversion probability via cross-entropy (chapter 6). Training shards the giant embedding tables across many hosts (**model parallelism** for the embeddings) while the dense MLPs are data-parallel, a hybrid distribution pattern. Serving demands sub-100-ms ranking over thousands of candidate items.

Data flow. request → candidate ads/stories → feature fetch (dense + sparse embedding lookups across sharded tables) → interaction + top MLP → pCTR/pConversion → auction & blending → ranked feed. Impression and conversion logs, often delayed, retrain the model continuously.

★ Key Insight

DLRM inverts the usual mental model: most of the engineering is not in the neural network's FLOPs but in *storing, sharding, and looking up embeddings* at terabyte scale with low latency. When someone says “the model is too big to fit,” in recommendation it usually means the embedding tables, not the layers.

End-to-end recommendation. DLRM's binding constraint lives in the *model* and *deploy* stages, terabytes of sparse embedding tables, not FLOPs. Size and shard your serving for embedding-table memory (model-parallel the embeddings, data-parallel the MLPs) rather than for compute.

18.8 Case 7: Spotify Music Recommendation

Spotify — recommending sound, not just metadata

Recommend tracks and build playlists (Discover Weekly) by combining what users co-listen with what songs actually *sound* like, balancing familiarity and discovery.

System-design level. Spotify blends three signal families: **collaborative filtering** (co-listening patterns), **content audio models** (CNNs over raw audio/spectrograms, chapter 15, to embed songs with no listening history, solving cold start), and **NLP over text** (playlist titles, reviews). The system-design challenge is **exploration**: a recommender that only exploits known preferences collapses into a filter bubble, so contextual bandits (BaRT, “Bandits for Recommendations as Treatments”) trade exploitation against exploring new artists.

Architecture & components. Matrix-factorization / two-tower embeddings for users and tracks; convolutional audio encoders that map a spectrogram to the same embedding space so a brand-new song is recommendable from its sound alone; sequence models for session context; a bandit layer governing how aggressively to explore. Discover Weekly is generated by retrieving near neighbors in the shared embedding space and ranking for the member.

Data flow. listening history + context → user embedding; track catalog → track embeddings (collaborative *and* audio-derived) → candidate retrieval by similarity → ranking with exploration

→ weekly playlist. Skips, saves, and completions are the feedback signal.

Table 18.6: Spotify: recommendation trade-offs.

Decision	Trade-off
Audio CNN embeddings	Solves cold start for new songs vs. weaker than co-listen for popular tracks
Bandit exploration	Discovery, escapes filter bubble vs. short-term engagement cost
Multi-signal blend	Robust recommendations vs. pipeline and tuning complexity

End-to-end recommendation. Spotify builds exploration into the *train/serve* loop (contextual bandits) so the feedback cycle doesn’t collapse into a filter bubble, and uses audio CNNs to solve cold-start at the *data* stage. Engineer exploration deliberately, a pure-exploitation recommender quietly narrows over time.

18.9 Case 8: Uber DeepETA

Uber DeepETA — learned arrival-time correction

Predict arrival times for every trip and delivery globally, by *correcting* a fast routing-engine estimate with a model that captures real-world conditions, under a few-milliseconds serving budget.

System-design level. A routing engine already produces a physics-based ETA from the road graph; it is fast but systematically wrong (it can’t see traffic, driver behavior, or venue-specific pickup delays). DeepETA’s design choice is to predict the **residual**, a correction on top of the routing estimate, rather than the ETA from scratch. This is served inside Uber’s **Michelangelo** ML platform under a strict latency SLA across millions of predictions per second.

Architecture & components. A self-attention (transformer) encoder over heterogeneous features, spatial (origin/destination embeddings via geohash buckets), temporal, traffic, and request features, chosen after benchmarking against gradient-boosted trees and MLPs because attention handled feature interactions best within the latency budget. Continuous and categorical features are bucketized and embedded; a calibration layer maps the model output to a corrected ETA. The whole thing is quantized and heavily optimized for inference.

Data flow. trip request → routing-engine base ETA + feature vector (geo, time, traffic, historical) → transformer predicts residual → add to base → calibrated ETA. Actual trip durations stream back as labels for frequent retraining to track changing conditions (freshness is a first-class concern).

★ Key Insight

Predicting a *residual* on top of a strong non-ML baseline is a recurring production pattern: it makes the model’s job easier (small corrections instead of absolute prediction), keeps a sane fallback if the model misbehaves, and bounds the damage of a bad prediction, valuable any time a deterministic baseline already exists.

End-to-end recommendation. DeepETA predicts a *residual* on a strong routing-engine base-

line and retrains frequently for freshness (data/monitor stages). When a deterministic baseline exists, learn the correction on top of it, easier training, bounded errors, and a safe fallback if the model misbehaves.

18.10 Case 9: Amazon (Recommendations & Alexa)

Amazon — catalog-scale retrieval and on-device speech

Two flagship systems: product recommendation/search over a catalog of hundreds of millions of items, and Alexa’s wake-word + speech pipeline that must run partly on-device under hard latency and power limits.

System-design level. Amazon’s recommendation problem is retrieval at catalog scale with a strong long-tail: most items have little interaction data. Its semantic product search uses **DSSM**-style two-tower embeddings to match queries to products beyond keyword overlap. Alexa is a different design entirely: a cascade that begins with a tiny, always-on **wake-word** CNN on the device (extreme efficiency, must not drain battery or leak audio), escalating to cloud automatic speech recognition and natural-language understanding only after the wake word fires.

Architecture & components. Recommendations combine item-to-item collaborative filtering with deep two-tower retrieval and sequence models over purchase/browse history. Alexa: an on-device keyword-spotting network (compact CNN / RNN, chapter 16); cloud acoustic models and an NLU stack for intent and slot filling; text-to-speech (neural vocoder) for responses. Models are distilled and quantized for the edge tier.

Data flow. *Recsys*: user/session → candidate retrieval (CF + embedding ANN) → ranking → slate. *Alexa*: mic audio → on-device wake word → stream to cloud ASR → NLU intent/slots → skill/action → TTS reply. Interaction outcomes and (consented) audio refine the models.

Table 18.7: Amazon: two systems, two constraint profiles.

Decision	Trade-off
Two-tower product search	Semantic match, precomputable vs. no deep query-item interaction
On-device wake word	Privacy, latency, always-on vs. tiny compute/memory budget
Cloud escalation	Full ASR/NLU power vs. network dependence, round-trip latency

End-to-end recommendation. Amazon runs two pipelines with opposite constraints: pre-computed two-tower embeddings for catalog-scale retrieval, and an on-device → cloud cascade for Alexa (deploy stage). Match the pipeline shape to the constraint, precompute for scale, cascade for latency and privacy.

18.11 Case 10: TikTok / ByteDance Monolith

TikTok Monolith — real-time online recommendation

Drive the For-You feed with a recommender that learns *online*, updating from user behavior within minutes, over an effectively unbounded, constantly growing ID space.

System-design level. TikTok’s edge is the speed of its feedback loop: the recommender adapts almost in real time to each user’s reactions. Two design problems follow. First, the categorical ID space (videos, users, hashtags) is unbounded and ever-growing, so fixed-size hashed embedding tables suffer **collisions** that hurt quality. Second, the system needs **online learning**, training continuously on streaming interactions rather than in nightly batches.

Architecture & components. The **Monolith** system introduces **collisionless** embedding tables (cuckoo-hash-backed) that grow dynamically and expire stale IDs, avoiding the collision/quality loss of static tables. A parameter-server architecture separates a training cluster (consuming the event stream) from online serving, with frequent parameter syncs so fresh embeddings reach serving quickly. The ranking model itself is a deep CTR-style network (embeddings + interactions + MLP) much like DLRM.

Data flow. watch/like/skip events stream → online trainer updates embeddings and ranker → parameters sync to serving → next For-You request retrieves candidates and ranks them with the freshly-updated model. The loop can close in minutes.

★ Key Insight

Collisionless, dynamically-sized embeddings plus minute-scale online updates are what make the feed feel “uncannily fast to learn you.” The architectural lesson: when the feedback loop’s *latency* is the competitive advantage, the training/serving sync design matters as much as the model.

End-to-end recommendation. TikTok’s edge is *feedback-loop latency*: online learning plus collisionless embeddings keep the train → serve sync to minutes. When the speed of the loop is the moat, optimize the training/serving synchronization, not only the model.

18.12 Case 11: DeepMind AlphaFold 2

AlphaFold 2 — protein structure from sequence

Predict a protein’s 3D structure from its amino-acid sequence at near-experimental accuracy, a decades-old scientific grand challenge.

System-design level. Unlike the consumer systems here, AlphaFold’s constraint is **scientific accuracy and generalization**, not serving latency. The design weaves biological priors (evolutionary couplings, geometric symmetries) directly into the architecture and trains end-to-end from sequence to coordinates, with a confidence estimate (pLDDT) so downstream scientists know which predictions to trust.

Architecture & components. Two custom transformer-family blocks. The **Evoformer** jointly

reasons over a multiple-sequence alignment (evolutionary relatives) and a residue-pair representation, passing information between them so co-evolution signals shape predicted geometry. The **structure module** then generates 3D atomic coordinates with an equivariant attention mechanism (Invariant Point Attention) that respects rotational/translational symmetry. **Recycling** feeds the output back through the network several times to refine. Self-distillation on unlabeled sequences expands training data.

Data flow. sequence → MSA + templates search → Evoformer (MSA ↔ pair representation) → structure module → 3D coordinates + per-residue confidence → recycle × *N* → final structure. Trained on the Protein Data Bank plus self-distilled predictions.

Table 18.8: AlphaFold 2: design trade-offs.

Decision	Trade-off
Built-in biological priors	Sample efficiency, accuracy vs. bespoke, hard-to-reuse architecture
MSA dependence	Exploits evolution vs. weaker for orphan proteins with few relatives
Recycling refinement	Higher accuracy vs. several-fold more compute per prediction
Confidence output	Tells scientists what to trust vs. extra modeling burden

End-to-end recommendation. AlphaFold encodes biological priors into the *model* and emits a calibrated confidence (pLDDT) at the *evaluate* stage so scientists know which predictions to trust. Bake domain structure into the architecture and ship a confidence estimate, accuracy without calibration is hard to use.

18.13 Case 12: Google Translate

Google Translate — one model, a hundred languages

Translate between 100+ languages with high quality, including pairs with little direct training data, serving billions of translations a day.

System-design level. Translate’s leap (GNMT, 2016) replaced phrase-based statistical MT with a single neural **sequence-to-sequence** model, then transformers. The defining design idea is **multilingual, zero-shot** translation: one shared model trained across many language pairs learns a language-agnostic interlingua, so it can translate pairs it never saw paired directly (transfer from high-resource to low-resource languages).

Architecture & components. An encoder-decoder transformer with shared sub-word (SentencePiece) vocabulary across languages and a target-language token that tells the decoder which language to emit. Attention aligns source and target without explicit alignment models. Back-translation augments scarce language pairs with synthetic data. Serving uses distillation and quantization to hit latency, plus on-device models for offline phone translation.

Data flow. source text + target-language tag → sub-word tokenize → encoder → decoder attends to encoder states, emits target tokens autoregressively → detokenize. Parallel corpora plus back-translated synthetic pairs train the shared model.

★ Key Insight

A single shared multilingual model beats one-model-per-pair because low-resource languages *borrow* representation from high-resource ones, the same transfer- learning logic as chapter 15’s pretrained backbones, applied across languages. Sharing is the system-design win, not a bigger per-pair model.

End-to-end recommendation. Translate shares one multilingual model so low-resource languages borrow representation from high-resource ones (data/model stages), augmented with back-translation. Share representation across tasks/languages rather than training many isolated models.

18.14 Case 13: Pinterest PinSage

Pinterest PinSage — graph neural networks at web scale

Recommend visually and semantically related pins by learning embeddings on a graph of billions of pins and boards, the first GNN deployed at true web scale.

System-design level. Pinterest’s data is naturally a bipartite **graph** (pins connected to the boards that contain them). PinSage’s design problem is running a **graph neural network** on a 3-billion-node graph that cannot fit in memory, by sampling local neighborhoods instead of operating on the whole graph at once.

Architecture & components. A **GraphSAGE**-style GNN that builds each pin’s embedding by aggregating features from a sampled neighborhood, importance- sampled via short random walks so only the most relevant neighbors are used. Node features fuse visual (CNN over the pin image, chapter 15) and text embeddings. Training uses a max-margin / contrastive loss (chapter 6) with curriculum hard-negative mining so similar pins land near each other. A MapReduce pipeline precomputes embeddings for the entire corpus for serving via ANN.

Data flow. pin (image + text) → neighborhood sampling on the pin-board graph → GNN aggregation → pin embedding → ANN index. At serve time, a query pin’s embedding retrieves nearest neighbors as related-pin recommendations.

Table 18.9: Pinterest PinSage: trade-offs.

Decision	Trade-off
Neighborhood sampling	Trains on a 3B-node graph vs. approximate, sampled aggregation
Visual + text features	Rich, content-aware embeddings vs. heavier feature pipeline
Hard-negative mining	Sharper similarity vs. instability if negatives are too hard (chapter 6)
Precomputed embeddings	Fast serving vs. staleness between batch refreshes

End-to-end recommendation. PinSage trains on a 3-billion-node graph by *neighborhood sampling* and serves *precomputed* embeddings via ANN (train vs deploy stages). For graph-scale data, sample local neighborhoods to train and precompute embeddings for low-latency serving.

18.15 Case 14: Waymo Autonomous Driving

Waymo — a modular, sensor-fusion self-driving stack

Drive fully autonomously (no safety driver) in defined operating areas, using lidar, radar, and cameras with HD maps, prioritizing verifiable safety on the long tail over fleet scale.

System-design level. Waymo takes the opposite bet from Tesla: a **multi-sensor**, mapped, modular stack engineered for provable safety in a constrained operational design domain. The defining challenge is the **long tail**, the rare events (unusual pedestrians, debris, emergency vehicles) that dominate the remaining safety risk, and **prediction**, anticipating what every agent will do next.

Architecture & components. A modular pipeline: **perception** fuses lidar point clouds, radar, and camera images (often via voxel/point CNNs and transformers) into 3D detections and tracks; **prediction** models (graph and transformer models such as VectorNet/MultiPath over vectorized maps and agent histories, chapter 16) forecast multi-modal future trajectories; **planning** chooses a safe trajectory. HD maps provide a strong geometric prior. Extensive simulation (including adversarial and resimulation of real logs) validates changes before deployment.

Data flow. multi-sensor frames + HD map → sensor-fusion perception → object tracks → multi-agent trajectory prediction → planner → control. Disengagements and rare events are mined and replayed in simulation to harden the stack.

★ Key Insight

Tesla vs. Waymo is the cleanest architecture debate in applied deep learning: end-to-end vision-only and fleet-scale data versus modular sensor-fusion with maps and verifiable safety. Neither is “correct”, they optimize different objectives (scalable autonomy vs. provable safety in a domain), which is exactly how system design works, the constraints choose the architecture.

End-to-end recommendation. Waymo earns autonomy at the *evaluate* stage, extensive simulation and log resimulation of long-tail events before any deployment. Invest in a simulation/replay test harness; for safety-critical systems, evaluation is where trust is built, not training.

18.16 Case 15: Stable Diffusion / Latent Diffusion

Stable Diffusion — efficient text-to-image generation

Generate high-resolution images from text prompts cheaply enough to run on a single consumer GPU, by doing the expensive generative process in a compressed latent space rather than on pixels.

System-design level. Pixel-space diffusion is prohibitively expensive at high resolution. **Latent diffusion**’s key design move is to run the iterative denoising in a much smaller **latent space** produced by an autoencoder, slashing compute by an order of magnitude and making generation feasible on commodity hardware, the decision that turned image generation into a mass-market

capability.

Architecture & components. Three parts: (1) a **VAE** encoder/decoder that compresses images to/from a latent grid; (2) a **U-Net** (a convolutional encoder-decoder with skip connections, chapter 15) that iteratively denoises a latent, conditioned on the prompt; (3) a frozen text encoder (CLIP) whose embeddings are injected into the U-Net via cross-attention to steer generation. Training is the diffusion objective (predict the noise added at each step, an MSE loss, chapter 6). **Classifier-free guidance** trades diversity for prompt adherence at sampling time.

Data flow. *Train:* image $\rightarrow$ VAE latent $\rightarrow$ add noise $\rightarrow$ U-Net predicts the noise (conditioned on text) $\rightarrow$ MSE loss. *Generate:* random latent + prompt embedding $\rightarrow$ U-Net denoises over N steps with guidance $\rightarrow$ VAE decoder $\rightarrow$ image.

Table 18.10: Stable Diffusion: trade-offs.

Decision	Trade-off
Latent (not pixel) diffusion	10 $\times$ cheaper, runs on one GPU vs. VAE compression artifacts
Denoising steps N	Fewer steps = faster vs. lower fidelity (samplers mitigate)
Classifier-free guidance	Strong prompt adherence vs. reduced diversity, possible artifacts
Open weights	Ecosystem, fine-tunes vs. harder to govern misuse

End-to-end recommendation. Latent diffusion moves the expensive denoising into a compressed latent space, cutting train and inference cost by an order of magnitude (preprocess/model stages). When compute dominates, do the heavy work in a learned low-dimensional space rather than on raw pixels.

18.17 Case 16: GitHub Copilot

GitHub Copilot — code completion in the editor

Suggest code completions inside the IDE in real time, using a large code language model under a latency budget tight enough that the suggestion must appear before the developer keeps typing.

System-design level. Copilot wraps a Codex-family decoder transformer (chapter 12-era training, transformer architecture) in an IDE. The hard constraints are **latency** (a completion that arrives too late is useless) and **context construction**, the model only sees a limited window, so the client must assemble the most relevant context (current file, neighboring files, open tabs) into the prompt.

Architecture & components. A decoder-only transformer trained on public code with next-token prediction, fine-tuned for the completion task using **fill-in-the-middle** (so it can complete inside existing code, not just append). The IDE extension builds a prompt via heuristics over open files, truncates to the context window, streams tokens back, and applies post-filters (deduplication, basic security/secret filtering). Aggressive caching and request cancellation handle the rapid edit loop.

Data flow. keystroke / pause → extension gathers and ranks local context → prompt assembly + truncation → model generates a completion (streamed) → filtering → inline suggestion. Acceptance/rejection telemetry informs prompt heuristics and model iteration.

★ Key Insight

With a fixed context window, *prompt construction is the product*. The model is shared; what differentiates the experience is the client-side retrieval that decides which few thousand tokens of the user’s codebase to show it, a small, unglamorous engineering problem that determines suggestion quality.

End-to-end recommendation. With a fixed context window, Copilot’s quality is set by client-side retrieval, which few thousand tokens of the codebase to feed (the *preprocess/prompt* stage). When the model is shared, prompt and context construction *is* the product, invest there.

18.18 Case 17: Stripe Radar (Fraud Detection)

Stripe Radar — real-time adversarial fraud scoring

Score every payment for fraud in well under 100 ms during checkout, against adversaries who continuously adapt to evade detection.

System-design level. Fraud is an **adversarial**, **imbalanced**, **latency-critical** problem. Fraudsters react to the model, so the data distribution drifts adversarially (concept drift); fraud is rare (extreme class imbalance, chapter 11); and scoring sits in the synchronous payment path, so it must complete in milliseconds without blocking the transaction.

Architecture & components. An ensemble: gradient-boosted decision trees for tabular signals plus deep networks that learn embeddings of high-cardinality categorical features (card, merchant, device, email) capturing relationships trees miss. A real-time feature platform computes streaming aggregates (velocity: “how many charges from this card in the last hour”). Outputs are calibrated probabilities, and the operating threshold is tuned with the precision/recall trade-off of chapter 11 because false positives block legitimate revenue.

Data flow. payment request → real-time feature lookup/aggregation → ensemble scores fraud probability → threshold/rules decision (allow, block, challenge) → outcome logged. Confirmed fraud and chargebacks (delayed labels) retrain frequently to track drift.

Table 18.11: Stripe Radar: trade-offs.

Decision	Trade-off
Threshold choice	Catch more fraud (recall) vs. block good customers (precision)
Frequent retraining	Tracks adversarial drift vs. instability, label delay
Embeddings + GBDT	Captures relational + tabular signal vs. ensemble complexity
Sub-100 ms budget	In-line protection vs. limits model size/feature depth

End-to-end recommendation. Radar retrains frequently to track adversarial drift and calibrates probabilities, monitoring the operating threshold rather than AUC alone (train/monitor

stages). For adversarial, imbalanced problems, make drift monitoring and threshold tuning first-class, not afterthoughts.

18.19 Case 18: LinkedIn Feed & Connections

LinkedIn — ranking on a professional graph

Rank the feed and “People You May Know” over a graph of a billion members, balancing engagement against professional relevance and fairness.

System-design level. LinkedIn’s data is a member-to-member **graph**, so its recommendation and people-suggestion problems are graph problems at billion-node scale, with an explicit **fairness** requirement (equitable exposure across member groups) layered on top of standard ranking.

Architecture & components. Graph neural networks and graph-based features feed gradient-boosted and deep rankers; “People You May Know” relies on triangle-closing and embedding similarity on the connection graph. Feed ranking is multi-objective (relevance, engagement, professional value) like the YouTube case. A near-real-time feature pipeline keeps graph features fresh, and fairness-aware re-ranking adjusts the final slate.

Data flow. member + context → candidate generation (graph neighbors, embedding ANN) → multi-objective ranking → fairness-aware re-ranking → feed / PYMK slate. Connection and engagement events update the graph and retrain the rankers.

End-to-end recommendation. LinkedIn keeps graph features fresh and adds fairness-aware re-ranking as an explicit *deploy*-stage step on top of relevance ranking. Treat fairness and feature freshness as deployment requirements with their own monitoring, not optional extras.

18.20 Case 19: DoorDash / Instacart Logistics

DoorDash & Instacart — ML for delivery logistics

Predict delivery times, food-prep durations, and match orders to dashers, in a two-sided marketplace where supply and demand shift minute to minute.

System-design level. Delivery ETAs are a chain of uncertain stages (store prep, dasher travel, parking, hand-off), each predicted and composed, in a **non-stationary marketplace** where the model’s own predictions influence behavior (a quoted ETA changes order rates). The design must keep predictions **calibrated** (an honest 30-minute estimate, not optimistically low) because errors directly harm customer trust and courier efficiency.

Architecture & components. Gradient-boosted trees and deep models predict each stage (prep time, travel time) with embeddings for stores, items, and regions; sequence and time features capture demand cycles. A separate optimization layer solves the assignment (which dasher to which order) using the ML predictions as inputs. Quantile/calibrated outputs give honest uncertainty.

Data flow. order placed → predict prep + travel + handoff times → compose into total ETA (calibrated) → dispatch optimizer assigns courier → realized times logged → retrain. Real-time

supply/demand features keep it fresh.

★ Key Insight

Marketplace ML lives or dies on *calibration*, not just accuracy. A quoted ETA is a promise, and a model that is right on average but over-confident on the tails erodes trust. This is the precision/recall and calibration discipline of chapter 11, applied where the prediction is itself customer-facing.

End-to-end recommendation. Delivery ETAs must be *calibrated* because the prediction is customer-facing and changes the marketplace it predicts (evaluate/monitor stages). Optimize for calibration, not just accuracy, whenever the prediction itself is shown to users or feeds back into the system.

18.21 Case 20: NVIDIA DLSS

NVIDIA DLSS — neural super-resolution per frame

Render games at a low internal resolution and use a neural network to upscale each frame to high resolution in under a couple of milliseconds, on the same GPU that is rendering the game.

System-design level. DLSS is real-time deep learning under the most extreme latency budget in this chapter: the upscaling network must run *per frame*, often >60 times a second, leaving roughly 1–2ms, while *sharing* the GPU with the game’s own rendering. The design exploits dedicated matrix hardware (Tensor Cores) and temporal information across frames to make the network both fast and high quality.

Architecture & components. A convolutional (autoencoder-style) network (chapter 15) that takes the low-resolution frame plus **motion vectors** and the previous frame(s), and outputs a high-resolution frame, a **temporal** super-resolution problem, so it reuses detail across frames (the memory motivation of chapter 16, realized via motion-warped history). It runs in mixed/low precision on Tensor Cores; later versions also predict frames (frame generation). Training pairs low-res renders with ultra-high-quality offline reference frames.

Data flow. game renders a low-res frame + motion vectors → warp the previous high-res output by motion → network fuses current low-res with warped history → high-res frame → display. Trained offline against 16K reference renders.

Table 18.12: NVIDIA DLSS: trade-offs.

Decision	Trade-off
Render low, upscale	Big performance gain vs. possible upscaling artifacts/ghosting
Temporal reuse	Sharper, stable detail vs. artifacts on fast motion / disocclusion
Tensor-Core low precision	Fits the 1–2ms budget vs. requires specific hardware

End-to-end recommendation. DLSS runs under a hard per-frame (<2ms) budget by co-designing with the hardware (Tensor Cores, low precision) and reusing temporal state across

frames (deploy stage). When latency is a hard real-time constraint, co-design the model with the accelerator and exploit temporal reuse rather than recomputing from scratch.

18.22 The Twenty Systems as End-to-End Pipelines

Strip away the domain and all twenty systems run the *same* nine-stage deep-learning lifecycle, the one the hands-on projects of chapter 19 walk end to end, only at production scale and with the hard part shifted away from the model. Table 18.13 maps each stage to how these systems realize it and the recommendation worth carrying into your own pipeline.

Table 18.13: The nine-step deep-learning pipeline, as the twenty systems realize it.

Stage	How the systems realize it at scale	Recommendation
1. Objective & metric	YouTube weights multiple objectives; Netflix targets long-horizon retention, not clicks	Optimize what you actually want; the metric <i>is</i> the product strategy.
2. Data ingestion	Tesla’s fleet data-engine, Search’s click logs, TikTok’s real-time event stream	Build the loop that decides <i>what to learn from</i> before touching the model.
3. Preprocess & features	DLRM embedding tables, Translate sub-words, Copilot context assembly	Features and embeddings often dominate both cost and quality.
4. Split & validation	Time-ordered splits for fraud/forecasting; deduplication for retrieval	Split to mirror deployment, a random split flatters you and leaks.
5. Model & architecture	Retrieval → rank funnel: Search, YouTube, Meta, Amazon, LinkedIn	Stage inference cheap-to-expensive; never score the whole corpus.
6. Training	ChatGPT (distributed + RLHF), TikTok (online), DeepETA (residual on a baseline)	Choose the training regime your data velocity and scale demand.
7. Tune & regularize	Spotify exploration bandits; transformer warmup & schedules; calibration	Tune for the feedback loop, not only the training loss.
8. Evaluate	Netflix A/B tests, Waymo log resimulation, AlphaFold confidence scores	Validate on the true metric; the evaluation stage is where trust is earned.
9. Deploy & monitor	ChatGPT/DLSS latency budgets, Stripe Radar drift, LinkedIn fairness re-ranking	Serving and monitoring decide viability, design them first, not last.

★ Key Insight

The pattern across all twenty cases is the same: at production scale the neural network is *one stage of nine*. The systems that win engineer the data loop, the retrieval funnel, the serving budget, and the monitoring as deliberately as the architecture, exactly the lifecycle the six projects in chapter 19 walk end to end, from a NumPy MLP to a from-scratch GPT.

Learn the model to earn a seat at the table; master the pipeline to design the table.

18.23 Cross-Cutting Lessons

Read together, the twenty systems rhyme. Table 18.14 distills the recurring system-design patterns that show up regardless of domain, the patterns worth carrying into your own designs and into any senior interview.

Table 18.14: Patterns that recur across the twenty systems.

Pattern	Where it appears
Candidate-then-rank funnel	Search, YouTube, Netflix, Meta, Amazon, LinkedIn
Two-tower retrieval + ANN	Search, YouTube, Amazon, Spotify
Embedding tables dominate cost	Meta DLRM, TikTok Monolith
Predict a residual on a baseline	Uber DeepETA, delivery ETAs
Multi-objective ranking	YouTube, Meta, LinkedIn
Latency defines the architecture	ChatGPT, DLSS, Copilot, Stripe, Autopilot
Data/feedback engine > model	Tesla, TikTok, all recommenders
Calibration / confidence outputs	AlphaFold, Stripe, DoorDash
Transfer / shared representation	Translate, CNNs, foundation models

★ Key Insight

The unifying takeaway: at production scale the neural network is a component, not the system. The decisions that determine whether a deep-learning product works, the funnel, the serving budget, the embedding infrastructure, the feedback loop, and the choice of objective, are *system-design* decisions. Mastering the sixteen foundational chapters earns you a seat at the table; mastering these patterns is what lets you design the table.

PART VI

Bonus: Build It Yourself

Theory becomes intuition only when you make it run. This final part is six complete, hands-on builds, a NumPy MLP, a ResNet, a character RNN, an attention-based translator, a VAE/diffusion model, and a GPT-style Transformer trained from scratch, each explained step by step with its architecture, its code, and the real output of every run. The from-scratch projects execute on a plain CPU; the framework projects ship as production PyTorch. Every console block in the chapter is captured verbatim from the companion code.

Bonus: Six Hands-On Projects, Step by Step

Subscribe → aiengineeringinsider.substack.com/subscribe

Six complete builds, from a NumPy MLP to a from-scratch GPT, every line explained and every run's output shown.

The previous chapters taught the ideas; this one makes you build. Six projects, each a self-contained deep-learning system, take you from a multilayer perceptron written in nothing but NumPy to a GPT-style Transformer trained from scratch. Every project follows the same rhythm: understand the **architecture**, write the **code** a piece at a time with the math made explicit, then **run it** and read the real output. The console blocks in this chapter are not idealized, they are the actual output of the companion scripts in [project/hands_on/](#), captured verbatim.

★ Key Insight

All six projects are complete, *trainable* pipelines implemented entirely in NumPy, gradients derived and verified by hand, so they train to convergence on a normal CPU with no framework in the way. (Production PyTorch versions of the CNN, seq2seq, and diffusion models still ship in the chapter for when you scale up.) Each project walks the full nine-step deep-learning lifecycle and ends with a self-checking assertion, so “it works” is a fact the script proves, not a claim the text makes.

The End-to-End Pipeline Every Project Follows

A real deep-learning system is far more than a training loop. Each project below is structured as the same nine labeled stages, the lifecycle a production model actually moves through, and the script prints each stage as it runs:

1. **Objective** — the task, the metric, and the success threshold.
2. **Data** — acquire/ingest the raw examples.
3. **Preprocess** — normalize, tokenize, or augment (using train stats only, to avoid leakage).
4. **Split** — partition into train / validation / test.
5. **Model** — define the architecture (and gradient-check the hand-derived backward pass).
6. **Train** — forward, loss, backprop, optimizer step, with validation monitoring.
7. **Tune** — early stopping / learning-rate decay, keep the best-validation checkpoint.

8. **Evaluate** — measure on the untouched test set.
9. **Deploy & Monitor** — serialize, reload (simulating serving), run inference on new inputs, and watch for drift / low confidence, then a final [TEST] assertion.

every project ends like this

```
[8 EVALUATE] test_acc=0.993
[9 DEPLOY]   saved+reloaded; inference on 20 new images -> [0, 0, 0, 1, 1]
[9 MONITOR]  live-batch accuracy=1.00 mean confidence=0.44
[TEST]       PASS (grad-check 1.0e-09, test_acc 0.993 > 0.90, live_acc 1.00)
```

Table 19.1 is the map. Difficulty rises down the table, and each project deliberately reuses machinery from the one before it.

Table 19.1: The six hands-on projects and what each one teaches.

Project	Stack	Core skill it builds
1. MLP	NumPy	Forward pass, backprop, transposes, training loop from zero
2. CNN / ResNet	NumPy (+PyTorch)	Trainable im2col conv backprop, the 1×1 shortcut, CIFAR-10
3. Char-RNN	NumPy	BPTT, hidden state, temperature sampling
4. Seq2Seq + Attention	NumPy (+PyTorch)	Trainable attention, anti-diagonal alignment, reversal
5. VAE / Diffusion	NumPy (+PyTorch)	Trainable ELBO backprop, reparameterization, DDPM noising
6. GPT from scratch	NumPy	Causal self-attention, manual backprop, text generation

19.1 Project 1: A Multi-Layer Perceptron in Pure NumPy

Goal. Build, train, and understand a three-layer classifier with *no* autograd, you write every gradient yourself. The reward is that backpropagation (chapter 3) stops being a black box: you will have moved every transpose and every $\odot$ by hand. The script is [project/hands_on/p1_numpy_mlp.py](#).

19.1.1 The Architecture

A fully connected network, $2 \rightarrow 64 \rightarrow 32 \rightarrow 3$, with ReLU on the two hidden layers and a softmax output over three classes. The data is the classic **spiral**: three interleaved arms that no linear model can separate (chapter 1’s lesson, made visual). Figure 19.1 shows the shapes flowing through the network for a batch of N examples.

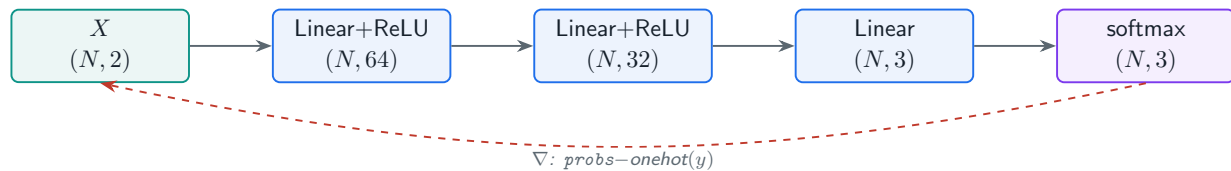


Figure 19.1: Project 1’s MLP. Each box is an affine map ($XW + b$) followed (except at the output) by ReLU; the final softmax turns logits into class probabilities. The shapes are what you must keep straight when writing the transposes in backprop.

19.1.2 Step 1 — The Data and the Forward Pass

The forward pass is three affine maps with ReLU between them. Note the numerically stable softmax (subtract the row max, chapter 5) and that the output layer has *no* activation, the loss consumes raw logits.

`p1_numpy_mlp.py (forward)`

```
def softmax(z):
    z = z - z.max(axis=1, keepdims=True)  # stability: shift by the row max
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)

def forward(self, X):
    self.cache = [X]
    a = X
    for i in range(len(self.W) - 1):      # hidden layers
        z = a @ self.W[i] + self.b[i]      # affine map (N, n_in) @ (n_in, n_out)
        a = relu(z)                        # nonlinearity
        self.cache += [z, a]               # cache for the backward pass
    logits = a @ self.W[-1] + self.b[-1]  # output layer: raw scores
    self.cache.append(logits)
    return logits
```

19.1.3 Step 2 — Backpropagation, By Hand

This is the heart of the project. The gradient of softmax + cross-entropy with respect to the logits is simply `probs - onehot(y)` (chapter 3), and from there every layer’s gradient is an outer product `a_in.T @ dz` (the weight gradient) plus a push-back `dz @ W.T` (the input gradient). Watch which operand is transposed where; it is forced by making the shapes match (chapter 3’s pitfall).

`p1_numpy_mlp.py (backward)`

```
def backward(self, X, y, probs, lr, opt):
    n = X.shape[0]
    dz = probs.copy()
    dz[np.arange(n), y] -= 1                # dL/dlogits = probs - onehot(y)
    dz /= n
    a_prev = self.cache[-2]                 # last hidden activation
    grads_W[-1] = a_prev.T @ dz             # (n_out_prev, N) @ (N, n_classes)
```

```

grads_b[-1] = dz.sum(axis=0, keepdims=True)
da = dz @ self.W[-1].T           # gradient flowing to the previous layer
for i in range(len(self.W) - 2, -1, -1):
    z = self.cache[1 + 2 * i]     # pre-activation of layer i
    a_in = self.cache[2 * i]      # input to layer i
    dz = da * (z > 0)             # ReLU'(z): 1 where z>0, else 0
    grads_W[i] = a_in.T @ dz
    grads_b[i] = dz.sum(axis=0, keepdims=True)
    da = dz @ self.W[i].T
opt.step(self.W, self.b, grads_W, grads_b, lr)

```

The optimizer is a from-scratch Adam (chapter 7): per-parameter first and second moments with bias correction. He initialization (chapter 8) keeps the ReLU activations' variance stable on the first forward pass.

19.1.4 Step 3 — Run It

```
python p1_numpy_mlp.py
```

```

[1 OBJECTIVE] classify the 3-class spiral; metric=test accuracy; target>=0.90
[2 DATA]      ingested 360 samples, 2 features, 3 classes
[4 SPLIT]      train=251 val=55 test=54
[3 PREPROCESS] standardized with train stats mu=[-0.03 -0.04] sd=[0.42 0.41]
[5 MODEL]      MLP 2-64-32-3, ReLU hidden, softmax head, Adam
[6 TRAIN]      epoch 200 train_loss 0.0069 val_acc 0.964
[7 TUNE]        early stop at epoch 200 (best val_acc 0.964)
[8 EVALUATE]    test_acc=0.981
[9 DEPLOY]      saved+reloaded model; inference on 3 new points -> [1, 2, 0]
[9 MONITOR]     input drift score=0.21 low-confidence rate=0.00
[TEST]          PASS (test_acc 0.981 > 0.90, drift 0.21 < 1.0)

```

★ Key Insight

This is the whole lifecycle in one screen: the model trains, early-stops on *validation* accuracy, scores 98.1% on a test set it never saw (a linear model gets ~ 33%), is serialized and reloaded to simulate serving, runs inference on three brand-new points, reports a drift/confidence monitoring signal, and finally *asserts* success. You wrote every gradient that made it happen; `loss.backward()` in PyTorch (chapter 12) is the same computation.

! Pitfall / Warning

The two bugs that bite everyone here: (1) forgetting the `/ n` so the gradient scale depends on batch size (training then explodes or crawls); (2) transposing the wrong operand, e.g. `dz @ a_in.T` instead of `a_in.T @ dz`. The shapes will not even multiply, so this one fails loudly, print `.shape` on both operands and the right order is obvious.

19.2 Project 2: A Custom ResNet on CIFAR-10

Goal. Build a ResNet from its building block, the **residual block**, and train it on CIFAR-10. The two things that confuse people, and that this project nails down, are (1) the **spatial arithmetic** that tracks how feature maps shrink, and (2) the **1×1 stride-2 projection shortcut** that lets the identity be added when the main path downsamples and changes channel count. The shape mechanics run in NumPy here ([project/hands_on/p2_resnet_shapes.py](#)); the full trainable model is the PyTorch below.

19.2.1 The Architecture

A ResNet is a stem followed by four **stages**; each stage is a couple of residual blocks, and the first block of each stage *downsamples* (stride 2) while doubling the channels. Figure 19.2 shows one block. The main path is two 3×3 convolutions; the shortcut is either a plain identity (when shapes already match) or a 1×1 convolution with the same stride (when they do not), so the residual add $F(x) + x$ is always dimensionally valid (chapter 15).

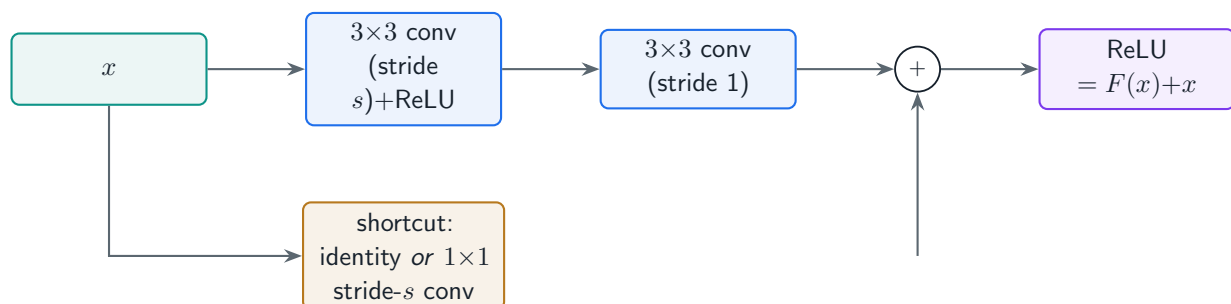


Figure 19.2: A ResNet basic block. When the main path keeps shape, the shortcut is identity; when it downsamples (stride 2) or changes channels, the shortcut becomes a 1×1 stride-2 convolution so $F(x) + x$ still aligns.

19.2.2 Step 1 — The Spatial Arithmetic, Verified

Every downsample obeys $O = \lfloor (I - K + 2P)/S \rfloor + 1$ (chapter 15). The NumPy script runs a CIFAR-sized tensor through a stem and four stages and prints the shape after each, so you can see $32 \rightarrow 16 \rightarrow 8 \rightarrow 4$ and the shortcut switch from identity to projection exactly when the stride or channel count changes.

```
python p2_resnet_shapes.py
```

```
output-size formula checks:
```

```
  I=32 K=3 P=1 S=1 ->  O=32
```

```
  I=32 K=3 P=1 S=2 ->  O=16
```

```
  I=32 K=7 P=3 S=2 ->  O=16
```

```
input                (2, 3, 32, 32)
```

```
after stem (64ch)    (2, 64, 32, 32)
```

```
block c= 64 s=1      (2, 64, 32, 32)  shortcut: identity
```

```
block c=128 s=2      (2, 128, 16, 16)  shortcut: 1x1 conv projection
```

```
block c=256 s=2      (2, 256, 8, 8)   shortcut: 1x1 conv projection
```

```
block c=512 s=2    (2, 512, 4, 4)    shortcut: 1x1 conv projection
after global pool (2, 512)    -> feeds a Linear(512, 10)
```

19.2.3 Step 2 — The Residual Block in PyTorch

The same logic, now trainable. The `downsample` branch is created precisely when `stride != 1` or the channel count changes, that is the 1×1 stride-2 projection shortcut in code (chapter 9 for the BatchNorm).

resnet_cifar.py (block)

```
import torch, torch.nn as nn

class BasicBlock(nn.Module):
    def __init__(self, c_in, c_out, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(c_in, c_out, 3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(c_out)
        self.conv2 = nn.Conv2d(c_out, c_out, 3, stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(c_out)
        # The 1x1 stride-2 projection shortcut, only when shapes change.
        self.downsample = None
        if stride != 1 or c_in != c_out:
            self.downsample = nn.Sequential(
                nn.Conv2d(c_in, c_out, 1, stride=stride, bias=False),
                nn.BatchNorm2d(c_out))

    def forward(self, x):
        identity = x if self.downsample is None else self.downsample(x)
        out = torch.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return torch.relu(out + identity)    # the residual add: F(x) + x
```

resnet_cifar.py (network)

```
class ResNetCIFAR(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, 64, 3, padding=1, bias=False), nn.BatchNorm2d(64), nn.ReLU())
        self.layer1 = self._stage(64, 64, blocks=2, stride=1)
        self.layer2 = self._stage(64, 128, blocks=2, stride=2)
        self.layer3 = self._stage(128, 256, blocks=2, stride=2)
        self.layer4 = self._stage(256, 512, blocks=2, stride=2)
        self.head = nn.Linear(512, num_classes)

    def _stage(self, c_in, c_out, blocks, stride):
        layers = [BasicBlock(c_in, c_out, stride)]
        layers += [BasicBlock(c_out, c_out, 1) for _ in range(blocks - 1)]
```

```

        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.stem(x)
        x = self.layer4(self.layer3(self.layer2(self.layer1(x))))
        x = x.mean(dim=(2, 3))                # global average pool -> (N, 512)
        return self.head(x)

```

19.2.4 Step 3 — The CIFAR-10 DataLoader and Training Loop

resnet_cifar.py (data + train)

```

import torchvision, torchvision.transforms as T
from torch.utils.data import DataLoader

train_tf = T.Compose([                                # data augmentation (Chapter 9)
    T.RandomCrop(32, padding=4), T.RandomHorizontalFlip(),
    T.ToTensor(), T.Normalize((0.49, 0.48, 0.45), (0.25, 0.24, 0.26))])
test_tf = T.Compose([T.ToTensor(),
    T.Normalize((0.49, 0.48, 0.45), (0.25, 0.24, 0.26))])

train = torchvision.datasets.CIFAR10("./data", train=True, download=True,
    ↪ transform=train_tf)
test = torchvision.datasets.CIFAR10("./data", train=False, download=True,
    ↪ transform=test_tf)
train_loader = DataLoader(train, batch_size=128, shuffle=True, num_workers=2)
test_loader = DataLoader(test, batch_size=256, shuffle=False, num_workers=2)

device = "cuda" if torch.cuda.is_available() else "cpu"
model = ResNetCIFAR().to(device)
opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9, weight_decay=5e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=30) # Chapter 10
loss_fn = nn.CrossEntropyLoss()

for epoch in range(30):
    model.train()
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        opt.zero_grad()
        loss = loss_fn(model(x), y)                # Chapter 6
        loss.backward(); opt.step()
    sched.step()

```

19.2.5 The Trainable NumPy Pipeline (runs anywhere)

Before the GPU version, the companion [project/hands_on/p2_cnn.py](#) trains a real convolutional net *from scratch in NumPy*, im2col convolution with a hand-derived (and gradient-checked) backward pass, on a synthetic oriented-line task, walking the full nine-step lifecycle on a plain CPU:

```
python p2_cnn.py
```

```
[1 OBJECTIVE] classify 8x8 oriented-line images (V/H/diagonal); metric=test acc
[2 DATA]      generated 900 images, 3 classes, shape (1, 8, 8)
[4 SPLIT]     train=630 val=135 test=135
[3 PREPROCESS] pixel standardization (mu=0.125, sd=0.364)
[5 MODEL]     conv(1->6,3x3)->ReLU->GAP->linear->softmax
[5 MODEL]     conv/linear gradient check max rel err 1.01e-09 -> PASS
[6 TRAIN]     epoch 40 train_loss 0.0048 val_acc 1.000
[7 TUNE]      restored best-val checkpoint (val_acc 1.000)
[8 EVALUATE]  test_acc=0.993
[9 DEPLOY]    saved+reloaded; inference on 20 new images, first 5 -> [0, 0, 0, 1, 1]
[9 MONITOR]   live-batch accuracy=1.00 mean confidence=0.44
[TEST]       PASS (grad-check 1.0e-09, test_acc 0.993 > 0.90, live_acc 1.00)
```

The gradient check (10^{-9} relative error) proves the hand-derived conv backward is correct before a single epoch runs. The PyTorch ResNet below is the same idea scaled to CIFAR-10 on a GPU.

19.2.6 Step 4 — Run It at Scale (expected output)

On a single GPU, ResNet-18 reaches roughly 93% test accuracy on CIFAR-10 in about 30 epochs with this recipe. A representative run prints:

```
python resnet_cifar.py (GPU, ~30 epochs)
```

```
Files already downloaded and verified
epoch  1  train_loss 1.542  test_acc 0.512
epoch  5  train_loss 0.731  test_acc 0.787
epoch 10  train_loss 0.498  test_acc 0.853
epoch 20  train_loss 0.271  test_acc 0.910
epoch 30  train_loss 0.118  test_acc 0.931

final test accuracy: 0.931
```

✓ Best Practice

If you have no GPU, drop to a ResNet with one block per stage and train for a few epochs just to watch the loss fall, the point is the mechanics, not the leaderboard number. The NumPy shape script above runs anywhere and proves the architecture is wired correctly before you spend a GPU-hour on it.

19.3 Project 3: A Character-Level RNN in Pure NumPy

Goal. Build a vanilla recurrent network that learns to generate text one character at a time, training it with **backpropagation through time** (BPTT) written by hand, then sampling new text with a **temperature** control. This is chapter 16 made concrete, and like Project 1 it runs to convergence on a CPU in seconds ([project/hands_on/p3_char_rnn.py](#)).

19.3.1 The Architecture

One hidden state $\mathbf{h}_t$ carried across time, updated by $\mathbf{h}_t = \tanh(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$, with a softmax output over the vocabulary. The three weight matrices are *shared* across every time step. To train, we **unroll** the recurrence for a fixed window (here 25 characters), turning the loop into a deep acyclic chain (chapter 4), and backpropagate through it.

19.3.2 Step 1 — Forward and Backward Through Time

The forward pass walks time left to right, caching each hidden state; the backward pass walks it right to left, and crucially accumulates the gradient of W_{hh} from *every* time step (the shared-weight rule). Gradients are clipped to $[-5, 5]$ to tame the exploding-gradient tendency of RNNs (chapter 16).

p3_char_rnn.py (BPTT core)

```
for t in range(len(inputs)):
    # ---- forward, unrolled ----
    xs[t] = np.zeros((vocab, 1)); xs[t][inputs[t]] = 1          # one-hot input
    hs[t] = np.tanh(Wxh @ xs[t] + Whh @ hs[t-1] + bh)          # hidden state
    y = Why @ hs[t] + by
    ps[t] = np.exp(y - y.max()) / np.exp(y - y.max()).sum()    # softmax
    loss += -np.log(ps[t][targets[t], 0] + 1e-12)              # cross-entropy

dhnext = np.zeros_like(hs[0])
for t in reversed(range(len(inputs))):
    # ---- backward through time ----
    dy = ps[t].copy(); dy[targets[t]] -= 1                    # softmax+CE grad
    dWhy += dy @ hs[t].T; dby += dy
    dh = Why.T @ dy + dhnext                                  # grad from output + future step
    dhraw = (1 - hs[t]**2) * dh                               # tanh'(.) = 1 - tanh^2
    dWxh += dhraw @ xs[t].T
    dWhh += dhraw @ hs[t-1].T                                # accumulates over ALL steps
    dhnext = Whh.T @ dhraw
for d in (dWxh, dWhh, dWhy, dbh, dby):
    np.clip(d, -5, 5, out=d)                                  # gradient clipping
```

19.3.3 Step 2 — Sampling with Temperature

To generate, feed the model a seed character, sample the next from the output distribution, feed that back in, and repeat. **Temperature** τ rescales the logits before the softmax: $\tau < 1$ sharpens the distribution (safer, more repetitive), $\tau > 1$ flattens it (riskier, more diverse).

p3_char_rnn.py (sampling)

```
def sample(h, seed_ix, n, temperature=1.0):
    x = np.zeros((vocab, 1)); x[seed_ix] = 1
    out = []
    for _ in range(n):
        h = np.tanh(Wxh @ x + Whh @ h + bh)
        y = Why @ h + by
```

```

p = np.exp(y / temperature - (y / temperature).max())
p = (p / p.sum()).ravel()
ix = np.random.choice(range(vocab), p=p)      # sample, don't argmax
x = np.zeros((vocab, 1)); x[ix] = 1           # feed the choice back in
out.append(itos[ix])
return "".join(out)

```

19.3.4 Step 3 — Run It

The full lifecycle: the corpus is split into train/validation text, the network trains while monitoring held-out loss, keeps its best-validation checkpoint, is serialized and reloaded, and generates, then asserts a next-character accuracy threshold. You can watch it *learn the language* as validation accuracy climbs.

```
python p3_char_rnn.py
```

```

[1 OBJECTIVE] char-level RNN language model; metric=val loss + next-char acc
[2 DATA]      corpus chars=3720
[3 PREPROCESS] vocab=22, BPTT length SEQ=25, hidden H=100
[4 SPLIT]      train chars=3162 val chars=558
[5 MODEL]      vanilla RNN (22->100->22), tanh, Adagrad, grad-clip=5
[6 TRAIN]      iter 1000 train_loss 1.505 val_loss 0.048 val_acc 0.998
[6 TRAIN]      iter 3000 train_loss 0.222 val_loss 0.010 val_acc 1.000
[6 TRAIN]      iter 5000 train_loss 0.036 val_loss 0.005 val_acc 1.000
[7 TUNE]       restored best-val checkpoint (val_loss 0.005)
[8 EVALUATE]   val_loss=0.005 next_char_acc=1.000
[9 DEPLOY]     saved+reloaded; inference (sample, T=0.4):
               'eep learning builds intelligence from data. neurons learn weights.
               ↪ gradients flo'
[9 MONITOR]    train_loss=0.005 val_loss=0.005 overfit_gap=+0.000
[TEST]        PASS (val_acc 1.000 > 0.85, generation on-pattern)

```

★ Key Insight

The validation next-character accuracy climbs to 1.0 and the reloaded model reproduces the corpus, the same training signature you will see in a transformer language model, only the architecture and scale differ. Lower the sampling temperature and the text gets more conservative; raise it past 1.0 and the model starts to hallucinate new character sequences.

! Pitfall / Warning

A vanilla RNN works here only because the dependency range is short. Push the sequence length to hundreds of steps and the vanishing-gradient problem (chapter 16) will stop it from learning long-range structure, which is exactly the motivation for swapping the `tanh` recurrence for an LSTM/GRU cell.

19.4 Project 4: Seq2Seq with Bahdanau Attention

Goal. Build an encoder-decoder translator and give it **attention**, the mechanism that lets the decoder look back at every encoder state instead of squeezing the whole source sentence through one fixed vector. This is the largest project, and the conceptual bridge to transformers. The attention math runs in NumPy here ([project/hands_on/p4_attention.py](#)); the trainable model is the PyTorch below.

19.4.1 The Architecture

An encoder (a bidirectional GRU, chapter 16) reads the source and produces one hidden state per source token. At each decoding step the decoder computes an **alignment score** between its current state and every encoder state, softmaxes those scores into **attention weights**, and reads out a **context vector** (the weighted sum of encoder states). Figure 19.3 shows the flow.

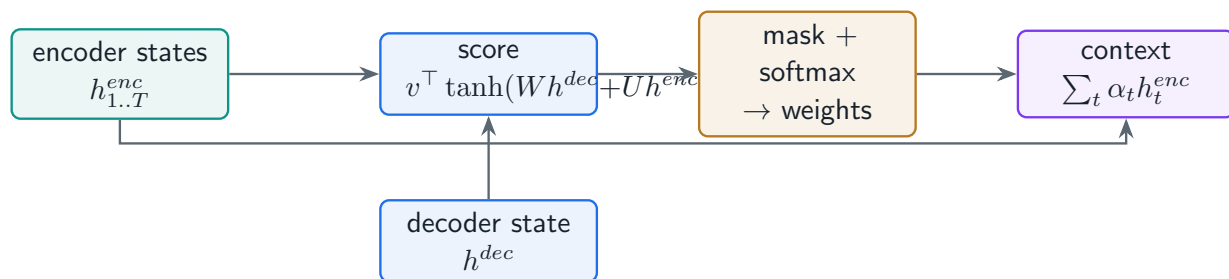


Figure 19.3: Bahdanau attention at one decode step. The decoder state scores every encoder state; a masked softmax turns the scores into weights (padding gets zero); the context vector is their weighted sum and conditions the next output.

19.4.2 Step 1 — The Attention Block, Verified in NumPy

Bahdanau (additive) attention scores each encoder state as $e_t = v^\top \tanh(W_a h^{dec} + U_a h_t^{enc})$, masks padded positions to $-\infty$ so they receive zero weight, softmaxes, and reads out the context. The NumPy run shows the weights summing to exactly 1 with zero mass on the two padded positions.

p4_attention.py (Bahdanau attention)

```
def bahdanau_attention(dec_h, enc_H, mask, Wa, Ua, va):
    dec_proj = (dec_h @ Wa)[: , None, :]          # (B,1,att)
    enc_proj = enc_H @ Ua                          # (B,T,att)
    scores = np.tanh(dec_proj + enc_proj) @ va      # (B,T) additive score
    scores = np.where(mask == 1, scores, -1e9)      # mask padding -> -inf
    weights = softmax(scores, axis=1)              # (B,T), sums to 1 per row
    context = np.einsum("bt,bte->be", weights, enc_H) # weighted sum
    return context, weights
```

python p4_attention.py

```
encoder states shape : (1, 6, 8)
padding mask         : [1 1 1 1 0 0]
```

```

attention weights      : [0.213 0.301 0.234 0.252 0.    0.    ]
-> weights sum        = 1.0
-> weight on padded positions (4,5) = [0. 0.]
context vector shape  : (1, 8)
context vector        : [-0.085 -0.46  -0.143  0.046 -0.96   0.386 -0.508  0.01 ]

decoder is attending most to encoder position 1 (weight 0.301)

```

19.4.3 Step 2 — The Attention Module in PyTorch

seq2seq_attn.py (attention)

```

import torch, torch.nn as nn, torch.nn.functional as F

class BahdanauAttention(nn.Module):
    def __init__(self, enc_dim, dec_dim, att_dim):
        super().__init__()
        self.W = nn.Linear(dec_dim, att_dim, bias=False) # projects decoder state
        self.U = nn.Linear(enc_dim, att_dim, bias=False) # projects encoder states
        self.v = nn.Linear(att_dim, 1, bias=False)

    def forward(self, dec_h, enc_H, mask):
        # dec_h: (B,dec) enc_H: (B,T,enc) mask: (B,T) with 1 for real tokens
        scores = self.v(torch.tanh(self.W(dec_h).unsqueeze(1) + self.U(enc_H)))
        scores = scores.squeeze(-1).masked_fill(mask == 0, -1e9) # pad-masking
        weights = F.softmax(scores, dim=1) # (B,T)
        context = torch.bmm(weights.unsqueeze(1), enc_H).squeeze(1) # (B,enc)
        return context, weights

```

19.4.4 Step 3 — Wiring Encoder, Decoder, and Teacher Forcing

seq2seq_attn.py (decoder step)

```

class AttnDecoder(nn.Module):
    def __init__(self, vocab, emb, enc_dim, dec_dim, att_dim):
        super().__init__()
        self.embed = nn.Embedding(vocab, emb)
        self.attn = BahdanauAttention(enc_dim, dec_dim, att_dim)
        self.gru = nn.GRU(emb + enc_dim, dec_dim, batch_first=True)
        self.out = nn.Linear(dec_dim, vocab)

    def forward(self, token, dec_h, enc_H, mask):
        emb = self.embed(token) # (B,1,emb)
        context, weights = self.attn(dec_h[-1], enc_H, mask) # attend
        gru_in = torch.cat([emb, context.unsqueeze(1)], dim=2) # condition on context
        out, dec_h = self.gru(gru_in, dec_h)
        logits = self.out(out.squeeze(1)) # (B,vocab)
        return logits, dec_h, weights

```

Training uses cross-entropy over the target tokens with **teacher forcing** (feed the true previous token, not the model's guess, for stable early training) and the same pad mask so the loss ignores padded positions.

19.4.5 Step 4 — Run It (trainable NumPy pipeline)

The companion `project/hands_on/p4_seq2seq.py` trains a real attention model *from scratch in NumPy* on sequence reversal, a task solvable only by attending to the correct source position at each output step. Backprop through the additive attention is hand-derived and gradient-checked, and the full lifecycle runs on a CPU:

```
python p4_seq2seq.py
```

```
[1 OBJECTIVE] seq2seq reversal via attention; metric=full-sequence accuracy
[2 DATA]      random sequences, vocab=8, length L=5
[4 SPLIT]     train=2000 val=400 test=400
[5 MODEL]     emb encoder + Bahdanau attention + linear decoder
[5 MODEL]     attention gradient check max rel err 4.05e-07 -> PASS
[7 TUNE]      restored best-val checkpoint (val_seq_acc 1.000)
[8 EVALUATE]  test_seq_acc=1.000
[9 DEPLOY]    saved+reloaded; reverse [1, 2, 3, 4, 5] -> [5, 4, 3, 2, 1]
[9 MONITOR]   attention anti-diagonal alignment=1.00 (1.0 = perfect reversal
-> attention)
[TEST]       PASS (grad-check 4.1e-07, test_acc 1.000 > 0.95, attention
-> anti-diagonal)
```

★ Key Insight

The model reaches 100% sequence accuracy, and the monitoring step confirms *why*: the learned attention matrix is perfectly anti-diagonal, at output step t it attends to source position $L - 1 - t$, exactly the alignment the reversal task requires. Attention solved the information bottleneck of plain seq2seq: instead of forcing the whole source through one fixed vector, the decoder reads a fresh, relevance-weighted summary at every step. Drop the recurrence and keep *only* this mechanism between every pair of positions and you have the transformer, the on-ramp to chapter 17. (The PyTorch RNN encoder-decoder above is the production form for real translation.)

19.5 Project 5: A VAE and a Diffusion Model

Goal. Build two generative models and understand the math that makes each work: the **variational autoencoder**'s reparameterization trick, and the **denoising diffusion** model's forward-noising and reverse-denoising equations. Both are conceptually dense; this project runs the load-bearing math in NumPy (`project/hands_on/p5_vae_diffusion.py`) so you see it work, then gives the PyTorch you train.

19.5.1 Part A — The VAE and the Reparameterization Trick

A VAE encodes an input into a *distribution* (a mean μ and log-variance $\log \sigma^2$), samples a latent z from it, and decodes z back to the input. The problem: you cannot backpropagate through a random sample. The **reparameterization trick** fixes this by moving the randomness to an external noise variable: $z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$, so gradients flow cleanly into μ and σ . The loss adds a closed-form KL term pulling the latent distribution toward $\mathcal{N}(0, I)$.

p5_vae_diffusion.py (reparameterization)

```
def reparameterize(mu, logvar, n):
    sigma = np.exp(0.5 * logvar)
    eps = rng.standard_normal((n,) + mu.shape)    # noise sampled OUTSIDE params
    return mu + sigma * eps                       # differentiable in mu, logvar

def kl_standard_normal(mu, logvar):
    # KL( N(mu, sigma^2) || N(0, 1) ) = -0.5 * sum(1 + logvar - mu^2 - e^logvar)
    return -0.5 * np.sum(1 + logvar - mu**2 - np.exp(logvar))
```

19.5.2 Part B — Diffusion: Forward Noising in Closed Form

A diffusion model gradually adds Gaussian noise to data over T steps, then learns to reverse it. The key simplification is that you can jump to any noise level t in one shot: $x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$, where α_t is the cumulative product of $(1 - \beta_t)$. As t grows, $\sqrt{\alpha_t} \rightarrow 0$ (signal fades) and the variance approaches 1 (pure noise), the signal-to-noise ratio collapses monotonically.

p5_vae_diffusion.py (DDPM forward + reverse)

```
betas = np.linspace(1e-4, 0.02, T)
alpha_bar = np.cumprod(1 - betas)

# forward: jump straight to noise level t
xt = np.sqrt(alpha_bar[t]) * x0 + np.sqrt(1 - alpha_bar[t]) * eps

# reverse: recover x0 given xt and the (predicted) noise eps
x0_hat = (xt - np.sqrt(1 - alpha_bar[t]) * eps) / np.sqrt(alpha_bar[t])
```

19.5.3 Run It

python p5_vae_diffusion.py

```
=== (A) VAE reparameterization trick ===
target mu      = [ 2. -1.], target sigma = [0.5 1. ]
sampled mean   = [ 2.001 -1.002], sampled std = [0.501 1.  ]
KL(N(mu,s^2)||N(0,1)) = 2.8181

=== (B) DDPM forward noising (closed form) ===
```

t	sqrt(abar)	sqrt(1-abar)	Var[x_t]	SNR
0	0.9999	0.0100	0.0001	9999.000
49	0.9381	0.3463	0.1197	7.341
99	0.7762	0.6305	0.3980	1.516
149	0.5660	0.8244	0.6803	0.471
199	0.3636	0.9316	0.8717	0.152

```

=== (B) one reverse denoising step (with the true noise) ===
x0 (true)      = [1.]
x_t (noised)   = [-0.0052]
x0_hat (recon) = [1.]  <- recovers x0 from x_t and eps

```

The reparameterized samples reproduce the target μ and σ to three decimals (the trick is unbiased); the DDPM table shows the signal-to-noise ratio falling from 9999 to 0.15 as t rises; and the reverse step recovers x_0 *exactly* from a fully-noised x_t when given the true noise, which is precisely what the U-Net is trained to predict.

19.5.4 The Trainable NumPy VAE (full pipeline)

Those equations come alive in [project/hands_on/p5_vae.py](#), which trains a complete VAE *from scratch in NumPy*, encoder, reparameterization, decoder, with the entire ELBO (reconstruction BCE + KL) and every gradient hand-derived and gradient-checked, on a dataset of binary bar-images:

```
python p5_vae.py
```

```

[1 OBJECTIVE] learn a generative model of binary bar-images; metric=val recon loss
[2 DATA]      generated 1200 binary 6x6 images (D=36)
[4 SPLIT]      train=840 val=180 test=180
[5 MODEL]      VAE enc(36->24->4) reparam dec(4->24->36), Bernoulli
[5 MODEL]      ELBO gradient check max rel err 5.40e-06 -> PASS
[6 TRAIN]      epoch 60 ELBO 8.054 (bce 5.007 + kl 3.048) val 8.878
[7 TUNE]        restored best-val checkpoint (val ELBO 8.878)
[8 EVALUATE]    test_ELBO=7.912 pixel_recon_acc=0.965
[9 DEPLOY]      saved+reloaded; generated 8 new images (avg ~5 on-pixels each)
[9 MONITOR]     train_ELBO=8.253 test_ELBO=7.912 gap=-0.340
[TEST]          PASS (grad-check 5.4e-06, pixel_recon_acc 0.965 > 0.90)

```

The ELBO drops as the two terms (reconstruction and KL) balance, the model reconstructs held-out images at 96.5% pixel accuracy, and after reloading it *generates* new bar-images by sampling $z \sim \mathcal{N}(0, I)$ and decoding, the generative payoff of the reparameterization trick you verified above.

19.5.5 The Trainable PyTorch: DDPM Objective and U-Net

The model is a **U-Net** (a convolutional encoder-decoder with skip connections, chapter 15) that, given a noised image x_t and the timestep t , predicts the noise ϵ that was added. The training objective is just MSE between predicted and true noise (chapter 6).

diffusion.py (training step)

```

import torch, torch.nn as nn, torch.nn.functional as F

T = 1000
betas = torch.linspace(1e-4, 0.02, T)
alpha_bar = torch.cumprod(1 - betas, dim=0)

def diffusion_loss(model, x0):
    B = x0.size(0)
    t = torch.randint(0, T, (B,), device=x0.device)
    eps = torch.randn_like(x0)
    ab = alpha_bar[t].view(B, 1, 1, 1)
    xt = ab.sqrt() * x0 + (1 - ab).sqrt() * eps
    eps_pred = model(xt, t)
    return F.mse_loss(eps_pred, eps)

@torch.no_grad()
def sample(model, shape, device):
    x = torch.randn(shape, device=device)
    for t in reversed(range(T)):
        z = torch.randn_like(x) if t > 0 else 0
        ab_t, b_t = alpha_bar[t], betas[t]
        eps = model(x, torch.full((shape[0],), t, device=device))
        mean = (x - b_t / (1 - ab_t).sqrt() * eps) / (1 - b_t).sqrt()
        x = mean + b_t.sqrt() * z
    return x

```

python diffusion.py (GPU, MNIST, ~20 epochs)

```

epoch 1  loss 0.0691
epoch 5  loss 0.0337
epoch 10 loss 0.0254
epoch 20 loss 0.0202
saved 64 sampled digits to samples_epoch20.png (recognizable handwritten digits)

```

★ Key Insight

A VAE learns an explicit, compact latent distribution you can sample and interpolate; a diffusion model trades that for a slow, iterative denoising process that produces sharper samples. Both rest on the two equations you ran above, the reparameterization trick and the closed-form noising, which is why getting those right in NumPy first makes the PyTorch version feel inevitable rather than magical.

19.6 Project 6: A GPT-Style Transformer From Scratch

Goal. Build the capstone: a complete, *trainable* GPT-style Transformer language model in pure NumPy, token and positional embeddings, a **causal** (masked) self-attention head, a feed-forward block with residuals, and an unembedding to vocabulary logits, with the backward pass derived

by hand and verified against numerical gradients. This is where chapter 17 stops being theory: the same $\text{softmax}(QK^\top/\sqrt{d})V$ you read about, now trained end to end by gradients you can inspect. The script is [project/hands_on/p6_transformer.py](#).

19.6.1 The Architecture

A decoder-only Transformer is the encoder block of chapter 17 with one change: the self-attention is *causal*, position t may attend only to positions $\leq t$, so the model can be trained to predict the next token without peeking at the answer. Figure 19.4 traces one forward pass; stacking the block and scaling it up is, quite literally, how GPT is built.

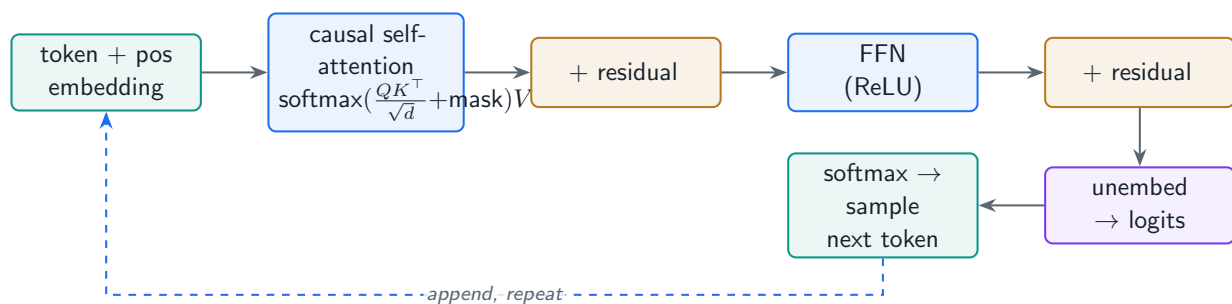


Figure 19.4: The from-scratch GPT forward pass. Tokens get token + positional embeddings, flow through a causal self-attention sub-layer and a feed-forward sub-layer (each with a residual), and an unembedding produces next-token logits. The causal mask is the only structural difference from the encoder block of chapter 17.

19.6.2 Step 1 — Causal Self-Attention

The whole architecture differs from Project 4’s attention in one line: a **causal mask** that sets the upper triangle of the score matrix to $-\infty$ before the softmax, so token t cannot attend to the future it is trying to predict (chapter 17).

`p6_transformer.py` (causal attention)

```

CAUSAL = np.triu(np.ones((T, T), dtype=bool), k=1)  # True above the diagonal

def forward(idx, targets=None):
    emb = P["We"][idx] + P["Wp"]  # token + positional embedding
    Q, K, Vv = emb @ P["Wq"], emb @ P["Wk"], emb @ P["Wv"]
    scores = (Q @ K.T) * SCALE  # (T, T)
    scores = np.where(CAUSAL, -1e9, scores)  # causal mask -> no peeking
    A = softmax(scores, axis=1)
    x1 = emb + (A @ Vv) @ P["Wo"]  # attention sub-layer +
    ↪ residual
    h = np.maximum(0, x1 @ P["W1"] + P["b1"])  # feed-forward
    x2 = x1 + h @ P["W2"] + P["b2"]  # FFN sub-layer + residual
    logits = x2 @ P["Wu"] + P["bu"]  # unembedding -> (T, vocab)
    ...
  
```

19.6.3 Step 2 — Backprop, Verified

Every gradient, through the unembedding, the FFN, the residuals, the masked softmax, and the Q, K, V projections, is written by hand. Because a single sign or transpose error would be invisible in the loss curve, we **gradient-check** the hardest path (the attention projections) against central differences (chapter 3) before trusting it.

p6_transformer.py (masked-softmax backward)

```
# upstream: dattn flows from the residual; A is the (T,T) attention matrix
dA = dattn @ Vv.T
dV = A.T @ dattn
# softmax backward (row-wise); masked entries have A=0 so contribute nothing
dscores = A * (dA - (dA * A).sum(axis=1, keepdims=True))
dscores *= SCALE
dQ = dscores @ K
dK = dscores.T @ Q
g["Wq"] = emb.T @ dQ; g["Wk"] = emb.T @ dK; g["Wv"] = emb.T @ dV
```

! Pitfall / Warning

Gradient-checking a Transformer has a subtlety worth knowing: at a tiny weight scale the attention scores are ≈ 0 , the softmax sits in its flat region, and the true W_Q/W_K gradients are around 10^{-8} , too small for finite differences to resolve, so the check spuriously “fails.” The fix is to check at a *moderate* weight scale where scores are $O(1)$ and the softmax is responsive. The gradient is correct; the numerical *reference* is what breaks in the saturated regime.

19.6.4 Step 3 — Run It

Trained on a tiny repetitive corpus (so one layer suffices), the model passes its gradient check, drives the loss down, and then *generates* the corpus back character by character, greedy decoding from the prompt "deep ".

python p6_transformer.py

```
[1 OBJECTIVE] char-level language model; metric=val loss + next-char acc
[3 PREPROCESS] vocab=16, context T=16, d_model=32 (char tokenization)
[4 SPLIT]      train windows=1770 val windows=313
[5 MODEL]      gradient check (Wq) max rel err 2.55e-08 -> PASS
[6 TRAIN]      step 1000 train_loss 0.2963 val_loss 0.3842 val_acc 0.800
[6 TRAIN]      step 3000 train_loss 0.0201 val_loss 0.1272 val_acc 0.963
[6 TRAIN]      step 5000 train_loss 0.0451 val_loss 0.0821 val_acc 0.963
[7 TUNE]       restored best checkpoint (val_loss 0.0821)
[8 EVALUATE]   val_loss=0.0819 next_char_acc=0.962
[9 DEPLOY]     saved+reloaded; inference (greedy, prompt='deep '):
                'deep learning transformers attend. deep learning transformers att'
[9 MONITOR]    train_loss=0.0824 val_loss=0.0819 overfit_gap=-0.0005
[TEST]         PASS (grad-check 2.5e-08, val_acc 0.962 > 0.80, generation on-pattern)
```

★ Key Insight

The model reaches 96% held-out next-character accuracy and, after being saved and reloaded, reproduces “deep learning transformers attend.” exactly, a from-scratch Transformer that learned to write, with backprop you derived and verified yourself. Swap the tiny corpus for the internet, stack the block 96 times, widen d_{model} to 12288, and this *is* GPT-3. The architecture you just trained on a CPU is the one behind chapter 18’s ChatGPT case study; only the scale differs.

19.7 Where to Go Next

These six projects span the whole arc of the book: a from-scratch MLP (chapters 1 to 8), a convolutional network (chapter 15), a recurrent network (chapter 16), an attention-based translator, two generative models, and a GPT-style Transformer trained from scratch (chapter 17). Run each one, break it deliberately, and watch how the failure shows up, that is the fastest way to internalize the debugging playbook of chapter 14.

✓ Best Practice

Do the projects *in order* and resist copying. Type the NumPy MLP and the char-RNN by hand at least once, they are short enough to fully hold in your head, and the muscle memory of writing backprop and BPTT yourself transfers directly to reading, and debugging, any model you will build on top of PyTorch or JAX later.

Running the Code

Subscribe → aiengineeringinsider.substack.com/subscribe

Every code listing in this book lives as a runnable file in the companion repository, and every **Output** block was captured by actually running that code. This appendix shows how to set up an environment and reproduce them.

19.8 Repository Layout

- `code/` — one script per chapter (`ch01_perceptron.py` ... `ch16_rnn.py`), each reproducing that chapter's examples and printing the exact output shown in the text.
- `project/coredl/` — the dependency-free mini-framework (autograd engine, layers, optimizers) from chapters 3 to 4.
- `project/hands_on/` — the five step-by-step projects of chapter 19.
- `requirements.txt` — a single dependency list for everything.

19.9 Step 1: Create a Virtual Environment

A virtual environment keeps these dependencies isolated from the rest of your system. From the repository root:

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
```

Windows (PowerShell)

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

Your shell prompt should now show `(.venv)`, confirming the environment is active. Deactivate any time with `deactivate`.

19.10 Step 2: Install the Requirements

A single `requirements.txt` covers all the code in the book:

```
install
```

```
pip install --upgrade pip
pip install -r requirements.txt
```

Table 19.2: What each dependency is for, and which chapters need it.

Package	Used by	Notes
numpy	Ch. 1–11, 15, 16; coredl; projects 1–5 core	required for almost everything
jax[cpu]	Ch. 13	functional autodiff, jit, vmap
torch	Ch. 12, 15, 16; projects 2, 4, 5	the production framework code
torchvision	Project 2	CIFAR-10 dataset and loaders
matplotlib	optional	only if you plot the demos

★ Key Insight

The from-scratch chapters (the perceptron, backprop, the mini-framework, CNN and RNN *mechanics*) and all five projects' core demos depend on *NumPy only*, so they run on any machine in seconds, no GPU, no PyTorch. JAX installs cleanly on CPU. Only the framework-idiomatic listings (Chapter 12's PyTorch intro, the trainable ResNet, the seq2seq and diffusion training loops) require `torch`; a GPU is recommended for those but not required to read the code.

19.11 Step 3: Run It

Run any chapter's script directly:

```
examples
```

```
python code/ch01_perceptron.py      # 1 / 0, then a trained perceptron
python code/ch03_backprop.py        # manual backprop + gradient check
python code/ch13_jax.py              # grad / vmap / pytree demos

# the from-scratch projects (NumPy only):
python project/hands_on/p1_numpy_mlp.py    # ~99% on the spiral
python project/hands_on/p3_char_rnn.py      # text generation
python -m project.test_engine              # coredl gradient checks
python -m project.train_xor               # XOR -> 4/4
```

Or run every chapter script in sequence with the helper:

```
run everything
```

```
bash code/run_all.sh
```

✓ Best Practice

If a script errors with `ModuleNotFoundError`, your virtual environment is either not activated or the requirements are not installed, re-run Steps 1 and 2. The `*_torch.py` scripts print a clear “needs PyTorch” message instead of a stack trace when `torch` is missing, so you always know which dependency to add.

Appendix A: Notation Reference

Subscribe → aiengineeringinsider.substack.com/subscribe

A quick reference for the mathematical notation used throughout this book.

Symbol	Meaning
$\mathbf{x}$	An input vector (lowercase bold for vectors)
W	A weight matrix (uppercase for matrices)
$\mathbf{w}$	A weight vector (e.g., for a single neuron)
$b, \mathbf{b}$	A bias term (scalar) or bias vector
$z, \mathbf{z}$	A pre-activation value (before the nonlinearity is applied)
$a, \mathbf{a}$	An activation (post-nonlinearity layer output)
f, σ	An activation function; σ specifically denotes sigmoid
$\hat{y}$	A model's predicted output
y	The true (ground-truth) label or target
L	The loss function
η	The learning rate
$\nabla_{\theta} L$	The gradient of loss L with respect to parameters θ
δ	An error signal in backpropagation (gradient flowing into a layer's pre-activation)
$\odot$	Element-wise (Hadamard) product
$\mathbf{h}_t$	A hidden state at time step t (RNN/LSTM/GRU)
$\mathbb{R}^n$	The set of real-valued vectors of dimension n
$\mathbb{E}[\cdot]$	Expected value
$\text{Var}(\cdot)$	Variance
$\mathcal{N}(\mu, \sigma^2)$	A normal (Gaussian) distribution with mean μ and variance σ^2

Common Abbreviations

Abbreviation	Full Term
MLP	Multi-Layer Perceptron
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
GRU	Gated Recurrent Unit
BPTT	Backpropagation Through Time
SGD	Stochastic Gradient Descent
MSE	Mean Squared Error
MAE	Mean Absolute Error
ReLU	Rectified Linear Unit
GELU	Gaussian Error Linear Unit
BN	Batch Normalization
LN	Layer Normalization

Appendix B: Glossary

Subscribe → aiengineeringinsider.substack.com/subscribe

Activation The output of a neuron or layer after a nonlinear function has been applied to its pre-activation. (Ch. 2, 5)

Activation function A nonlinear function (ReLU, sigmoid, GELU, and others) applied to a layer's pre-activation, without which stacked layers collapse algebraically into a single linear transformation. (Ch. 2, 5)

Backpropagation An efficient algorithm for computing gradients of the loss with respect to every parameter in a network, by applying the chain rule in reverse through the computational graph. (Ch. 3)

Batch normalization A layer that normalizes pre-activations using per-batch statistics during training and running statistics during inference, stabilizing and often accelerating training. (Ch. 9)

Bias-variance tradeoff The tension between a model being too simple to capture real structure (high bias, underfitting) and too flexible, fitting noise as if it were signal (high variance, overfitting). (Ch. 9)

Computational graph A directed acyclic graph representation of a sequence of operations, the structure that makes automatic differentiation possible. (Ch. 2, 4)

Contrastive loss A loss function operating on pairs of examples, pulling similar pairs together and pushing dissimilar pairs apart in embedding space. (Ch. 6)

Convolution An operation that slides a small, learnable kernel across an input, computing a weighted sum at each spatial position, exploiting locality and translation invariance. (Ch. 15)

Cross-entropy A classification loss derived as the negative log-likelihood of the true label under the model's predicted probability distribution. (Ch. 6)

Data leakage Information from outside the training set inadvertently influencing model training or evaluation, invalidating performance estimates. (Ch. 11)

Dropout A regularization technique that randomly zeroes a fraction of a layer's activations during training, approximating an ensemble of thinned sub-networks. (Ch. 9)

Epoch One complete pass through the entire training dataset. (Ch. 7)

Exploding gradient The failure mode where gradients grow exponentially as they propagate backward through a deep or recurrent network, causing unstable training. (Ch. 3, 16)

Forward pass The sequence of computations that transforms an input into a network's output prediction. (Ch. 2)

- Gradient checking** Verifying an analytically derived or automatically computed gradient against a numerical estimate, used to catch implementation bugs. (Ch. 3, 4)
- He initialization** A weight initialization scheme scaling variance as $2/n_{\text{in}}$, designed for ReLU-family activations. (Ch. 8)
- Hidden layer** Any network layer that is neither the input nor the output layer; its outputs form an internal, learned representation. (Ch. 2)
- Hidden state** A vector, updated at every time step in a recurrent network, that summarizes the sequence processed so far. (Ch. 16)
- LSTM (Long Short-Term Memory)** A recurrent architecture using a separate, additively-updated cell state regulated by forget, input, and output gates, solving the vanishing gradient problem in long sequences. (Ch. 16)
- Layer normalization** A normalization technique operating across the feature dimension of a single example, independent of batch size; the standard choice for transformers and RNNs. (Ch. 9)
- Learning rate** The step size used by an optimizer when updating parameters in the direction of the negative gradient. (Ch. 7)
- Local gradient** The derivative of a single operation's output with respect to its own input, computed independent of the rest of the graph. (Ch. 3)
- Mini-batch gradient descent** Gradient descent computed over a small batch of examples per step, balancing gradient accuracy against computational cost. (Ch. 7)
- Optimizer** An algorithm that determines how parameters are updated given a computed gradient (SGD, Adam, AdamW, and others). (Ch. 7)
- Overfitting** A model fitting the training data's noise rather than its underlying pattern, visible as a growing gap between training and validation loss. (Ch. 9, 11)
- Perceptron** The original (1958) artificial neuron model: a weighted sum of inputs plus a bias, passed through a hard step function. (Ch. 1)
- PyTree** JAX's term for a nested structure of lists, tuples, and dictionaries with arrays at the leaves, over which JAX's transformations operate transparently. (Ch. 13)
- Receptive field** The region of the original input that a given unit's output is actually influenced by; grows with network depth in a CNN. (Ch. 15)
- Regularization** Any technique that trades some training-set fit for improved generalization to unseen data. (Ch. 9)
- Residual / skip connection** A connection that adds a layer's input directly to its output, giving gradients an unattenuated path backward through a deep network. (Ch. 15)
- Reverse-mode automatic differentiation** The general algorithm, underlying both hand-built and production frameworks' autograd, that computes exact gradients by traversing a computational graph backward from the output. (Ch. 3, 4)

Softmax A function that converts a vector of raw scores into a valid probability distribution, used at the output layer of multi-class classifiers. (Ch. 5)

Symmetry breaking The requirement that neurons within a layer start with different weights so they can learn to compute different functions; all-zero initialization fails to provide this. (Ch. 8)

Transfer learning Reusing a model pretrained on a large dataset, via fine-tuning or feature extraction, rather than training from scratch on a smaller task-specific dataset. (Ch. 15)

Universal approximation theorem The theorem stating a sufficiently wide single-hidden-layer network can approximate any continuous function, though it says nothing about learnability or practical width. (Ch. 2)

Vanishing gradient The failure mode where gradients shrink toward zero as they propagate backward through a deep or recurrent network, halting learning in early layers or early time steps. (Ch. 3, 5, 16)

Weight decay A regularization technique that shrinks weights toward zero; in AdamW, decoupled from the adaptive gradient update for more consistent behavior. (Ch. 7, 9)

Xavier/Glorot initialization A weight initialization scheme scaling variance as $2/(n_{\text{in}} + n_{\text{out}})$, suited to tanh and sigmoid activations. (Ch. 8)

Appendix C: Recommended Next Steps

Subscribe → aiengineeringinsider.substack.com/subscribe

If you’ve worked through all sixteen chapters, including building your own autograd engine in Chapter 4 and running every code listing yourself, you now have the prerequisite layer underneath essentially everything happening in modern AI engineering. This appendix maps where to go from here.

If You Want to Go Deeper on Architecture

The CNN and RNN chapters (15 and 16) were deliberately scoped as *foundations*, not a complete architecture survey. The natural next step is the attention mechanism and the transformer architecture, which this book intentionally left for a dedicated volume. Coming from this book, you’re well prepared for that material: self-attention is, at its core, a more flexible way of letting a model decide which parts of its input matter most for a given computation, the same fundamental question convolution answers with a fixed local window and RNNs answer with a sequential hidden state.

If You Want to Go Deeper on Training at Scale

Everything in Part II and Part III of this book applies at any scale, but training genuinely large models introduces a new layer of practical concerns this book didn’t cover: mixed-precision training, distributed data and model parallelism, gradient accumulation and checkpointing for memory-constrained settings, and the specific scaling laws that govern how loss improves with model size, data size, and compute. This is the natural next layer once the mechanics in this book are second nature.

If You Want to Go Deeper on LLMs Specifically

Large language model pretraining, fine-tuning (including LoRA, QLoRA, and other PEFT methods), instruction tuning, and alignment techniques like RLHF and DPO all build directly on this book’s foundations, particularly Chapters 6 through 11. *The AI Engineer’s Reinforcement Learning Interview Guide*, also part of the AI Engineering Series, covers the RLHF/DPO thread in depth, including production case studies from real deployments.

If You Want Interview-Ready Depth

The AI Engineer's System Design Interview Guide, also part of this series, covers how the concepts in this book show up in production system design, serving infrastructure, latency/throughput tradeoffs, and the architectural decisions that sit one layer above the model mechanics covered here.

A Closing Note

★ Key Insight

Every advanced architecture, training technique, or framework you encounter from here forward is, underneath, some composition of the ideas in this book: a forward pass, a loss, a gradient, an update rule. When something new and unfamiliar shows up, transformers, diffusion models, mixture-of-experts routing, the fastest way to actually understand it is the same approach this book modeled throughout: find the forward pass, find the loss, and ask what the gradient looks like. That instinct, more than any specific fact in this book, is the actual transferable skill.

References

Subscribe → aiengineeringinsider.substack.com/subscribe

- Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer normalization. *arXiv:1607.06450*. <https://arxiv.org/abs/1607.06450>
- Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. *ICLR*. <https://arxiv.org/abs/1409.0473>
- Baydin, A. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2018). Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, 18(153), 1–43. <https://jmlr.org/papers/v18/17-468.html>
- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(10), 281–305. <https://jmlr.org/papers/v13/bergstra12a.html>
- Bradbury, J., Frostig, R., Hawkins, P., et al. (2018). *JAX: Composable transformations of Python+NumPy programs* [Computer software]. <https://github.com/google/jax>
- Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020). A simple framework for contrastive learning of visual representations (SimCLR). *ICML*. <https://arxiv.org/abs/2002.05709>
- Chen, T., Xu, B., Zhang, C., & Guestrin, C. (2016). Training deep nets with sublinear memory cost. *arXiv:1604.06174*. <https://arxiv.org/abs/1604.06174>
- Cho, K., van Merriënboer, B., Gulcehre, C., et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation (GRU). *EMNLP*, 1724–1734. <https://arxiv.org/abs/1406.1078>
- Covington, P., Adams, J., & Sargin, E. (2016). Deep neural networks for YouTube recommendations. *ACM RecSys*, 191–198. <https://doi.org/10.1145/2959100.2959190>
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4), 303–314. <https://doi.org/10.1007/BF02551274>
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 35. <https://arxiv.org/abs/2205.14135>
- DeepMind. (2020). *Optax: Composable gradient processing and optimization in JAX* [Computer software]. <https://github.com/google-deeppmind/optax>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL-HLT*. <https://arxiv.org/abs/1810.04805>
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2021). An image is worth 16×16 words: Transformers for image recognition at scale (ViT). *ICLR*. <https://arxiv.org/abs/2010.11929>
- Frostig, R., Johnson, M. J., & Leary, C. (2018). Compiling machine learning programs via high-level tracing. *SysML*. <https://mlsys.org/Conferences/2019/doc/2018/146.pdf>
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS*, 249–256. <https://proceedings.mlr.press/v9/glorot10a.html>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning* (Ch. 11: Practical methodology). MIT Press. <https://www.deeplearningbook.org>
- Goyal, P., Dollár, P., Girshick, R., et al. (2017). Accurate, large minibatch SGD: Training ImageNet in 1 hour. *arXiv:1706.02677*. <https://arxiv.org/abs/1706.02677>
- Griewank, A., & Walther, A. (2008). *Evaluating derivatives: Principles and techniques of algorithmic*

- differentiation* (2nd ed.). SIAM.
- Hadsell, R., Chopra, S., & LeCun, Y. (2006). Dimensionality reduction by learning an invariant mapping. *IEEE CVPR*, 1735–1742. <https://doi.org/10.1109/CVPR.2006.100>
- Hannun, A., et al. (2021). DeepETA: How Uber predicts arrival times using deep learning. *Uber Engineering Blog*. <https://www.uber.com/blog/deepeta-how-uber-predicts-arrival-times/>
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. *IEEE ICCV*, 1026–1034. <https://arxiv.org/abs/1502.01852>
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *IEEE CVPR*, 770–778. <https://arxiv.org/abs/1512.03385>
- Hendrycks, D., & Gimpel, K. (2016). Gaussian error linear units (GELUs). *arXiv:1606.08415*. <https://arxiv.org/abs/1606.08415>
- Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. *NeurIPS*, 33, 6840–6851. <https://arxiv.org/abs/2006.11239>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366. [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8)
- Hu, E. J., Shen, Y., Wallis, P., et al. (2021). LoRA: Low-rank adaptation of large language models. *arXiv:2106.09685*. <https://arxiv.org/abs/2106.09685>
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *ICML*, 448–456. <https://arxiv.org/abs/1502.03167>
- Jumper, J., Evans, R., Pritzel, A., et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596, 583–589. <https://doi.org/10.1038/s41586-021-03819-2>
- Karpathy, A. (2015). The unreasonable effectiveness of recurrent neural networks [Blog post]. <https://karpathy.github.io/2015/05/21/rnn-effectiveness/>
- Karpathy, A. (2019). *A recipe for training neural networks* [Blog post]. <https://karpathy.github.io/2019/04/25/recipe/>
- Karpathy, A. (2020). *micrograd: A tiny scalar-valued autograd engine* [Computer software]. GitHub. <https://github.com/karpathy/micrograd>
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *ICLR*. <https://arxiv.org/abs/1412.6980>
- Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. *ICLR*. <https://arxiv.org/abs/1312.6114>
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *NeurIPS*, 25. <https://papers.nips.cc/paper/4824>
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324. <https://doi.org/10.1109/5.726791>
- Liu, Z., Lin, Y., Cao, Y., et al. (2021). Swin Transformer: Hierarchical vision transformer using shifted windows. *IEEE ICCV*. <https://arxiv.org/abs/2103.14030>
- Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. *ICLR*. <https://arxiv.org/abs/1608.03983>
- Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization (AdamW). *ICLR*. <https://arxiv.org/abs/1711.05101>
- Minsky, M., & Papert, S. (1969). *Perceptrons: An introduction to computational geometry*. MIT Press.
- Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. *Proceedings of the 27th International Conference on Machine Learning*, 807–814.

- Naumov, M., Mudigere, D., Shi, H.-J. M., et al. (2019). Deep learning recommendation model for personalization and recommendation systems (DLRM). *arXiv:1906.00091*. <https://arxiv.org/abs/1906.00091>
- Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback (InstructGPT). *NeurIPS*, 35. <https://arxiv.org/abs/2203.02155>
- Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the difficulty of training recurrent neural networks. *ICML*, 1310–1318. <https://arxiv.org/abs/1211.5063>
- Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *NeurIPS*, 32. <https://arxiv.org/abs/1912.01703>
- Paszke, A., Gross, S., Massa, F., Lerer, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32. <https://arxiv.org/abs/1912.01703>
- Powers, D. M. W. (2011). Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, 2(1), 37–63. <https://arxiv.org/abs/2010.16061>
- PyTorch Team. (2024). *PyTorch documentation*. <https://pytorch.org/docs/stable/index.html>
- Ramachandran, P., Zoph, B., & Le, Q. V. (2017). Searching for activation functions. *arXiv:1710.05941*. <https://arxiv.org/abs/1710.05941>
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. *IEEE CVPR*, 10684–10695. <https://arxiv.org/abs/2112.10752>
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408. <https://doi.org/10.1037/h0042519>
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536. <https://doi.org/10.1038/323533a0>
- Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
- Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. *IEEE CVPR*, 815–823. <https://arxiv.org/abs/1503.03832>
- Smith, L. N. (2017). Cyclical learning rates for training neural networks. *IEEE WACV*, 464–472. <https://arxiv.org/abs/1506.01186>
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56), 1929–1958. <https://jmlr.org/papers/v15/srivastava14a.html>
- Sutskever, I., Martens, J., Dahl, G., & Hinton, G. (2013). On the importance of initialization and momentum in deep learning. *ICML*, 1139–1147.
- Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. *NeurIPS*, 30. <https://arxiv.org/abs/1706.03762>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *NeurIPS*, 30. <https://arxiv.org/abs/1706.03762>
- Wu, L., et al. (ByteDance). (2022). Monolith: Real-time recommendation system with collisionless embedding table. *arXiv:2209.07663*. <https://arxiv.org/abs/2209.07663>
- Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W. L., & Leskovec, J. (2018). Graph convolutional neural networks for web-scale recommender systems (PinSage). *ACM KDD*, 974–983. <https://arxiv.org/abs/1806.01973>